

33 Command Line Editing

This chapter describes the basic features of the GNU command line editing interface.

33.1 Introduction to Line Editing

The following paragraphs describe the notation used to represent keystrokes.

The text *C-k* is read as ‘Control-K’ and describes the character produced when the *k* key is pressed while the Control key is depressed.

The text *M-k* is read as ‘Meta-K’ and describes the character produced when the Meta key (if you have one) is depressed, and the *k* key is pressed. The Meta key is labeled **ALT** on many keyboards. On keyboards with two keys labeled **ALT** (usually to either side of the space bar), the **ALT** on the left side is generally set to work as a Meta key. The **ALT** key on the right may also be configured to work as a Meta key or may be configured as some other modifier, such as a Compose key for typing accented characters.

If you do not have a Meta or **ALT** key, or another key working as a Meta key, the identical keystroke can be generated by typing **ESC** *first*, and then typing *k*. Either process is known as *metafying* the *k* key.

The text *M-C-k* is read as ‘Meta-Control-k’ and describes the character produced by *metafying* *C-k*.

In addition, several keys have their own names. Specifically, **DEL**, **ESC**, **LFD**, **SPC**, **RET**, and **TAB** all stand for themselves when seen in this text, or in an init file (see Section 33.3 [Readline Init File], page 736). If your keyboard lacks a **LFD** key, typing *C-j* will produce the desired character. The **RET** key may be labeled **Return** or **Enter** on some keyboards.

33.2 Readline Interaction

Often during an interactive session you type in a long line of text, only to notice that the first word on the line is misspelled. The Readline library gives you a set of commands for manipulating the text as you type it in, allowing you to just fix your typo, and not forcing you to retype the majority of the line. Using these editing commands, you move the cursor to the place that needs correction, and delete or insert the text of the corrections. Then, when you are satisfied with the line, you simply press **RET**. You do not have to be at the end of the line to press **RET**; the entire line is accepted regardless of the location of the cursor within the line.

33.2.1 Readline Bare Essentials

In order to enter characters into the line, simply type them. The typed character appears where the cursor was, and then the cursor moves one space to the right. If you mistype a character, you can use your erase character to back up and delete the mistyped character.

Sometimes you may mistype a character, and not notice the error until you have typed several other characters. In that case, you can type *C-b* to move the cursor to the left, and then correct your mistake. Afterwards, you can move the cursor to the right with *C-f*.

When you add text in the middle of a line, you will notice that characters to the right of the cursor are ‘pushed over’ to make room for the text that you have inserted. Likewise, when you delete text behind the cursor, characters to the right of the cursor are ‘pulled

back' to fill in the blank space created by the removal of the text. A list of the bare essentials for editing the text of an input line follows.

C-b Move back one character.

C-f Move forward one character.

DEL or **Backspace**

Delete the character to the left of the cursor.

C-d Delete the character underneath the cursor.

Printing characters

Insert the character into the line at the cursor.

C-_ or **C-x C-u**

Undo the last editing command. You can undo all the way back to an empty line.

(Depending on your configuration, the **Backspace** key might be set to delete the character to the left of the cursor and the **DEL** key set to delete the character underneath the cursor, like **C-d**, rather than the character to the left of the cursor.)

33.2.2 Readline Movement Commands

The above table describes the most basic keystrokes that you need in order to do editing of the input line. For your convenience, many other commands have been added in addition to **C-b**, **C-f**, **C-d**, and **DEL**. Here are some commands for moving more rapidly about the line.

C-a Move to the start of the line.

C-e Move to the end of the line.

M-f Move forward a word, where a word is composed of letters and digits.

M-b Move backward a word.

C-l Clear the screen, reprinting the current line at the top.

Notice how **C-f** moves forward a character, while **M-f** moves forward a word. It is a loose convention that control keystrokes operate on characters while meta keystrokes operate on words.

33.2.3 Readline Killing Commands

Killing text means to delete the text from the line, but to save it away for later use, usually by *yanking* (re-inserting) it back into the line. ('Cut' and 'paste' are more recent jargon for 'kill' and 'yank'.)

If the description for a command says that it 'kills' text, then you can be sure that you can get the text back in a different (or the same) place later.

When you use a kill command, the text is saved in a *kill-ring*. Any number of consecutive kills save all of the killed text together, so that when you yank it back, you get it all. The kill ring is not line specific; the text that you killed on a previously typed line is available to be yanked back later, when you are typing another line.

Here is the list of commands for killing text.

C-k	Kill the text from the current cursor position to the end of the line.
M-d	Kill from the cursor to the end of the current word, or, if between words, to the end of the next word. Word boundaries are the same as those used by M-f .
M-DEL	Kill from the cursor to the start of the current word, or, if between words, to the start of the previous word. Word boundaries are the same as those used by M-b .
C-w	Kill from the cursor to the previous whitespace. This is different than M-DEL because the word boundaries differ.

Here is how to *yank* the text back into the line. Yanking means to copy the most-recently-killed text from the kill buffer.

C-y	Yank the most recently killed text back into the buffer at the cursor.
M-y	Rotate the kill-ring, and yank the new top. You can only do this if the prior command is C-y or M-y .

33.2.4 Readline Arguments

You can pass numeric arguments to Readline commands. Sometimes the argument acts as a repeat count, other times it is the *sign* of the argument that is significant. If you pass a negative argument to a command which normally acts in a forward direction, that command will act in a backward direction. For example, to kill text back to the start of the line, you might type ‘**M-- C-k**’.

The general way to pass numeric arguments to a command is to type meta digits before the command. If the first ‘digit’ typed is a minus sign (‘-’), then the sign of the argument will be negative. Once you have typed one meta digit to get the argument started, you can type the remainder of the digits, and then the command. For example, to give the **C-d** command an argument of 10, you could type ‘**M-1 0 C-d**’, which will delete the next ten characters on the input line.

33.2.5 Searching for Commands in the History

Readline provides commands for searching through the command history for lines containing a specified string. There are two search modes: *incremental* and *non-incremental*.

Incremental searches begin before the user has finished typing the search string. As each character of the search string is typed, Readline displays the next entry from the history matching the string typed so far. An incremental search requires only as many characters as needed to find the desired history entry. To search backward in the history for a particular string, type **C-r**. Typing **C-s** searches forward through the history. The characters present in the value of the `isearch-terminators` variable are used to terminate an incremental search. If that variable has not been assigned a value, the **ESC** and **C-J** characters will terminate an incremental search. **C-g** will abort an incremental search and restore the original line. When the search is terminated, the history entry containing the search string becomes the current line.

To find other matching entries in the history list, type **C-r** or **C-s** as appropriate. This will search backward or forward in the history for the next entry matching the search string

typed so far. Any other key sequence bound to a Readline command will terminate the search and execute that command. For instance, a **RET** will terminate the search and accept the line, thereby executing the command from the history list. A movement command will terminate the search, make the last line found the current line, and begin editing.

Readline remembers the last incremental search string. If two **C-rs** are typed without any intervening characters defining a new search string, any remembered search string is used.

Non-incremental searches read the entire search string before starting to search for matching history lines. The search string may be typed by the user or be part of the contents of the current line.

33.3 Readline Init File

Although the Readline library comes with a set of Emacs-like keybindings installed by default, it is possible to use a different set of keybindings. Any user can customize programs that use Readline by putting commands in an *inputrc* file, conventionally in their home directory. The name of this file is taken from the value of the environment variable **INPUTRC**. If that variable is unset, the default is **~/.inputrc**. If that file does not exist or cannot be read, the ultimate default is **/etc/inputrc**.

When a program which uses the Readline library starts up, the init file is read, and the key bindings are set.

In addition, the **C-x C-r** command re-reads this init file, thus incorporating any changes that you might have made to it.

33.3.1 Readline Init File Syntax

There are only a few basic constructs allowed in the Readline init file. Blank lines are ignored. Lines beginning with a **#** are comments. Lines beginning with a **\$** indicate conditional constructs (see Section 33.3.2 [Conditional Init Constructs], page 745). Other lines denote variable settings and key bindings.

Variable Settings

You can modify the run-time behavior of Readline by altering the values of variables in Readline using the **set** command within the init file. The syntax is simple:

```
set variable value
```

Here, for example, is how to change from the default Emacs-like key binding to use **vi** line editing commands:

```
set editing-mode vi
```

Variable names and values, where appropriate, are recognized without regard to case. Unrecognized variable names are ignored.

Boolean variables (those that can be set to on or off) are set to on if the value is null or empty, *on* (case-insensitive), or 1. Any other value results in the variable being set to off.

A great deal of run-time behavior is changeable with the following variables.

active-region-start-color

A string variable that controls the text color and background when displaying the text in the active region (see the description of **enable-active-region** below). This string must not take up any physical character positions on the display, so it should consist only of terminal escape sequences. It is output to the terminal before displaying the text in the active region. This variable is reset to the default value whenever the terminal type changes. The default value is the string that puts the terminal in standout mode, as obtained from the terminal's terminfo description. A sample value might be `'\e[01;33m'`.

active-region-end-color

A string variable that "undoes" the effects of **active-region-start-color** and restores "normal" terminal display appearance after displaying text in the active region. This string must not take up any physical character positions on the display, so it should consist only of terminal escape sequences. It is output to the terminal after displaying the text in the active region. This variable is reset to the default value whenever the terminal type changes. The default value is the string that restores the terminal from stand-out mode, as obtained from the terminal's terminfo description. A sample value might be `'\e[0m'`.

bell-style

Controls what happens when Readline wants to ring the terminal bell. If set to `'none'`, Readline never rings the bell. If set to `'visible'`, Readline uses a visible bell if one is available. If set to `'audible'` (the default), Readline attempts to ring the terminal's bell.

bind-tty-special-chars

If set to `'on'` (the default), Readline attempts to bind the control characters treated specially by the kernel's terminal driver to their Readline equivalents.

blink-matching-paren

If set to `'on'`, Readline attempts to briefly move the cursor to an opening parenthesis when a closing parenthesis is inserted. The default is `'off'`.

colored-completion-prefix

If set to `'on'`, when listing completions, Readline displays the common prefix of the set of possible completions using a different color. The color definitions are taken from the value of the `LS_COLORS` environment variable. If there is a color definition in `LS_COLORS` for the custom suffix `'readline-colored-completion-prefix'`, Readline uses this color for the common prefix instead of its default. The default is `'off'`.

colored-stats

If set to 'on', Readline displays possible completions using different colors to indicate their file type. The color definitions are taken from the value of the `LS_COLORS` environment variable. The default is 'off'.

comment-begin

The string to insert at the beginning of the line when the `insert-comment` command is executed. The default value is "#".

completion-display-width

The number of screen columns used to display possible matches when performing completion. The value is ignored if it is less than 0 or greater than the terminal screen width. A value of 0 will cause matches to be displayed one per line. The default value is -1.

completion-ignore-case

If set to 'on', Readline performs filename matching and completion in a case-insensitive fashion. The default value is 'off'.

completion-map-case

If set to 'on', and *completion-ignore-case* is enabled, Readline treats hyphens ('-') and underscores ('_') as equivalent when performing case-insensitive filename matching and completion. The default value is 'off'.

completion-prefix-display-length

The length in characters of the common prefix of a list of possible completions that is displayed without modification. When set to a value greater than zero, common prefixes longer than this value are replaced with an ellipsis when displaying possible completions.

completion-query-items

The number of possible completions that determines when the user is asked whether the list of possibilities should be displayed. If the number of possible completions is greater than or equal to this value, Readline will ask whether or not the user wishes to view them; otherwise, they are simply listed. This variable must be set to an integer value greater than or equal to zero. A zero value means Readline should never ask; negative values are treated as zero. The default limit is 100.

convert-meta

If set to 'on', Readline will convert characters with the eighth bit set to an ASCII key sequence by stripping the eighth bit and prefixing an ESC character, converting them to a meta-prefixed key sequence. The default value is 'on', but will be set to 'off' if the locale is one that contains eight-bit characters. This variable is dependent on the `LC_CTYPE` locale category, and may change if the locale is changed.

disable-completion

If set to **‘On’**, Readline will inhibit word completion. Completion characters will be inserted into the line as if they had been mapped to **self-insert**. The default is **‘off’**.

echo-control-characters

When set to **‘on’**, on operating systems that indicate they support it, Readline echoes a character corresponding to a signal generated from the keyboard. The default is **‘on’**.

editing-mode

The **editing-mode** variable controls which default set of key bindings is used. By default, Readline starts up in Emacs editing mode, where the keystrokes are most similar to Emacs. This variable can be set to either **‘emacs’** or **‘vi’**.

emacs-mode-string

If the *show-mode-in-prompt* variable is enabled, this string is displayed immediately before the last line of the primary prompt when emacs editing mode is active. The value is expanded like a key binding, so the standard set of meta- and control prefixes and backslash escape sequences is available. Use the **‘\1’** and **‘\2’** escapes to begin and end sequences of non-printing characters, which can be used to embed a terminal control sequence into the mode string. The default is **‘@’**.

enable-active-region

The *point* is the current cursor position, and *mark* refers to a saved cursor position (see Section 33.4.1 [Commands For Moving], page 749). The text between the point and mark is referred to as the *region*. When this variable is set to **‘On’**, Readline allows certain commands to designate the region as *active*. When the region is active, Readline highlights the text in the region using the value of the **active-region-start-color**, which defaults to the string that enables the terminal’s standout mode. The active region shows the text inserted by bracketed-paste and any matching text found by incremental and non-incremental history searches. The default is **‘On’**.

enable-bracketed-paste

When set to **‘On’**, Readline configures the terminal to insert each paste into the editing buffer as a single string of characters, instead of treating each character as if it had been read from the keyboard. This is called putting the terminal into *bracketed paste mode*; it prevents Readline from executing any editing commands bound to key sequences appearing in the pasted text. The default is **‘On’**.

enable-keypad

When set to **‘on’**, Readline will try to enable the application keypad when it is called. Some systems need this to enable the arrow keys. The default is **‘off’**.

enable-meta-key

When set to ‘on’, Readline will try to enable any meta modifier key the terminal claims to support when it is called. On many terminals, the meta key is used to send eight-bit characters. The default is ‘on’.

expand-tilde

If set to ‘on’, tilde expansion is performed when Readline attempts word completion. The default is ‘off’.

history-preserve-point

If set to ‘on’, the history code attempts to place the point (the current cursor position) at the same location on each history line retrieved with `previous-history` or `next-history`. The default is ‘off’.

history-size

Set the maximum number of history entries saved in the history list. If set to zero, any existing history entries are deleted and no new entries are saved. If set to a value less than zero, the number of history entries is not limited. By default, the number of history entries is not limited. If an attempt is made to set *history-size* to a non-numeric value, the maximum number of history entries will be set to 500.

horizontal-scroll-mode

This variable can be set to either ‘on’ or ‘off’. Setting it to ‘on’ means that the text of the lines being edited will scroll horizontally on a single screen line when they are longer than the width of the screen, instead of wrapping onto a new screen line. This variable is automatically set to ‘on’ for terminals of height 1. By default, this variable is set to ‘off’.

input-meta

If set to ‘on’, Readline will enable eight-bit input (it will not clear the eighth bit in the characters it reads), regardless of what the terminal claims it can support. The default value is ‘off’, but Readline will set it to ‘on’ if the locale contains eight-bit characters. The name `meta-flag` is a synonym for this variable. This variable is dependent on the `LC_CTYPE` locale category, and may change if the locale is changed.

isearch-terminators

The string of characters that should terminate an incremental search without subsequently executing the character as a command (see Section 33.2.5 [Searching], page 735). If this variable has not been given a value, the characters `ESC` and `C-J` will terminate an incremental search.

keymap

Sets Readline’s idea of the current keymap for key binding commands. Built-in `keymap` names are `emacs`, `emacs-standard`,

`emacs-meta`, `emacs-ctlx`, `vi`, `vi-move`, `vi-command`, and `vi-insert`. `vi` is equivalent to `vi-command` (`vi-move` is also a synonym); `emacs` is equivalent to `emacs-standard`. Applications may add additional names. The default value is `emacs`. The value of the `editing-mode` variable also affects the default keymap.

keyseq-timeout

Specifies the duration Readline will wait for a character when reading an ambiguous key sequence (one that can form a complete key sequence using the input read so far, or can take additional input to complete a longer key sequence). If no input is received within the timeout, Readline will use the shorter but complete key sequence. Readline uses this value to determine whether or not input is available on the current input source (`rl_instream` by default). The value is specified in milliseconds, so a value of 1000 means that Readline will wait one second for additional input. If this variable is set to a value less than or equal to zero, or to a non-numeric value, Readline will wait until another key is pressed to decide which key sequence to complete. The default value is 500.

mark-directories

If set to `'on'`, completed directory names have a slash appended. The default is `'on'`.

mark-modified-lines

This variable, when set to `'on'`, causes Readline to display an asterisk (`'*'`) at the start of history lines which have been modified. This variable is `'off'` by default.

mark-symlinked-directories

If set to `'on'`, completed names which are symbolic links to directories have a slash appended (subject to the value of `mark-directories`). The default is `'off'`.

match-hidden-files

This variable, when set to `'on'`, causes Readline to match files whose names begin with a `'.'` (hidden files) when performing filename completion. If set to `'off'`, the leading `'.'` must be supplied by the user in the filename to be completed. This variable is `'on'` by default.

menu-complete-display-prefix

If set to `'on'`, menu completion displays the common prefix of the list of possible completions (which may be empty) before cycling through the list. The default is `'off'`.

output-meta

If set to `'on'`, Readline will display characters with the eighth bit set directly rather than as a meta-prefixed escape sequence. The default is `'off'`, but Readline will set it to `'on'` if the locale contains

eight-bit characters. This variable is dependent on the `LC_CTYPE` locale category, and may change if the locale is changed.

page-completions

If set to ‘on’, Readline uses an internal `more`-like pager to display a screenful of possible completions at a time. This variable is ‘on’ by default.

print-completions-horizontally

If set to ‘on’, Readline will display completions with matches sorted horizontally in alphabetical order, rather than down the screen. The default is ‘off’.

revert-all-at-newline

If set to ‘on’, Readline will undo all changes to history lines before returning when `accept-line` is executed. By default, history lines may be modified and retain individual undo lists across calls to `readline()`. The default is ‘off’.

show-all-if-ambiguous

This alters the default behavior of the completion functions. If set to ‘on’, words which have more than one possible completion cause the matches to be listed immediately instead of ringing the bell. The default value is ‘off’.

show-all-if-unmodified

This alters the default behavior of the completion functions in a fashion similar to *show-all-if-ambiguous*. If set to ‘on’, words which have more than one possible completion without any possible partial completion (the possible completions don’t share a common prefix) cause the matches to be listed immediately instead of ringing the bell. The default value is ‘off’.

show-mode-in-prompt

If set to ‘on’, add a string to the beginning of the prompt indicating the editing mode: `emacs`, `vi command`, or `vi insertion`. The mode strings are user-settable (e.g., *emacs-mode-string*). The default value is ‘off’.

skip-completed-text

If set to ‘on’, this alters the default completion behavior when inserting a single match into the line. It’s only active when performing completion in the middle of a word. If enabled, Readline does not insert characters from the completion that match characters after point in the word being completed, so portions of the word following the cursor are not duplicated. For instance, if this is enabled, attempting completion when the cursor is after the ‘e’ in ‘`Makefile`’ will result in ‘`Makefile`’ rather than ‘`Makefilefile`’, assuming there is a single possible completion. The default value is ‘off’.

vi-cmd-mode-string

If the *show-mode-in-prompt* variable is enabled, this string is displayed immediately before the last line of the primary prompt when vi editing mode is active and in command mode. The value is expanded like a key binding, so the standard set of meta- and control prefixes and backslash escape sequences is available. Use the ‘\1’ and ‘\2’ escapes to begin and end sequences of non-printing characters, which can be used to embed a terminal control sequence into the mode string. The default is ‘(cmd)’.

vi-ins-mode-string

If the *show-mode-in-prompt* variable is enabled, this string is displayed immediately before the last line of the primary prompt when vi editing mode is active and in insertion mode. The value is expanded like a key binding, so the standard set of meta- and control prefixes and backslash escape sequences is available. Use the ‘\1’ and ‘\2’ escapes to begin and end sequences of non-printing characters, which can be used to embed a terminal control sequence into the mode string. The default is ‘(ins)’.

visible-stats

If set to ‘on’, a character denoting a file’s type is appended to the filename when listing possible completions. The default is ‘off’.

Key Bindings

The syntax for controlling key bindings in the init file is simple. First you need to find the name of the command that you want to change. The following sections contain tables of the command name, the default keybinding, if any, and a short description of what the command does.

Once you know the name of the command, simply place on a line in the init file the name of the key you wish to bind the command to, a colon, and then the name of the command. There can be no space between the key name and the colon – that will be interpreted as part of the key name. The name of the key can be expressed in different ways, depending on what you find most comfortable.

In addition to command names, Readline allows keys to be bound to a string that is inserted when the key is pressed (a *macro*).

keyname: *function-name* or *macro*

keyname is the name of a key spelled out in English. For example:

```
Control-u: universal-argument
Meta-Rubout: backward-kill-word
Control-o: "> output"
```

In the example above, *C-u* is bound to the function *universal-argument*, *M-DEL* is bound to the function *backward-kill-word*, and *C-o* is bound to run the macro expressed on the right hand side (that is, to insert the text ‘> output’ into the line).

A number of symbolic character names are recognized while processing this key binding syntax: *DEL*, *ESC*, *ESCAPE*, *LFD*, *NEWLINE*, *RET*, *RETURN*, *RUBOUT*, *SPACE*, *SPC*, and *TAB*.

"*keyseq*": *function-name* or *macro*

keyseq differs from *keyname* above in that strings denoting an entire key sequence can be specified, by placing the key sequence in double quotes. Some GNU Emacs style key escapes can be used, as in the following example, but the special character names are not recognized.

```
"\C-u": universal-argument
"\C-x\C-r": re-read-init-file
"\e[11~": "Function Key 1"
```

In the above example, *C-u* is again bound to the function *universal-argument* (just as it was in the first example), '*C-x C-r*' is bound to the function *re-read-init-file*, and '*ESC [1 1 ~*' is bound to insert the text '*Function Key 1*'.

The following GNU Emacs style escape sequences are available when specifying key sequences:

<i>\C-</i>	control prefix
<i>\M-</i>	meta prefix
<i>\e</i>	an escape character
<i>\\</i>	backslash
<i>\"</i>	", a double quotation mark
<i>\'</i>	', a single quote or apostrophe

In addition to the GNU Emacs style escape sequences, a second set of backslash escapes is available:

<i>\a</i>	alert (bell)
<i>\b</i>	backspace
<i>\d</i>	delete
<i>\f</i>	form feed
<i>\n</i>	newline
<i>\r</i>	carriage return
<i>\t</i>	horizontal tab
<i>\v</i>	vertical tab
<i>\nnn</i>	the eight-bit character whose value is the octal value <i>nnn</i> (one to three digits)
<i>\xHH</i>	the eight-bit character whose value is the hexadecimal value <i>HH</i> (one or two hex digits)

When entering the text of a macro, single or double quotes must be used to indicate a macro definition. Unquoted text is assumed to be a function name. In the macro body, the backslash escapes described above are expanded. Backslash will quote any other character in the macro text, including ‘”’ and ‘’’. For example, the following binding will make ‘C-x \’ insert a single ‘\’ into the line:

```
"\C-x\\": "\\\""
```

33.3.2 Conditional Init Constructs

Readline implements a facility similar in spirit to the conditional compilation features of the C preprocessor which allows key bindings and variable settings to be performed as the result of tests. There are four parser directives used.

\$if The **\$if** construct allows bindings to be made based on the editing mode, the terminal being used, or the application using Readline. The text of the test, after any comparison operator, extends to the end of the line; unless otherwise noted, no characters are required to isolate it.

mode The **mode=** form of the **\$if** directive is used to test whether Readline is in **emacs** or **vi** mode. This may be used in conjunction with the ‘**set keymap**’ command, for instance, to set bindings in the **emacs-standard** and **emacs-ctlx** keymaps only if Readline is starting out in **emacs** mode.

term The **term=** form may be used to include terminal-specific key bindings, perhaps to bind the key sequences output by the terminal’s function keys. The word on the right side of the ‘=’ is tested against both the full name of the terminal and the portion of the terminal name before the first ‘-’. This allows **sun** to match both **sun** and **sun-cmd**, for instance.

version The **version** test may be used to perform comparisons against specific Readline versions. The **version** expands to the current Readline version. The set of comparison operators includes ‘=’ (and ‘==’), ‘!=’, ‘<=’, ‘>=’, ‘<’, and ‘>’. The version number supplied on the right side of the operator consists of a major version number, an optional decimal point, and an optional minor version (e.g., ‘7.1’). If the minor version is omitted, it is assumed to be ‘0’. The operator may be separated from the string **version** and from the version number argument by whitespace. The following example sets a variable if the Readline version being used is 7.0 or newer:

```
$if version >= 7.0
set show-mode-in-prompt on
$endif
```

application

The *application* construct is used to include application-specific settings. Each program using the Readline library sets the *application name*, and you can test for a particular value. This could be used to bind key sequences to functions useful for a specific program. For

instance, the following command adds a key sequence that quotes the current or previous word in Bash:

```
$if Bash
# Quote the current or previous word
"\C-xq": "\eb\""\ef\""\
$endif
```

variable The *variable* construct provides simple equality tests for Readline variables and values. The permitted comparison operators are '=', '==', and '!='. The variable name must be separated from the comparison operator by whitespace; the operator may be separated from the value on the right hand side by whitespace. Both string and boolean variables may be tested. Boolean variables must be tested against the values *on* and *off*. The following example is equivalent to the `mode=emacs` test described above:

```
$if editing-mode == emacs
set show-mode-in-prompt on
$endif
```

\$endif This command, as seen in the previous example, terminates an `$if` command.

\$else Commands in this branch of the `$if` directive are executed if the test fails.

\$include This directive takes a single filename as an argument and reads commands and bindings from that file. For example, the following directive reads from `/etc/inputrc`:

```
$include /etc/inputrc
```

33.3.3 Sample Init File

Here is an example of an *inputrc* file. This illustrates key binding, variable assignment, and conditional syntax.

```

# This file controls the behaviour of line input editing for
# programs that use the GNU Readline library. Existing
# programs include FTP, Bash, and GDB.
#
# You can re-read the inputrc file with C-x C-r.
# Lines beginning with '#' are comments.
#
# First, include any system-wide bindings and variable
# assignments from /etc/Inputrc
$include /etc/Inputrc

#
# Set various bindings for emacs mode.

set editing-mode emacs

$if mode=emacs

Meta-Control-h: backward-kill-word Text after the function name is ignored

#
# Arrow keys in keypad mode
#
#"M-OD":      backward-char
#"M-OC":      forward-char
#"M-OA":      previous-history
#"M-OB":      next-history
#
# Arrow keys in ANSI mode
#
"M-[D":      backward-char
"M-[C":      forward-char
"M-[A":      previous-history
"M-[B":      next-history
#
# Arrow keys in 8 bit keypad mode
#
#"M-\C-OD":   backward-char
#"M-\C-OC":   forward-char
#"M-\C-OA":   previous-history
#"M-\C-OB":   next-history
#
# Arrow keys in 8 bit ANSI mode
#
#"M-\C-[D":   backward-char
#"M-\C-[C":   forward-char

```

```

#\M-\C-[A":      previous-history
#\M-\C-[B":      next-history

C-q: quoted-insert

$endif

# An old-style binding.  This happens to be the default.
TAB: complete

# Macros that are convenient for shell interaction
$if Bash
# edit the path
"\C-xp": "PATH=${PATH}\e\C-e\C-a\ef\C-f"
# prepare to type a quoted word --
# insert open and close double quotes
# and move to just after the open quote
"\C-x\"": "\""\C-b"
# insert a backslash (testing backslash escapes
# in sequences and macros)
"\C-x\\": "\\\"
# Quote the current or previous word
"\C-xq": "\eb\""\ef\"
# Add a binding to refresh the line, which is unbound
"\C-xr": redraw-current-line
# Edit variable on current line.
#\M-\C-v": "\C-a\C-k$\C-y\M-\C-e\C-a\C-y="
$endif

# use a visible bell if one is available
set bell-style visible

# don't strip characters to 7 bits when reading
set input-meta on

# allow iso-latin1 characters to be inserted rather
# than converted to prefix-meta sequences
set convert-meta off

# display characters with the eighth bit set directly
# rather than as meta-prefixed characters
set output-meta on

# if there are 150 or more possible completions for a word,
# ask whether or not the user wants to see all of them
set completion-query-items 150

```

```
# For FTP
$if Ftp
"\C-xg": "get \M-?"
"\C-xt": "put \M-?"
"\M-.": yank-last-arg
$endif
```

33.4 Bindable Readline Commands

This section describes Readline commands that may be bound to key sequences. Command names without an accompanying key sequence are unbound by default.

In the following descriptions, *point* refers to the current cursor position, and *mark* refers to a cursor position saved by the `set-mark` command. The text between the point and mark is referred to as the *region*.

33.4.1 Commands For Moving

`beginning-of-line (C-a)`

Move to the start of the current line.

`end-of-line (C-e)`

Move to the end of the line.

`forward-char (C-f)`

Move forward a character.

`backward-char (C-b)`

Move back a character.

`forward-word (M-f)`

Move forward to the end of the next word. Words are composed of letters and digits.

`backward-word (M-b)`

Move back to the start of the current or previous word. Words are composed of letters and digits.

`previous-screen-line ()`

Attempt to move point to the same physical screen column on the previous physical screen line. This will not have the desired effect if the current Readline line does not take up more than one physical line or if point is not greater than the length of the prompt plus the screen width.

`next-screen-line ()`

Attempt to move point to the same physical screen column on the next physical screen line. This will not have the desired effect if the current Readline line does not take up more than one physical line or if the length of the current Readline line is not greater than the length of the prompt plus the screen width.

`clear-display (M-C-l)`

Clear the screen and, if possible, the terminal's scrollbar buffer, then redraw the current line, leaving the current line at the top of the screen.

clear-screen (C-l)

Clear the screen, then redraw the current line, leaving the current line at the top of the screen.

redraw-current-line ()

Refresh the current line. By default, this is unbound.

33.4.2 Commands For Manipulating The History

accept-line (Newline or Return)

Accept the line regardless of where the cursor is. If this line is non-empty, it may be added to the history list for future recall with `add_history()`. If this line is a modified history line, the history line is restored to its original state.

previous-history (C-p)

Move ‘back’ through the history list, fetching the previous command.

next-history (C-n)

Move ‘forward’ through the history list, fetching the next command.

beginning-of-history (M-<)

Move to the first line in the history.

end-of-history (M->)

Move to the end of the input history, i.e., the line currently being entered.

reverse-search-history (C-r)

Search backward starting at the current line and moving ‘up’ through the history as necessary. This is an incremental search. This command sets the region to the matched text and activates the mark.

forward-search-history (C-s)

Search forward starting at the current line and moving ‘down’ through the history as necessary. This is an incremental search. This command sets the region to the matched text and activates the mark.

non-incremental-reverse-search-history (M-p)

Search backward starting at the current line and moving ‘up’ through the history as necessary using a non-incremental search for a string supplied by the user. The search string may match anywhere in a history line.

non-incremental-forward-search-history (M-n)

Search forward starting at the current line and moving ‘down’ through the history as necessary using a non-incremental search for a string supplied by the user. The search string may match anywhere in a history line.

history-search-forward ()

Search forward through the history for the string of characters between the start of the current line and the point. The search string must match at the beginning of a history line. This is a non-incremental search. By default, this command is unbound.

history-search-backward ()

Search backward through the history for the string of characters between the start of the current line and the point. The search string must match at the

beginning of a history line. This is a non-incremental search. By default, this command is unbound.

history-substring-search-forward ()

Search forward through the history for the string of characters between the start of the current line and the point. The search string may match anywhere in a history line. This is a non-incremental search. By default, this command is unbound.

history-substring-search-backward ()

Search backward through the history for the string of characters between the start of the current line and the point. The search string may match anywhere in a history line. This is a non-incremental search. By default, this command is unbound.

yank-nth-arg (M-C-y)

Insert the first argument to the previous command (usually the second word on the previous line) at point. With an argument *n*, insert the *n*th word from the previous command (the words in the previous command begin with word 0). A negative argument inserts the *n*th word from the end of the previous command. Once the argument *n* is computed, the argument is extracted as if the ‘!*n*’ history expansion had been specified.

yank-last-arg (M-. or M-_)

Insert last argument to the previous command (the last word of the previous history entry). With a numeric argument, behave exactly like **yank-nth-arg**. Successive calls to **yank-last-arg** move back through the history list, inserting the last word (or the word specified by the argument to the first call) of each line in turn. Any numeric argument supplied to these successive calls determines the direction to move through the history. A negative argument switches the direction through the history (back or forward). The history expansion facilities are used to extract the last argument, as if the ‘!\$’ history expansion had been specified.

operate-and-get-next (C-o)

Accept the current line for return to the calling application as if a newline had been entered, and fetch the next line relative to the current line from the history for editing. A numeric argument, if supplied, specifies the history entry to use instead of the current line.

fetch-history ()

With a numeric argument, fetch that entry from the history list and make it the current line. Without an argument, move back to the first entry in the history list.

33.4.3 Commands For Changing Text

end-of-file (usually C-d)

The character indicating end-of-file as set, for example, by **stty**. If this character is read when there are no characters on the line, and point is at the beginning of the line, Readline interprets it as the end of input and returns EOF.

delete-char (C-d)

Delete the character at point. If this function is bound to the same character as the tty EOF character, as **C-d** commonly is, see above for the effects.

backward-delete-char (Rubout)

Delete the character behind the cursor. A numeric argument means to kill the characters instead of deleting them.

forward-backward-delete-char ()

Delete the character under the cursor, unless the cursor is at the end of the line, in which case the character behind the cursor is deleted. By default, this is not bound to a key.

quoted-insert (C-q or C-v)

Add the next character typed to the line verbatim. This is how to insert key sequences like **C-q**, for example.

tab-insert (M-TAB)

Insert a tab character.

self-insert (a, b, A, 1, !, ...)

Insert yourself.

bracketed-paste-begin ()

This function is intended to be bound to the "bracketed paste" escape sequence sent by some terminals, and such a binding is assigned by default. It allows Readline to insert the pasted text as a single unit without treating each character as if it had been read from the keyboard. The characters are inserted as if each one was bound to **self-insert** instead of executing any editing commands.

Bracketed paste sets the region (the characters between point and the mark) to the inserted text. It uses the concept of an *active mark*: when the mark is active, Readline redisplay uses the terminal's standout mode to denote the region.

transpose-chars (C-t)

Drag the character before the cursor forward over the character at the cursor, moving the cursor forward as well. If the insertion point is at the end of the line, then this transposes the last two characters of the line. Negative arguments have no effect.

transpose-words (M-t)

Drag the word before point past the word after point, moving point past that word as well. If the insertion point is at the end of the line, this transposes the last two words on the line.

upcase-word (M-u)

Uppercase the current (or following) word. With a negative argument, uppercase the previous word, but do not move the cursor.

downcase-word (M-l)

Lowercase the current (or following) word. With a negative argument, lowercase the previous word, but do not move the cursor.

capitalize-word (M-c)

Capitalize the current (or following) word. With a negative argument, capitalize the previous word, but do not move the cursor.

overwrite-mode ()

Toggle overwrite mode. With an explicit positive numeric argument, switches to overwrite mode. With an explicit non-positive numeric argument, switches to insert mode. This command affects only **emacs** mode; **vi** mode does overwrite differently. Each call to **readline()** starts in insert mode.

In overwrite mode, characters bound to **self-insert** replace the text at point rather than pushing the text to the right. Characters bound to **backward-delete-char** replace the character before point with a space.

By default, this command is unbound.

33.4.4 Killing And Yanking

kill-line (C-k)

Kill the text from point to the end of the line. With a negative numeric argument, kill backward from the cursor to the beginning of the current line.

backward-kill-line (C-x Rubout)

Kill backward from the cursor to the beginning of the current line. With a negative numeric argument, kill forward from the cursor to the end of the current line.

unix-line-discard (C-u)

Kill backward from the cursor to the beginning of the current line.

kill-whole-line ()

Kill all characters on the current line, no matter where point is. By default, this is unbound.

kill-word (M-d)

Kill from point to the end of the current word, or if between words, to the end of the next word. Word boundaries are the same as **forward-word**.

backward-kill-word (M-DEL)

Kill the word behind point. Word boundaries are the same as **backward-word**.

shell-transpose-words (M-C-t)

Drag the word before point past the word after point, moving point past that word as well. If the insertion point is at the end of the line, this transposes the last two words on the line. Word boundaries are the same as **shell-forward-word** and **shell-backward-word**.

unix-word-rubout (C-w)

Kill the word behind point, using white space as a word boundary. The killed text is saved on the kill-ring.

unix-filename-rubout ()

Kill the word behind point, using white space and the slash character as the word boundaries. The killed text is saved on the kill-ring.

delete-horizontal-space ()

Delete all spaces and tabs around point. By default, this is unbound.

kill-region ()

Kill the text in the current region. By default, this command is unbound.

copy-region-as-kill ()

Copy the text in the region to the kill buffer, so it can be yanked right away. By default, this command is unbound.

copy-backward-word ()

Copy the word before point to the kill buffer. The word boundaries are the same as **backward-word**. By default, this command is unbound.

copy-forward-word ()

Copy the word following point to the kill buffer. The word boundaries are the same as **forward-word**. By default, this command is unbound.

yank (C-y)

Yank the top of the kill ring into the buffer at point.

yank-pop (M-y)

Rotate the kill-ring, and yank the new top. You can only do this if the prior command is **yank** or **yank-pop**.

33.4.5 Specifying Numeric Arguments

digit-argument (*M-0*, *M-1*, ... *M--*)

Add this digit to the argument already accumulating, or start a new argument. *M--* starts a negative argument.

universal-argument ()

This is another way to specify an argument. If this command is followed by one or more digits, optionally with a leading minus sign, those digits define the argument. If the command is followed by digits, executing **universal-argument** again ends the numeric argument, but is otherwise ignored. As a special case, if this command is immediately followed by a character that is neither a digit nor minus sign, the argument count for the next command is multiplied by four. The argument count is initially one, so executing this function the first time makes the argument count four, a second time makes the argument count sixteen, and so on. By default, this is not bound to a key.

33.4.6 Letting Readline Type For You

complete (TAB)

Attempt to perform completion on the text before point. The actual completion performed is application-specific. The default is filename completion.

possible-completions (M-?)

List the possible completions of the text before point. When displaying completions, Readline sets the number of columns used for display to the value of **completion-display-width**, the value of the environment variable **COLUMNS**, or the screen width, in that order.

insert-completions (M-*)

Insert all completions of the text before point that would have been generated by **possible-completions**.

menu-complete ()

Similar to **complete**, but replaces the word to be completed with a single match from the list of possible completions. Repeated execution of **menu-complete** steps through the list of possible completions, inserting each match in turn. At the end of the list of completions, the bell is rung (subject to the setting of **bell-style**) and the original text is restored. An argument of *n* moves *n* positions forward in the list of matches; a negative argument may be used to move backward through the list. This command is intended to be bound to TAB, but is unbound by default.

menu-complete-backward ()

Identical to **menu-complete**, but moves backward through the list of possible completions, as if **menu-complete** had been given a negative argument.

delete-char-or-list ()

Deletes the character under the cursor if not at the beginning or end of the line (like **delete-char**). If at the end of the line, behaves identically to **possible-completions**. This command is unbound by default.

33.4.7 Keyboard Macros

start-kbd-macro (C-x)

Begin saving the characters typed into the current keyboard macro.

end-kbd-macro (C-x)

Stop saving the characters typed into the current keyboard macro and save the definition.

call-last-kbd-macro (C-x e)

Re-execute the last keyboard macro defined, by making the characters in the macro appear as if typed at the keyboard.

print-last-kbd-macro ()

Print the last keyboard macro defined in a format suitable for the *inputrc* file.

33.4.8 Some Miscellaneous Commands

re-read-init-file (C-x C-r)

Read in the contents of the *inputrc* file, and incorporate any bindings or variable assignments found there.

abort (C-g)

Abort the current editing command and ring the terminal's bell (subject to the setting of **bell-style**).

do-lowercase-version (M-A, M-B, M-x, ...)

If the metafiled character *x* is upper case, run the command that is bound to the corresponding metafiled lower case character. The behavior is undefined if *x* is already lower case.

prefix-meta (ESC)

Metafy the next character typed. This is for keyboards without a meta key. Typing 'ESC f' is equivalent to typing *M-f*.

undo (C-_ or C-x C-u)

Incremental undo, separately remembered for each line.

revert-line (M-r)

Undo all changes made to this line. This is like executing the **undo** command enough times to get back to the beginning.

tilde-expand (M-~)

Perform tilde expansion on the current word.

set-mark (C-@)

Set the mark to the point. If a numeric argument is supplied, the mark is set to that position.

exchange-point-and-mark (C-x C-x)

Swap the point with the mark. The current cursor position is set to the saved position, and the old cursor position is saved as the mark.

character-search (C-])

A character is read and point is moved to the next occurrence of that character. A negative argument searches for previous occurrences.

character-search-backward (M-C-])

A character is read and point is moved to the previous occurrence of that character. A negative argument searches for subsequent occurrences.

skip-csi-sequence ()

Read enough characters to consume a multi-key sequence such as those defined for keys like Home and End. Such sequences begin with a Control Sequence Indicator (CSI), usually ESC[. If this sequence is bound to "\e[", keys producing such sequences will have no effect unless explicitly bound to a Readline command, instead of inserting stray characters into the editing buffer. This is unbound by default, but usually bound to ESC[.

insert-comment (M-#)

Without a numeric argument, the value of the **comment-begin** variable is inserted at the beginning of the current line. If a numeric argument is supplied, this command acts as a toggle: if the characters at the beginning of the line do not match the value of **comment-begin**, the value is inserted, otherwise the characters in **comment-begin** are deleted from the beginning of the line. In either case, the line is accepted as if a newline had been typed.

dump-functions ()

Print all of the functions and their key bindings to the Readline output stream. If a numeric argument is supplied, the output is formatted in such a way that it can be made part of an *inputrc* file. This command is unbound by default.

dump-variables ()

Print all of the settable variables and their values to the Readline output stream. If a numeric argument is supplied, the output is formatted in such a way that it can be made part of an *inputrc* file. This command is unbound by default.

dump-macros ()

Print all of the Readline key sequences bound to macros and the strings they output. If a numeric argument is supplied, the output is formatted in such a way that it can be made part of an *inputrc* file. This command is unbound by default.

emacs-editing-mode (C-e)

When in *vi* command mode, this causes a switch to **emacs** editing mode.

vi-editing-mode (M-C-j)

When in **emacs** editing mode, this causes a switch to *vi* editing mode.

33.5 Readline *vi* Mode

While the Readline library does not have a full set of *vi* editing functions, it does contain enough to allow simple editing of the line. The Readline *vi* mode behaves as specified in the POSIX standard.

In order to switch interactively between **emacs** and *vi* editing modes, use the command *M-C-j* (bound to *emacs-editing-mode* when in *vi* mode and to *vi-editing-mode* in **emacs** mode). The Readline default is **emacs** mode.

When you enter a line in *vi* mode, you are already placed in ‘insertion’ mode, as if you had typed an ‘i’. Pressing **ESC** switches you into ‘command’ mode, where you can edit the text of the line with the standard *vi* movement keys, move to previous history lines with ‘k’ and subsequent lines with ‘j’, and so forth.

34 Using History Interactively

This chapter describes how to use the GNU History Library interactively, from a user's standpoint. It should be considered a user's guide. For information on using the GNU History Library in your own programs, see Section "Programming with GNU History" in *GNU History Library*.

34.1 History Expansion

The History library provides a history expansion feature that is similar to the history expansion provided by `csh`. This section describes the syntax used to manipulate the history information.

History expansions introduce words from the history list into the input stream, making it easy to repeat commands, insert the arguments to a previous command into the current input line, or fix errors in previous commands quickly.

History expansion takes place in two parts. The first is to determine which line from the history list should be used during substitution. The second is to select portions of that line for inclusion into the current one. The line selected from the history is called the *event*, and the portions of that line that are acted upon are called *words*. Various *modifiers* are available to manipulate the selected words. The line is broken into words in the same fashion that Bash does, so that several words surrounded by quotes are considered one word. History expansions are introduced by the appearance of the history expansion character, which is `'!`' by default.

History expansion implements shell-like quoting conventions: a backslash can be used to remove the special handling for the next character; single quotes enclose verbatim sequences of characters, and can be used to inhibit history expansion; and characters enclosed within double quotes may be subject to history expansion, since backslash can escape the history expansion character, but single quotes may not, since they are not treated specially within double quotes.

34.1.1 Event Designators

An event designator is a reference to a command line entry in the history list. Unless the reference is absolute, events are relative to the current position in the history list.

<code>!</code>	Start a history substitution, except when followed by a space, tab, the end of the line, or <code>'='</code> .
<code>!n</code>	Refer to command line <i>n</i> .
<code>!-n</code>	Refer to the command <i>n</i> lines back.
<code>!!</code>	Refer to the previous command. This is a synonym for <code>'!-1'</code> .
<code>!string</code>	Refer to the most recent command preceding the current position in the history list starting with <i>string</i> .
<code>!?string[?]</code>	Refer to the most recent command preceding the current position in the history list containing <i>string</i> . The trailing <code>'?'</code> may be omitted if the <i>string</i> is followed immediately by a newline. If <i>string</i> is missing, the string from the most recent search is used; it is an error if there is no previous search string.

`^string1^string2^`

Quick Substitution. Repeat the last command, replacing *string1* with *string2*.
Equivalent to `!!:s^string1^string2^`.

`!#` The entire command line typed so far.

34.1.2 Word Designators

Word designators are used to select desired words from the event. A ‘:’ separates the event specification from the word designator. It may be omitted if the word designator begins with a ‘^’, ‘\$’, ‘*’, ‘-’, or ‘%’. Words are numbered from the beginning of the line, with the first word being denoted by 0 (zero). Words are inserted into the current line separated by single spaces.

For example,

`!!` designates the preceding command. When you type this, the preceding command is repeated in toto.

`!!: $` designates the last argument of the preceding command. This may be shortened to `!$`.

`!fi:2` designates the second argument of the most recent command starting with the letters `fi`.

Here are the word designators:

`0 (zero)` The 0th word. For many applications, this is the command word.

`n` The *n*th word.

`^` The first argument; that is, word 1.

`$` The last argument.

`%` The first word matched by the most recent ‘?*string*?’ search, if the search string begins with a character that is part of a word.

`x-y` A range of words; ‘-y’ abbreviates ‘0-y’.

`*` All of the words, except the 0th. This is a synonym for ‘1-\$’. It is not an error to use ‘*’ if there is just one word in the event; the empty string is returned in that case.

`x*` Abbreviates ‘x-\$’

`x-` Abbreviates ‘x-\$’ like ‘x*’, but omits the last word. If ‘x’ is missing, it defaults to 0.

If a word designator is supplied without an event specification, the previous command is used as the event.

34.1.3 Modifiers

After the optional word designator, you can add a sequence of one or more of the following modifiers, each preceded by a ‘:’. These modify, or edit, the word or words selected from the history event.

`h` Remove a trailing pathname component, leaving only the head.

- t Remove all leading pathname components, leaving the tail.
- r Remove a trailing suffix of the form '*.suffix*', leaving the basename.
- e Remove all but the trailing suffix.
- p Print the new command but do not execute it.
- s/*old/new*/ Substitute *new* for the first occurrence of *old* in the event line. Any character may be used as the delimiter in place of '/'. The delimiter may be quoted in *old* and *new* with a single backslash. If '&' appears in *new*, it is replaced by *old*. A single backslash will quote the '&'. If *old* is null, it is set to the last *old* substituted, or, if no previous history substitutions took place, the last *string* in a *!?string[?]* search. If *new* is null, each matching *old* is deleted. The final delimiter is optional if it is the last character on the input line.
- & Repeat the previous substitution.
- g
- a Cause changes to be applied over the entire event line. Used in conjunction with 's', as in *gs/old/new/*, or with '&'.
- G Apply the following 's' or '&' modifier once to each word in the event.

Appendix A In Memoriam

The GDB project mourns the loss of the following long-time contributors:

Fred Fish Fred was a long-standing contributor to GDB (1991-2006), and to Free Software in general. Outside of GDB, he was known in the Amiga world for his series of Fish Disks, and the GeekGadget project.

Michael Snyder

Michael was one of the Global Maintainers of the GDB project, with contributions recorded as early as 1996, until 2011. In addition to his day to day participation, he was a large driving force behind adding Reverse Debugging to GDB.

Beyond their technical contributions to the project, they were also enjoyable members of the Free Software Community. We will miss them.

Appendix B Formatting Documentation

The GDB 4 release includes an already-formatted reference card, ready for printing with PostScript or Ghostscript, in the `gdb` subdirectory of the main source directory¹. If you can use PostScript or Ghostscript with your printer, you can print the reference card immediately with `refcard.ps`.

The release also includes the source for the reference card. You can format it, using $\text{\TeX}$, by typing:

```
make refcard.dvi
```

The GDB reference card is designed to print in *landscape* mode on US “letter” size paper; that is, on a sheet 11 inches wide by 8.5 inches high. You will need to specify this form of printing as an option to your DVI output program.

All the documentation for GDB comes as part of the machine-readable distribution. The documentation is written in Texinfo format, which is a documentation system that uses a single source file to produce both on-line information and a printed manual. You can use one of the Info formatting commands to create the on-line version of the documentation and $\text{\TeX}$ (or `texi2roff`) to typeset the printed version.

GDB includes an already formatted copy of the on-line Info version of this manual in the `gdb` subdirectory. The main Info file is `gdb-18.0.50.20260503-git/gdb/gdb.info`, and it refers to subordinate files matching ‘`gdb.info*`’ in the same directory. If necessary, you can print out these files, or read them with any editor; but they are easier to read using the `info` subsystem in GNU Emacs or the standalone `info` program, available as part of the GNU Texinfo distribution.

If you want to format these Info files yourself, you need one of the Info formatting programs, such as `texinfo-format-buffer` or `makeinfo`.

If you have `makeinfo` installed, and are in the top level GDB source directory (`gdb-18.0.50.20260503-git`, in the case of version 18.0.50.20260503-git), you can make the Info file by typing:

```
cd gdb
make gdb.info
```

If you want to typeset and print copies of this manual, you need $\text{\TeX}$, a program to print its DVI output files, and `texinfo.tex`, the Texinfo definitions file.

$\text{\TeX}$ is a typesetting program; it does not print files directly, but produces output files called DVI files. To print a typeset document, you need a program to print DVI files. If your system has $\text{\TeX}$ installed, chances are it has such a program. The precise command to use depends on your system; `lpr -d` is common; another (for PostScript devices) is `dvips`. The DVI print command may require a file name without any extension or a ‘`.dvi`’ extension.

$\text{\TeX}$ also requires a macro definitions file called `texinfo.tex`. This file tells $\text{\TeX}$ how to typeset a document written in Texinfo format. On its own, $\text{\TeX}$ cannot either read or typeset a Texinfo file. `texinfo.tex` is distributed with GDB and is located in the `gdb-version-number/texinfo` directory.

¹ In `gdb-18.0.50.20260503-git/gdb/refcard.ps` of the version 18.0.50.20260503-git release.

If you have $\text{\TeX}$ and a DVI printer program installed, you can typeset and print this manual. First switch to the `gdb` subdirectory of the main source directory (for example, to `gdb-18.0.50.20260503-git/gdb`) and type:

```
make gdb.dvi
```

Then give `gdb.dvi` to your DVI printing program.

Appendix C Installing GDB

C.1 Requirements for Building GDB

Building GDB requires various tools and packages to be available. Other packages will be used only if they are found.

Tools/Packages Necessary for Building GDB

C++17 compiler

GDB is written in C++17. It should be buildable with any recent C++17 compiler, e.g. GCC.

GNU make

GDB's build system relies on features only found in the GNU make program. Other variants of **make** will not work.

Libraries The following libraries are mandatory for building GDB. The **configure** script searches for each of these libraries in several standard locations; if some library is installed in an unusual place, you can use either the **--with-lib** **configure** option to specify its installation directory, or the two separate options **---with-library-include** (to specify the location of its header files) and **--with-library-lib** (to specify the location of its libraries). For example, for the GMP library, the 3 options are **--with-gmp**, **--with-gmp-include**, and **--with-gmp-lib**. See Section C.5 [Configure Options], page 771. We mention below the home site of each library, so that you could download and install them if your system doesn't already include them.

GMP (The GNU Multiple Precision arithmetic library)

GDB uses GMP to perform some of its extended-precision arithmetic. The latest version of GMP is available from <https://gmplib.org/>.

MPFR (The GNU Multiple-precision floating-point library)

GDB uses MPFR to emulate the target floating-point arithmetic during expression evaluation, if the target uses different floating-point formats than the host. The latest version of MPFR is available from <http://www.mpfr.org>.

Tools/Packages Optional for Building GDB

The tools/packages and libraries listed below are optional; GDB can be build without them, at the expense of some run-time functionality that will be missing. As above, we list the home sites for each package/library, and the command-line options supported by the **configure** script to specify their installation directories if they are non-standard. In addition, for each package you can use the option **--with-package** to force GDB to be compiled with the named *package*, and **--without-package** to disable building with it even if it is available. See Section C.5 [Configure Options], page 771, for detailed description of the options to **configure**.

- Python GDB can be scripted using Python language. See Section 23.3 [Python], page 407. The latest version is available from <https://www.python.org/downloads/>. Use the `--with-python=dir` to specify the non-standard directory where Python is installed.
- Guile GDB can also be scripted using GNU Guile. See Section 23.4 [Guile], page 538. The latest version can be found on <https://www.gnu.org/software/guile/download/>. If you have more than one version of Guile installed, use the `--with-guile=guile-version` to specify the Guile version to include in the build.
- Expat If available, GDB uses the Expat library for parsing XML files. GDB uses XML files for the following functionalities:
- Remote protocol memory maps (see Section E.17 [Memory Map Format], page 867)
 - Target descriptions (see Appendix G [Target Descriptions], page 883)
 - Remote shared library lists (See Section E.15 [Library List Format], page 865, or alternatively see Section E.16 [Library List Format for SVR4 Targets], page 866)
 - MS-Windows shared libraries (see [Shared Libraries], page 298)
 - Traceframe info (see Section E.19 [Traceframe Info Format], page 869)
 - Branch trace (see Section E.20 [Branch Trace Format], page 869, see Section E.21 [Branch Trace Configuration Format], page 870)
- The latest version of Expat is available from <http://expat.sourceforge.net>. Use the `--with-libexpat-prefix` to specify non-standard installation places for Expat.
- iconv GDB's features related to character sets (see Section 10.21 [Character Sets], page 190) require a functioning `iconv` implementation. If you are on a GNU system, then this is provided by the GNU C Library. Some other systems also provide a working `iconv`. Use the option `--with-iconv-bin` to specify where to find the `iconv` program.
- On systems without `iconv`, you can install the GNU Libiconv library; its latest version can be found on <https://ftp.gnu.org/pub/gnu/libiconv/> if your system doesn't provide it. Use the `--with-libiconv-prefix` option to `configure` to specify non-standard installation place for it.
- Alternatively, GDB's top-level `configure` and `Makefile` will arrange to build Libiconv if a directory named `libiconv` appears in the top-most source directory. If Libiconv is built this way, and if the operating system does not provide a suitable `iconv` implementation, then the just-built library will automatically be used by GDB. One easy way to set this up is to download GNU Libiconv, unpack it inside the top-level directory of the GDB source tree, and then rename the directory holding the Libiconv source code to `'libiconv'`.
- lzma GDB can support debugging sections that are compressed with the LZMA library. See Section 18.4 [MiniDebugInfo], page 306. If this library is not included with your operating system, you can find it in the xz package at <http://>

`tukaani.org/xz/`. Use the `--with-liblzma-prefix` option to specify its non-standard location.

zlib GDB will use the ‘**zlib**’ library, if available, to read compressed debug sections. Some linkers, such as GNU **gold**, are capable of producing binaries with compressed debug sections. If GDB is compiled with ‘**zlib**’, it will be able to read the debug information in such binaries.

The ‘**zlib**’ library is likely included with your operating system distribution; if it is not, you can get the latest version from <http://zlib.net>.

C.2 Invoking the GDB configure Script

GDB comes with a **configure** script that automates the process of preparing GDB for installation; you can then use **make** to build the **gdb** program.¹

The GDB distribution includes all the source code you need for GDB in a single directory, whose name is usually composed by appending the version number to ‘**gdb**’.

For example, the GDB version 18.0.50.20260503-git distribution is in the **gdb-18.0.50.20260503-git** directory. That directory contains:

```
gdb-18.0.50.20260503-git/configure (and supporting files)
    script for configuring GDB and all its supporting libraries

gdb-18.0.50.20260503-git/gdb
    the source specific to GDB itself

gdb-18.0.50.20260503-git/bfd
    source for the Binary File Descriptor library

gdb-18.0.50.20260503-git/include
    GNU include files

gdb-18.0.50.20260503-git/libiberty
    source for the ‘-liberty’ free software library

gdb-18.0.50.20260503-git/opcodes
    source for the library of opcode tables and disassemblers

gdb-18.0.50.20260503-git/readline
    source for the GNU command-line interface
```

There may be other subdirectories as well.

The simplest way to configure and build GDB is to run **configure** from the **gdb-version-number** source directory, which in this example is the **gdb-18.0.50.20260503-git** directory.

First switch to the **gdb-version-number** source directory if you are not already in it; then run **configure**. Pass the identifier for the platform on which GDB will run as an argument.

For example:

```
cd gdb-18.0.50.20260503-git
```

¹ If you have a more recent version of GDB than 18.0.50.20260503-git, look at the **README** file in the sources; we may have improved the installation procedures since publishing this manual.

```
./configure
make
```

Running `configure` and then running `make` builds the included supporting libraries, then `gdb` itself. The configured source files, and the binaries, are left in the corresponding source directories.

`configure` is a Bourne-shell (`/bin/sh`) script; if your system does not recognize this automatically when you run a different shell, you may need to run `sh` on it explicitly:

```
sh configure
```

You should run the `configure` script from the top directory in the source tree, the `gdb-version-number` directory. If you run `configure` from one of the subdirectories, you will configure only that subdirectory. That is usually not what you want. In particular, if you run the first `configure` from the `gdb` subdirectory of the `gdb-version-number` directory, you will omit the configuration of `bfd`, `readline`, and other sibling directories of the `gdb` subdirectory. This leads to build errors about missing include files such as `bfd/bfd.h`.

You can install GDB anywhere. The best way to do this is to pass the `--prefix` option to `configure`, and then install it with `make install`.

C.3 Compiling GDB in Another Directory

If you want to run GDB versions for several host or target machines, you need a different `gdb` compiled for each combination of host and target. `configure` is designed to make this easy by allowing you to generate each configuration in a separate subdirectory, rather than in the source directory. If your `make` program handles the `'VPATH'` feature (GNU `make` does), running `make` in each of these directories builds the `gdb` program specified there.

To build `gdb` in a separate directory, run `configure` with the `'--srcdir'` option to specify where to find the source. (You also need to specify a path to find `configure` itself from your working directory. If the path to `configure` would be the same as the argument to `'--srcdir'`, you can leave out the `'--srcdir'` option; it is assumed.)

For example, with version 18.0.50.20260503-git, you can build GDB in a separate directory for a Sun 4 like this:

```
cd gdb-18.0.50.20260503-git
mkdir ../gdb-sun4
cd ../gdb-sun4
../gdb-18.0.50.20260503-git/configure
make
```

When `configure` builds a configuration using a remote source directory, it creates a tree for the binaries with the same structure (and using the same names) as the tree under the source directory. In the example, you'd find the Sun 4 library `libiberty.a` in the directory `gdb-sun4/libiberty`, and GDB itself in `gdb-sun4/gdb`.

Make sure that your path to the `configure` script has just one instance of `gdb` in it. If your path to `configure` looks like `../gdb-18.0.50.20260503-git/gdb/configure`, you are configuring only one subdirectory of GDB, not the whole package. This leads to build errors about missing include files such as `bfd/bfd.h`.

One popular reason to build several GDB configurations in separate directories is to configure GDB for cross-compiling (where GDB runs on one machine—the *host*—while debugging

programs that run on another machine—the *target*). You specify a cross-debugging target by giving the ‘`--target=target`’ option to `configure`.

When you run `make` to build a program or library, you must run it in a configured directory—whatever directory you were in when you called `configure` (or one of its subdirectories).

The Makefile that `configure` generates in each source directory also runs recursively. If you type `make` in a source directory such as `gdb-18.0.50.20260503-git` (or in a separate configured directory configured with ‘`--srcdir=dirname/gdb-18.0.50.20260503-git`’), you will build all the required libraries, and then build GDB.

When you have multiple hosts or targets configured in separate directories, you can run `make` on them in parallel (for example, if they are NFS-mounted on each of the hosts); they will not interfere with each other.

C.4 Specifying Names for Hosts and Targets

The specifications used for hosts and targets in the `configure` script are based on a three-part naming scheme, but some short predefined aliases are also supported. The full naming scheme encodes three pieces of information in the following pattern:

```
architecture-vendor-os
```

For example, you can use the alias `sun4` as a *host* argument, or as the value for *target* in a `--target=target` option. The equivalent full name is ‘`sparc-sun-sunos4`’.

The `configure` script accompanying GDB does not provide any query facility to list all supported host and target names or aliases. `configure` calls the Bourne shell script `config.sub` to map abbreviations to full names; you can read the script, if you wish, or you can use it to test your guesses on abbreviations—for example:

```
% sh config.sub i386-linux
i386-pc-linux-gnu
% sh config.sub alpha-linux
alpha-unknown-linux-gnu
% sh config.sub hp9k700
hppa1.1-hp-hpux
% sh config.sub sun4
sparc-sun-sunos4.1.1
% sh config.sub sun3
m68k-sun-sunos4.1.1
% sh config.sub i986v
Invalid configuration 'i986v': machine 'i986v' not recognized
```

`config.sub` is also distributed in the GDB source directory (`gdb-18.0.50.20260503-git`, for version 18.0.50.20260503-git).

C.5 configure Options

Here is a summary of the `configure` options and arguments that are most often useful for building GDB. `configure` also has several other options not listed here. See Section “Running configure Scripts” in `autoconf`, for a full explanation of `configure`.

```
configure [--help]
           [--prefix=dir]
           [--exec-prefix=dir]
           [--srcdir=dirname]
```

`[--target=target]`

You may introduce options with a single ‘-’ rather than ‘--’ if you prefer; but you may abbreviate option names if you use ‘--’.

`--help` Display a quick summary of how to invoke `configure`.

`--prefix=dir`

Configure the source to install programs and files under directory *dir*.

`--exec-prefix=dir`

Configure the source to install programs under directory *dir*.

`--srcdir=dirname`

Use this option to make configurations in directories separate from the GDB source directories. Among other things, you can use this to build (or maintain) several configurations simultaneously, in separate directories. `configure` writes configuration-specific files in the current directory, but arranges for them to use the source in the directory *dirname*. `configure` creates directories under the working directory in parallel to the source directories below *dirname*.

`--target=target`

Configure GDB for cross-debugging programs running on the specified *target*. Without this option, GDB is configured to debug programs that run on the same machine (*host*) as GDB itself.

There is no convenient way to generate a list of all available targets. Also see the `--enable-targets` option, below.

There are many other options that are specific to GDB. This lists just the most common ones; there are some very specialized options not described here.

`--enable-targets=[target]`...

`--enable-targets=all`

Configure GDB for cross-debugging programs running on the specified list of targets. The special value ‘all’ configures GDB for debugging programs running on any target it supports.

`--enable-binary-file-formats=[format]`...

`--enable-binary-file-formats=all`

Configure GDB to support certain binary file formats. If a format is the main (or only) file format for a given target, the configure script may add support to it anyway, and warn the user. If not given, all file formats that GDB supports are compiled in.

`--with-gdb-datadir=path`

Set the GDB-specific data directory. GDB will look here for certain supporting files or scripts. This defaults to the `gdb` subdirectory of ‘*datadir*’ (which can be set using `--datadir`).

`--with-relocated-sources=dir`

Sets up the default source path substitution rule so that directory names recorded in debug information will be automatically adjusted for any directory under *dir*. *dir* should be a subdirectory of GDB’s configured prefix, the one

mentioned in the `--prefix` or `--exec-prefix` options to configure. This option is useful if GDB is supposed to be moved to a different place after it is built.

`--enable-64-bit-bfd`

Enable 64-bit support in BFD on 32-bit hosts.

`--disable-gdbmi`

Build GDB without the GDB/MI machine interface (see Chapter 27 [GDB/MI], page 609).

`--enable-tui`

Build GDB with the text-mode full-screen user interface (TUI). Requires a curses library (ncurses and cursesX are also supported).

`--with-curses`

Use the curses library instead of the termcap library, for text-mode terminal operations.

`--with-debuginfod`

Build GDB with `libdebuginfod`, the `debuginfod` client library. Used to automatically fetch ELF, DWARF and source files from `debuginfod` servers using build IDs associated with any missing files. Enabled by default if `libdebuginfod` is installed and found at configure time. For more information regarding `debuginfod` see Appendix K [Debuginfod], page 913.

`--with-libunwind-ia64`

Use the `libunwind` library for unwinding function call stack on ia64 target platforms. See <http://www.nongnu.org/libunwind/index.html> for details.

`--with-system-readline`

Use the readline library installed on the host, rather than the library supplied as part of GDB. Readline 7 or newer is required; this is enforced by the build system.

`--with-system-zlib`

Use the zlib library installed on the host, rather than the library supplied as part of GDB.

`--with-expat`

Build GDB with Expat, a library for XML parsing. (Done by default if `libexpat` is installed and found at configure time.) This library is used to read XML files supplied with GDB. If it is unavailable, some features, such as remote protocol memory maps, target descriptions, and shared library lists, that are based on XML files, will not be available in GDB. If your host does not have `libexpat` installed, you can get the latest version from <http://expat.sourceforge.net>.

`--with-libiconv-prefix[=dir]`

Build GDB with GNU `libiconv`, a character set encoding conversion library. This is not done by default, as on GNU systems the `iconv` that is built in to the C library is sufficient. If your host does not have a working `iconv`, you can

get the latest version of GNU iconv from <https://www.gnu.org/software/libiconv/>.

GDB's build system also supports building GNU libiconv as part of the overall build. See Section C.1 [Requirements], page 767.

--with-lzma

Build GDB with LZMA, a compression library. (Done by default if liblzma is installed and found at configure time.) LZMA is used by GDB's "mini debuginfo" feature, which is only useful on platforms using the ELF object file format. If your host does not have liblzma installed, you can get the latest version from <https://tukaani.org/xz/>.

--with-python[=python]

Build GDB with Python scripting support. (Done by default if libpython is present and found at configure time.) Python makes GDB scripting much more powerful than the restricted CLI scripting language. If your host does not have Python installed, you can find it on <http://www.python.org/download/>. The oldest version of Python supported by GDB is 3.4. The optional argument *python* is used to find the Python headers and libraries. It can be either the name of a Python executable, or the name of the directory in which Python is installed.

--with-guile[=guile]

Build GDB with GNU Guile scripting support. (Done by default if libguile is present and found at configure time.) If your host does not have Guile installed, you can find it at <https://www.gnu.org/software/guile/>. The optional argument *guile* can be a version number, which will cause **configure** to try to use that version of Guile; or the file name of a **pkg-config** executable, which will be queried to find the information needed to compile and link against Guile.

--without-included-regex

Don't use the regex library included with GDB (as part of the libiberty library). This is the default on hosts with version 2 of the GNU C library.

--with-sysroot=dir

Use *dir* as the default system root directory for libraries whose file names begin with */lib*' or */usr/lib*'. (The value of *dir* can be modified at run time by using the **set sysroot** command.) If *dir* is under the GDB configured prefix (set with **--prefix** or **--exec-prefix** options, the default system root will be automatically adjusted if and when GDB is moved to a different location.

--with-system-gdbinit=file

Configure GDB to automatically load a system-wide init file. *file* should be an absolute file name. If *file* is in a directory under the configured prefix, and GDB is moved to another location after being built, the location of the system-wide init file will be adjusted accordingly.

--with-system-gdbinit-dir=directory

Configure GDB to automatically load init files from a system-wide directory. *directory* should be an absolute directory name. If *directory* is in a directory

under the configured prefix, and GDB is moved to another location after being built, the location of the system-wide init directory will be adjusted accordingly.

--enable-build-warnings

When building the GDB sources, ask the compiler to warn about any code which looks even vaguely suspicious. It passes many different warning flags, depending on the exact version of the compiler you are using.

--enable-werror

Treat compiler warnings as errors. It adds the `-Werror` flag to the compiler, which will fail the compilation if the compiler outputs any warning messages.

--enable-ubsan

Enable the GCC undefined behavior sanitizer. This is disabled by default, but passing `--enable-ubsan=yes` or `--enable-ubsan=auto` to `configure` will enable it. The undefined behavior sanitizer checks for C++ undefined behavior. It has a performance cost, so if you are looking at GDB's performance, you should disable it. The undefined behavior sanitizer was first introduced in GCC 4.9.

C.6 System-wide configuration and settings

GDB can be configured to have a system-wide init file and a system-wide init file directory; this file and files in that directory (if they have a recognized file extension) will be read and executed at startup (see Section 2.1.3 [What GDB does during startup], page 17).

Here are the corresponding configure options:

--with-system-gdbinit=*file*

Specify that the default location of the system-wide init file is *file*.

--with-system-gdbinit-dir=*directory*

Specify that the default location of the system-wide init file directory is *directory*.

If GDB has been configured with the option `--prefix=$prefix`, they may be subject to relocation. Two possible cases:

- If the default location of this init file/directory contains `$prefix`, it will be subject to relocation. Suppose that the configure options are `--prefix=$prefix --with-system-gdbinit=$prefix/etc/gdbinit`; if GDB is moved from `$prefix` to `$install`, the system init file is looked for as `$install/etc/gdbinit` instead of `$prefix/etc/gdbinit`.
- By contrast, if the default location does not contain the prefix, it will not be relocated. E.g. if GDB has been configured with `--prefix=/usr/local --with-system-gdbinit=/usr/share/gdb/gdbinit`, then GDB will always look for `/usr/share/gdb/gdbinit`, wherever GDB is installed.

If the configured location of the system-wide init file (as given by the `--with-system-gdbinit` option at configure time) is in the data-directory (as specified by `--with-gdb-datadir` at configure time) or in one of its subdirectories, then GDB will look for the system-wide init file in the directory specified by the `--data-directory` command-line option. Note that the system-wide init file is only read once, during GDB initialization. If the data-directory is changed after GDB has started with the `set data-directory` command, the file will not be reread.

This applies similarly to the system-wide directory specified in `--with-system-gdbinit-dir`.

Any supported scripting language can be used for these init files, as long as the file extension matches the scripting language. To be interpreted as regular GDB commands, the files need to have a `.gdb` extension.

C.6.1 Installed System-wide Configuration Scripts

The `system-gdbinit` directory, located inside the data-directory (as specified by `--with-gdb-datadir` at configure time) contains a number of scripts which can be used as system-wide init files. To automatically source those scripts at startup, GDB should be configured with `--with-system-gdbinit`. Otherwise, any user should be able to source them by hand as needed.

The following scripts are currently available:

- `elinos.py` This script is useful when debugging a program on an ELinOS target. It takes advantage of the environment variables defined in a standard ELinOS environment in order to determine the location of the system shared libraries, and then sets the `'solib-absolute-prefix'` and `'solib-search-path'` variables appropriately.
- `wrs-linux.py` This script is useful when debugging a program on a target running Wind River Linux. It expects the `ENV_PREFIX` to be set to the host-side sysroot used by the target system.

Appendix D Maintenance Commands

In addition to commands intended for GDB users, GDB includes a number of commands intended for GDB developers, that are not documented elsewhere in this manual. These commands are provided here for reference. (For commands that turn on debugging messages, see Section 22.10 [Debugging Output], page 387.)

maint agent [-at *linespec*,] *expression*

maint agent-eval [-at *linespec*,] *expression*

Translate the given *expression* into remote agent bytecodes. This command is useful for debugging the Agent Expression mechanism (see Appendix F [Agent Expressions], page 871). The ‘**agent**’ version produces an expression useful for data collection, such as by tracepoints, while ‘**maint agent-eval**’ produces an expression that evaluates directly to a result. For instance, a collection expression for **globa + globb** will include bytecodes to record four bytes of memory at each of the addresses of **globa** and **globb**, while discarding the result of the addition, while an evaluation expression will do the addition and return the sum. If **-at** is given, generate remote agent bytecode for all the addresses to which *linespec* resolves (see Section 9.2.1 [Linespec Locations], page 129). If not, generate remote agent bytecode for current frame PC address.

maint agent-printf *format,expr,...*

Translate the given format string and list of argument expressions into remote agent bytecodes and display them as a disassembled list. This command is useful for debugging the agent version of dynamic printf (see Section 5.1.8 [Dynamic Printf], page 81).

maint canonicalize *name*

Print the canonical form of *name*, a C++ name. Because a C++ name may have multiple possible spellings, GDB computes a canonical form of a name for internal use. For example, **short int** and **short** are two ways to name the same type.

maint info **breakpoints**

Using the same format as ‘**info breakpoints**’, display both the breakpoints you’ve set explicitly, and those GDB is using for internal purposes. Internal breakpoints are shown with negative breakpoint numbers. The type column identifies what kind of breakpoint is shown:

breakpoint

Normal, explicitly set breakpoint.

watchpoint

Normal, explicitly set watchpoint.

longjmp Internal breakpoint, used to handle correctly stepping through **longjmp** calls.

longjmp resume

Internal breakpoint at the target of a **longjmp**.

until Temporary internal breakpoint used by the GDB **until** command.

finish Temporary internal breakpoint used by the GDB **finish** command.

shlib events
Shared library events.

maint info btrace
Print information about raw branch tracing data.

maint btrace packet-history
Print the raw branch trace packets that are used to compute the execution history for the **'record btrace'** command. Both the information and the format in which it is printed depend on the btrace recording format.

bts For the BTS recording format, print a list of blocks of sequential code. For each block, the following information is printed:

- Block number
 - Newer blocks have higher numbers. The oldest block has number zero.
- Lowest 'PC'
- Highest 'PC'

pt For the Intel Processor Trace recording format, print a list of Intel Processor Trace packets. For each packet, the following information is printed:

- Packet number
 - Newer packets have higher numbers. The oldest packet has number zero.
- Trace offset
 - The packet's offset in the trace stream.
- Packet opcode and payload

maint btrace clear-packet-history
Discards the cached packet history printed by the **'maint btrace packet-history'** command. The history will be computed again when needed.

maint btrace clear
Discard the branch trace data. The data will be fetched anew and the branch trace will be recomputed when needed.

This implicitly truncates the branch trace to a single branch trace buffer. When updating branch trace incrementally, the branch trace available to GDB may be bigger than a single branch trace buffer.

maint set btrace pt skip-pad
maint show btrace pt skip-pad
Control whether GDB will skip PAD packets when computing the packet history.

maint info jit
Print information about JIT code objects loaded in the current inferior.

maint info python-disassemblers

This command is defined within the `gdb.disassembler` Python module (see Section 23.3.2.42 [Disassembly In Python], page 517), and will only be present after that module has been imported. To force the module to be imported do the following:

maint info linux-lwps

Print information about LWPs under control of the Linux native target.

```
(gdb) python import gdb.disassembler
```

This command lists all the architectures for which a disassembler is currently registered, and the name of the disassembler. If a disassembler is registered for all architectures, then this is listed last against the ‘GLOBAL’ architecture.

If one of the disassemblers would be selected for the architecture of the current inferior, then this disassembler will be marked.

The following example shows a situation in which two disassemblers are registered, initially the ‘i386’ disassembler matches the current architecture, then the architecture is changed, now the ‘GLOBAL’ disassembler matches.

```
(gdb) show architecture
The target architecture is set to "auto" (currently "i386").
(gdb) maint info python-disassemblers
Architecture      Disassembler Name
i386               Disassembler_1 (Matches current architecture)
GLOBAL            Disassembler_2
(gdb) set architecture arm
The target architecture is set to "arm".
(gdb) maint info python-disassemblers
quit
Architecture      Disassembler Name
i386               Disassembler_1
GLOBAL            Disassembler_2 (Matches current architecture)
```

set displaced-stepping**show displaced-stepping**

Control whether or not GDB will do *displaced stepping* if the target supports it. Displaced stepping is a way to single-step over breakpoints without removing them from the inferior, by executing an out-of-line copy of the instruction that was originally at the breakpoint location. It is also known as out-of-line single-stepping.

set displaced-stepping on

If the target architecture supports it, GDB will use displaced stepping to step over breakpoints.

set displaced-stepping off

GDB will not use displaced stepping to step over breakpoints, even if such is supported by the target architecture.

set displaced-stepping auto

This is the default mode. GDB will use displaced stepping only if non-stop mode is active (see Section 5.5.2 [Non-Stop Mode], page 97) and the target architecture supports displaced stepping.

`maint check symtabs`

Check the consistency of currently expanded symtabs.

`maint expand-symtabs [regex]`

Expand symbol tables. If *regex* is specified, only expand symbol tables for file names matching *regex*.

`maint set catch-demangler-crashes [on|off]`

`maint show catch-demangler-crashes`

Control whether GDB should attempt to catch crashes in the symbol name demangler. The default is to attempt to catch crashes. If enabled, the first time a crash is caught, a core file is created, the offending symbol is displayed and the user is presented with the option to terminate the current session.

`maint cplus first_component name`

Print the first C++ class/namespace component of *name*.

`maint cplus namespace`

Print the list of possible C++ namespaces.

`maint deprecate command [replacement]`

`maint undeprecate command`

Deprecate or undeprecate the named *command*. Deprecated commands cause GDB to issue a warning when you use them. The optional argument *replacement* says which newer command should be used in favor of the deprecated one; if it is given, GDB will mention the replacement as part of the warning.

`maint dump-me`

Cause a fatal signal in the debugger and force it to dump its core. This is supported only on systems which support aborting a program with the SIGQUIT signal.

`maint internal-error [message-text]`

`maint internal-warning [message-text]`

`maint demangler-warning [message-text]`

Cause GDB to call the internal function `internal_error`, `internal_warning` or `demangler_warning` and hence behave as though an internal problem has been detected. In addition to reporting the internal problem, these functions give the user the opportunity to either quit GDB or (for `internal_error` and `internal_warning`) create a core file of the current GDB session.

These commands take an optional parameter *message-text* that is used as the text of the error or warning message.

Here's an example of using `internal-error`:

```
(gdb) maint internal-error testing, 1, 2
.../maint.c:121: internal-error: testing, 1, 2
A problem internal to GDB has been detected. Further
debugging may prove unreliable.
Quit this debugging session? (y or n) n
Create a core file? (y or n) n
(gdb)
```

```
maint set debuginfod download-sections
maint set debuginfod download-sections [on|off]
maint show debuginfod download-sections
```

Controls whether GDB will attempt to download individual ELF/DWARF sections from `debuginfod`. If disabled, only whole debug info files will be downloaded; this could result in GDB downloading larger amounts of data.

```
maint set internal-error action [ask|yes|no]
maint show internal-error action
maint set internal-warning action [ask|yes|no]
maint show internal-warning action
maint set demangler-warning action [ask|yes|no]
maint show demangler-warning action
```

When GDB reports an internal problem (error or warning) it gives the user the opportunity to both quit GDB and create a core file of the current GDB session. These commands let you override the default behavior for each particular *action*, described in the table below.

‘quit’ You can specify that GDB should always (yes) or never (no) quit. The default is to ask the user what to do.

‘corefile’ You can specify that GDB should always (yes) or never (no) create a core file. The default is to ask the user what to do. Note that there is no `corefile` option for `demangler-warning`: demangler warnings always create a core file and this cannot be disabled.

```
maint set internal-error backtrace [on|off]
maint show internal-error backtrace
maint set internal-warning backtrace [on|off]
maint show internal-warning backtrace
```

When GDB reports an internal problem (error or warning) it is possible to have a backtrace of GDB printed to the standard error stream. This is ‘on’ by default for `internal-error` and ‘off’ by default for `internal-warning`.

```
maint packet text
```

If GDB is talking to an inferior via the serial protocol, then this command sends the string *text* to the inferior, and displays the response packet. GDB supplies the initial ‘\$’ character, the terminating ‘#’ character, and the checksum.

Any non-printable characters in the reply are printed as escaped hex, e.g. ‘\x00’, ‘\x01’, etc.

```
maint print architecture [file]
```

Print the entire architecture configuration. The optional argument *file* names the file where the output goes.

```
maint print c-tdesc [-single-feature] [file]
```

Print the target description (see Appendix G [Target Descriptions], page 883) as a C source file. By default, the target description is for the current target, but if the optional argument *file* is provided, that file is used to produce the description. The *file* should be an XML document, of the form described in

Section G.2 [Target Description Format], page 883. The created source file is built into GDB when GDB is built again. This command is used by developers after they add or modify XML target descriptions.

When the optional flag ‘**-single-feature**’ is provided then the target description being processed (either the default, or from *file*) must only contain a single feature. The source file produced is different in this case.

The *file* argument supports escaping and quoting, see Section 3.4 [Filenames As Command Arguments], page 27.

maint print xml-tdesc [*file*]

Print the target description (see Appendix G [Target Descriptions], page 883) as an XML file. By default print the target description for the current target, but if the optional argument *file* is provided, then that file is read in by GDB and then used to produce the description. The *file* should be an XML document, of the form described in Section G.2 [Target Description Format], page 883.

maint check xml-descriptions *dir*

Check that the target descriptions dynamically created by GDB equal the descriptions created from XML files found in *dir*.

maint check libthread-db

Run integrity checks on the current inferior’s thread debugging library. This exercises all **libthread_db** functionality used by GDB on GNU/Linux systems, and by extension also exercises the **proc_service** functions provided by GDB that **libthread_db** uses. Note that parts of the test may be skipped on some platforms when debugging core files.

maint print core-file-backed-mappings

Print the file-backed mappings which were loaded from a core file note. This output represents state internal to GDB and should be similar to the mappings displayed by the **info proc mappings** command.

maint print dummy-frames

Prints the contents of GDB’s internal dummy-frame stack.

```
(gdb) b add
...
(gdb) print add(2,3)
Breakpoint 2, add (a=2, b=3) at ...
58  return (a + b);
The program being debugged stopped while in a function called from GDB.
...
(gdb) maint print dummy-frames
0xa8206d8: id={stack=0xbffff734,code=0xbffff73f,!special}, ptid=process 9353
(gdb)
```

Takes an optional file parameter.

maint print frame-id

maint print frame-id level

Print GDB’s internal frame-id for the frame at relative *level*, or for the currently selected frame when *level* is not given.

If used, *level* should be an integer, as displayed in the **backtrace** output.

```
(gdb) maint print frame-id
```

```

frame-id for frame #0: {stack=0x7fffffffac70,code=0x000000000401106,!special}
(gdb) maint print frame-id 2
frame-id for frame #2: {stack=0x7fffffffac90,code=0x00000000040111c,!special}

```

```
maint info inline-frames
```

```
maint info inline-frames address
```

Print information about inlined frames which start at the current address, or *address* if specified.

In order to allow the user to correctly step into inlined functions, GDB needs to identify which inlined functions start at a particular address, and GDB also needs to track which of these functions was last displayed to the user as the current frame.

Imagine a situation where function `main` calls `foo`, which then calls `bar`, something like this:

```

int
main ()
{
    /* Some interesting code here... */

    foo ();

    /* More interesting code here... */
}

void
foo ()
{
    bar ();
}

void
bar ()
{
    /* Some interesting code here... */
}

```

As both `foo` and `bar` are inlined within `main` then there could be one address within `main` which is also the start of `foo` and also the start of `bar`. When the user stops at this address they will initially be told the inferior is in `main`, if the user does a *step* then GDB doesn't actually step the inferior, instead the user is told the inferior entered `foo`. After the next *step* the user is told the inferior entered `bar`. The *maint info inline-frames* command can be used to view this internal GDB state, like this:

```

(gdb) step
24   foo ();
(gdb) maintenance info inline-frames
Cached inline state information for thread 1.
program counter = 0x401137
skipped frames = 2
    bar
    foo
> main

```

Here the user is stopped in `main` at the call to `foo`. The inline-frames information shows that at this address GDB has found the start of inlined functions

`bar` and `foo`, but currently GDB has skipped 2 frames and considers `main` to be the current frame, this is indicated with the `>`.

If the user performs a *step* to enter `foo` then the situation is updated:

```
(gdb) step
foo () at inline.c:14
14  bar ();
(gdb) maintenance info inline-frames
Cached inline state information for thread 1.
program counter = 0x401137
skipped frames = 1
  bar
> foo
  main
```

Notice that the program counter value `0x401137` hasn't change, but now GDB considers `foo` to be the current frame, and it is marked as such with the `>`.

Finally, the user performs another *step* to enter `bar`:

```
(gdb) step
bar () at inline.c:6
6  ++global_counter;
(gdb) maintenance info inline-frames
Cached inline state information for thread 1.
program counter = 0x401137
skipped frames = 0
> bar
  foo
  main
```

`maint info blocks`

`maint info blocks address`

Print information about all blocks at *address*, or at the current `$pc` if *address* is not given.

For information about what blocks are in GDB see Section 23.3.2.29 [Blocks In Python], page 493.

Blocks are listed starting from the global block, then the static block, and then proceeding through progressively narrower scopes.

Here is an example of the command's output:

```
(gdb) maintenance info blocks
Blocks at 0x401137:
  from objfile: [(objfile *) 0x50507d0] /tmp/inline_func_demo

[(block *) 0x504da90] 0x401106..0x40119a
  entry pc: 0x401106
  is global block
  symbol count: 2
  is contiguous
[(block *) 0x504d9f0] 0x401106..0x40119a
  entry pc: 0x401106
  is static block
  symbol count: 1
  is contiguous
[(block *) 0x504d9a0] 0x401106..0x40119a
  entry pc: 0x401106
  function: main
  is contiguous
```

```

[(block *) 0x504d900] 0x401137..0x401166
  entry pc: 0x401137
  inline function: foo
  symbol count: 1
  is contiguous
[(block *) 0x504d860] 0x401137..0x401165
  entry pc: 0x401137
  inline function: bar
  symbol count: 1
  is contiguous

```

The command *maint info blocks* lists the symbol count for each block but doesn't print the symbols themselves. The symbol names can be found using *maint print symbols* (see [maint print symbols], page 275).

```

maint print registers [file]
maint print raw-registers [file]
maint print cooked-registers [file]
maint print register-groups [file]
maint print remote-registers [file]

```

Print GDB's internal register data structures.

The command *maint print raw-registers* includes the contents of the raw register cache; the command *maint print cooked-registers* includes the (cooked) value of all registers, including registers which aren't available on the target nor visible to user; the command *maint print register-groups* includes the groups that each register is a member of; and the command *maint print remote-registers* includes the remote target's register numbers and offsets in the 'G' packets, as well as an indication of which registers were included in the last stop reply packet received by GDB (see Section E.4 [Stop Reply Packets], page 807). Please note that the list of registers included in a stop reply can change from one stop to the next.

These commands take an optional parameter, a file name to which to write the information.

```

maint print reggroups [file]

```

Print GDB's internal register group data structures. The optional argument *file* tells to what file to write the information.

The register groups info looks like this:

```

(gdb) maint print reggroups
Group      Type
general    user
float      user
all        user
vector     user
system     user
save       internal
restore    internal

```

```

maint flush register-cache
flushregs

```

Flush the contents of the register cache and as a consequence the frame cache. This command is useful when debugging issues related to register fetching, or

frame unwinding. The command `flushregs` is deprecated in favor of `maint flush register-cache`.

`maint flush source-cache`

Flush GDB's cache of source code file contents. After GDB reads a source file, and optionally applies styling (see Section 22.5 [Output Styling], page 375), the file contents are cached. This command clears that cache. The next time GDB wants to show lines from a source file, the content will be re-read.

This command is useful when debugging issues related to source code styling. After flushing the cache any source code displayed by GDB will be re-read and re-styled.

`maint print objfiles [regex]`

Print a dump of all known object files. If *regex* is specified, only print object files whose names match *regex*. For each object file, this command prints its name, address in memory, and all of its symtabs.

`maint print user-registers`

List all currently available *user registers*. User registers typically provide alternate names for actual hardware registers. They include the four “standard” registers `$fp`, `$pc`, `$sp`, and `$ps`. See [standard registers], page 181. User registers can be used in expressions in the same way as the canonical register names, but only the latter are listed by the `info registers` and `maint print registers` commands.

`maint print section-scripts [regex]`

Print a dump of scripts specified in the `.debug_gdb_section` section. If *regex* is specified, only print scripts loaded by object files matching *regex*. For each script, this command prints its name as specified in the objfile, and the full path if known. See Section 23.5.2 [dotdebug_gdb_scripts section], page 593.

`maint print statistics`

This command prints, for each object file in the program, various data about that object file followed by the byte cache (*bcache*) statistics for the object file. The objfile data includes the number of minimal symbols, of full symbols, of types defined by the objfile, of line tables and of string tables, and the amount of memory used by the various tables. The bcache statistics include the counts, sizes, and counts of duplicates of all and unique objects, max, average, and median entry size, total memory used and its overhead and savings, and various measures of the hash table size and chain lengths.

`maint print target-stack`

A *target* is an interface between the debugger and a particular kind of file or process. Targets can be stacked in *strata*, so that more than one target can potentially respond to a request. In particular, memory accesses will walk down the stack of targets until they find a target that is interested in handling that particular address.

This command prints a short description of each layer that was pushed on the *target stack*, starting from the top layer down to the bottom one.

maint print type *expr*

Print the type chain for a type specified by *expr*. The argument can be either a type name or a symbol. If it is a symbol, the type of that symbol is described. The type chain produced by this command is a recursive definition of the data type as stored in GDB's data structures, including its flags and contained types.

maint print record-instruction

maint print record-instruction *N*

print how GDB recorded a given instruction. If *n* is not positive number, it prints the values stored by the inferior before the *n*-th previous instruction was executed. If *n* is positive, print the values after the *n*-th following instruction is executed. If *n* is not given, 0 is assumed.

maint selftest [-verbose] [*filter*]

Run any self tests that were compiled in to GDB. This will print a message showing how many tests were run, and how many failed. If a *filter* is passed, only the tests with *filter* in their name will be ran. If **-verbose** is passed, the self tests can be more verbose.

maint set selftest verbose

maint show selftest verbose

Control whether self tests are run verbosely or not.

maint info selftests

List the selftests compiled in to GDB.

maint set dwarf always-disassemble

maint show dwarf always-disassemble

Control the behavior of **info address** when using DWARF debugging information.

The default is **off**, which means that GDB should try to describe a variable's location in an easily readable format. When **on**, GDB will instead display the DWARF location expression in an assembly-like format. Note that some locations are too complex for GDB to describe simply; in this case you will always see the disassembly form.

Here is an example of the resulting disassembly:

```
(gdb) info addr argc
Symbol "argc" is a complex DWARF expression:
1: DW_OP_fbreg 0
```

For more information on these expressions, see the DWARF standard (<http://www.dwarfstd.org/>).

maint set dwarf max-cache-age

maint show dwarf max-cache-age

Control the DWARF compilation unit cache.

In object files with inter-compilation-unit references, such as those produced by the GCC option **'-feliminate-dwarf2-dups'**, the DWARF reader needs to frequently refer to previously read compilation units. This setting controls how long a compilation unit will remain in the cache if it is not referenced. A higher limit means that cached compilation units will be stored in memory longer, and

more total memory will be used. Setting it to zero disables caching, which will slow down GDB startup, but reduce memory consumption.

maint set dwarf synchronous

maint show dwarf synchronous

Control whether DWARF is read asynchronously.

On hosts where threading is available, the DWARF reader is mostly asynchronous with respect to the rest of GDB. That is, the bulk of the reading is done in the background, and GDB will only pause for completion of this task when absolutely necessary.

When this setting is enabled, GDB will instead wait for DWARF processing to complete before continuing.

On hosts without threading, or where worker threads have been disabled at runtime, this setting has no effect, as DWARF reading is always done on the main thread, and is therefore always synchronous.

maint info frame-unwinders

List the frame unwinders currently in effect, starting with the highest priority. This also lists the unwinder class, which is mostly defined by which area of GDB uses it. The currently available classes are:

‘GDB’ Internal unwinders, added by GDB core.

‘EXTENSION’ Unwinders added by extension languages.

‘DEBUGINFO’ Unwinders installed by debug information readers.

‘ARCH’ Unwinders installed by the architecture specific code.

maint frame-unwinder disable [-all | -name *name* | [-class] *class*]

maint frame-unwinder enable [-all | -name *name* | [-class] *class*]

Disable or enable frame unwinders. This is meant only as a testing tool, and GDB is not guaranteed to work properly if it is unable to find valid frame unwinders.

The meaning of each of the invocations are as follows:

‘-all’ Disable or enable all unwinders.

‘-name *name*’ *name* is the exact name of the unwinder to be disabled or enabled.

‘[-class] *class*’ *class* is the class of frame unwinders to be disabled or enabled. The class may include the prefix `FRAME_UNWINDER_`, but it is not required.

name and *class* are always case insensitive. If no option starting with - is given, GDB assumes -class.

maint set worker-threads

maint show worker-threads

Control the number of worker threads that may be used by GDB. On capable hosts, GDB may use multiple threads to speed up certain CPU-intensive operations, such as demangling symbol names. While the number of threads used by GDB may vary, this command can be used to set an upper bound on this number. The default is **unlimited**, which lets GDB choose a reasonable number. Note that this only controls worker threads started by GDB itself; libraries used by GDB may start threads of their own.

maint set profile

maint show profile

Control profiling of GDB.

Profiling will be disabled until you use the ‘**maint set profile**’ command to enable it. When you enable profiling, the system will begin collecting timing and execution count data; when you disable profiling or exit GDB, the results will be written to a log file. Remember that if you use profiling, GDB will overwrite the profiling log file (often called **gmon.out**). If you have a record of important profiling data in a **gmon.out** file, be sure to move it to a safe location.

Configuring with ‘**--enable-profiling**’ arranges for GDB to be compiled with the ‘**-pg**’ compiler option.

maint set show-debug-regs

maint show show-debug-regs

Control whether to show variables that mirror the hardware debug registers. Use **on** to enable, **off** to disable. If enabled, the debug registers values are shown when GDB inserts or removes a hardware breakpoint or watchpoint, and when the inferior triggers a hardware-assisted breakpoint or watchpoint.

maint set show-all-tib

maint show show-all-tib

Control whether to show all non zero areas within a 1k block starting at thread local base, when using the ‘**info w32 thread-information-block**’ command.

maint set target-async

maint show target-async

This controls whether GDB targets operate in synchronous or asynchronous mode (see Section 5.5.3 [Background Execution], page 98). Normally the default is asynchronous, if it is available; but this can be changed to more easily debug problems occurring only in synchronous mode.

maint set target-non-stop

maint show target-non-stop

This controls whether GDB targets (e.g., the native target, or a remote target) operate in non-stop mode even if **set non-stop** is **off** (see Section 5.5.2 [Non-Stop Mode], page 97). The default is **auto**.

This affects GDB internal operation and is largely invisible to users. Normally users should not need to change this setting, but it can be changed to more easily debug problems occurring only in a specific mode.

maint set target-non-stop auto

This is the default mode. GDB controls the target in non-stop mode if **set non-stop** is on, or the target tells infrun that it wants to operate in non-stop mode even with **set non-stop** is set to off.

maint set target-non-stop on

GDB controls the target in non-stop mode even if the target does not indicate support.

maint set target-non-stop off

GDB does not control the target in non-stop mode even if the target supports it.

Here is how **set non-stop** and **maint set target-non-stop** settings combine:

set non-stop off, target operating in all-stop mode

When a thread hits a breakpoint, finishes a step, etc., the target stops all threads, and reports the event to the infrun module in the core of GDB. If infrun decides the stop is not to be seen by the user, infrun re-resumes all threads again. In other words, all threads stop and are re-resumed for every debug event, even for debug events that are internal and do not cause a user-visible stop.

set non-stop off, target operating in non-stop mode

When a thread hits a breakpoint, finishes a step, etc., the target does not immediately stop all other threads. If, while processing the event, infrun decides the stop should be reported to the user, it then explicitly stops all threads, just before presenting the stop to the user; otherwise, infrun re-resumes the stopped thread. This scenario is also called “all-stop on top of non-stop”.

set non-stop on, target operating in all-stop mode

This combination is invalid.

set non-stop on, target operating in non-stop mode

When a thread hits a breakpoint, finishes a step, etc., neither the target, nor infrun stop any other thread.

maint set tui-resize-message

maint show tui-resize-message

Control whether GDB displays a message each time the terminal is resized when in TUI mode. The default is off, which means that GDB is silent during resizes. When on, GDB will display a message after a resize is completed; the message will include a number indicating how many times the terminal has been resized. This setting is intended for use by the test suite, where it would otherwise be difficult to determine when a resize and refresh has been completed.

maint set tui-left-margin-verbose

maint show tui-left-margin-verbose

Control whether the left margin of the TUI source and disassembly windows uses ‘_’ and ‘0’ at locations where otherwise there would be a space. The default is off, which means spaces are used. The setting is intended to make it clear

where the left margin begins and ends, to avoid incorrectly interpreting a space as being part of the the left margin.

`maint set per-command`

`maint show per-command`

GDB can display the resources used by each command. This is useful in debugging performance problems.

`maint set per-command space [on|off]`

`maint show per-command space`

Enable or disable the printing of the memory used by GDB for each command. If enabled, GDB will display how much memory each command took, following the command's own output. This can also be requested by invoking GDB with the `--statistics` command-line switch (see Section 2.1.2 [Mode Options], page 13).

`maint set per-command time [on|off]`

`maint show per-command time`

Enable or disable the printing of the execution time of GDB for each command. If enabled, GDB will display how much time it took to execute each command, following the command's own output. Both CPU time and wallclock time are printed. Printing both is useful when trying to determine whether the cost is CPU or, e.g., disk/network latency. Note that the CPU time printed is for GDB only, it does not include the execution time of the inferior because there's no mechanism currently to compute how much time was spent by GDB and how much time was spent by the program been debugged. This can also be requested by invoking GDB with the `--statistics` command-line switch (see Section 2.1.2 [Mode Options], page 13).

`maint set per-command symtab [on|off]`

`maint show per-command symtab`

Enable or disable the printing of basic symbol table statistics for each command. If enabled, GDB will display the following information:

- a. number of symbol tables
- b. number of primary symbol tables
- c. number of blocks in the blockvector

`maint set check-libthread-db [on|off]`

`maint show check-libthread-db`

Control whether GDB should run integrity checks on inferior specific thread debugging libraries as they are loaded. The default is not to perform such checks. If any check fails GDB will unload the library and continue searching for a suitable candidate as described in `[set libthread-db-search-path]`, page 52. For more information about the tests, see `[maint check libthread-db]`, page 782.

```
maint set gnu-source-highlight enabled [on|off]
maint show gnu-source-highlight enabled
```

Control whether GDB should use the GNU Source Highlight library for applying styling to source code (see Section 22.5 [Output Styling], page 375). This will be ‘on’ by default if the GNU Source Highlight library is available. If the GNU Source Highlight library is not available, then this will be ‘off’ by default, and attempting to change this value to ‘on’ will give an error.

If the GNU Source Highlight library is not being used, then GDB will use the Python Pygments package for source code styling, if it is available.

This option is useful for debugging GDB’s use of the Pygments library when GDB is linked against the GNU Source Highlight library.

```
maint set libopcodes-styling enabled [on|off]
maint show libopcodes-styling enabled
```

Control whether GDB should use its builtin disassembler (`libopcodes`) to style disassembler output (see Section 22.5 [Output Styling], page 375). The builtin disassembler does not support styling for all architectures.

When this option is ‘off’ the builtin disassembler will not be used for styling, GDB will fall back to using the Python Pygments package if possible.

Trying to set this option ‘on’ for an architecture that the builtin disassembler is unable to style will give an error, otherwise, the builtin disassembler will be used to style disassembler output.

This option is ‘on’ by default for supported architectures.

This option is useful for debugging GDB’s use of the Pygments library when GDB is built for an architecture that supports styling with the builtin disassembler

```
maint info screen
```

Print various characteristics of the screen, such as various notions of width and height.

```
maint space value
```

An alias for `maint set per-command space`. A non-zero value enables it, zero disables it.

```
maint time value
```

An alias for `maint set per-command time`. A non-zero value enables it, zero disables it.

```
maint translate-address [section] addr
```

Find the symbol stored at the location specified by the address `addr` and an optional section name `section`. If found, GDB prints the name of the closest symbol and an offset from the symbol’s location to the specified address. This is similar to the `info address` command (see Chapter 16 [Symbols], page 265), except that this command also allows to find symbols in other sections.

If section was not specified, the section in which the symbol was found is also printed. For dynamically linked executables, the name of executable or shared library containing the symbol is printed as well.

```
maint test-options require-delimiter
maint test-options unknown-is-error
maint test-options unknown-is-operand
```

These commands are used by the testsuite to validate the command options framework. The `require-delimiter` variant requires a double-dash delimiter to indicate end of options. The `unknown-is-error` and `unknown-is-operand` do not. The `unknown-is-error` variant throws an error on unknown option, while `unknown-is-operand` treats unknown options as the start of the command's operands. When run, the commands output the result of the processed options. When completed, the commands store the internal result of completion in a variable exposed by the `maint show test-options-completion-result` command.

```
maint show test-options-completion-result
```

Shows the result of completing the `maint test-options` subcommands. This is used by the testsuite to validate completion support in the command options framework.

```
maint set test-settings kind
maint show test-settings kind
```

These are representative commands for each *kind* of setting type GDB supports. They are used by the testsuite for exercising the settings infrastructure.

```
maint test-remote-args args
```

For targets that don't support passing inferior arguments as a single string (see [single-inf-arg], page 824), GDB will attempt to split the inferior arguments before passing them to the remote target, and the remote target might choose to join the inferior arguments upon receipt. Historically gdbserver did join inferior arguments, but now it will request inferior arguments be passed as a single string if GDB supports this feature.

This maintenance command splits *args* as GDB would normally split such an argument string before passing the arguments to a remote target, the split arguments are then printed.

The split arguments are then joined together as gdbserver would join them, and the result is printed.

This command is intended to help diagnose issues passing inferior arguments to remote targets.

```
maint set backtrace-on-fatal-signal [on|off]
maint show backtrace-on-fatal-signal
```

When this setting is `on`, if GDB itself terminates with a fatal signal (e.g. SIGSEGV), then a limited backtrace will be printed to the standard error stream. This backtrace can be used to help diagnose crashes within GDB in situations where a user is unable to share a corefile with the GDB developers.

If the functionality to provide this backtrace is not available for the platform on which GDB is running then this feature will be `off` by default, and attempting to turn this feature on will give an error.

For platforms that do support creating the backtrace this feature is `on` by default.

maint wait-for-index-cache

Wait until all pending writes to the index cache have completed. This is used by the test suite to avoid races when the index cache is being updated by a worker thread.

maint with *setting* [*value*] [-- *command*]

Like the **with** command, but works with **maintenance set** variables. This is used by the testsuite to exercise the **with** command's infrastructure.

maint ignore-probes [-v|-verbose] [*provider* [*name* [*objfile*]]]

maint ignore-probes -reset

Set or reset the ignore-probes filter. The *provider*, *name* and *objfile* arguments are as in **enable probes** and **disable probes** (see [enable probes], page 84). Only supported for SystemTap probes.

Here's an example of using **maint ignore-probes**:

```
(gdb) maint ignore-probes -verbose libc ^longjmp$
ignore-probes filter has been set to:
PROVIDER: 'libc'
PROBE_NAME: '^longjmp$'
OBJNAME: ''
(gdb) start
<... more output ...>
Ignoring SystemTap probe libc longjmp in /lib64/libc.so.6.^M
Ignoring SystemTap probe libc longjmp in /lib64/libc.so.6.^M
Ignoring SystemTap probe libc longjmp in /lib64/libc.so.6.^M
```

The following command is useful for non-interactive invocations of GDB, such as in the test suite.

set watchdog *nsec*

Set the maximum number of seconds GDB will wait for the target operation to finish. If this time expires, GDB reports an error and the command is aborted.

show watchdog

Show the current setting of the target wait timeout.

Appendix E GDB Remote Serial Protocol

E.1 Overview

There may be occasions when you need to know something about the protocol—for example, if there is only one serial port to your target machine, you might want your program to do something special if it recognizes a packet meant for GDB.

In the examples below, ‘->’ and ‘<-’ are used to indicate transmitted and received data, respectively.

All GDB commands and responses (other than acknowledgments and notifications, see Section E.10 [Notification Packets], page 850) are sent as a *packet*. A *packet* is introduced with the character ‘\$’, the actual *packet-data*, and the terminating character ‘#’ followed by a two-digit *checksum*:

```
$packet-data#checksum
```

The two-digit *checksum* is computed as the modulo 256 sum of all characters between the leading ‘\$’ and the trailing ‘#’ (an eight bit unsigned checksum).

Implementors should note that prior to GDB 5.0 the protocol specification also included an optional two-digit *sequence-id*:

```
$sequence-id:packet-data#checksum
```

That *sequence-id* was appended to the acknowledgment. GDB has never output *sequence-ids*. Stubs that handle packets added since GDB 5.0 must not accept *sequence-id*.

When either the host or the target machine receives a packet, the first response expected is an acknowledgment: either ‘+’ (to indicate the package was received correctly) or ‘-’ (to request retransmission):

```
-> $packet-data#checksum
<- +
```

The ‘+’/‘-’ acknowledgments can be disabled once a connection is established. See Section E.12 [Packet Acknowledgment], page 853, for details.

The host (GDB) sends *commands*, and the target (the debugging stub incorporated in your program) sends a *response*. In the case of step and continue *commands*, the response is only sent when the operation has completed, and the target has again stopped all threads in all attached processes. This is the default all-stop mode behavior, but the remote protocol also supports GDB’s non-stop execution mode; see Section E.11 [Remote Non-Stop], page 852, for details.

packet-data consists of a sequence of characters with the exception of ‘#’ and ‘\$’ (see ‘X’ packet for additional exceptions).

Fields within the packet should be separated using ‘,’ ‘;’ or ‘:’. Except where otherwise noted all numbers are represented in HEX with leading zeros suppressed.

Implementors should note that prior to GDB 5.0, the character ‘:’ could not appear as the third character in a packet (as it would potentially conflict with the *sequence-id*).

Binary data in most packets is encoded as two hexadecimal digits per byte of binary data. This allowed the traditional remote protocol to work over connections which were only seven-bit clean. Some packets designed more recently assume an eight-bit clean connection, and use a more efficient encoding to send and receive binary data.

The binary data representation uses 7d (ASCII ‘}’) as an escape character. Any escaped byte is transmitted as the escape character followed by the original character XORed with 0x20. For example, the byte 0x7d would be transmitted as the two bytes 0x7d 0x5d. The bytes 0x23 (ASCII ‘#’), 0x24 (ASCII ‘\$’), and 0x7d (ASCII ‘}’) must always be escaped. Responses sent by the stub must also escape 0x2a (ASCII ‘*’), so that it is not interpreted as the start of a run-length encoded sequence (described next).

Response *data* can be run-length encoded to save space. Run-length encoding replaces runs of identical characters with one instance of the repeated character, followed by a ‘*’ and a repeat count. The repeat count is itself sent encoded, to avoid binary characters in *data*: a value of *n* is sent as *n*+29. For a repeat count greater or equal to 3, this produces a printable ASCII character, e.g. a space (ASCII code 32) for a repeat count of 3. (This is because run-length encoding starts to win for counts 3 or more.) Thus, for example, ‘0* ’ is a run-length encoding of “0000”: the space character after ‘*’ means repeat the leading 0 32 - 29 = 3 more times.

The printable characters ‘#’ and ‘\$’ or with a numeric value greater than 126 must not be used. Runs of six repeats (‘#’) or seven repeats (‘\$’) can be expanded using a repeat count of only five (‘’’). For example, ‘00000000’ can be encoded as ‘0*’’00’.

See Section E.2 [Standard Replies], page 796, for standard error responses, and how to respond indicating a command is not supported.

In describing packets (commands and responses), each description has a template showing the overall syntax, followed by an explanation of the packet’s meaning. We include spaces in some of the templates for clarity; these are not part of the packet’s syntax. No GDB packet uses spaces to separate its components. For example, a template like ‘foo bar baz’ describes a packet beginning with the three ASCII bytes ‘foo’, followed by a *bar*, followed directly by a *baz*. GDB does not transmit a space character between the ‘foo’ and the *bar*, or between the *bar* and the *baz*.

We place optional portions of a packet in [square brackets]; for example, a template like ‘c [addr]’ describes a packet beginning with the single ASCII character ‘c’, possibly followed by an *addr*.

At a minimum, a stub is required to support the ‘?’ command to tell GDB the reason for halting, ‘g’ and ‘G’ commands for register access, and the ‘m’ and ‘M’ commands for memory access. Stubs that only control single-threaded targets can implement run control with the ‘c’ (continue) command, and if the target architecture supports hardware-assisted single-stepping, the ‘s’ (step) command. Stubs that support multi-threading targets should support the ‘vCont’ command. All other commands are optional.

E.2 Standard Replies

The remote protocol specifies a few standard replies. All commands support these, except as noted in the individual command descriptions.

empty response

An empty response (raw character sequence ‘\$#00’) means the *command* is not supported by the stub. This way it is possible to extend the protocol. A newer GDB can tell if a command is supported based on that response (but see also [qSupported], page 822).

‘E *xx*’ An error has occurred; *xx* is a two-digit hexadecimal error number. In almost all cases, the protocol does not specify the meaning of the error numbers; GDB usually ignores the numbers, or displays them to the user without further interpretation.

‘E.*errtext*’ An error has occurred; *errtext* is the textual error message, encoded in ASCII.

E.3 Packets

The following table provides a complete list of all currently defined *commands* and their corresponding response *data*. See Section E.14 [File-I/O Remote Protocol Extension], page 854, for details about the File I/O extension of the remote protocol.

Each packet’s description has a template showing the packet’s overall syntax, followed by an explanation of the packet’s meaning. We include spaces in some of the templates for clarity; these are not part of the packet’s syntax. No GDB packet uses spaces to separate its components. For example, a template like ‘**foo bar baz**’ describes a packet beginning with the three ASCII bytes ‘foo’, followed by a *bar*, followed directly by a *baz*. GDB does not transmit a space character between the ‘foo’ and the *bar*, or between the *bar* and the *baz*.

Several packets and replies include a *thread-id* field to identify a thread. Normally these are positive numbers with a target-specific interpretation, formatted as big-endian hex strings. A *thread-id* can also be a literal ‘-1’ to indicate all threads, or ‘0’ to pick any thread.

In addition, the remote protocol supports a multiprocess feature in which the *thread-id* syntax is extended to optionally include both process and thread ID fields, as ‘**ppid.tid**’. The *pid* (process) and *tid* (thread) components each have the format described above: a positive number with target-specific interpretation formatted as a big-endian hex string, literal ‘-1’ to indicate all processes or threads (respectively), or ‘0’ to indicate an arbitrary process or thread. Specifying just a process, as ‘**ppid**’, is equivalent to ‘**ppid.-1**’. It is an error to specify all processes but a specific thread, such as ‘**p-1.tid**’. Note that the ‘p’ prefix is *not* used for those packets and replies explicitly documented to include a process ID, rather than a *thread-id*.

The multiprocess *thread-id* syntax extensions are only used if both GDB and the stub report support for the ‘**multiprocess**’ feature using ‘**qSupported**’. See [multiprocess extensions], page 829, for more information.

Note that all packet forms beginning with an upper- or lower-case letter, other than those described here, are reserved for future use.

Here are the packet descriptions.

‘!’ Enable extended mode. In extended mode, the remote server is made persistent. The ‘**R**’ packet is used to restart the program being debugged.

Reply:

‘OK’ The remote target both supports and has enabled extended mode.

‘?’ This is sent when connection is first established to query the reason the target halted. The reply is the same as for step and continue. This packet has a special

interpretation when the target is in non-stop mode; see Section E.11 [Remote Non-Stop], page 852.

Reply: See Section E.4 [Stop Reply Packets], page 807, for the reply specifications.

'A arglen, argnum, arg, ...'

Initialized `argv[]` array passed into program. *arglen* specifies the number of bytes in the hex encoded byte stream *arg*. See `gdbserver` for more details.

Reply:

'OK' The arguments were set.

'b baud' (Don't use this packet; its behavior is not well-defined.) Change the serial line speed to *baud*.

JTC: *When does the transport layer state change? When it's received, or after the ACK is transmitted. In either case, there are problems if the command or the acknowledgment packet is dropped.*

Stan: *If people really wanted to add something like this, and get it working for the first time, they ought to modify ser-unix.c to send some kind of out-of-band message to a specially-setup stub and have the switch happen "in between" packets, so that from remote protocol's point of view, nothing actually happened.*

'B addr, mode'

Set (*mode* is **'S'**) or clear (*mode* is **'C'**) a breakpoint at *addr*.

Don't use this packet. Use the **'Z'** and **'z'** packets instead (see [insert breakpoint or watchpoint packet], page 806).

'bc' Backward continue. Execute the target system in reverse. No parameter. See Chapter 6 [Reverse Execution], page 103, for more information.

Reply: See Section E.4 [Stop Reply Packets], page 807, for the reply specifications.

'bs' Backward single step. Execute one instruction in reverse. No parameter. See Chapter 6 [Reverse Execution], page 103, for more information.

Reply: See Section E.4 [Stop Reply Packets], page 807, for the reply specifications.

'c [addr]' Continue at *addr*, which is the address to resume. If *addr* is omitted, resume at current address.

This packet is deprecated for multi-threading support. See [vCont packet], page 802.

Reply: See Section E.4 [Stop Reply Packets], page 807, for the reply specifications.

'C sig[, addr]'

Continue with signal *sig* (hex signal number). If **','***addr* is omitted, resume at same address.

This packet is deprecated for multi-threading support. See [vCont packet], page 802.

Reply: See Section E.4 [Stop Reply Packets], page 807, for the reply specifications.

‘d’ Toggle debug flag.

Don’t use this packet; instead, define a general set packet (see Section E.5 [General Query Packets], page 811).

‘D’

‘D;pid’ The first form of the packet is used to detach GDB from the remote system. It is sent to the remote target before GDB disconnects via the `detach` command.

The second form, including a process ID, is used when multiprocess protocol extensions are enabled (see [multiprocess extensions], page 829), to detach only a specific process. The *pid* is specified as a big-endian hex string.

Reply:

‘OK’ for success

‘F RC,EE,CF;XX’

A reply from GDB to an ‘F’ packet sent by the target. This is part of the File-I/O protocol extension. See Section E.14 [File-I/O Remote Protocol Extension], page 854, for the specification.

‘g’

Read general registers.

Reply:

‘XX...’ Each byte of register data is described by two hex digits. The bytes with the register are transmitted in target byte order. The size of each register and their position within the ‘g’ packet are determined by the target description (see Appendix G [Target Descriptions], page 883); in the absence of a target description, this is done using code internal to GDB; typically this is some customary register layout for the architecture in question.

When reading registers, the stub may also return a string of literal ‘x’'s in place of the register data digits, to indicate that the corresponding register’s value is unavailable. For example, when reading registers from a trace frame (see Section 13.2 [Using the Collected Data], page 217), this means that the register has not been collected in the trace frame. When reading registers from a live program, this indicates that the stub has no means to access the register contents, even though the corresponding register is known to exist. Note that if a register truly does not exist on the target, then it is better to not include it in the target description in the first place.

For example, for an architecture with 4 registers of 4 bytes each, the following reply indicates to GDB that registers 0 and 2 are unavailable, while registers 1 and 3 are available, and both have zero value:

```
-> g
<- xxxxxxxx00000000xxxxxxx00000000
```

‘G *XX...*’ Write general registers. See [read registers packet], page 799, for a description of the *XX...* data.

Reply:

‘OK’ for success

‘H *op thread-id*’

Set thread for subsequent operations (‘m’, ‘M’, ‘g’, ‘G’, et.al.). Depending on the operation to be performed, *op* should be ‘c’ for step and continue operations (note that this is deprecated, supporting the ‘vCont’ command is a better option), and ‘g’ for other operations. The thread designator *thread-id* has the format and interpretation described in [thread-id syntax], page 797.

Reply:

‘OK’ for success

‘i [*addr[,nnn]*’

Step the remote target by a single clock cycle. If ‘,*nnn*’ is present, cycle step *nnn* cycles. If *addr* is present, cycle step starting at that address.

‘I’ Signal, then cycle step. See [step with signal packet], page 801. See [cycle step packet], page 800.

‘k’ Kill request.

The exact effect of this packet is not specified.

For a bare-metal target, it may power cycle or reset the target system. For that reason, the ‘k’ packet has no reply.

For a single-process target, it may kill that process if possible.

A multiple-process target may choose to kill just one process, or all that are under GDB’s control. For more precise control, use the vKill packet (see [vKill packet], page 804).

If the target system immediately closes the connection in response to ‘k’, GDB does not consider the lack of packet acknowledgment to be an error, and assumes the kill was successful.

If connected using *target extended-remote*, and the target does not close the connection in response to a kill request, GDB probes the target state as if a new connection was opened (see [? packet], page 797).

‘m *addr,length*’

Read *length* addressable memory units starting at address *addr* (see [addressable memory unit], page 156). Note that *addr* does not have to be aligned to any particular boundary.

The stub need not use any particular size or alignment when gathering data from memory for the response; even if *addr* is word-aligned and *length* is a multiple of the word size, the stub is free to use byte accesses, or not. For this reason, this packet may not be suitable for accessing memory-mapped I/O devices.

Reply:

‘XX...’ Memory contents; each byte is transmitted as a two-digit hexadecimal number. The reply may contain fewer addressable memory

units than requested if the server was reading from a trace frame memory and was able to read only part of the region of memory.

Unlike most packets, this packet does not support ‘E.errtext’-style textual error replies (see [textual error reply], page 797) by default. Stubs should be careful to only send such a reply if GDB reported support for it with the `error-message` feature (see [error-message], page 823).

‘M *addr,length:XX...*’

Write *length* addressable memory units starting at address *addr* (see [addressable memory unit], page 156). The data is given by *XX...*; each byte is transmitted as a two-digit hexadecimal number.

Reply:

‘OK’ All the data was written successfully. (If only part of the data was written, this command returns an error.)

‘p *n*’ Read the value of register *n*; *n* is in hex. See [read registers packet], page 799, for a description of how the returned register value is encoded.

Reply:

‘*XX...*’ the register’s value

‘P *n...=r...*’

Write register *n...* with value *r...*. The register number *n* is in hexadecimal, and *r...* contains two hex digits for each byte in the register (target byte order).

Reply:

‘OK’ for success

‘q *name params...*’

‘Q *name params...*’

General query (‘q’) and set (‘Q’). These packets are described fully in Section E.5 [General Query Packets], page 811.

‘r’ Reset the entire system.

Don’t use this packet; use the ‘R’ packet instead.

‘R *XX*’ Restart the program being debugged. The *XX*, while needed, is ignored. This packet is only available in extended mode (see [extended mode], page 797).

The ‘R’ packet has no reply.

‘s [*addr*]’ Single step, resuming at *addr*. If *addr* is omitted, resume at same address.

This packet is deprecated for multi-threading support. See [vCont packet], page 802.

Reply: See Section E.4 [Stop Reply Packets], page 807, for the reply specifications.

‘S *sig[;addr]*’

Step with signal. This is analogous to the ‘C’ packet, but requests a single-step, rather than a normal resumption of execution.

This packet is deprecated for multi-threading support. See [vCont packet], page 802.

Reply: See Section E.4 [Stop Reply Packets], page 807, for the reply specifications.

‘t addr:PP,MM’

Search backwards starting at address *addr* for a match with pattern *PP* and mask *MM*, both of which are 4 byte long. There must be at least 3 digits in *addr*.

‘T thread-id’

Find out if the thread *thread-id* is alive. See [thread-id syntax], page 797.

Reply:

‘OK’ thread is still alive

‘v’

Packets starting with ‘v’ are identified by a multi-letter name, up to the first ‘;’ or ‘?’ (or the end of the packet).

‘vAttach;pid’

Attach to a new process with the specified process ID *pid*. The process ID is a hexadecimal integer identifying the process. In all-stop mode, all threads in the attached process are stopped; in non-stop mode, it may be attached without being stopped if that is supported by the target.

This packet is only available in extended mode (see [extended mode], page 797).

Reply:

‘Any stop packet’

for success in all-stop mode (see Section E.4 [Stop Reply Packets], page 807)

‘OK’

for success in non-stop mode (see Section E.11 [Remote Non-Stop], page 852)

‘vCont[:action[:thread-id]]...’

Resume the inferior, specifying different actions for each thread.

For each inferior thread, the leftmost action with a matching *thread-id* is applied. Threads that don’t match any action remain in their current state. Thread IDs are specified using the syntax described in [thread-id syntax], page 797. If multiprocess extensions (see [multiprocess extensions], page 829) are supported, actions can be specified to match all threads in a process by using the ‘*ppid*.-1’ form of the *thread-id*. An action with no *thread-id* matches all threads. Specifying no actions is an error.

Currently supported actions are:

‘c’ Continue.

‘C *sig*’ Continue with signal *sig*. The signal *sig* should be two hex digits.

‘s’ Step.

‘S *sig*’ Step with signal *sig*. The signal *sig* should be two hex digits.

‘t’ Stop.

‘r *start,end*’

Step once, and then keep stepping as long as the thread stops at addresses between *start* (inclusive) and *end* (exclusive). The remote stub reports a stop reply when either the thread goes out of the range or is stopped due to an unrelated reason, such as hitting a breakpoint. See [range stepping], page 89.

If the range is empty (*start* == *end*), then the action becomes equivalent to the ‘s’ action. In other words, single-step once, and report the stop (even if the stepped instruction jumps to *start*).

(A stop reply may be sent at any point even if the PC is still within the stepping range; for example, it is valid to implement this packet in a degenerate way as a single instruction step operation.)

The optional argument *addr* normally associated with the ‘c’, ‘C’, ‘s’, and ‘S’ packets is not supported in ‘vCont’.

The ‘t’ action is only relevant in non-stop mode (see Section E.11 [Remote Non-Stop], page 852) and may be ignored by the stub otherwise. A stop reply should be generated for any affected thread not already stopped. When a thread is stopped by means of a ‘t’ action, the corresponding stop reply should indicate that the thread has stopped with signal ‘0’, regardless of whether the target uses some other signal as an implementation detail.

The server must ignore ‘c’, ‘C’, ‘s’, ‘S’, and ‘r’ actions for threads that are already running. Conversely, the server must ignore ‘t’ actions for threads that are already stopped.

Note: In non-stop mode, a thread is considered running until GDB acknowledges an asynchronous stop notification for it with the ‘vStopped’ packet (see Section E.11 [Remote Non-Stop], page 852).

The stub must support ‘vCont’ if it reports support for multiprocess extensions (see [multiprocess extensions], page 829).

Reply: See Section E.4 [Stop Reply Packets], page 807, for the reply specifications.

‘vCont?’ Request a list of actions supported by the ‘vCont’ packet.

Reply:

‘vCont[;*action*...]

The ‘vCont’ packet is supported. Each *action* is a supported command in the ‘vCont’ packet.

‘vCtrlC’ Interrupt remote target as if a control-C was pressed on the remote terminal. This is the equivalent to reacting to the ^C (‘\003’, the control-C character) character in all-stop mode while the target is running, except this works in non-stop mode. See [interrupting remote targets], page 849, for more info on the all-stop variant.

Reply:

‘OK’ for success

`'vFile:operation:parameter...'`

Perform a file operation on the target system. For details, see Section E.8 [Host I/O Packets], page 847.

`'vFlashErase:addr,length'`

Direct the stub to erase *length* bytes of flash starting at *addr*. The region may enclose any number of flash blocks, but its start and end must fall on block boundaries, as indicated by the flash block size appearing in the memory map (see Section E.17 [Memory Map Format], page 867). GDB groups flash memory programming operations together, and sends a `'vFlashDone'` request after each group; the stub is allowed to delay erase operation until the `'vFlashDone'` packet is received.

Reply:

`'OK'` for success

`'vFlashWrite:addr:XX...'`

Direct the stub to write data to flash address *addr*. The data is passed in binary form using the same encoding as for the `'X'` packet (see [Binary Data], page 795). The memory ranges specified by `'vFlashWrite'` packets preceding a `'vFlashDone'` packet must not overlap, and must appear in order of increasing addresses (although `'vFlashErase'` packets for higher addresses may already have been received; the ordering is guaranteed only between `'vFlashWrite'` packets). If a packet writes to an address that was neither erased by a preceding `'vFlashErase'` packet nor by some other target-specific method, the results are unpredictable.

Reply:

`'OK'` for success

`'E.memtype'`

for vFlashWrite addressing non-flash memory

`'vFlashDone'`

Indicate to the stub that flash programming operation is finished. The stub is permitted to delay or batch the effects of a group of `'vFlashErase'` and `'vFlashWrite'` packets until a `'vFlashDone'` packet is received. The contents of the affected regions of flash memory are unpredictable until the `'vFlashDone'` request is completed.

`'vKill;pid'`

Kill the process with the specified process ID *pid*, which is a hexadecimal integer identifying the process. This packet is used in preference to `'k'` when multiprocess protocol extensions are supported; see [multiprocess extensions], page 829.

Reply:

`'OK'` for success

‘vMustReplyEmpty’

The correct reply to an unknown ‘v’ packet is to return the empty string, however, some older versions of **gdbserver** would incorrectly return ‘OK’ for unknown ‘v’ packets.

The ‘vMustReplyEmpty’ is used as a feature test to check how **gdbserver** handles unknown packets, it is important that this packet be handled in the same way as other unknown ‘v’ packets. If this packet is handled differently to other unknown ‘v’ packets then it is possible that GDB may run into problems in other areas, specifically around use of ‘vFile:setfs:’.

‘vRun;filename[;argument]...’

Run the program *filename*, passing it each *argument* on its command line. The file and arguments are hex-encoded strings. If *filename* is an empty string, the stub may use a default program (e.g. the last program run). The program is created in the stopped state.

If GDB sent the ‘single-inf-arg’ feature in the ‘qSupported’ packet (see [single-inf-arg], page 824), and the stub replied with ‘single-inf-arg+’, then there will only be a single *argument* string, which includes all inferior arguments, separated with whitespace.

This packet is only available in extended mode (see [extended mode], page 797).

Reply:

‘Any stop packet’

for success (see Section E.4 [Stop Reply Packets], page 807)

‘vStopped’

See Section E.10 [Notification Packets], page 850.

‘x addr,length’

Read *length* addressable memory units starting at address *addr* (see [addressable memory unit], page 156). Note that *addr* does not have to be aligned to any particular boundary.

The stub need not use any particular size or alignment when gathering data from memory for the response; even if *addr* is word-aligned and *length* is a multiple of the word size, the stub is free to use byte accesses, or not. For this reason, this packet may not be suitable for accessing memory-mapped I/O devices.

GDB will only use this packet if the stub reports the ‘binary-upload’ feature is supported in its ‘qSupported’ reply (see [qSupported], page 822).

Reply:

‘b XX...’ Memory contents as binary data (see [Binary Data], page 795). The reply may contain fewer addressable memory units than requested if the server was reading from a trace frame memory and was able to read only part of the region of memory.

‘E NN’ for an error

`'X addr,length:XX...'`

Write data to memory, where the data is transmitted in binary. Memory is specified by its address *addr* and number of addressable memory units *length* (see [addressable memory unit], page 156); `'XX...'` is binary data (see [Binary Data], page 795).

Reply:

`'OK'` for success

`'z type,addr,kind'`

`'Z type,addr,kind'`

Insert ('Z') or remove ('z') a *type* breakpoint or watchpoint starting at address *address* of kind *kind*.

Each breakpoint and watchpoint packet *type* is documented separately.

Implementation notes: A remote target shall return an empty string for an unrecognized breakpoint or watchpoint packet type. A remote target shall support either both or neither of a given `'Ztype...'` and `'ztype...'` packet pair. To avoid potential problems with duplicate packets, the operations should be implemented in an idempotent way.

`'z0,addr,kind'`

`'Z0,addr,kind[;cond_list...][;cmds:persist,cmd_list...]`

Insert ('Z0') or remove ('z0') a software breakpoint at address *addr* of type *kind*.

A software breakpoint is implemented by replacing the instruction at *addr* with a software breakpoint or trap instruction. The *kind* is target-specific and typically indicates the size of the breakpoint in bytes that should be inserted. E.g., the ARM and MIPS can insert either a 2 or 4 byte breakpoint. Some architectures have additional meanings for *kind* (see Section E.6 [Architecture-Specific Protocol Details], page 839); if no architecture-specific value is being used, it should be '0'. *kind* is hex-encoded. *cond_list* is an optional list of conditional expressions in bytecode form that should be evaluated on the target's side. These are the conditions that should be taken into consideration when deciding if the breakpoint trigger should be reported back to GDB.

See also the `'swbreak'` stop reason (see [swbreak stop reason], page 809) for how to best report a software breakpoint event to GDB.

The *cond_list* parameter is comprised of a series of expressions, concatenated without separators. Each expression has the following form:

`'X len,expr'`

len is the length of the bytecode expression and *expr* is the actual conditional expression in bytecode form.

The optional *cmd_list* parameter introduces commands that may be run on the target, rather than being reported back to GDB. The parameter starts with a numeric flag *persist*; if the flag is nonzero, then the breakpoint may remain active and the commands continue to be run even when GDB disconnects from the target. Following this flag is a series of expressions concatenated with no separators. Each expression has the following form:

`'X len,expr'`

len is the length of the bytecode expression and *expr* is the actual commands expression in bytecode form.

Implementation note: It is possible for a target to copy or move code that contains software breakpoints (e.g., when implementing overlays). The behavior of this packet, in the presence of such a target, is not defined.

Reply:

`'OK'` success

`'z1,addr,kind'`

`'Z1,addr,kind[;cond_list...][;cmds:persist,cmd_list...]`

Insert (`'Z1'`) or remove (`'z1'`) a hardware breakpoint at address *addr*.

A hardware breakpoint is implemented using a mechanism that is not dependent on being able to modify the target's memory. The *kind*, *cond_list*, and *cmd_list* arguments have the same meaning as in `'Z0'` packets.

Implementation note: A hardware breakpoint is not affected by code movement.

Reply:

`'OK'` success

`'z2,addr,kind'`

`'Z2,addr,kind'`

Insert (`'Z2'`) or remove (`'z2'`) a write watchpoint at *addr*. The number of bytes to watch is specified by *kind*.

Reply:

`'OK'` success

`'z3,addr,kind'`

`'Z3,addr,kind'`

Insert (`'Z3'`) or remove (`'z3'`) a read watchpoint at *addr*. The number of bytes to watch is specified by *kind*.

Reply:

`'OK'` success

`'z4,addr,kind'`

`'Z4,addr,kind'`

Insert (`'Z4'`) or remove (`'z4'`) an access watchpoint at *addr*. The number of bytes to watch is specified by *kind*.

Reply:

`'OK'` success

E.4 Stop Reply Packets

The `'C'`, `'c'`, `'S'`, `'s'`, `'vCont'`, `'vAttach'`, `'vRun'`, `'vStopped'`, and `'?'` packets can receive any of the below as a reply. Except for `'?'` and `'vStopped'`, that reply is only returned when the target halts. In the below the exact meaning of *signal number* is defined by the header `include/gdb/signals.h` in the GDB source code.

In non-stop mode, the server will simply reply ‘OK’ to commands such as ‘vCont’; any stop will be the subject of a future notification. See Section E.11 [Remote Non-Stop], page 852.

As in the description of request packets, we include spaces in the reply templates for clarity; these are not part of the reply packet’s syntax. No GDB stop reply packet uses spaces to separate its components.

‘S AA’ The program received signal number AA (a two-digit hexadecimal number). This is equivalent to a ‘T’ response with no *n:r* pairs.

‘T AA n1:r1;n2:r2;...’

The program received signal number AA (a two-digit hexadecimal number). This is equivalent to an ‘S’ response, except that the ‘*n:r*’ pairs can carry values of important registers and other information directly in the stop reply packet, reducing round-trip latency. Single-step and breakpoint traps are reported this way. Each ‘*n:r*’ pair is interpreted as follows:

- If *n* is a hexadecimal number, it is a register number, and the corresponding *r* gives that register’s value. The data *r* is a series of bytes in target byte order, with each byte given by a two-digit hex number.
- If *n* is ‘thread’, then *r* is the thread ID of the stopped thread, as specified in [thread-id syntax], page 797.
- If *n* is ‘core’, then *r* is the hexadecimal number of the core on which the stop event was detected.
- If *n* is a recognized *stop reason*, it describes a more specific event that stopped the target. The currently defined stop reasons are listed below. The *aa* should be ‘05’, the trap signal. At most one stop reason should be present.
- Otherwise, GDB should ignore this ‘*n:r*’ pair and go on to the next; this allows us to extend the protocol in the future.

The currently defined stop reasons are:

‘watch’

‘rwatch’

‘awatch’ The packet indicates a watchpoint hit, and *r* is the data address, in hex.

Some targets, for example AArch64, are unable to accurately report the address which triggered a watchpoint trap. As a consequence, multiple watched addresses could explain a single watchpoint trap.

If GDB sent the ‘multi-wp-addr’ feature flag in its ‘qSupported’ packet (see [multi-wp-addr feature], page 824), then multiple instances of these stop reasons can appear in a single ‘T’ stop reply packet. GDB will select between the multiple reported watchpoint addresses when displaying the stop to the user.

If the ‘multi-wp-addr’ was not sent by GDB, then GDB only expects one watchpoint related stop address in a single ‘T’ packet. The server must select the most likely watchpoint address.

- ‘syscall_entry’**
‘syscall_return’ The packet indicates a syscall entry or return, and *r* is the syscall number, in hex.
- ‘library’** The packet indicates that the loaded libraries have changed. GDB should use **‘qXfer:libraries:read’** to fetch a new list of loaded libraries. The *r* part is ignored.
- ‘replaylog’** The packet indicates that the target cannot continue replaying logged execution events, because it has reached the end (or the beginning when executing backward) of the log. The value of *r* will be either **‘begin’** or **‘end’**. See Chapter 6 [Reverse Execution], page 103, for more information.
- ‘swbreak’** The packet indicates a software breakpoint instruction was executed, irrespective of whether it was GDB that planted the breakpoint or the breakpoint is hardcoded in the program. The *r* part must be left empty.
- On some architectures, such as x86, at the architecture level, when a breakpoint instruction executes the program counter points at the breakpoint address plus an offset. On such targets, the stub is responsible for adjusting the PC to point back at the breakpoint address.
- This packet should not be sent by default; older GDB versions did not support it. GDB requests it, by supplying an appropriate **‘qSupported’** feature (see [qSupported], page 822). The remote stub must also supply the appropriate **‘qSupported’** feature indicating support.
- This packet is required for correct non-stop mode operation.
- ‘hwbreak’** The packet indicates the target stopped for a hardware breakpoint. The *r* part must be left empty.
- The same remarks about **‘qSupported’** and non-stop mode above apply.
- ‘fork’** The packet indicates that **fork** was called, and *r* is the thread ID of the new child process, as specified in [thread-id syntax], page 797. This packet is only applicable to targets that support fork events.
- This packet should not be sent by default; older GDB versions did not support it. GDB requests it, by supplying an appropriate **‘qSupported’** feature (see [qSupported], page 822). The remote stub must also supply the appropriate **‘qSupported’** feature indicating support.
- ‘vfork’** The packet indicates that **vfork** was called, and *r* is the thread ID of the new child process, as specified in [thread-id syntax], page 797. This packet is only applicable to targets that support vfork events.

This packet should not be sent by default; older GDB versions did not support it. GDB requests it, by supplying an appropriate ‘qSupported’ feature (see [qSupported], page 822). The remote stub must also supply the appropriate ‘qSupported’ feature indicating support.

‘vforkdone’

The packet indicates that a child process created by a vfork has either called `exec` or terminated, so that the address spaces of the parent and child process are no longer shared. The *r* part is ignored. This packet is only applicable to targets that support vforkdone events.

This packet should not be sent by default; older GDB versions did not support it. GDB requests it, by supplying an appropriate ‘qSupported’ feature (see [qSupported], page 822). The remote stub must also supply the appropriate ‘qSupported’ feature indicating support.

‘exec’

The packet indicates that `execve` was called, and *r* is the absolute pathname of the file that was executed, in hex. This packet is only applicable to targets that support exec events.

This packet should not be sent by default; older GDB versions did not support it. GDB requests it, by supplying an appropriate ‘qSupported’ feature (see [qSupported], page 822). The remote stub must also supply the appropriate ‘qSupported’ feature indicating support.

‘clone’

The packet indicates that `clone` was called, and *r* is the thread ID of the new child thread, as specified in [thread-id syntax], page 797. This packet is only applicable to targets that support clone events.

This packet should not be sent by default; GDB requests it with the [QThreadOptions], page 820, packet.

‘create’

The packet indicates that the thread was just created. The new thread is stopped until GDB sets it running with a resumption packet (see [vCont packet], page 802). This packet should not be sent by default; GDB requests it with the [QThreadEvents], page 820, packet. See also the ‘w’ (see [thread exit event], page 811) remote reply below. The *r* part is ignored.

‘W AA’

‘W AA ; process:pid’

The process exited, and AA is the exit status. This is only applicable to certain targets.

The second form of the response, including the process ID of the exited process, can be used only when GDB has reported support for multiprocess protocol extensions; see [multiprocess extensions], page 829. Both AA and *pid* are formatted as big-endian hex strings.

‘X AA’

‘X AA ; process:pid’

The process terminated with signal AA.

The second form of the response, including the process ID of the terminated process, can be used only when GDB has reported support for multiprocess protocol extensions; see [multiprocess extensions], page 829. Both AA and pid are formatted as big-endian hex strings.

‘w AA ; tid’

The thread exited, and AA is the exit status. This response should not be sent by default; GDB requests it with either the [QThreadEvents], page 820, or [QThreadOptions], page 820, packets. See also [thread create event], page 810, above. AA is formatted as a big-endian hex string.

‘N’

There are no resumed threads left in the target. In other words, even though the process is alive, the last resumed thread has exited. For example, say the target process has two threads: thread 1 and thread 2. The client leaves thread 1 stopped, and resumes thread 2, which subsequently exits. At this point, even though the process is still alive, and thus no ‘W’ stop reply is sent, no thread is actually executing either. The ‘N’ stop reply thus informs the client that it can stop waiting for stop replies. This packet should not be sent by default; older GDB versions did not support it. GDB requests it, by supplying an appropriate ‘qSupported’ feature (see [qSupported], page 822). The remote stub must also supply the appropriate ‘qSupported’ feature indicating support.

‘O XX...’ ‘XX...’ is hex encoding of ASCII data, to be written as the program’s console output. This can happen at any time while the program is running and the debugger should continue to wait for ‘W’, ‘T’, etc. This reply is not permitted in non-stop mode.

‘F call-id,parameter...’

call-id is the identifier which says which host system call should be called. This is just the name of the function. Translation into the correct system call is only applicable as it’s defined in GDB. See Section E.14 [File-I/O Remote Protocol Extension], page 854, for a list of implemented system calls.

‘parameter...’ is a list of parameters as defined for this very system call.

The target replies with this packet when it expects GDB to call a host system call on behalf of the target. GDB replies with an appropriate ‘F’ packet and keeps up waiting for the next reply packet from the target. The latest ‘C’, ‘c’, ‘S’ or ‘s’ action is expected to be continued. See Section E.14 [File-I/O Remote Protocol Extension], page 854, for more details.

E.5 General Query Packets

Packets starting with ‘q’ are *general query packets*; packets starting with ‘Q’ are *general set packets*. General query and set packets are a semi-unified form for retrieving and sending information to and from the stub.

The initial letter of a query or set packet is followed by a name indicating what sort of thing the packet applies to. For example, GDB may use a ‘qSymbol’ packet to exchange symbol definitions with the stub. These packet names follow some conventions:

- The name must not contain commas, colons or semicolons.
- Most GDB query and set packets have a leading upper case letter.
- The names of custom vendor packets should use a company prefix, in lower case, followed by a period. For example, packets designed at the Acme Corporation might begin with ‘qacme.foo’ (for querying foos) or ‘Qacme.bar’ (for setting bars).

The name of a query or set packet should be separated from any parameters by a ‘:’; the parameters themselves should be separated by ‘,’ or ‘;’. Stubs must be careful to match the full packet name, and check for a separator or the end of the packet, in case two packet names share a common prefix. New packets should not begin with ‘qC’, ‘qP’, or ‘qL’¹.

Like the descriptions of the other packets, each description here has a template showing the packet’s overall syntax, followed by an explanation of the packet’s meaning. We include spaces in some of the templates for clarity; these are not part of the packet’s syntax. No GDB packet uses spaces to separate its components.

Here are the currently defined query and set packets:

‘QAgent:1’

‘QAgent:0’

Turn on or off the agent as a helper to perform some debugging operations delegated from GDB (see [Control Agent], page 725).

‘QAllow:op:val...’

Specify which operations GDB expects to request of the target, as a semicolon-separated list of operation name and value pairs. Possible values for *op* include ‘WriteReg’, ‘WriteMem’, ‘InsertBreak’, ‘InsertTrace’, ‘InsertFastTrace’, and ‘Stop’. *val* is either 0, indicating that GDB will not request the operation, or 1, indicating that it may. (The target can then use this to set up its own internals optimally, for instance if the debugger never expects to insert breakpoints, it may not need to install its own trap handler.)

‘qC’

Return the current thread ID.

Reply:

‘QC *thread-id*’

Where *thread-id* is a thread ID as documented in [thread-id syntax], page 797.

‘(anything else)’

Any other reply implies the old thread ID.

‘qCRC:addr,length’

Compute the CRC checksum of a block of memory using CRC-32 defined in IEEE 802.3. The CRC is computed byte at a time, taking the most significant

¹ The ‘qP’ and ‘qL’ packets predate these conventions, and have arguments without any terminator for the packet name; we suspect they are in widespread use in places that are difficult to upgrade. The ‘qC’ packet has no arguments, but some existing stubs (e.g. RedBoot) are known to not check for the end of the packet.

bit of each byte first. The initial pattern code `0xffffffff` is used to ensure leading zeros affect the CRC.

Note: This is the same CRC used in validating separate debug files (see Section 18.3 [Debugging Information in Separate Files], page 303). However the algorithm is slightly different. When validating separate debug files, the CRC is computed taking the *least* significant bit of each byte first, and the final result is inverted to detect trailing zeros.

Reply:

`'C crc32'` The specified memory region's checksum is `crc32`.

`'QDisableRandomization:value'`

Some target operating systems will randomize the virtual address space of the inferior process as a security feature, but provide a feature to disable such randomization, e.g. to allow for a more deterministic debugging experience. On such systems, this packet with a *value* of 1 directs the target to disable address space randomization for processes subsequently started via `'vRun'` packets, while a packet with a *value* of 0 tells the target to enable address space randomization. This packet is only available in extended mode (see [extended mode], page 797).

Reply:

`'OK'` The request succeeded.

This packet is not probed by default; the remote stub must request it, by supplying an appropriate `'qSupported'` response (see [qSupported], page 822). This should only be done on targets that actually support disabling address space randomization.

`'QStartupWithShell:value'`

On UNIX-like targets, it is possible to start the inferior using a shell program. This is the default behavior on both GDB and `gdbserver` (see [set startup-with-shell], page 36). This packet is used to inform `gdbserver` whether it should start the inferior using a shell or not.

If *value* is `'0'`, `gdbserver` will not use a shell to start the inferior. If *value* is `'1'`, `gdbserver` will use a shell to start the inferior. All other values are considered an error.

This packet is only available in extended mode (see [extended mode], page 797).

Reply:

`'OK'` The request succeeded.

This packet is not probed by default; the remote stub must request it, by supplying an appropriate `'qSupported'` response (see [qSupported], page 822). This should only be done on targets that actually support starting the inferior using a shell.

Use of this packet is controlled by the `set startup-with-shell` command; see [set startup-with-shell], page 36.

`'QEnvironmentHexEncoded:hex-value'`

On UNIX-like targets, it is possible to set environment variables that will be passed to the inferior during the startup process. This packet is used to inform

gdbserver of an environment variable that has been defined by the user on GDB (see [set environment], page 39).

The packet is composed by *hex-value*, an hex encoded representation of the *name=value* format representing an environment variable. The name of the environment variable is represented by *name*, and the value to be assigned to the environment variable is represented by *value*. If the variable has no value (i.e., the value is **null**), then *value* will not be present.

This packet is only available in extended mode (see [extended mode], page 797).

Reply:

‘OK’ The request succeeded.

This packet is not probed by default; the remote stub must request it, by supplying an appropriate ‘**qSupported**’ response (see [qSupported], page 822). This should only be done on targets that actually support passing environment variables to the starting inferior.

This packet is related to the **set environment** command; see [set environment], page 39.

‘QEnvironmentUnset:hex-value’

On UNIX-like targets, it is possible to unset environment variables before starting the inferior in the remote target. This packet is used to inform **gdbserver** of an environment variable that has been unset by the user on GDB (see [unset environment], page 39).

The packet is composed by *hex-value*, an hex encoded representation of the name of the environment variable to be unset.

This packet is only available in extended mode (see [extended mode], page 797).

Reply:

‘OK’ The request succeeded.

This packet is not probed by default; the remote stub must request it, by supplying an appropriate ‘**qSupported**’ response (see [qSupported], page 822). This should only be done on targets that actually support passing environment variables to the starting inferior.

This packet is related to the **unset environment** command; see [unset environment], page 39.

‘QEnvironmentReset’

On UNIX-like targets, this packet is used to reset the state of environment variables in the remote target before starting the inferior. In this context, reset means unsetting all environment variables that were previously set by the user (i.e., were not initially present in the environment). It is sent to **gdbserver** before the ‘**QEnvironmentHexEncoded**’ (see [QEnvironmentHexEncoded], page 813) and the ‘**QEnvironmentUnset**’ (see [QEnvironmentUnset], page 814) packets.

This packet is only available in extended mode (see [extended mode], page 797).

Reply:

‘OK’ The request succeeded.

This packet is not probed by default; the remote stub must request it, by supplying an appropriate ‘**qSupported**’ response (see [qSupported], page 822). This should only be done on targets that actually support passing environment variables to the starting inferior.

‘**QSetWorkingDir**:*[directory]*’

This packet is used to inform the remote server of the intended current working directory for programs that are going to be executed.

The packet is composed by *directory*, an hex encoded representation of the directory that the remote inferior will use as its current working directory. If *directory* is an empty string, the remote server should reset the inferior’s current working directory to its original, empty value.

This packet is only available in extended mode (see [extended mode], page 797).

Reply:

‘OK’ The request succeeded.

‘**qfThreadInfo**’

‘**qsThreadInfo**’

Obtain a list of all active thread IDs from the target (OS). Since there may be too many active threads to fit into one reply packet, this query works iteratively: it may require more than one query/reply sequence to obtain the entire list of threads. The first query of the sequence will be the ‘**qfThreadInfo**’ query; subsequent queries in the sequence will be the ‘**qsThreadInfo**’ query.

NOTE: This packet replaces the ‘**qL**’ query (see below).

Reply:

‘**m thread-id**’

A single thread ID

‘**m thread-id,thread-id...**’

a comma-separated list of thread IDs

‘**l**’

(lower case letter ‘L’) denotes end of list.

In response to each query, the target will reply with a list of one or more thread IDs, separated by commas. GDB will respond to each reply with a request for more thread ids (using the ‘**qs**’ form of the query), until the target responds with ‘**l**’ (lower-case ell, for *last*). Refer to [thread-id syntax], page 797, for the format of the *thread-id* fields.

*Note: GDB will send the **qfThreadInfo** query during the initial connection with the remote target, and the very first thread ID mentioned in the reply will be stopped by GDB in a subsequent message. Therefore, the stub should ensure that the first thread ID in the **qfThreadInfo** reply is suitable for being stopped by GDB.*

‘**qGetTLSAddr**:*thread-id,offset,lm*’

Fetch the address associated with thread local storage specified by *thread-id*, *offset*, and *lm*.

thread-id is the thread ID associated with the thread for which to fetch the TLS address. See [thread-id syntax], page 797.

offset is the (big endian, hex encoded) offset associated with the thread local variable. (This offset is obtained from the debug information associated with the variable.)

lm is the (big endian, hex encoded) OS/ABI-specific encoding of the load module associated with the thread local storage. For example, a GNU/Linux system will pass the link map address of the shared object associated with the thread local storage under consideration. Other operating environments may choose to represent the load module differently, so the precise meaning of this parameter will vary.

Reply:

‘XX...’ Hex encoded (big endian) bytes representing the address of the thread local storage requested.

‘qGetTIBAddr:thread-id’

Fetch address of the Windows OS specific Thread Information Block.

thread-id is the thread ID associated with the thread.

Reply:

‘XX...’ Hex encoded (big endian) bytes representing the linear address of the thread information block.

‘qL startflag threadcount nextthread’

Obtain thread information from RTOS. Where: *startflag* (one hex digit) is one to indicate the first query and zero to indicate a subsequent query; *threadcount* (two hex digits) is the maximum number of threads the response packet can contain; and *nextthread* (eight hex digits), for subsequent queries (*startflag* is zero), is returned in the response as *argthread*.

Don’t use this packet; use the ‘qfThreadInfo’ query instead (see above).

Reply:

‘qM count done argthread thread...’

Where: *count* (two hex digits) is the number of threads being returned; *done* (one hex digit) is zero to indicate more threads and one indicates no further threads; *argthreadid* (eight hex digits) is *nextthread* from the request packet; *thread...* is a sequence of thread IDs, *threadid* (eight hex digits), from the target. See `remote.c:parse_threadlist_response()`.

‘qMemTags:start address,length:type’

Fetch memory tags of type *type* from the address range [*start address*, *start address* + *length*). The target is responsible for calculating how many tags will be returned, as this is architecture-specific.

start address is the starting address of the memory range.

length is the length, in bytes, of the memory range.

type is the type of tag the request wants to fetch. The type is a signed integer.

GDB will only send this packet if the stub has advertised support for memory tagging via ‘qSupported’.

Reply:

‘*mxx...*’ Hex encoded sequence of uninterpreted bytes, *xx...*, representing the tags found in the requested memory range.

‘qIsAddressTagged:*address*’

Check if address *address* is in a memory tagged region; if it is, it’s said to be *tagged*. The target is responsible for checking it, as this is architecture-specific. *address* is the address to be checked.

Reply:

Replies to this packet should all be in two hex digit format, as follows:

‘‘01’’ Address *address* is tagged.

‘‘00’’ Address *address* is not tagged.

‘QMemTags:*start address,length:type:tag bytes*’

Store memory tags of type *type* to the address range [*start address*, *start address* + *length*). The target is responsible for interpreting the type, the tag bytes and modifying the memory tag granules accordingly, given this is architecture-specific.

The interpretation of how many tags (*nt*) should be written to how many memory tag granules (*ng*) is also architecture-specific. The behavior is implementation-specific, but the following is suggested.

If the number of memory tags, *nt*, is greater than or equal to the number of memory tag granules, *ng*, only *ng* tags will be stored.

If *nt* is less than *ng*, the behavior is that of a fill operation, and the tag bytes will be used as a pattern that will get repeated until *ng* tags are stored.

start address is the starting address of the memory range. The address does not have any restriction on alignment or size.

length is the length, in bytes, of the memory range.

type is the type of tag the request wants to fetch. The type is a signed integer.

tag bytes is a sequence of hex encoded uninterpreted bytes which will be interpreted by the target. Each pair of hex digits is interpreted as a single byte.

GDB will only send this packet if the stub has advertised support for memory tagging via ‘qSupported’.

Reply:

‘OK’ The request was successful and the memory tag granules were modified accordingly.

‘qOffsets’

Get section offsets that the target used when relocating the downloaded image.

Reply:

‘Text=*xxx*;Data=*yyy*[:Bss=*zzz*]’

Relocate the **Text** section by *xxx* from its original address. Relocate the **Data** section by *yyy* from its original address. If the object file

format provides segment information (e.g. ELF ‘PT_LOAD’ program headers), GDB will relocate entire segments by the supplied offsets.

Note: while a Bss offset may be included in the response, GDB ignores this and instead applies the Data offset to the Bss section.

‘TextSeg=xxx[;DataSeg=yyy]’

Relocate the first segment of the object file, which conventionally contains program code, to a starting address of xxx. If ‘DataSeg’ is specified, relocate the second segment, which conventionally contains modifiable data, to a starting address of yyy. GDB will report an error if the object file does not contain segment information, or does not contain at least as many segments as mentioned in the reply. Extra segments are kept at fixed offsets relative to the last relocated segment.

‘qP mode thread-id’

Returns information on *thread-id*. Where: *mode* is a hex encoded 32 bit mode; *thread-id* is a thread ID (see [thread-id syntax], page 797).

Don’t use this packet; use the ‘qThreadExtraInfo’ query instead (see below).

Reply: see `remote.c:remote_unpack_thread_info_response()`.

‘QNonStop:1’

‘QNonStop:0’

Enter non-stop (‘QNonStop:1’) or all-stop (‘QNonStop:0’) mode. See Section E.11 [Remote Non-Stop], page 852, for more information.

Reply:

‘OK’ The request succeeded.

This packet is not probed by default; the remote stub must request it, by supplying an appropriate ‘qSupported’ response (see [qSupported], page 822). Use of this packet is controlled by the `set non-stop` command; see Section 5.5.2 [Non-Stop Mode], page 97.

‘QCatchSyscalls:1 [;sysno]...’

‘QCatchSyscalls:0’

Enable (‘QCatchSyscalls:1’) or disable (‘QCatchSyscalls:0’) catching syscalls from the inferior process.

For ‘QCatchSyscalls:1’, each listed syscall *sysno* (encoded in hex) should be reported to GDB. If no syscall *sysno* is listed, every system call should be reported.

Note that if a syscall not in the list is reported, GDB will still filter the event according to its own list from all corresponding `catch syscall` commands. However, it is more efficient to only report the requested syscalls.

Multiple ‘QCatchSyscalls:1’ packets do not combine; any earlier ‘QCatchSyscalls:1’ list is completely replaced by the new list.

If the inferior process execs, the state of ‘QCatchSyscalls’ is kept for the new process too. On targets where exec may affect syscall numbers, for example

with `exec` between 32 and 64-bit processes, the client should send a new packet with the new syscall list.

Reply:

‘OK’ The request succeeded.

Use of this packet is controlled by the `set remote catch-syscalls` command (see Section 20.4 [Remote Configuration], page 328). This packet is not probed by default; the remote stub must request it, by supplying an appropriate ‘qSupported’ response (see [qSupported], page 822).

‘QPassSignals: *signal* [*;signal*]...’

Each listed *signal* should be passed directly to the inferior process. Signals are numbered identically to continue packets and stop replies (see Section E.4 [Stop Reply Packets], page 807). Each *signal* list item should be strictly greater than the previous item. These signals do not need to stop the inferior, or be reported to GDB. All other signals should be reported to GDB. Multiple ‘QPassSignals’ packets do not combine; any earlier ‘QPassSignals’ list is completely replaced by the new list. This packet improves performance when using ‘handle *signal* nostop noprint pass’.

Reply:

‘OK’ The request succeeded.

Use of this packet is controlled by the `set remote pass-signals` command (see Section 20.4 [Remote Configuration], page 328). This packet is not probed by default; the remote stub must request it, by supplying an appropriate ‘qSupported’ response (see [qSupported], page 822).

‘QProgramSignals: *signal* [*;signal*]...’

Each listed *signal* may be delivered to the inferior process. Others should be silently discarded.

In some cases, the remote stub may need to decide whether to deliver a signal to the program or not without GDB involvement. One example of that is while detaching — the program’s threads may have stopped for signals that haven’t yet had a chance of being reported to GDB, and so the remote stub can use the signal list specified by this packet to know whether to deliver or ignore those pending signals.

This does not influence whether to deliver a signal as requested by a resumption packet (see [vCont packet], page 802).

Signals are numbered identically to continue packets and stop replies (see Section E.4 [Stop Reply Packets], page 807). Each *signal* list item should be strictly greater than the previous item. Multiple ‘QProgramSignals’ packets do not combine; any earlier ‘QProgramSignals’ list is completely replaced by the new list.

Reply:

‘OK’ The request succeeded.

Use of this packet is controlled by the `set remote program-signals` command (see Section 20.4 [Remote Configuration], page 328). This packet is not

probed by default; the remote stub must request it, by supplying an appropriate ‘qSupported’ response (see [qSupported], page 822).

‘QThreadEvents:1’

‘QThreadEvents:0’

Enable (‘QThreadEvents:1’) or disable (‘QThreadEvents:0’) reporting of thread create and exit events. See [thread create event], page 810, for the reply specifications. For example, this is used in non-stop mode when GDB stops a set of threads and synchronously waits for the their corresponding stop replies. Without exit events, if one of the threads exits, GDB would hang forever not knowing that it should no longer expect a stop for that same thread. GDB does not enable this feature unless the stub reports that it supports it by including ‘QThreadEvents+’ in its ‘qSupported’ reply.

This packet always enables/disables event reporting for all threads of all processes under control of the remote stub. For per-thread control of optional event reporting, see the [QThreadOptions], page 820, packet.

Reply:

‘OK’ The request succeeded.

Use of this packet is controlled by the `set remote thread-events` command (see Section 20.4 [Remote Configuration], page 328).

‘QThreadOptions[:options[:thread-id]]...’

For each inferior thread, the last *options* in the list with a matching *thread-id* are applied. Any options previously set on a thread are discarded and replaced by the new options specified. Threads that do not match any *thread-id* retain their previously-set options. Thread IDs are specified using the syntax described in [thread-id syntax], page 797. If multiprocess extensions (see [multiprocess extensions], page 829) are supported, options can be specified to apply to all threads of a process by using the ‘ppid.-1’ form of *thread-id*. Options with no *thread-id* apply to all threads. Specifying no options value is an error. Zero is a valid value.

options is an hexadecimal integer specifying the enabled thread options, and is the bitwise OR of the following values. All values are given in hexadecimal representation.

GDB_THREAD_OPTION_CLONE (0x1)

Report thread clone events (see [thread clone event], page 810). This is only meaningful for targets that support clone events (e.g., GNU/Linux systems).

GDB_THREAD_OPTION_EXIT (0x2)

Report thread exit events (see [thread exit event], page 811).

For example, GDB enables the GDB_THREAD_OPTION_EXIT and GDB_THREAD_OPTION_CLONE options when single-stepping a thread past a breakpoint, for the following reasons:

- If the single-stepped thread exits (e.g., it executes a thread exit system call), enabling GDB_THREAD_OPTION_EXIT prevents GDB from waiting for-

ever, not knowing that it should no longer expect a stop for that same thread, and blocking other threads from progressing.

- If the single-stepped thread spawns a new clone child (i.e., it executes a clone system call), enabling `GDB_THREAD_OPTION_CLONE` halts the cloned thread before it executes any instructions, and thus prevents the following problematic situations:
 - If the breakpoint is stepped-over in-line, the spawned thread would incorrectly run free while the breakpoint being stepped over is not inserted, and thus the cloned thread may potentially run past the breakpoint without stopping for it;
 - If displaced (out-of-line) stepping is used, the cloned thread starts running at the out-of-line PC, leading to undefined behavior, usually crashing or corrupting data.

New threads start with thread options cleared.

GDB does not enable this feature unless the stub reports that it supports it by including `'QThreadOptions=supported_options'` in its `'qSupported'` reply.

Reply:

`'OK'` The request succeeded.

Use of this packet is controlled by the `set remote thread-options` command (see Section 20.4 [Remote Configuration], page 328).

`'qRcmd,command'`

command (hex encoded) is passed to the local interpreter for execution. Invalid commands should be reported using the output string. Before the final result packet, the target may also respond with a number of intermediate `'Ooutput'` console output packets. *Implementors should note that providing access to a stubs's interpreter may have security implications.*

Reply:

`'OK'` A command response with no output.

`'OUTPUT'` A command response with the hex encoded output string *OUTPUT*.

Unlike most packets, this packet does not support `'E.errtext'`-style textual error replies (see [textual error reply], page 797) by default. Stubs should be careful to only send such a reply if GDB reported support for it with the `error-message` feature (see [error-message], page 823).

(Note that the `qRcmd` packet's name is separated from the command by a `','`, not a `':'`, contrary to the naming conventions above. Please don't use this packet as a model for new packets.)

`'qSearch:memory:address:length;search-pattern'`

Search *length* bytes at *address* for *search-pattern*. Both *address* and *length* are encoded in hex; *search-pattern* is a sequence of binary-encoded bytes (see [Binary Data], page 795).

Reply:

‘0’ The pattern was not found.

‘1, *address*’
 The pattern was found at *address*.

‘QStartNoAckMode’

Request that the remote stub disable the normal ‘+’/‘-’ protocol acknowledgments (see Section E.12 [Packet Acknowledgment], page 853).

Reply:

‘OK’ The stub has switched to no-acknowledgment mode. GDB acknowledges this response, but neither the stub nor GDB shall send or expect further ‘+’/‘-’ acknowledgments in the current connection.

‘qSupported [;*gdbfeature* [;*gdbfeature*]...]’

Tell the remote stub about features supported by GDB, and query the stub for features it supports. This packet allows GDB and the remote stub to take advantage of each others’ features. ‘qSupported’ also consolidates multiple feature probes at startup, to improve GDB performance—a single larger packet performs better than multiple smaller probe packets on high-latency links. Some features may enable behavior which must not be on by default, e.g. because it would confuse older clients or stubs. Other features may describe packets which could be automatically probed for, but are not. These features must be reported before GDB will use them. This “default unsupported” behavior is not appropriate for all packets, but it helps to keep the initial connection time under control with new versions of GDB which support increasing numbers of packets.

Reply:

‘*stubfeature* [;*stubfeature*]...’

The stub supports or does not support each returned *stubfeature*, depending on the form of each *stubfeature* (see below for the possible forms).

The allowed forms for each feature (either a *gdbfeature* in the ‘qSupported’ packet, or a *stubfeature* in the response) are:

‘*name=value*’

The remote protocol feature *name* is supported, and associated with the specified *value*. The format of *value* depends on the feature, but it must not include a semicolon.

‘*name+*’ The remote protocol feature *name* is supported, and does not need an associated value.

‘*name-*’ The remote protocol feature *name* is not supported.

‘*name?*’ The remote protocol feature *name* may be supported, and GDB should auto-detect support in some other way when it is needed. This form will not be used for *gdbfeature* notifications, but may be used for *stubfeature* responses.

Whenever the stub receives a ‘qSupported’ request, the supplied set of GDB features should override any previous request. This allows GDB to put the stub in a known state, even if the stub had previously been communicating with a different version of GDB.

The following values of *gdbfeature* (for the packet sent by GDB) are defined:

‘multiprocess’

This feature indicates whether GDB supports multiprocess extensions to the remote protocol. GDB does not use such extensions unless the stub also reports that it supports them by including ‘multiprocess+’ in its ‘qSupported’ reply. See [multiprocess extensions], page 829, for details.

‘xmlRegisters’

This feature indicates that GDB supports the XML target description. If the stub sees ‘xmlRegisters=’ with target specific strings separated by a comma, it will report register description.

‘qRelocInsn’

This feature indicates whether GDB supports the ‘qRelocInsn’ packet (see Section E.7 [Relocate instruction reply packet], page 840).

‘swbreak’ This feature indicates whether GDB supports the swbreak stop reason in stop replies. See [swbreak stop reason], page 809, for details.

‘hwbreak’ This feature indicates whether GDB supports the hwbreak stop reason in stop replies. See [swbreak stop reason], page 809, for details.

‘fork-events’

This feature indicates whether GDB supports fork event extensions to the remote protocol. GDB does not use such extensions unless the stub also reports that it supports them by including ‘fork-events+’ in its ‘qSupported’ reply.

‘vfork-events’

This feature indicates whether GDB supports vfork event extensions to the remote protocol. GDB does not use such extensions unless the stub also reports that it supports them by including ‘vfork-events+’ in its ‘qSupported’ reply.

‘exec-events’

This feature indicates whether GDB supports exec event extensions to the remote protocol. GDB does not use such extensions unless the stub also reports that it supports them by including ‘exec-events+’ in its ‘qSupported’ reply.

‘vContSupported’

This feature indicates whether GDB wants to know the supported actions in the reply to ‘vCont?’ packet.

‘error-message’

This feature indicates whether GDB supports accepting a reply in ‘E.errtext’ format (See [textual error reply], page 797) from the ‘qRcmd’ and ‘m’ packets. These packets, historically, didn’t support ‘E.errtext’, and older versions of GDB didn’t expect to see a reply in this format.

New packets should be written to support ‘E.errtext’ regardless of this feature being true or not.

‘single-inf-arg’

This feature indicates that GDB would like to send the inferior arguments as a single string within the ‘vRun’ packet. GDB will not send the arguments as a single string unless the stub also reports that it supports this behaviour by including ‘single-inf-arg+’ in its ‘qSupported’ reply.

‘multi-wp-addr’

This features indicates that GDB supports receiving multiple watchpoint addresses in the ‘T’ stop reply packet (see Section E.4 [Stop Reply Packets], page 807).

Use of this feature is controlled by the `set remote multiple-watchpoint-addresses-packet` command (see Section 20.4 [Remote Configuration], page 328).

Stubs should ignore any unknown values for *gdbfeature*. Any GDB which sends a ‘qSupported’ packet supports receiving packets of unlimited length (earlier versions of GDB may reject overly long responses). Additional values for *gdbfeature* may be defined in the future to let the stub take advantage of new features in GDB, e.g. incompatible improvements in the remote protocol—the ‘multiprocess’ feature is an example of such a feature. The stub’s reply should be independent of the *gdbfeature* entries sent by GDB; first GDB describes all the features it supports, and then the stub replies with all the features it supports. Similarly, GDB will silently ignore unrecognized stub feature responses, as long as each response uses one of the standard forms.

Some features are flags. A stub which supports a flag feature should respond with a ‘+’ form response. Other features require values, and the stub should respond with an ‘=’ form response.

Each feature has a default value, which GDB will use if ‘qSupported’ is not available or if the feature is not mentioned in the ‘qSupported’ response. The default values are fixed; a stub is free to omit any feature responses that match the defaults.

Not all features can be probed, but for those which can, the probing mechanism is useful: in some cases, a stub’s internal architecture may not allow the protocol layer to know some information about the underlying target in advance. This is especially common in stubs which may be configured for multiple targets.

These are the currently defined stub features and their properties:

Feature Name	Value Required	Default	Probe Allowed
'PacketSize'	Yes	'_'	No
'qXfer:auxv:read'	No	'_'	Yes
'qXfer:btrace:read'	No	'_'	Yes
'qXfer:btrace-conf:read'	No	'_'	Yes
'qXfer:exec-file:read'	No	'_'	Yes
'qXfer:features:read'	No	'_'	Yes
'qXfer:libraries:read'	No	'_'	Yes
'qXfer:libraries-svr4:read'	No	'_'	Yes
'augmented-libraries-svr4:read'	No	'_'	No
'qXfer:memory-map:read'	No	'_'	Yes
'qXfer:sdata:read'	No	'_'	Yes
'qXfer:signinfo:read'	No	'_'	Yes
'qXfer:signinfo:write'	No	'_'	Yes
'qXfer:threads:read'	No	'_'	Yes
'qXfer:traceframe-info:read'	No	'_'	Yes
'qXfer:uib:read'	No	'_'	Yes
'qXfer:fdpic:read'	No	'_'	Yes
'Qbtrace:off'	Yes	'_'	Yes
'Qbtrace:bts'	Yes	'_'	Yes
'Qbtrace:pt'	Yes	'_'	Yes
'Qbtrace-conf:bts:size'	Yes	'_'	Yes
'Qbtrace-conf:pt:size'	Yes	'_'	Yes

'Qbtrace-conf:pt:ptwrite'	Yes	'_'	Yes
'Qbtrace-conf:pt:event-tracing'	Yes	'_'	Yes
'QNonStop'	No	'_'	Yes
'QCatchSyscalls'	No	'_'	Yes
'QPassSignals'	No	'_'	Yes
'QStartNoAckMode'	No	'_'	Yes
'multiprocess'	No	'_'	No
'ConditionalBreakpoints'	No	'_'	No
'ConditionalTracepoints'	No	'_'	No
'ReverseContinue'	No	'_'	No
'ReverseStep'	No	'_'	No
'TracepointSource'	No	'_'	No
'QAgent'	No	'_'	No
'QAllow'	No	'_'	No
'QDisableRandomization'	No	'_'	No
'EnableDisableTracepoints'	No	'_'	No
'QTBuffer:size'	No	'_'	No
'tracenz'	No	'_'	No
'BreakpointCommands'	No	'_'	No
'swbreak'	No	'_'	No
'hwbreak'	No	'_'	No
'fork-events'	No	'_'	No
'vfork-events'	No	'_'	No

<code>'exec-events'</code>	No	<code>'_'</code>	No
<code>'QThreadEvents'</code>	No	<code>'_'</code>	No
<code>'QThreadOptions'</code>	Yes	<code>'_'</code>	No
<code>'no-resumed'</code>	No	<code>'_'</code>	No
<code>'memory-tagging'</code>	No	<code>'_'</code>	No
<code>'error-message'</code>	No	<code>'+'</code>	No
<code>'binary-upload'</code>	No	<code>'_'</code>	No
<code>'single-inf-arg'</code>	No	<code>'_'</code>	No
<code>'multi-wp-addr'</code>	No	<code>'+'</code>	No

These are the currently defined stub features, in more detail:

`'PacketSize=bytes'`

The remote stub can accept packets up to at least *bytes* in length. GDB will send packets up to this size for bulk transfers, and will never send larger packets. This is a limit on the data characters in the packet, not including the frame and checksum. There is no trailing NUL byte in a remote protocol packet; if the stub stores packets in a NUL-terminated format, it should allow an extra byte in its buffer for the NUL. If this stub feature is not supported, GDB guesses based on the size of the `'g'` packet response.

`'qXfer:auxv:read'`

The remote stub understands the `'qXfer:auxv:read'` packet (see [qXfer auxiliary vector read], page 834).

`'qXfer:btrace:read'`

The remote stub understands the `'qXfer:btrace:read'` packet (see [qXfer btrace read], page 834).

`'qXfer:btrace-conf:read'`

The remote stub understands the `'qXfer:btrace-conf:read'` packet (see [qXfer btrace-conf read], page 834).

`'qXfer:exec-file:read'`

The remote stub understands the `'qXfer:exec-file:read'` packet (see [qXfer executable filename read], page 834).

`'qXfer:features:read'`

The remote stub understands the `'qXfer:features:read'` packet (see [qXfer target description read], page 835).

‘qXfer:libraries:read’

The remote stub understands the ‘qXfer:libraries:read’ packet (see [qXfer library list read], page 835).

‘qXfer:libraries-svr4:read’

The remote stub understands the ‘qXfer:libraries-svr4:read’ packet (see [qXfer svr4 library list read], page 835).

‘augmented-libraries-svr4-read’

The remote stub understands the augmented form of the ‘qXfer:libraries-svr4:read’ packet (see [qXfer svr4 library list read], page 835).

‘qXfer:memory-map:read’

The remote stub understands the ‘qXfer:memory-map:read’ packet (see [qXfer memory map read], page 836).

‘qXfer:sdata:read’

The remote stub understands the ‘qXfer:sdata:read’ packet (see [qXfer sdata read], page 836).

‘qXfer:signinfo:read’

The remote stub understands the ‘qXfer:signinfo:read’ packet (see [qXfer signinfo read], page 836).

‘qXfer:signinfo:write’

The remote stub understands the ‘qXfer:signinfo:write’ packet (see [qXfer signinfo write], page 837).

‘qXfer:threads:read’

The remote stub understands the ‘qXfer:threads:read’ packet (see [qXfer threads read], page 836).

‘qXfer:traceframe-info:read’

The remote stub understands the ‘qXfer:traceframe-info:read’ packet (see [qXfer traceframe info read], page 836).

‘qXfer:uib:read’

The remote stub understands the ‘qXfer:uib:read’ packet (see [qXfer unwind info block], page 837).

‘qXfer:fdpic:read’

The remote stub understands the ‘qXfer:fdpic:read’ packet (see [qXfer fdpic loadmap read], page 837).

‘QNonStop’

The remote stub understands the ‘QNonStop’ packet (see [QNonStop], page 818).

‘QCatchSyscalls’

The remote stub understands the ‘QCatchSyscalls’ packet (see [QCatchSyscalls], page 818).

‘QPassSignals’

The remote stub understands the ‘QPassSignals’ packet (see [QPassSignals], page 819).

‘QStartNoAckMode’

The remote stub understands the ‘QStartNoAckMode’ packet and prefers to operate in no-acknowledgment mode. See Section E.12 [Packet Acknowledgment], page 853.

‘multiprocess’

The remote stub understands the multiprocess extensions to the remote protocol syntax. The multiprocess extensions affect the syntax of thread IDs in both packets and replies (see [thread-id syntax], page 797), and add process IDs to the ‘D’ packet and ‘W’ and ‘X’ replies. Note that reporting this feature indicates support for the syntactic extensions only, not that the stub necessarily supports debugging of more than one process at a time. The stub must not use multiprocess extensions in packet replies unless GDB has also indicated it supports them in its ‘qSupported’ request.

‘qXfer:osdata:read’

The remote stub understands the ‘qXfer:osdata:read’ packet ((see [qXfer osdata read], page 837).

‘ConditionalBreakpoints’

The target accepts and implements evaluation of conditional expressions defined for breakpoints. The target will only report breakpoint triggers when such conditions are true (see Section 5.1.6 [Break Conditions], page 78).

‘ConditionalTracepoints’

The remote stub accepts and implements conditional expressions defined for tracepoints (see Section 13.1.4 [Tracepoint Conditions], page 209).

‘ReverseContinue’

The remote stub accepts and implements the reverse continue packet (see [bc], page 798).

‘ReverseStep’

The remote stub accepts and implements the reverse step packet (see [bs], page 798).

‘TracepointSource’

The remote stub understands the ‘QTDPsrc’ packet that supplies the source form of tracepoint definitions.

‘QAgent’ The remote stub understands the ‘QAgent’ packet.

‘QAllow’ The remote stub understands the ‘QAllow’ packet.

‘QDisableRandomization’

The remote stub understands the ‘QDisableRandomization’ packet.

‘StaticTracepoint’

The remote stub supports static tracepoints.

‘InstallInTrace’

The remote stub supports installing tracepoint in tracing.

‘EnableDisableTracepoints’

The remote stub supports the ‘QTEnable’ (see [QTEnable], page 843) and ‘QTDisable’ (see [QTDisable], page 843) packets that allow tracepoints to be enabled and disabled while a trace experiment is running.

‘QTBuffer:size’

The remote stub supports the ‘QTBuffer:size’ (see [QTBuffer-size], page 846) packet that allows to change the size of the trace buffer.

‘tracenz’ The remote stub supports the ‘tracenz’ bytecode for collecting strings. See Section F.2 [Bytecode Descriptions], page 873, for details about the bytecode.

‘BreakpointCommands’

The remote stub supports running a breakpoint’s command list itself, rather than reporting the hit to GDB.

‘Qbtrace:off’

The remote stub understands the ‘Qbtrace:off’ packet.

‘Qbtrace:bts’

The remote stub understands the ‘Qbtrace:bts’ packet.

‘Qbtrace:pt’

The remote stub understands the ‘Qbtrace:pt’ packet.

‘Qbtrace-conf:bts:size’

The remote stub understands the ‘Qbtrace-conf:bts:size’ packet.

‘Qbtrace-conf:pt:size’

The remote stub understands the ‘Qbtrace-conf:pt:size’ packet.

‘Qbtrace-conf:pt:ptwrite’

The remote stub understands the ‘Qbtrace-conf:pt:ptwrite’ packet.

‘Qbtrace-conf:pt:event-tracing’

The remote stub understands the ‘Qbtrace-conf:pt:event-tracing’ packet.

‘swbreak’ The remote stub reports the ‘swbreak’ stop reason for memory breakpoints.

‘hwbreak’ The remote stub reports the ‘hwbreak’ stop reason for hardware breakpoints.

‘fork-events’

The remote stub reports the ‘fork’ stop reason for fork events.

‘vfork-events’

The remote stub reports the ‘vfork’ stop reason for vfork events and vforkdone events.

‘exec-events’

The remote stub reports the ‘exec’ stop reason for exec events.

‘vContSupported’

The remote stub reports the supported actions in the reply to ‘vCont?’ packet.

‘QThreadEvents’

The remote stub understands the ‘QThreadEvents’ packet.

‘QThreadOptions=*supported_options*’

The remote stub understands the ‘QThreadOptions’ packet. *supported_options* indicates the set of thread options the remote stub supports. *supported_options* has the same format as the *options* parameter of the QThreadOptions packet, described at [QThreadOptions], page 820.

‘no-resumed’

The remote stub reports the ‘N’ stop reply.

‘memory-tagging’

The remote stub supports and implements the required memory tagging functionality and understands the ‘qMemTags’ (see [qMemTags], page 816) and ‘QMemTags’ (see [QMemTags], page 817) packets.

For AArch64 GNU/Linux systems, this feature can require access to the `/proc/pid/smmaps` file so memory mapping page flags can be inspected, if ‘qIsAddressTagged’ (see [qIsAddressTagged], page 817) packet is not supported by the stub. Access to the `/proc/pid/smmaps` file is done via ‘vFile’ requests.

‘error-message’

The remote stub supports replying with an error in a ‘E.errtext’ (See [textual error reply], page 797) format from the ‘m’ and ‘qRcmd’ packets. It is not usually necessary to send this feature back to GDB in the ‘qSupported’ reply, GDB will always support ‘E.errtext’ format replies if it sent the ‘error-message’ feature.

‘binary-upload’

The remote stub supports the ‘x’ packet (see [x packet], page 805).

‘single-inf-arg’

The remote stub would like to receive the inferior arguments as a single string within the ‘vRun’ packet. The stub should only send this feature if GDB sent ‘single-inf-arg+’ in the ‘qSupported’ packet.

`'multi-wp-addr'`

The remote stub supports sending multiple watchpoint addresses within 'T' stop reply packet. Stubs that don't support this feature don't need to tell GDB. Not supporting this feature just means sending back one watchpoint address instead of multiple, and GDB has always supported receiving a single watchpoint address.

`'qSymbol::'`

Notify the target that GDB is prepared to serve symbol lookup requests. Accept requests from the target for the values of symbols.

Reply:

`'OK'` The target does not need to look up any (more) symbols.

`'qSymbol:sym_name'`

The target requests the value of symbol *sym_name* (hex encoded). GDB may provide the value by using the `'qSymbol:sym_value:sym_name'` message, described below.

`'qSymbol:sym_value:sym_name'`

Set the value of *sym_name* to *sym_value*.

sym_name (hex encoded) is the name of a symbol whose value the target has previously requested.

sym_value (hex) is the value for symbol *sym_name*. If GDB cannot supply a value for *sym_name*, then this field will be empty.

Reply:

`'OK'` The target does not need to look up any (more) symbols.

`'qSymbol:sym_name'`

The target requests the value of a new symbol *sym_name* (hex encoded). GDB will continue to supply the values of symbols (if available), until the target ceases to request them.

`'qTBuffer'`

`'QTBuffer'`

`'QTDisconnected'`

`'QTDP'`

`'QTDPsrc'`

`'QTDV'`

`'qTfP'`

`'qTfV'`

`'QTFrame'`

`'qTMinFTPILen'`

See Section E.7 [Tracepoint Packets], page 840.

`'qThreadExtraInfo,thread-id'`

Obtain from the target OS a printable string description of thread attributes for the thread *thread-id*; see [thread-id syntax], page 797, for the forms of *thread-id*. This string may contain anything that the target OS thinks is interesting

for GDB to tell the user about the thread. The string is displayed in GDB's `info threads` display. Some examples of possible thread extra info strings are 'Runnable', or 'Blocked on Mutex'.

Reply:

'XX...' Where 'XX...' is a hex encoding of ASCII data, comprising the printable string containing the extra information about the thread's attributes.

(Note that the `qThreadExtraInfo` packet's name is separated from the command by a ',', not a ':', contrary to the naming conventions above. Please don't use this packet as a model for new packets.)

'QTNotes'
'qTP'
'QTSave'
'qTsP'
'qTsV'
'QTStart'
'QTStop'
'QTEnable'
'QTDisable'
'QTinit'
'QTro'
'qTStatus'
'qTV'
'qTfSTM'
'qTsSTM'
'qTSTMat'

See Section E.7 [Tracepoint Packets], page 840.

'qXfer:object:read:annex:offset,length'

Read uninterpreted bytes from the target's special data area identified by the keyword *object*. Request *length* bytes starting at *offset* bytes into the data. The content and encoding of *annex* is specific to *object*; it can supply additional details about what data to access.

Reply:

'm data' Data *data* (see [Binary Data], page 795) has been read from the target. There may be more data at a higher address (although it is permitted to return 'm' even for the last valid block of data, as long as at least one byte of data was read). It is possible for *data* to have fewer bytes than the *length* in the request.

'1 data' Data *data* (see [Binary Data], page 795) has been read from the target. There is no more data to be read. It is possible for *data* to have fewer bytes than the *length* in the request.

'1' The *offset* in the request is at the end of the data. There is no more data to be read.

Here are the specific requests of this form defined so far. All the ‘qXfer:object:read:...’ requests use the same reply formats, listed above.

‘qXfer:auxv:read::offset,length’

Access the target’s *auxiliary vector*. See Section 10.17 [OS Information], page 183. Note *annex* must be empty.

This packet is not probed by default; the remote stub must request it, by supplying an appropriate ‘qSupported’ response (see [qSupported], page 822).

‘qXfer:btrace:read:annex:offset,length’

Return a description of the current branch trace. See Section E.20 [Branch Trace Format], page 869. The annex part of the generic ‘qXfer’ packet may have one of the following values:

- | | |
|-------|--|
| all | Returns all available branch trace. |
| new | Returns all available branch trace if the branch trace changed since the last read request. |
| delta | Returns the new branch trace since the last read request. Adds a new block to the end of the trace that begins at zero and ends at the source location of the first branch in the trace buffer. This extra block is used to stitch traces together.

If the trace buffer overflowed, returns an error indicating the overflow. |

This packet is not probed by default; the remote stub must request it by supplying an appropriate ‘qSupported’ response (see [qSupported], page 822).

‘qXfer:btrace-conf:read::offset,length’

Return a description of the current branch trace configuration. See Section E.21 [Branch Trace Configuration Format], page 870.

This packet is not probed by default; the remote stub must request it by supplying an appropriate ‘qSupported’ response (see [qSupported], page 822).

‘qXfer:exec-file:read:annex:offset,length’

Return the full absolute name of the file that was executed to create a process running on the remote system. The annex specifies the numeric process ID of the process to query, encoded as a hexadecimal number. If the annex part is empty the remote stub should return the filename corresponding to the currently executing process.

This packet is not probed by default; the remote stub must request it, by supplying an appropriate ‘qSupported’ response (see [qSupported], page 822).

‘qXfer:features:read:annex:offset,length’

Access the *target description*. See Appendix G [Target Descriptions], page 883. The annex specifies which XML document to access. The main description is always loaded from the ‘target.xml’ annex.

This packet is not probed by default; the remote stub must request it, by supplying an appropriate ‘qSupported’ response (see [qSupported], page 822).

‘qXfer:libraries:read:annex:offset,length’

Access the target’s list of loaded libraries. See Section E.15 [Library List Format], page 865. The annex part of the generic ‘qXfer’ packet must be empty (see [qXfer read], page 833).

Targets which maintain a list of libraries in the program’s memory do not need to implement this packet; it is designed for platforms where the operating system manages the list of loaded libraries.

This packet is not probed by default; the remote stub must request it, by supplying an appropriate ‘qSupported’ response (see [qSupported], page 822).

‘qXfer:libraries-svr4:read:annex:offset,length’

Access the target’s list of loaded libraries when the target is an SVR4 platform. See Section E.16 [Library List Format for SVR4 Targets], page 866. The annex part of the generic ‘qXfer’ packet must be empty unless the remote stub indicated it supports the augmented form of this packet by supplying an appropriate ‘qSupported’ response (see [qXfer read], page 833, [qSupported], page 822).

This packet is optional for better performance on SVR4 targets. GDB uses memory read packets to read the SVR4 library list otherwise.

This packet is not probed by default; the remote stub must request it, by supplying an appropriate ‘qSupported’ response (see [qSupported], page 822).

If the remote stub indicates it supports the augmented form of this packet then the annex part of the generic ‘qXfer’ packet may contain a semicolon-separated list of ‘*name=value*’ arguments. The currently supported arguments are:

start=address

A hexadecimal number specifying the address of the ‘struct link_map’ to start reading the library list from. If unset or zero then the first ‘struct link_map’ in the library list will be chosen as the starting point.

prev=address

A hexadecimal number specifying the address of the ‘struct link_map’ immediately preceding the ‘struct

`link_map` specified by the `'start'` argument. If unset or zero then the remote stub will expect that no `'struct link_map'` exists prior to the starting point.

`lmid=lmid`

A hexadecimal number specifying a namespace identifier. This is currently only used together with `'start'` to provide the namespace identifier back to GDB in the response. GDB will only provide values that were previously reported to it. If unset, the response will include `'lmid="0x0"'`.

Arguments that are not understood by the remote stub will be silently ignored.

`'qXfer:memory-map:read::offset,length'`

Access the target's *memory-map*. See Section E.17 [Memory Map Format], page 867. The annex part of the generic `'qXfer'` packet must be empty (see [qXfer read], page 833).

This packet is not probed by default; the remote stub must request it, by supplying an appropriate `'qSupported'` response (see [qSupported], page 822).

`'qXfer:sdata:read::offset,length'`

Read contents of the extra collected static tracepoint marker information. The annex part of the generic `'qXfer'` packet must be empty (see [qXfer read], page 833). See Section 13.1.6 [Tracepoint Action Lists], page 210.

This packet is not probed by default; the remote stub must request it, by supplying an appropriate `'qSupported'` response (see [qSupported], page 822).

`'qXfer:signinfo:read::offset,length'`

Read contents of the extra signal information on the target system. The annex part of the generic `'qXfer'` packet must be empty (see [qXfer read], page 833).

This packet is not probed by default; the remote stub must request it, by supplying an appropriate `'qSupported'` response (see [qSupported], page 822).

`'qXfer:threads:read::offset,length'`

Access the list of threads on target. See Section E.18 [Thread List Format], page 868. The annex part of the generic `'qXfer'` packet must be empty (see [qXfer read], page 833).

This packet is not probed by default; the remote stub must request it, by supplying an appropriate `'qSupported'` response (see [qSupported], page 822).

`'qXfer:traceframe-info:read::offset,length'`

Return a description of the current traceframe's contents. See Section E.19 [Traceframe Info Format], page 869. The annex part

of the generic ‘qXfer’ packet must be empty (see [qXfer read], page 833).

This packet is not probed by default; the remote stub must request it, by supplying an appropriate ‘qSupported’ response (see [qSupported], page 822).

‘qXfer:uib:read:pc:offset,length’

Return the unwind information block for *pc*. This packet is used on OpenVMS/ia64 to ask the kernel unwind information.

This packet is not probed by default.

‘qXfer:fdpic:read:annex:offset,length’

Read contents of *loadmaps* on the target system. The annex, either ‘exec’ or ‘interp’, specifies which *loadmap*, executable *loadmap* or interpreter *loadmap* to read.

This packet is not probed by default; the remote stub must request it, by supplying an appropriate ‘qSupported’ response (see [qSupported], page 822).

‘qXfer:osdata:read::offset,length’

Access the target’s *operating system information*. See Appendix H [Operating System Information], page 905.

‘qXfer:object:write:annex:offset:data...’

Write uninterpreted bytes into the target’s special data area identified by the keyword *object*, starting at *offset* bytes into the data. The binary-encoded data (see [Binary Data], page 795) to be written is given by *data...* The content and encoding of *annex* is specific to *object*; it can supply additional details about what data to access.

Reply:

‘nn’ *nn* (hex encoded) is the number of bytes written. This may be fewer bytes than supplied in the request.

Here are the specific requests of this form defined so far. All the ‘qXfer:object:write:...’ requests use the same reply formats, listed above.

‘qXfer:signfo:write::offset:data...’

Write *data* to the extra signal information on the target system. The annex part of the generic ‘qXfer’ packet must be empty (see [qXfer write], page 837).

This packet is not probed by default; the remote stub must request it, by supplying an appropriate ‘qSupported’ response (see [qSupported], page 822).

‘qXfer:object:operation:...’

Requests of this form may be added in the future. When a stub does not recognize the *object* keyword, or its support for *object* does not recognize the *operation* keyword, the stub must respond with an empty packet.

‘qAttached:pid’

Return an indication of whether the remote server attached to an existing process or created a new process. When the multiprocess protocol extensions are supported (see [multiprocess extensions], page 829), *pid* is an integer in hexadecimal format identifying the target process. Otherwise, GDB will omit the *pid* field and the query packet will be simplified as ‘qAttached’.

This query is used, for example, to know whether the remote process should be detached or killed when a GDB session is ended with the **quit** command.

Reply:

‘1’ The remote server attached to an existing process.

‘0’ The remote server created a new process.

‘qExecAndArgs’

Return the program filename and arguments string with which the remote server was started, if the remote server was started with such things. If the remote server was started without the filename of a program to execute, or without any arguments, then the reply indicates this.

Reply:

‘U’ The program filename and arguments are unset. If GDB wants to start a new inferior, for example with ‘vRun’, then it will need to provide the filename of a program to use.

‘S;prog;args;’

‘S;;args;’

The program filename provided to the remote server when it started was *prog*, which is a hex encoded string. The complete argument string passed to the inferior when it started as *args*, this is also a hex encoded string.

If no arguments were passed when the inferior started then *args* be an empty string.

It is valid for *prog* to be the empty string, this indicates that the server has no program set, GDB will need to supply a program name in order to start a new inferior. It is valid to reply with an empty *prog* and non-empty *args*, GDB will set the inferior arguments, but the user will need to supply a remote exec-file before an inferior can be started.

The argument string *args* is passed directly to GDB as if to the **set args** command. And escaping required should be present in *args*, no changes are made by GDB.

‘Qbtrace:bts’

Enable branch tracing for the current thread using Branch Trace Store.

Reply:

‘OK’ Branch tracing has been enabled.

‘Qbtrace:pt’
 Enable branch tracing for the current thread using Intel Processor Trace.
 Reply:
 ‘OK’ Branch tracing has been enabled.

‘Qbtrace:off’
 Disable branch tracing for the current thread.
 Reply:
 ‘OK’ Branch tracing has been disabled.

‘Qbtrace-conf:bts:size=value’
 Set the requested ring buffer size for new threads that use the btrace recording method in bts format.
 Reply:
 ‘OK’ The ring buffer size has been set.

‘Qbtrace-conf:pt:size=value’
 Set the requested ring buffer size for new threads that use the btrace recording method in pt format.
 Reply:
 ‘OK’ The ring buffer size has been set.

‘Qbtrace-conf:pt:ptwrite=(yes|no)’
 Indicate support for PTWRITE packets. This allows for backwards compatibility.
 Reply:
 ‘OK’ The ptwrite config parameter has been set.

‘E.errtext’
 A badly formed request or an error was encountered.

‘Qbtrace-conf:pt:event-tracing=(yes|no)’
 Indicate support for event-tracing packets. This allows for backwards compatibility.
 Reply:
 ‘OK’ The event-tracing config parameter has been set.

‘E.errtext’
 A badly formed request or an error was encountered.

E.6 Architecture-Specific Protocol Details

This section describes how the remote protocol is applied to specific target architectures. Also see Section G.5 [Standard Target Features], page 889, for details of XML target descriptions for each architecture.

E.6.1 ARM-specific Protocol Details

E.6.1.1 ARM Breakpoint Kinds

These breakpoint kinds are defined for the ‘Z0’ and ‘Z1’ packets.

- 2 16-bit Thumb mode breakpoint.
- 3 32-bit Thumb mode (Thumb-2) breakpoint.
- 4 32-bit ARM mode breakpoint.

E.6.1.2 ARM Memory Tag Types

These memory tag types are defined for the ‘qMemTag’ and ‘QMemTag’ packets.

- 0 MTE logical tag
- 1 MTE allocation tag

E.6.2 MIPS-specific Protocol Details

E.6.2.1 MIPS Register Packet Format

The following g/G packets have previously been defined. In the below, some thirty-two bit registers are transferred as sixty-four bits. Those registers should be zero/sign extended (which?) to fill the space allocated. Register bytes are transferred in target byte order. The two nibbles within a register byte are transferred most-significant – least-significant.

- MIPS32 All registers are transferred as thirty-two bit quantities in the order: 32 general-purpose; sr; lo; hi; bad; cause; pc; 32 floating-point registers; fsr; fir; fp.
- MIPS64 All registers are transferred as sixty-four bit quantities (including thirty-two bit registers such as **sr**). The ordering is the same as MIPS32.

E.6.2.2 MIPS Breakpoint Kinds

These breakpoint kinds are defined for the ‘Z0’ and ‘Z1’ packets.

- 2 16-bit MIPS16 mode breakpoint.
- 3 16-bit microMIPS mode breakpoint.
- 4 32-bit standard MIPS mode breakpoint.
- 5 32-bit microMIPS mode breakpoint.

E.7 Tracepoint Packets

Here we describe the packets GDB uses to implement tracepoints (see Chapter 13 [Tracepoints], page 205).

`‘QTDP:n:addr:ena:step:pass[:Fflen] [:S] [:Xlen,bytes] [-]’`

Create a new tracepoint, number *n*, at *addr*. If *ena* is ‘E’, then the tracepoint is enabled; if it is ‘D’, then the tracepoint is disabled. The *step* gives the tracepoint’s step count, and *pass* gives its pass count. If an ‘F’ is present, then the tracepoint is to be a fast tracepoint, and the *flen* is the number of bytes that the target should copy elsewhere to make room for the tracepoint. If an ‘S’ is present, the tracepoint is to be a static tracepoint. If an ‘X’ is

present, it introduces a tracepoint condition, which consists of a hexadecimal length, followed by a comma and hex-encoded bytes, in a manner similar to action encodings as described below. If the trailing ‘-’ is present, further ‘QTDP’ packets will follow to specify this tracepoint’s actions.

Replies:

‘OK’ The packet was understood and carried out.

‘qRelocInsn’

See Section E.7 [Relocate instruction reply packet], page 840.

‘QTDP:-n:addr:[S]action...[-]’

Define actions to be taken when a tracepoint is hit. The *n* and *addr* must be the same as in the initial ‘QTDP’ packet for this tracepoint. This packet may only be sent immediately after another ‘QTDP’ packet that ended with a ‘-’. If the trailing ‘-’ is present, further ‘QTDP’ packets will follow, specifying more actions for this tracepoint.

In the series of action packets for a given tracepoint, at most one can have an ‘S’ before its first *action*. If such a packet is sent, it and the following packets define “while-stepping” actions. Any prior packets define ordinary actions — that is, those taken when the tracepoint is first hit. If no action packet has an ‘S’, then all the packets in the series specify ordinary tracepoint actions.

The ‘*action...*’ portion of the packet is a series of actions, concatenated without separators. Each action has one of the following forms:

‘R *mask*’ Collect the registers whose bits are set in *mask*, a hexadecimal number whose *i*’th bit is set if register number *i* should be collected. (The least significant bit is numbered zero.) Note that *mask* may be any number of digits long; it may not fit in a 32-bit word.

‘M *basereg,offset,len*’

Collect *len* bytes of memory starting at the address in register number *basereg*, plus *offset*. If *basereg* is ‘-1’, then the range has a fixed address: *offset* is the address of the lowest byte to collect. The *basereg*, *offset*, and *len* parameters are all unsigned hexadecimal values (the ‘-1’ value for *basereg* is a special case).

‘X *len,expr*’

Evaluate *expr*, whose length is *len*, and collect memory as it directs. The agent expression *expr* is as described in Appendix F [Agent Expressions], page 871. Each byte of the expression is encoded as a two-digit hex number in the packet; *len* is the number of bytes in the expression (and thus one-half the number of hex digits in the packet).

‘L’ Collect static trace data.

Any number of actions may be packed together in a single ‘QTDP’ packet, as long as the packet does not exceed the maximum packet length (400 bytes, for many stubs). There may be only one ‘R’ action per tracepoint, and it must precede any ‘M’ or ‘X’ actions. Any registers referred to by ‘M’ and ‘X’ actions must be

collected by a preceding ‘R’ action. (The “while-stepping” actions are treated as if they were attached to a separate tracepoint, as far as these restrictions are concerned.)

Replies:

‘OK’ The packet was understood and carried out.

‘qRelocInsn’

See Section E.7 [Relocate instruction reply packet], page 840.

‘QTDPsrc:n:addr:type:start:slen:bytes’

Specify a source string of tracepoint *n* at address *addr*. This is useful to get accurate reproduction of the tracepoints originally downloaded at the beginning of the trace run. The *type* is the name of the tracepoint part, such as ‘cond’ for the tracepoint’s conditional expression (see below for a list of types), while *bytes* is the string, encoded in hexadecimal.

start is the offset of the *bytes* within the overall source string, while *slen* is the total length of the source string. This is intended for handling source strings that are longer than will fit in a single packet.

The available string types are ‘at’ for the location, ‘cond’ for the conditional, and ‘cmd’ for an action command. GDB sends a separate packet for each command in the action list, in the same order in which the commands are stored in the list.

The target does not need to do anything with source strings except report them back as part of the replies to the ‘qTfP’/‘qTsP’ query packets.

Although this packet is optional, and GDB will only send it if the target replies with ‘TracepointSource’ See Section E.5 [General Query Packets], page 811, it makes both disconnected tracing and trace files much easier to use. Otherwise the user must be careful that the tracepoints in effect while looking at trace frames are identical to the ones in effect during the trace run; even a small discrepancy could cause ‘tdump’ not to work, or a particular trace frame not be found.

‘QTDV:n:value:builtin:name’

Create a new trace state variable, number *n*, with an initial value of *value*, which is a 64-bit signed integer. Both *n* and *value* are encoded as hexadecimal values. GDB has the option of not using this packet for initial values of zero; the target should simply create the trace state variables as they are mentioned in expressions. The value *builtin* should be 1 (one) if the trace state variable is builtin and 0 (zero) if it is not builtin. GDB only sets *builtin* to 1 if a previous ‘qTfV’ or ‘qTsV’ packet had it set. The contents of *name* is the hex-encoded name (without the leading ‘\$’) of the trace state variable.

‘QTFrame:n’

Select the *n*’th tracepoint frame from the buffer, and use the register and memory contents recorded there to answer subsequent request packets from GDB.

A successful reply from the stub indicates that the stub has found the requested frame. The response is a series of parts, concatenated without separators, describing the frame we selected. Each part has one of the following forms:

- ‘F *f*’ The selected frame is number *n* in the trace frame buffer; *f* is a hexadecimal number. If *f* is ‘-1’, then there was no frame matching the criteria in the request packet.
- ‘T *t*’ The selected trace frame records a hit of tracepoint number *t*; *t* is a hexadecimal number.
- ‘QTFrame:pc:addr’
 Like ‘QTFrame:*n*’, but select the first tracepoint frame after the currently selected frame whose PC is *addr*; *addr* is a hexadecimal number.
- ‘QTFrame:tdp:*t*’
 Like ‘QTFrame:*n*’, but select the first tracepoint frame after the currently selected frame that is a hit of tracepoint *t*; *t* is a hexadecimal number.
- ‘QTFrame:range:start:end’
 Like ‘QTFrame:*n*’, but select the first tracepoint frame after the currently selected frame whose PC is between *start* (inclusive) and *end* (inclusive); *start* and *end* are hexadecimal numbers.
- ‘QTFrame:outside:start:end’
 Like ‘QTFrame:range:start:end’, but select the first frame *outside* the given range of addresses (exclusive).
- ‘qTMinFTPILen’
 This packet requests the minimum length of instruction at which a fast tracepoint (see Section 13.1 [Set Tracepoints], page 205) may be placed. For instance, on the 32-bit x86 architecture, it is possible to use a 4-byte jump, but it depends on the target system being able to create trampolines in the first 64K of memory, which might or might not be possible for that system. So the reply to this packet will be 4 if it is able to arrange for that.
 Replies:
- ‘0’ The minimum instruction length is currently unknown.
- ‘length’ The minimum instruction length is *length*, where *length* is a hexadecimal number greater or equal to 1. A reply of 1 means that a fast tracepoint may be placed on any instruction regardless of size.
- ‘E’ An error has occurred.
- ‘QTStart’ Begin the tracepoint experiment. Begin collecting data from tracepoint hits in the trace frame buffer. This packet supports the ‘qRelocInsn’ reply (see Section E.7 [Relocate instruction reply packet], page 840).
- ‘QTStop’ End the tracepoint experiment. Stop collecting trace frames.
- ‘QTEnable:*n*:addr’
 Enable tracepoint *n* at address *addr* in a started tracepoint experiment. If the tracepoint was previously disabled, then collection of data from it will resume.
- ‘QTDisable:*n*:addr’
 Disable tracepoint *n* at address *addr* in a started tracepoint experiment. No more data will be collected from the tracepoint unless ‘QTEnable:*n*:addr’ is subsequently issued.

‘QTinit’ Clear the table of tracepoints, and empty the trace frame buffer.

‘QTro:start1,end1:start2,end2:...’

Establish the given ranges of memory as “transparent”. The stub will answer requests for these ranges from memory’s current contents, if they were not collected as part of the tracepoint hit.

GDB uses this to mark read-only regions of memory, like those containing program code. Since these areas never change, they should still have the same contents they did when the tracepoint was hit, so there’s no reason for the stub to refuse to provide their contents.

‘QTDisconnected:value’

Set the choice to what to do with the tracing run when GDB disconnects from the target. A *value* of 1 directs the target to continue the tracing run, while 0 tells the target to stop tracing if GDB is no longer in the picture.

‘qTStatus’

Ask the stub if there is a trace experiment running right now.

The reply has the form:

‘Trunning[:field]...’

running is a single digit 1 if the trace is presently running, or 0 if not. It is followed by semicolon-separated optional fields that an agent may use to report additional status.

If the trace is not running, the agent may report any of several explanations as one of the optional fields:

‘tnotrun:0’

No trace has been run yet.

‘tstop[:text]:0’

The trace was stopped by a user-originated stop command. The optional *text* field is a user-supplied string supplied as part of the stop command (for instance, an explanation of why the trace was stopped manually). It is hex-encoded.

‘tfull:0’ The trace stopped because the trace buffer filled up.

‘tdisconnected:0’

The trace stopped because GDB disconnected from the target.

‘tpasscount:tpnum’

The trace stopped because tracepoint *tpnum* exceeded its pass count.

‘terror:text:tpnum’

The trace stopped because tracepoint *tpnum* had an error. The string *text* is available to describe the nature of the error (for instance, a divide by zero in the condition expression); it is hex encoded.

‘tunknown:0’

The trace stopped for some other reason.

Additional optional fields supply statistical and other information. Although not required, they are extremely useful for users monitoring the progress of a trace run. If a trace has stopped, and these numbers are reported, they must reflect the state of the just-stopped trace.

‘tframes:n’

The number of trace frames in the buffer.

‘tcreated:n’

The total number of trace frames created during the run. This may be larger than the trace frame count, if the buffer is circular.

‘tsize:n’ The total size of the trace buffer, in bytes.

‘tfree:n’ The number of bytes still unused in the buffer.

‘circular:n’

The value of the circular trace buffer flag. 1 means that the trace buffer is circular and old trace frames will be discarded if necessary to make room, 0 means that the trace buffer is linear and may fill up.

‘disconn:n’

The value of the disconnected tracing flag. 1 means that tracing will continue after GDB disconnects, 0 means that the trace run will stop.

‘qTP:tp:addr’

Ask the stub for the current state of tracepoint number *tp* at address *addr*.

Replies:

‘Vhits:usage’

The tracepoint has been hit *hits* times so far during the trace run, and accounts for *usage* in the trace buffer. Note that **while-stepping** steps are not counted as separate hits, but the steps’ space consumption is added into the usage number.

‘qTV:var’ Ask the stub for the value of the trace state variable number *var*.

Replies:

‘Vvalue’ The value of the variable is *value*. This will be the current value of the variable if the user is examining a running target, or a saved value if the variable was collected in the trace frame that the user is looking at. Note that multiple requests may result in different reply values, such as when requesting values while the program is running.

‘U’ The value of the variable is unknown. This would occur, for example, if the user is examining a trace frame in which the requested variable was not collected.

‘qTfP’

‘qTsP’ These packets request data about tracepoints that are being used by the target. GDB sends **qTfP** to get the first piece of data, and multiple **qTsP** to get additional

pieces. Replies to these packets generally take the form of the QTDV packets that define tracepoints. (FIXME add detailed syntax)

‘qTfV’

‘qTsV’

These packets request data about trace state variables that are on the target. GDB sends qTfV to get the first vari of data, and multiple qTsV to get additional variables. Replies to these packets follow the syntax of the QTDV packets that define trace state variables.

‘qTfSTM’

‘qTsSTM’

These packets request data about static tracepoint markers that exist in the target program. GDB sends qTfSTM to get the first piece of data, and multiple qTsSTM to get additional pieces. Replies to these packets take the following form:

Reply:

‘m *address:id:extra*’

A single marker

‘m *address:id:extra,address:id:extra...*’

a comma-separated list of markers

‘l’

(lower case letter ‘L’) denotes end of list.

The *address* is encoded in hex; *id* and *extra* are strings encoded in hex.

In response to each query, the target will reply with a list of one or more markers, separated by commas. GDB will respond to each reply with a request for more markers (using the ‘qs’ form of the query), until the target responds with ‘l’ (lower-case ell, for *last*).

‘qTSTMat:*address*’

This packet requests data about static tracepoint markers in the target program at *address*. Replies to this packet follow the syntax of the qTfSTM and qTsSTM packets that list static tracepoint markers.

‘QTSave:*filename*’

This packet directs the target to save trace data to the file name *filename* in the target’s filesystem. The *filename* is encoded as a hex string; the interpretation of the file name (relative vs absolute, wild cards, etc) is up to the target.

‘qTBuffer:*offset,len*’

Return up to *len* bytes of the current contents of trace buffer, starting at *offset*. The trace buffer is treated as if it were a contiguous collection of traceframes, as per the trace file format. The reply consists as many hex-encoded bytes as the target can deliver in a packet; it is not an error to return fewer than were asked for. A reply consisting of just l indicates that no bytes are available.

‘QTBuffer:*circular:value*’

This packet directs the target to use a circular trace buffer if *value* is 1, or a linear buffer if the value is 0.

‘QTBuffer:*size:size*’

This packet directs the target to make the trace buffer be of size *size* if possible. A value of -1 tells the target to use whatever size it prefers.

`'QTNotes:[type:text][;type:text]...'`

This packet adds optional textual notes to the trace run. Allowable types include `user`, `notes`, and `tstop`, the *text* fields are arbitrary strings, hex-encoded.

E.7.1 Relocate instruction reply packet

When installing fast tracepoints in memory, the target may need to relocate the instruction currently at the tracepoint address to a different address in memory. For most instructions, a simple copy is enough, but, for example, call instructions that implicitly push the return address on the stack, and relative branches or other PC-relative instructions require offset adjustment, so that the effect of executing the instruction at a different address is the same as if it had executed in the original location.

In response to several of the tracepoint packets, the target may also respond with a number of intermediate `'qRelocInsn'` request packets before the final result packet, to have GDB handle this relocation operation. If a packet supports this mechanism, its documentation will explicitly say so. See for example the above descriptions for the `'QTStart'` and `'QTDP'` packets. The format of the request is:

`'qRelocInsn:from;to'`

This requests GDB to copy instruction at address *from* to address *to*, possibly adjusted so that executing the instruction at *to* has the same effect as executing it at *from*. GDB writes the adjusted instruction to target memory starting at *to*.

Replies:

`'qRelocInsn:adjusted_size'`

Informs the stub the relocation is complete. The *adjusted_size* is the length in bytes of resulting relocated instruction sequence.

E.8 Host I/O Packets

The *Host I/O* packets allow GDB to perform I/O operations on the far side of a remote link. For example, Host I/O is used to upload and download files to a remote target with its own filesystem. Host I/O uses the same constant values and data structure layout as the target-initiated File-I/O protocol. However, the Host I/O packets are structured differently. The target-initiated protocol relies on target memory to store parameters and buffers. Host I/O requests are initiated by GDB, and the target's memory is not involved. See Section E.14 [File-I/O Remote Protocol Extension], page 854, for more details on the target-initiated protocol.

The Host I/O request packets all encode a single operation along with its arguments. They have this format:

`'vFile:operation: parameter...'`

operation is the name of the particular request; the target should compare the entire packet name up to the second colon when checking for a supported operation. The format of *parameter* depends on the operation. Numbers are always passed in hexadecimal. Negative numbers have an explicit minus sign (i.e. two's complement is not used). Strings (e.g. filenames) are encoded as a

series of hexadecimal bytes. The last argument to a system call may be a buffer of escaped binary data (see [Binary Data], page 795).

The valid responses to Host I/O packets are:

`'F result [, errno] [; attachment]'`

result is the integer value returned by this operation, usually non-negative for success and -1 for errors. If an error has occurred, *errno* will be included in the result specifying a value defined by the File-I/O protocol (see [Errno Values], page 864). For operations which return data, *attachment* supplies the data as a binary buffer. Binary buffers in response packets are escaped in the normal way (see [Binary Data], page 795). See the individual packet documentation for the interpretation of *result* and *attachment*.

`'` An empty response indicates that this operation is not recognized.

These are the supported Host I/O operations:

`'vFile:open: filename, flags, mode'`

Open a file at *filename* and return a file descriptor for it, or return -1 if an error occurs. The *filename* is a string, *flags* is an integer indicating a mask of open flags (see [Open Flags], page 864), and *mode* is an integer indicating a mask of mode bits to use if the file is created (see [mode_t Values], page 864). See [open], page 856, for details of the open flags and mode values.

`'vFile:close: fd'`

Close the open file corresponding to *fd* and return 0, or -1 if an error occurs.

`'vFile:pread: fd, count, offset'`

Read data from the open file corresponding to *fd*. Up to *count* bytes will be read from the file, starting at *offset* relative to the start of the file. The target may read fewer bytes; common reasons include packet size limits and an end-of-file condition. The number of bytes read is returned. Zero should only be returned for a successful read at the end of the file, or if *count* was zero.

The data read should be returned as a binary attachment on success. If zero bytes were read, the response should include an empty binary attachment (i.e. a trailing semicolon). The return value is the number of target bytes read; the binary attachment may be longer if some characters were escaped.

`'vFile:pwrite: fd, offset, data'`

Write *data* (a binary buffer) to the open file corresponding to *fd*. Start the write at *offset* from the start of the file. Unlike many `write` system calls, there is no separate *count* argument; the length of *data* in the packet is used. `'vFile:pwrite'` returns the number of bytes written, which may be shorter than the length of *data*, or -1 if an error occurred.

`'vFile:fstat: fd'`

Get information about the open file corresponding to *fd*. On success the information is returned as a binary attachment and the return value is the size of this attachment in bytes. If an error occurs the return value is -1. The format of the returned binary attachment is as described in [struct stat], page 863.

‘vFile:stat: *filename*’

Get information about the file *filename* on the target as if from a ‘stat’ call. On success the information is returned as a binary attachment and the return value is the size of this attachment in bytes. If an error occurs the return value is -1. The format of the returned binary attachment is as described in [struct stat], page 863.

If *filename* is a symbolic link, then the information returned is about the file the link refers to, this is inline with the ‘stat’ library call.

‘vFile:lstat: *filename*’

Get information about the file *filename* on the target as if from an ‘lstat’ call. On success the information is returned as a binary attachment and the return value is the size of this attachment in bytes. If an error occurs the return value is -1. The format of the returned binary attachment is as described in [struct stat], page 863.

This packet is identical to ‘vFile:stat’, except if *filename* is a symbolic link, then this packet returns information about the link itself, not the file that the link refers to, this is inline with the ‘lstat’ library call.

‘vFile:unlink: *filename*’

Delete the file at *filename* on the target. Return 0, or -1 if an error occurs. The *filename* is a string.

‘vFile:readlink: *filename*’

Read value of symbolic link *filename* on the target. Return the number of bytes read, or -1 if an error occurs.

The data read should be returned as a binary attachment on success. If zero bytes were read, the response should include an empty binary attachment (i.e. a trailing semicolon). The return value is the number of target bytes read; the binary attachment may be longer if some characters were escaped.

‘vFile:setfs: *pid*’

Select the filesystem on which vFile operations with *filename* arguments will operate. This is required for GDB to be able to access files on remote targets where the remote stub does not share a common filesystem with the inferior(s).

If *pid* is nonzero, select the filesystem as seen by process *pid*. If *pid* is zero, select the filesystem as seen by the remote stub. Return 0 on success, or -1 if an error occurs. If vFile:setfs: indicates success, the selected filesystem remains selected until the next successful vFile:setfs: operation.

E.9 Interrupts

In all-stop mode, when a program on the remote target is running, GDB may attempt to interrupt it by sending a ‘Ctrl-C’, BREAK or a BREAK followed by g, control of which is specified via GDB’s ‘interrupt-sequence’.

The precise meaning of BREAK is defined by the transport mechanism and may, in fact, be undefined. GDB does not currently define a BREAK mechanism for any of the network interfaces except for TCP, in which case GDB sends the telnet BREAK sequence.

‘Ctrl-C’, on the other hand, is defined and implemented for all transport mechanisms. It is represented by sending the single byte 0x03 without any of the usual packet overhead described in the Overview section (see Section E.1 [Overview], page 795). When a 0x03 byte is transmitted as part of a packet, it is considered to be packet data and does *not* represent an interrupt. E.g., an ‘X’ packet (see [X packet], page 806), used for binary downloads, may include an unescaped 0x03 as part of its packet.

BREAK followed by g is also known as Magic SysRq g. When Linux kernel receives this sequence from serial port, it stops execution and connects to gdb.

In non-stop mode, because packet resumptions are asynchronous (see [vCont packet], page 802), GDB is always free to send a remote command to the remote stub, even when the target is running. For that reason, GDB instead sends a regular packet (see [vCtrlC packet], page 803) with the usual packet framing instead of the single byte 0x03.

Stubs are not required to recognize these interrupt mechanisms and the precise meaning associated with receipt of the interrupt is implementation defined. If the target supports debugging of multiple threads and/or processes, it should attempt to interrupt all currently-executing threads and processes. If the stub is successful at interrupting the running program, it should send one of the stop reply packets (see Section E.4 [Stop Reply Packets], page 807) to GDB as a result of successfully stopping the program in all-stop mode, and a stop reply for each stopped thread in non-stop mode. Interrupts received while the program is stopped are queued and the program will be interrupted when it is resumed next time.

E.10 Notification Packets

The GDB remote serial protocol includes *notifications*, packets that require no acknowledgment. Both the GDB and the stub may send notifications (although the only notifications defined at present are sent by the stub). Notifications carry information without incurring the round-trip latency of an acknowledgment, and so are useful for low-impact communications where occasional packet loss is not a problem.

A notification packet has the form ‘% *data* # *checksum*’, where *data* is the content of the notification, and *checksum* is a checksum of *data*, computed and formatted as for ordinary GDB packets. A notification’s *data* never contains ‘\$’, ‘%’ or ‘#’ characters. Upon receiving a notification, the recipient sends no ‘+’ or ‘-’ to acknowledge the notification’s receipt or to report its corruption.

Every notification’s *data* begins with a name, which contains no colon characters, followed by a colon character.

Recipients should silently ignore corrupted notifications and notifications they do not understand. Recipients should restart timeout periods on receipt of a well-formed notification, whether or not they understand it.

Senders should only send the notifications described here when this protocol description specifies that they are permitted. In the future, we may extend the protocol to permit existing notifications in new contexts; this rule helps older senders avoid confusing newer recipients.

(Older versions of GDB ignore bytes received until they see the ‘\$’ byte that begins an ordinary packet, so new stubs may transmit notifications without fear of confusing older clients. There are no notifications defined for GDB to send at the moment, but we assume that most older stubs would ignore them, as well.)

Each notification is comprised of three parts:

‘name:event’

The notification packet is sent by the side that initiates the exchange (currently, only the stub does that), with *event* carrying the specific information about the notification, and *name* specifying the name of the notification.

‘ack’

The acknowledge sent by the other side, usually GDB, to acknowledge the exchange and request the event.

The purpose of an asynchronous notification mechanism is to report to GDB that something interesting happened in the remote stub.

The remote stub may send notification *name:event* at any time, but GDB acknowledges the notification when appropriate. The notification event is pending before GDB acknowledges. Only one notification at a time may be pending; if additional events occur before GDB has acknowledged the previous notification, they must be queued by the stub for later synchronous transmission in response to *ack* packets from GDB. Because the notification mechanism is unreliable, the stub is permitted to resend a notification if it believes GDB may not have received it.

Specifically, notifications may appear when GDB is not otherwise reading input from the stub, or when GDB is expecting to read a normal synchronous response or a ‘+’/‘-’ acknowledgment to a packet it has sent. Notification packets are distinct from any other communication from the stub so there is no ambiguity.

After receiving a notification, GDB shall acknowledge it by sending a *ack* packet as a regular, synchronous request to the stub. Such acknowledgment is not required to happen immediately, as GDB is permitted to send other, unrelated packets to the stub first, which the stub should process normally.

Upon receiving a *ack* packet, if the stub has other queued events to report to GDB, it shall respond by sending a normal *event*. GDB shall then send another *ack* packet to solicit further responses; again, it is permitted to send other, unrelated packets as well which the stub should process normally.

If the stub receives a *ack* packet and there are no additional *event* to report, the stub shall return an ‘OK’ response. At this point, GDB has finished processing a notification and the stub has completed sending any queued events. GDB won’t accept any new notifications until the final ‘OK’ is received. If further notification events occur, the stub shall send a new notification, GDB shall accept the notification, and the process shall be repeated.

The process of asynchronous notification can be illustrated by the following example:

```
<- %Stop:T0505:98e7ffbf;04:4ce6ffbf;08:b1b6e54c;thread:p7526.7526;core:0;
...
-> vStopped
<- T0505:68f37db7;04:40f37db7;08:63850408;thread:p7526.7528;core:0;
-> vStopped
<- T0505:68e3fdb6;04:40e3fdb6;08:63850408;thread:p7526.7529;core:0;
-> vStopped
<- OK
```

The following notifications are defined:

Notification	Ack	Event	Description
--------------	-----	-------	-------------

Stop	vStopped <i>reply.</i> The <i>reply</i> has the form of a stop reply, as described in Section E.4 [Stop Reply Packets], page 807. Refer to Section E.11 [Remote Non-Stop], page 852, for information on how these notifications are acknowledged by GDB.	Report an asynchronous stop event in non-stop mode.
------	--	---

E.11 Remote Protocol Support for Non-Stop Mode

GDB's remote protocol supports non-stop debugging of multi-threaded programs, as described in Section 5.5.2 [Non-Stop Mode], page 97. If the stub supports non-stop mode, it should report that to GDB by including 'QNonStop+' in its 'qSupported' response (see [qSupported], page 822).

GDB typically sends a 'QNonStop' packet only when establishing a new connection with the stub. Entering non-stop mode does not alter the state of any currently-running threads, but targets must stop all threads in any already-attached processes when entering all-stop mode. GDB uses the '?' packet as necessary to probe the target state after a mode change.

In non-stop mode, when an attached process encounters an event that would otherwise be reported with a stop reply, it uses the asynchronous notification mechanism (see Section E.10 [Notification Packets], page 850) to inform GDB. In contrast to all-stop mode, where all threads in all processes are stopped when a stop reply is sent, in non-stop mode only the thread reporting the stop event is stopped. That is, when reporting a 'S' or 'T' response to indicate completion of a step operation, hitting a breakpoint, or a fault, only the affected thread is stopped; any other still-running threads continue to run. When reporting a 'W' or 'X' response, all running threads belonging to other attached processes continue to run.

In non-stop mode, the target shall respond to the '?' packet as follows. First, any incomplete stop reply notification/'vStopped' sequence in progress is abandoned. The target must begin a new sequence reporting stop events for all stopped threads, whether or not it has previously reported those events to GDB. The first stop reply is sent as a synchronous reply to the '?' packet, and subsequent stop replies are sent as responses to 'vStopped' packets using the mechanism described above. The target must not send asynchronous stop reply notifications until the sequence is complete. If all threads are running when the target receives the '?' packet, or if the target is not attached to any process, it shall respond 'OK'.

If the stub supports non-stop mode, it should also support the 'swbreak' stop reason if software breakpoints are supported, and the 'hwbreak' stop reason if hardware breakpoints are supported (see [swbreak stop reason], page 809). This is because given the asynchronous nature of non-stop mode, between the time a thread hits a breakpoint and the time the event is finally processed by GDB, the breakpoint may have already been removed from the target. Due to this, GDB needs to be able to tell whether a trap stop was caused by a delayed breakpoint event, which should be ignored, as opposed to a random trap signal, which should be reported to the user. Note the 'swbreak' feature implies that the target is responsible for adjusting the PC when a software breakpoint triggers, if necessary, such as on the x86 architecture.

E.12 Packet Acknowledgment

By default, when either the host or the target machine receives a packet, the first response expected is an acknowledgment: either ‘+’ (to indicate the package was received correctly) or ‘-’ (to request retransmission). This mechanism allows the GDB remote protocol to operate over unreliable transport mechanisms, such as a serial line.

In cases where the transport mechanism is itself reliable (such as a pipe or TCP connection), the ‘+’/‘-’ acknowledgments are redundant. It may be desirable to disable them in that case to reduce communication overhead, or for other reasons. This can be accomplished by means of the ‘QStartNoAckMode’ packet; see [QStartNoAckMode], page 822.

When in no-acknowledgment mode, neither the stub nor GDB shall send or expect ‘+’/‘-’ protocol acknowledgments. The packet and response format still includes the normal checksum, as described in Section E.1 [Overview], page 795, but the checksum may be ignored by the receiver.

If the stub supports ‘QStartNoAckMode’ and prefers to operate in no-acknowledgment mode, it should report that to GDB by including ‘QStartNoAckMode+’ in its response to ‘qSupported’; see [qSupported], page 822. If GDB also supports ‘QStartNoAckMode’ and it has not been disabled via the `set remote noack-packet off` command (see Section 20.4 [Remote Configuration], page 328), GDB may then send a ‘QStartNoAckMode’ packet to the stub. Only then may the stub actually turn off packet acknowledgments. GDB sends a final ‘+’ acknowledgment of the stub’s ‘OK’ response, which can be safely ignored by the stub.

Note that `set remote noack-packet` command only affects negotiation between GDB and the stub when subsequent connections are made; it does not affect the protocol acknowledgment state for any current connection. Since ‘+’/‘-’ acknowledgments are enabled by default when a new connection is established, there is also no protocol request to re-enable the acknowledgments for the current connection, once disabled.

E.13 Examples

Example sequence of a target being re-started. Notice how the restart does not get any direct output:

```
-> R00
<- +
target restarts
-> ?
<- +
<- T001:1234123412341234
-> +
```

Example sequence of a target being stepped by a single instruction:

```
-> G1445...
<- +
-> s
<- +
time passes
<- T001:1234123412341234
-> +
-> g
<- +
<- 1455...
-> +
```

E.14 File-I/O Remote Protocol Extension

E.14.1 File-I/O Overview

The *File I/O remote protocol extension* (short: File-I/O) allows the target to use the host's file system and console I/O to perform various system calls. System calls on the target system are translated into a remote protocol packet to the host system, which then performs the needed actions and returns a response packet to the target system. This simulates file system operations even on targets that lack file systems.

The protocol is defined to be independent of both the host and target systems. It uses its own internal representation of datatypes and values. Both GDB and the target's GDB stub are responsible for translating the system-dependent value representations into the internal protocol representations when data is transmitted.

The communication is synchronous. A system call is possible only when GDB is waiting for a response from the 'C', 'c', 'S' or 's' packets. While GDB handles the request for a system call, the target is stopped to allow deterministic access to the target's memory. Therefore File-I/O is not interruptible by target signals. On the other hand, it is possible to interrupt File-I/O by a user interrupt ('Ctrl-C') within GDB.

The target's request to perform a host system call does not finish the latest 'C', 'c', 'S' or 's' action. That means, after finishing the system call, the target returns to continuing the previous activity (continue, step). No additional continue or step request from GDB is required.

```
(gdb) continue
<- target requests 'system call X'
target is stopped, GDB executes system call
-> GDB returns result
... target continues, GDB returns to wait for the target
<- target hits breakpoint and sends a Txx packet
```

The protocol only supports I/O on the console and to regular files on the host file system. Character or block special devices, pipes, named pipes, sockets or any other communication method on the host system are not supported by this protocol.

File I/O is not supported in non-stop mode.

E.14.2 Protocol Basics

The File-I/O protocol uses the F packet as the request as well as reply packet. Since a File-I/O system call can only occur when GDB is waiting for a response from the continuing or stepping target, the File-I/O request is a reply that GDB has to expect as a result of a previous 'C', 'c', 'S' or 's' packet. This F packet contains all information needed to allow GDB to call the appropriate host system call:

- A unique identifier for the requested system call.
- All parameters to the system call. Pointers are given as addresses in the target memory address space. Pointers to strings are given as pointer/length pair. Numerical values are given as they are. Numerical control flags are given in a protocol-specific representation.

At this point, GDB has to perform the following actions.

- If the parameters include pointer values to data needed as input to a system call, GDB requests this data from the target with a standard m packet request. This additional

communication has to be expected by the target implementation and is handled as any other *m* packet.

- GDB translates all value from protocol representation to host representation as needed. Datatypes are coerced into the host types.
- GDB calls the system call.
- It then coerces datatypes back to protocol representation.
- If the system call is expected to return data in buffer space specified by pointer parameters to the call, the data is transmitted to the target using a *M* or *X* packet. This packet has to be expected by the target implementation and is handled as any other *M* or *X* packet.

Eventually GDB replies with another *F* packet which contains all necessary information for the target to continue. This at least contains

- Return value.
- *errno*, if has been changed by the system call.
- “Ctrl-C” flag.

After having done the needed type and value coercion, the target continues the latest continue or step action.

E.14.3 The F Request Packet

The *F* request packet has the following format:

‘Fcall-id,parameter...’

call-id is the identifier to indicate the host system call to be called. This is just the name of the function.

parameter... are the parameters to the system call. Parameters are hexadecimal integer values, either the actual values in case of scalar datatypes, pointers to target buffer space in case of compound datatypes and unspecified memory areas, or pointer/length pairs in case of string parameters. These are appended to the *call-id* as a comma-delimited list. All values are transmitted in ASCII string representation, pointer/length pairs separated by a slash.

E.14.4 The F Reply Packet

The *F* reply packet has the following format:

‘Fretcode,errno,Ctrl-C flag;call-specific attachment’

retcode is the return code of the system call as hexadecimal value.

errno is the *errno* set by the call, in protocol-specific representation. This parameter can be omitted if the call was successful.

Ctrl-C flag is only sent if the user requested a break. In this case, *errno* must be sent as well, even if the call was successful. The *Ctrl-C flag* itself consists of the character ‘C’:

F0,0,C

or, if the call was interrupted before the host call has been performed:

F-1,4,C

assuming 4 is the protocol-specific representation of *EINTR*.

E.14.5 The ‘Ctrl-C’ Message

If the ‘Ctrl-C’ flag is set in the GDB reply packet (see Section E.14.4 [The F Reply Packet], page 855), the target should behave as if it had gotten a break message. The meaning for the target is “system call interrupted by SIGINT”. Consequently, the target should actually stop (as with a break message) and return to GDB with a T02 packet.

It’s important for the target to know in which state the system call was interrupted. There are two possible cases:

- The system call hasn’t been performed on the host yet.
- The system call on the host has been finished.

These two states can be distinguished by the target by the value of the returned `errno`. If it’s the protocol representation of `EINTR`, the system call hasn’t been performed. This is equivalent to the `EINTR` handling on POSIX systems. In any other case, the target may presume that the system call has been finished — successfully or not — and should behave as if the break message arrived right after the system call.

GDB must behave reliably. If the system call has not been called yet, GDB may send the F reply immediately, setting `EINTR` as `errno` in the packet. If the system call on the host has been finished before the user requests a break, the full action must be finished by GDB. This requires sending M or X packets as necessary. The F packet may only be sent when either nothing has happened or the full action has been completed.

E.14.6 Console I/O

By default and if not explicitly closed by the target system, the file descriptors 0, 1 and 2 are connected to the GDB console. Output on the GDB console is handled as any other file output operation (`write(1, ...)` or `write(2, ...)`). Console input is handled by GDB so that after the target read request from file descriptor 0 all following typing is buffered until either one of the following conditions is met:

- The user types `Ctrl-c`. The behavior is as explained above, and the `read` system call is treated as finished.
- The user presses `RET`. This is treated as end of input with a trailing newline.
- The user types `Ctrl-d`. This is treated as end of input. No trailing character (neither newline nor ‘Ctrl-D’) is appended to the input.

If the user has typed more characters than fit in the buffer given to the `read` call, the trailing characters are buffered in GDB until either another `read(0, ...)` is requested by the target, or debugging is stopped at the user’s request.

E.14.7 List of Supported Calls

open

Synopsis:

```
int open(const char *pathname, int flags);
int open(const char *pathname, int flags, mode_t mode);
```

Request: ‘Fopen,pathptr/len,flags,mode’

flags is the bitwise OR of the following values:

<code>O_CREAT</code>	If the file does not exist it will be created. The host rules apply as far as file ownership and time stamps are concerned.
<code>O_EXCL</code>	When used with <code>O_CREAT</code> , if the file already exists it is an error and <code>open()</code> fails.
<code>O_TRUNC</code>	If the file already exists and the open mode allows writing (<code>O_RDWR</code> or <code>O_WRONLY</code> is given) it will be truncated to zero length.
<code>O_APPEND</code>	The file is opened in append mode.
<code>O_RDONLY</code>	The file is opened for reading only.
<code>O_WRONLY</code>	The file is opened for writing only.
<code>O_RDWR</code>	The file is opened for reading and writing.

Other bits are silently ignored.

mode is the bitwise OR of the following values:

<code>S_IRUSR</code>	User has read permission.
<code>S_IWUSR</code>	User has write permission.
<code>S_IRGRP</code>	Group has read permission.
<code>S_IWGRP</code>	Group has write permission.
<code>S_IROTH</code>	Others have read permission.
<code>S_IWOTH</code>	Others have write permission.

Other bits are silently ignored.

Return value:

`open` returns the new file descriptor or -1 if an error occurred.

Errors:

<code>EEXIST</code>	<i>pathname</i> already exists and <code>O_CREAT</code> and <code>O_EXCL</code> were used.
<code>EISDIR</code>	<i>pathname</i> refers to a directory.
<code>EACCES</code>	The requested access is not allowed.
<code>ENAMETOOLONG</code>	<i>pathname</i> was too long.
<code>ENOENT</code>	A directory component in <i>pathname</i> does not exist.
<code>ENODEV</code>	<i>pathname</i> refers to a device, pipe, named pipe or socket.
<code>EROFS</code>	<i>pathname</i> refers to a file on a read-only filesystem and write access was requested.
<code>EFAULT</code>	<i>pathname</i> is an invalid pointer value.
<code>ENOSPC</code>	No space on device to create the file.
<code>EMFILE</code>	The process already has the maximum number of files open.

ENFILE The limit on the total number of files open on the system has been reached.

EINTR The call was interrupted by the user.

close

Synopsis:

```
int close(int fd);
```

Request: ‘Fclose,*fd*’

Return value:

`close` returns zero on success, or -1 if an error occurred.

Errors:

EBADF *fd* isn’t a valid open file descriptor.

EINTR The call was interrupted by the user.

read

Synopsis:

```
int read(int fd, void *buf, unsigned int count);
```

Request: ‘Fread,*fd,bufptr,count*’

Return value:

On success, the number of bytes read is returned. Zero indicates end of file. If count is zero, read returns zero as well. On error, -1 is returned.

Errors:

EBADF *fd* is not a valid file descriptor or is not open for reading.

EFAULT *bufptr* is an invalid pointer value.

EINTR The call was interrupted by the user.

write

Synopsis:

```
int write(int fd, const void *buf, unsigned int count);
```

Request: ‘Fwrite,*fd,bufptr,count*’

Return value:

On success, the number of bytes written are returned. Zero indicates nothing was written. On error, -1 is returned.

Errors:

EBADF *fd* is not a valid file descriptor or is not open for writing.

EFAULT *bufptr* is an invalid pointer value.

EFBIG An attempt was made to write a file that exceeds the host-specific maximum file size allowed.

ENOSPC No space on device to write the data.

EINTR The call was interrupted by the user.

lseek

Synopsis:

```
long lseek (int fd, long offset, int flag);
```

Request: `'Flseek,fd,offset,flag'`

flag is one of:

SEEK_SET The offset is set to *offset* bytes.

SEEK_CUR The offset is set to its current location plus *offset* bytes.

SEEK_END The offset is set to the size of the file plus *offset* bytes.

Return value:

On success, the resulting unsigned offset in bytes from the beginning of the file is returned. Otherwise, a value of -1 is returned.

Errors:

EBADF *fd* is not a valid open file descriptor.

ESPIPE *fd* is associated with the GDB console.

EINVAL *flag* is not a proper value.

EINTR The call was interrupted by the user.

rename

Synopsis:

```
int rename(const char *oldpath, const char *newpath);
```

Request: `'Frename,oldpathptr/len,newpathptr/len'`

Return value:

On success, zero is returned. On error, -1 is returned.

Errors:

EISDIR *newpath* is an existing directory, but *oldpath* is not a directory.

EEXIST *newpath* is a non-empty directory.

EBUSY *oldpath* or *newpath* is a directory that is in use by some process.

EINVAL An attempt was made to make a directory a subdirectory of itself.

ENOTDIR A component used as a directory in *oldpath* or new path is not a directory. Or *oldpath* is a directory and *newpath* exists but is not a directory.

EFAULT *oldpathptr* or *newpathptr* are invalid pointer values.

EACCES No access to the file or the path of the file.

ENAMETOOLONG

oldpath or *newpath* was too long.

ENOENT A directory component in *oldpath* or *newpath* does not exist.

EROFS	The file is on a read-only filesystem.
ENOSPC	The device containing the file has no room for the new directory entry.
EINTR	The call was interrupted by the user.

unlink

Synopsis:

```
int unlink(const char *pathname);
```

Request: ‘Funlink,*pathnameptr/len*’

Return value:

On success, zero is returned. On error, -1 is returned.

Errors:

EACCES	No access to the file or the path of the file.
EPERM	The system does not allow unlinking of directories.
EBUSY	The file <i>pathname</i> cannot be unlinked because it's being used by another process.
EFAULT	<i>pathnameptr</i> is an invalid pointer value.
ENAMETOOLONG	<i>pathname</i> was too long.
ENOENT	A directory component in <i>pathname</i> does not exist.
ENOTDIR	A component of the path is not a directory.
EROFS	The file is on a read-only filesystem.
EINTR	The call was interrupted by the user.

stat/fstat

Synopsis:

```
int stat(const char *pathname, struct stat *buf);
int fstat(int fd, struct stat *buf);
```

Request: ‘Fstat,*pathnameptr/len,bufptr*’
‘Ffstat,*fd,bufptr*’

Return value:

On success, zero is returned. On error, -1 is returned.

Errors:

EBADF	<i>fd</i> is not a valid open file.
ENOENT	A directory component in <i>pathname</i> does not exist or the path is an empty string.
ENOTDIR	A component of the path is not a directory.

EFAULT *pathnameptr* is an invalid pointer value.
 EACCES No access to the file or the path of the file.
 ENAMETOOLONG
 pathname was too long.
 EINTR The call was interrupted by the user.

gettimeofday

Synopsis:

```
int gettimeofday(struct timeval *tv, void *tz);
```

Request: ‘Fgettimeofday,*tvptr*,*tzptr*’

Return value:

On success, 0 is returned, -1 otherwise.

Errors:

EINVAL *tz* is a non-NULL pointer.
 EFAULT *tvptr* and/or *tzptr* is an invalid pointer value.

isatty

Synopsis:

```
int isatty(int fd);
```

Request: ‘Fisatty,*fd*’

Return value:

Returns 1 if *fd* refers to the GDB console, 0 otherwise.

Errors:

EINTR The call was interrupted by the user.

Note that the **isatty** call is treated as a special case: it returns 1 to the target if the file descriptor is attached to the GDB console, 0 otherwise. Implementing through system calls would require implementing `ioctl` and would be more complex than needed.

system

Synopsis:

```
int system(const char *command);
```

Request: ‘Fsystem,*commandptr/len*’

Return value:

If *len* is zero, the return value indicates whether a shell is available. A zero return value indicates a shell is not available. For non-zero *len*, the value returned is -1 on error and the return status of the command otherwise. Only the exit status of the command is returned, which is extracted from the host’s **system** return value by calling `WEXITSTATUS(retval)`. In case `/bin/sh` could not be executed, 127 is returned.

Errors:

EINTR The call was interrupted by the user.

GDB takes over the full task of calling the necessary host calls to perform the **system** call. The return value of **system** on the host is simplified before it's returned to the target. Any termination signal information from the child process is discarded, and the return value consists entirely of the exit status of the called command.

Due to security concerns, the **system** call is by default refused by GDB. The user has to allow this call explicitly with the **set remote system-call-allowed 1** command.

set remote system-call-allowed

Control whether to allow the **system** calls in the File I/O protocol for the remote target. The default is zero (disabled).

show remote system-call-allowed

Show whether the **system** calls are allowed in the File I/O protocol.

E.14.8 Protocol-specific Representation of Datatypes

Integral Datatypes

The integral datatypes used in the system calls are **int**, **unsigned int**, **long**, **unsigned long**, **mode_t**, and **time_t**.

int, **unsigned int**, **mode_t** and **time_t** are implemented as 32 bit values in this protocol.

long and **unsigned long** are implemented as 64 bit types.

See [Limits], page 865, for corresponding MIN and MAX values (similar to those in **limits.h**) to allow range checking on host and target.

time_t datatypes are defined as seconds since the Epoch.

All integral datatypes transferred as part of a memory read or write of a structured datatype e.g. a **struct stat** have to be given in big endian byte order.

Pointer Values

Pointers to target data are transmitted as they are. An exception is made for pointers to buffers for which the length isn't transmitted as part of the function call, namely strings. Strings are transmitted as a pointer/length pair, both as hex values, e.g.

1aaf/12

which is a pointer to data of length 18 bytes at position 0x1aaf. The length is defined as the full string length in bytes, including the trailing null byte. For example, the string "hello world" at address 0x123456 is transmitted as

123456/d

Memory Transfer

Structured data which is transferred using a memory read or write (for example, a **struct stat**) is expected to be in a protocol-specific format with all scalar multibyte datatypes being big endian. Translation to this representation needs to be done both by the target before the F packet is sent, and by GDB before it transfers memory to the target. Transferred pointers to structured data should point to the already-coerced data at any time.

struct stat

The buffer of type `struct stat` used by the target and GDB is defined as follows:

```

struct stat {
    unsigned int  st_dev;      /* device */
    unsigned int  st_ino;     /* inode */
    mode_t        st_mode;    /* protection */
    unsigned int  st_nlink;   /* number of hard links */
    unsigned int  st_uid;     /* user ID of owner */
    unsigned int  st_gid;     /* group ID of owner */
    unsigned int  st_rdev;    /* device type (if inode device) */
    unsigned long st_size;    /* total size, in bytes */
    unsigned long st_blksize; /* blocksize for filesystem I/O */
    unsigned long st_blocks;  /* number of blocks allocated */
    time_t        st_atime;   /* time of last access */
    time_t        st_mtime;   /* time of last modification */
    time_t        st_ctime;   /* time of last change */
};

```

The integral datatypes conform to the definitions given in the appropriate section (see [Integral Datatypes], page 862, for details) so this structure is of size 64 bytes.

The values of several fields have a restricted meaning and/or range of values.

st_dev A value of 0 represents a file, 1 the console.

st_ino No valid meaning for the target. Transmitted unchanged.

st_mode Valid mode bits are described in Section E.14.9 [Constants], page 864. Any other bits have currently no meaning for the target.

st_uid

st_gid

st_rdev No valid meaning for the target. Transmitted unchanged.

st_atime

st_mtime

st_ctime These values have a host and file system dependent accuracy. Especially on Windows hosts, the file system may not support exact timing values.

The target gets a `struct stat` of the above representation and is responsible for coercing it to the target representation before continuing.

Note that due to size differences between the host, target, and protocol representations of `struct stat` members, these members could eventually get truncated on the target.

struct timeval

The buffer of type `struct timeval` used by the File-I/O protocol is defined as follows:

```

struct timeval {
    time_t tv_sec; /* second */
    long   tv_usec; /* microsecond */
};

```

The integral datatypes conform to the definitions given in the appropriate section (see [Integral Datatypes], page 862, for details) so this structure is of size 8 bytes.

E.14.9 Constants

The following values are used for the constants inside of the protocol. GDB and target are responsible for translating these values before and after the call as needed.

Open Flags

All values are given in hexadecimal representation.

O_RDONLY	0x0
O_WRONLY	0x1
O_RDWR	0x2
O_APPEND	0x8
O_CREAT	0x200
O_TRUNC	0x400
O_EXCL	0x800

mode_t Values

All values are given in octal representation.

S_IFREG	0100000
S_IFDIR	040000
S_IRUSR	0400
S_IWUSR	0200
S_IXUSR	0100
S_IRGRP	040
S_IWGRP	020
S_IXGRP	010
S_IROTH	04
S_IWOTH	02
S_IXOTH	01

Errno Values

All values are given in decimal representation.

EPERM	1
ENOENT	2
EINTR	4
EIO	5
EBADF	9
EACCES	13
EFAULT	14
EBUSY	16
EEXIST	17
ENODEV	19
ENOTDIR	20
EISDIR	21
EINVAL	22
ENFILE	23
EMFILE	24
EFBIG	27
ENOSPC	28
ESPIPE	29
EROFS	30
ENOSYS	88
ENAMETOOLONG	91
EUNKNOWN	9999

EUNKNOWN is used as a fallback error value if a host system returns any error value not in the list of supported error numbers.

Lseek Flags

SEEK_SET	0
SEEK_CUR	1
SEEK_END	2

Limits

All values are given in decimal representation.

INT_MIN	-2147483648
INT_MAX	2147483647
UINT_MAX	4294967295
LONG_MIN	-9223372036854775808
LONG_MAX	9223372036854775807
ULONG_MAX	18446744073709551615

E.14.10 File-I/O Examples

Example sequence of a write call, file descriptor 3, buffer is at target address 0x1234, 6 bytes should be written:

```
<- Fwrite,3,1234,6
request memory read from target
-> m1234,6
<- XXXXXX
return "6 bytes written"
-> F6
```

Example sequence of a read call, file descriptor 3, buffer is at target address 0x1234, 6 bytes should be read:

```
<- Fread,3,1234,6
request memory write to target
-> X1234,6:XXXXXX
return "6 bytes read"
-> F6
```

Example sequence of a read call, call fails on the host due to invalid file descriptor (EBADF):

```
<- Fread,3,1234,6
-> F-1,9
```

Example sequence of a read call, user presses *Ctrl-c* before syscall on host is called:

```
<- Fread,3,1234,6
-> F-1,4,C
<- T02
```

Example sequence of a read call, user presses *Ctrl-c* after syscall on host is called:

```
<- Fread,3,1234,6
-> X1234,6:XXXXXX
<- T02
```

E.15 Library List Format

On some platforms, a dynamic loader (e.g. `ld.so`) runs in the same process as your application to manage libraries. In this case, GDB can use the loader's symbol table and normal memory operations to maintain a list of shared libraries. On other platforms, the operating system manages loaded libraries. GDB can not retrieve the list of currently loaded libraries

through memory operations, so it uses the ‘qXfer:libraries:read’ packet (see [qXfer library list read], page 835) instead. The remote stub queries the target’s operating system and reports which libraries are loaded.

The ‘qXfer:libraries:read’ packet returns an XML document which lists loaded libraries and their offsets. Each library has an associated name and one or more segment or section base addresses, which report where the library was loaded in memory.

For the common case of libraries that are fully linked binaries, the library should have a list of segments. If the target supports dynamic linking of a relocatable object file, its library XML element should instead include a list of allocated sections. The segment or section bases are start addresses, not relocation offsets; they do not depend on the library’s link-time base addresses.

GDB must be linked with the Expat library to support XML library lists. See [Expat], page 768.

A simple memory map, with one loaded library relocated by a single offset, looks like this:

```
<library-list>
  <library name="/lib/libc.so.6">
    <segment address="0x10000000"/>
  </library>
</library-list>
```

Another simple memory map, with one loaded library with three allocated sections (.text, .data, .bss), looks like this:

```
<library-list>
  <library name="sharedlib.o">
    <section address="0x10000000"/>
    <section address="0x20000000"/>
    <section address="0x30000000"/>
  </library>
</library-list>
```

The format of a library list is described by this DTD:

```
<!-- library-list: Root element with versioning -->
<!ELEMENT library-list (library)*>
<!ATTLIST library-list version CDATA #FIXED "1.0">
<!ELEMENT library (segment*, section*)>
<!ATTLIST library name CDATA #REQUIRED>
<!ELEMENT segment EMPTY>
<!ATTLIST segment address CDATA #REQUIRED>
<!ELEMENT section EMPTY>
<!ATTLIST section address CDATA #REQUIRED>
```

In addition, segments and section descriptors cannot be mixed within a single library element, and you must supply at least one segment or section for each library.

E.16 Library List Format for SVR4 Targets

On SVR4 platforms GDB can use the symbol table of a dynamic loader (e.g. `ld.so`) and normal memory operations to maintain a list of shared libraries. Still a special library list provided by this packet is more efficient for the GDB remote protocol.

The ‘qXfer:libraries-svr4:read’ packet returns an XML document which lists loaded libraries and their SVR4 linker parameters. For each library on SVR4 target, the following parameters are reported:

- **name**, the absolute file name from the **l_name** field of **struct link_map**.
- **lm** with address of **struct link_map** used for TLS (Thread Local Storage) access.
- **l_addr**, the displacement as read from the field **l_addr** of **struct link_map**. For prelinked libraries this is not an absolute memory address. It is a displacement of absolute memory address against address the file was prelinked to during the library load.
- **l_ld**, which is memory address of the **PT_DYNAMIC** segment
- **lmid**, which is an identifier for a linker namespace, such as the memory address of the **r_debug** object that contains this namespace's load map or the namespace identifier returned by **dldinfo** (3).

Additionally the single **main-lm** attribute specifies address of **struct link_map** used for the main executable. This parameter is used for TLS access and its presence is optional.

GDB must be linked with the Expat library to support XML SVR4 library lists. See [Expat], page 768.

A simple memory map, with two loaded libraries (which do not use prelink), looks like this:

```
<library-list-svr4 version="1.0" main-lm="0xe4f8f8">
  <library name="/lib/ld-linux.so.2" lm="0xe4f51c" l_addr="0xe2d000"
    l_ld="0xe4eefc" lmid="0xffffe0"/>
  <library name="/lib/libc.so.6" lm="0xe4fbe8" l_addr="0x154000"
    l_ld="0x152350" lmid="0xffffe0"/>
</library-list-svr4>
```

The format of an SVR4 library list is described by this DTD:

```
<!-- library-list-svr4: Root element with versioning -->
<!ELEMENT library-list-svr4 (library)*>
<!ATTLIST library-list-svr4 version CDATA #FIXED "1.0">
<!ATTLIST library-list-svr4 main-lm CDATA #IMPLIED>
<!ELEMENT library EMPTY>
<!ATTLIST library
  name CDATA #REQUIRED>
<!ATTLIST library
  lm CDATA #REQUIRED>
<!ATTLIST library
  l_addr CDATA #REQUIRED>
<!ATTLIST library
  l_ld CDATA #REQUIRED>
<!ATTLIST library
  lmid CDATA #IMPLIED>
```

E.17 Memory Map Format

To be able to write into flash memory, GDB needs to obtain a memory map from the target. This section describes the format of the memory map.

The memory map is obtained using the ‘**qXfer:memory-map:read**’ (see [qXfer memory map read], page 836) packet and is an XML document that lists memory regions.

GDB must be linked with the Expat library to support XML memory maps. See [Expat], page 768.

The top-level structure of the document is shown below:

```
<?xml version="1.0"?>
<!DOCTYPE memory-map
  PUBLIC "-//IDN gnu.org//DTD GDB Memory Map V1.0//EN"
  "http://sourceware.org/gdb/gdb-memory-map.dtd">
<memory-map>
```

```
    region...
  </memory-map>
```

Each region can be either:

- A region of RAM starting at *addr* and extending for *length* bytes from there:

```
<memory type="ram" start="addr" length="length"/>
```

- A region of read-only memory:

```
<memory type="rom" start="addr" length="length"/>
```

- A region of flash memory, with erasure blocks *blocksize* bytes in length:

```
<memory type="flash" start="addr" length="length">
  <property name="blocksize">blocksize</property>
</memory>
```

Regions must not overlap. GDB assumes that areas of memory not covered by the memory map are RAM, and uses the ordinary ‘M’ and ‘X’ packets to write to addresses in such ranges.

The formal DTD for memory map format is given below:

```
<!-- ..... -->
<!-- Memory Map XML DTD ..... -->
<!-- File: memory-map.dtd ..... -->
<!-- ..... -->
<!-- memory-map.dtd -->
<!-- memory-map: Root element with versioning -->
<!ELEMENT memory-map (memory)*>
<!ATTLIST memory-map    version CDATA    #FIXED    "1.0.0">
<!ELEMENT memory (property)*>
<!-- memory: Specifies a memory region,
      and its type, or device. -->
<!ATTLIST memory        type      (ram|rom|flash) #REQUIRED
                        start    CDATA    #REQUIRED
                        length   CDATA    #REQUIRED>
<!-- property: Generic attribute tag -->
<!ELEMENT property (#PCDATA | property)*>
<!ATTLIST property      name      (blocksize) #REQUIRED>
```

E.18 Thread List Format

To efficiently update the list of threads and their attributes, GDB issues the ‘qXfer:threads:read’ packet (see [qXfer threads read], page 836) and obtains the XML document with the following structure:

```
<?xml version="1.0"?>
<threads>
  <thread id="id" core="0" name="name" id_str="Thread 12.34" handle="1a2b3c">
    ... description ...
  </thread>
</threads>
```

Each ‘thread’ element must have the ‘id’ attribute that identifies the thread (see [thread-id syntax], page 797). The ‘core’ attribute, if present, specifies which processor core the thread was last executing on. The ‘name’ attribute, if present, specifies the human-readable name of the thread. The content of the ‘thread’ element is interpreted as human-readable auxiliary information. The ‘id_str’ attribute, if present, specifies what GDB should print as the target ID of the thread (e.g. in the ‘info threads’ command or when switching to the thread). The ‘handle’ attribute, if present, is a hex encoded representation of the thread handle.

E.19 Traceframe Info Format

To be able to know which objects in the inferior can be examined when inspecting a tracepoint hit, GDB needs to obtain the list of memory ranges, registers and trace state variables that have been collected in a traceframe.

This list is obtained using the ‘qXfer:traceframe-info:read’ (see [qXfer traceframe info read], page 836) packet and is an XML document.

GDB must be linked with the Expat library to support XML traceframe info discovery. See [Expat], page 768.

The top-level structure of the document is shown below:

```
<?xml version="1.0"?>
<!DOCTYPE traceframe-info
    PUBLIC "-//IDN gnu.org//DTD GDB Memory Map V1.0//EN"
    "http://sourceware.org/gdb/gdb-traceframe-info.dtd">
<traceframe-info>
  block...
</traceframe-info>
```

Each traceframe block can be either:

- A region of collected memory starting at *addr* and extending for *length* bytes from there:

```
<memory start="addr" length="length"/>
```

- A block indicating trace state variable numbered *number* has been collected:

```
<tvar id="number"/>
```

The formal DTD for the traceframe info format is given below:

```
<!ELEMENT traceframe-info (memory | tvar)* >
<!ATTLIST traceframe-info version CDATA #FIXED "1.0">

<!ELEMENT memory EMPTY>
<!ATTLIST memory
  start CDATA #REQUIRED
  length CDATA #REQUIRED>

<!ELEMENT tvar>
<!ATTLIST tvar
  id CDATA #REQUIRED>
```

E.20 Branch Trace Format

In order to display the branch trace of an inferior thread, GDB needs to obtain the list of branches. This list is represented as list of sequential code blocks that are connected via branches. The code in each block has been executed sequentially.

This list is obtained using the ‘qXfer:btrace:read’ (see [qXfer btrace read], page 834) packet and is an XML document.

GDB must be linked with the Expat library to support XML traceframe info discovery. See [Expat], page 768.

The top-level structure of the document is shown below:

```
<?xml version="1.0"?>
<!DOCTYPE btrace
    PUBLIC "-//IDN gnu.org//DTD GDB Branch Trace V1.0//EN"
    "http://sourceware.org/gdb/gdb-btrace.dtd">
<btrace>
  block...
```

```
</btrace>
```

- A block of sequentially executed instructions starting at *begin* and ending at *end*:

```
<block begin="begin" end="end"/>
```

The formal DTD for the branch trace format is given below:

```
<!ELEMENT btrace (block* | pt) >
<!ATTLIST btrace version CDATA #FIXED "1.0">

<!ELEMENT block EMPTY>
<!ATTLIST block begin CDATA #REQUIRED
end CDATA #REQUIRED>

<!ELEMENT pt (pt-config?, raw?)>

<!ELEMENT pt-config (cpu?)>

<!ELEMENT cpu EMPTY>
<!ATTLIST cpu vendor CDATA #REQUIRED
family CDATA #REQUIRED
model CDATA #REQUIRED
stepping CDATA #REQUIRED>

<!ELEMENT raw (#PCDATA)>
```

E.21 Branch Trace Configuration Format

For each inferior thread, GDB can obtain the branch trace configuration using the ‘qXfer:btrace-conf:read’ (see [qXfer btrace-conf read], page 834) packet.

The configuration describes the branch trace format and configuration settings for that format. The following information is described:

- | | |
|-------------|--|
| bts | This thread uses the <i>Branch Trace Store</i> (BTS) format. |
| size | The size of the BTS ring buffer in bytes. |
| pt | This thread uses the <i>Intel Processor Trace</i> (Intel PT) format. |
| size | The size of the Intel PT ring buffer in bytes. |

GDB must be linked with the Expat library to support XML branch trace configuration discovery. See [Expat], page 768.

The formal DTD for the branch trace configuration format is given below:

```
<!ELEMENT btrace-conf (bts?, pt?)>
<!ATTLIST btrace-conf version CDATA #FIXED "1.0">

<!ELEMENT bts EMPTY>
<!ATTLIST bts size CDATA #IMPLIED>

<!ELEMENT pt EMPTY>
<!ATTLIST pt size CDATA #IMPLIED>
<!ATTLIST pt ptwrite (yes | no) #IMPLIED>
<!ATTLIST pt event-tracing (yes | no) #IMPLIED>
```

Appendix F The GDB Agent Expression Mechanism

In some applications, it is not feasible for the debugger to interrupt the program's execution long enough for the developer to learn anything helpful about its behavior. If the program's correctness depends on its real-time behavior, delays introduced by a debugger might cause the program to fail, even when the code itself is correct. It is useful to be able to observe the program's behavior without interrupting it.

Using GDB's `trace` and `collect` commands, the user can specify locations in the program, and arbitrary expressions to evaluate when those locations are reached. Later, using the `tfind` command, she can examine the values those expressions had when the program hit the trace points. The expressions may also denote objects in memory — structures or arrays, for example — whose values GDB should record; while visiting a particular tracepoint, the user may inspect those objects as if they were in memory at that moment. However, because GDB records these values without interacting with the user, it can do so quickly and unobtrusively, hopefully not disturbing the program's behavior.

When GDB is debugging a remote target, the GDB *agent* code running on the target computes the values of the expressions itself. To avoid having a full symbolic expression evaluator on the agent, GDB translates expressions in the source language into a simpler bytecode language, and then sends the bytecode to the agent; the agent then executes the bytecode, and records the values for GDB to retrieve later.

The bytecode language is simple; there are forty-odd opcodes, the bulk of which are the usual vocabulary of C operands (addition, subtraction, shifts, and so on) and various sizes of literals and memory reference operations. The bytecode interpreter operates strictly on machine-level values — various sizes of integers and floating point numbers — and requires no information about types or symbols; thus, the interpreter's internal data structures are simple, and each bytecode requires only a few native machine instructions to implement it. The interpreter is small, and strict limits on the memory and time required to evaluate an expression are easy to determine, making it suitable for use by the debugging agent in real-time applications.

F.1 General Bytecode Design

The agent represents bytecode expressions as an array of bytes. Each instruction is one byte long (thus the term *bytecode*). Some instructions are followed by operand bytes; for example, the `goto` instruction is followed by a destination for the jump.

The bytecode interpreter is a stack-based machine; most instructions pop their operands off the stack, perform some operation, and push the result back on the stack for the next instruction to consume. Each element of the stack may contain either a integer or a floating point value; these values are as many bits wide as the largest integer that can be directly manipulated in the source language. Stack elements carry no record of their type; bytecode could push a value as an integer, then pop it as a floating point value. However, GDB will not generate code which does this. In C, one might define the type of a stack element as follows:

```
union agent_val {
    LONGEST l;
    DOUBLEST d;
```

```
};
```

where `LONGEST` and `DOUBLEST` are `typedef` names for the largest integer and floating point types on the machine.

By the time the bytecode interpreter reaches the end of the expression, the value of the expression should be the only value left on the stack. For tracing applications, `trace` bytecodes in the expression will have recorded the necessary data, and the value on the stack may be discarded. For other applications, like conditional breakpoints, the value may be useful.

Separate from the stack, the interpreter has two registers:

<code>pc</code>	The address of the next bytecode to execute.
<code>start</code>	The address of the start of the bytecode expression, necessary for interpreting the <code>goto</code> and <code>if_goto</code> instructions.

Neither of these registers is directly visible to the bytecode language itself, but they are useful for defining the meanings of the bytecode operations.

There are no instructions to perform side effects on the running program, or call the program's functions; we assume that these expressions are only used for unobtrusive debugging, not for patching the running code.

Most bytecode instructions do not distinguish between the various sizes of values, and operate on full-width values; the upper bits of the values are simply ignored, since they do not usually make a difference to the value computed. The exceptions to this rule are:

memory reference instructions (`refn`)

There are distinct instructions to fetch different word sizes from memory. Once on the stack, however, the values are treated as full-size integers. They may need to be sign-extended; the `ext` instruction exists for this purpose.

the sign-extension instruction (`ext n`)

These clearly need to know which portion of their operand is to be extended to occupy the full length of the word.

If the interpreter is unable to evaluate an expression completely for some reason (a memory location is inaccessible, or a divisor is zero, for example), we say that interpretation “terminates with an error”. This means that the problem is reported back to the interpreter's caller in some helpful way. In general, code using agent expressions should assume that they may attempt to divide by zero, fetch arbitrary memory locations, and misbehave in other ways.

Even complicated C expressions compile to a few bytecode instructions; for example, the expression `x + y * z` would typically produce code like the following, assuming that `x` and `y` live in registers, and `z` is a global variable holding a 32-bit `int`:

```
reg 1
reg 2
const32 address of z
ref32
ext 32
mul
add
```

end

In detail, these mean:

reg 1 Push the value of register 1 (presumably holding *x*) onto the stack.

reg 2 Push the value of register 2 (holding *y*).

const32 address of z
 Push the address of *z* onto the stack.

ref32 Fetch a 32-bit word from the address at the top of the stack; replace the address on the stack with the value. Thus, we replace the address of *z* with *z*'s value.

ext 32 Sign-extend the value on the top of the stack from 32 bits to full length. This is necessary because *z* is a signed integer.

mul Pop the top two numbers on the stack, multiply them, and push their product. Now the top of the stack contains the value of the expression *y * z*.

add Pop the top two numbers, add them, and push the sum. Now the top of the stack contains the value of *x + y * z*.

end Stop executing; the value left on the stack top is the value to be recorded.

F.2 Bytecode Descriptions

Each bytecode description has the following form:

add (0x02): *a b* $\Rightarrow$ *a+b*

Pop the top two stack items, *a* and *b*, as integers; push their sum, as an integer.

In this example, **add** is the name of the bytecode, and (0x02) is the one-byte value used to encode the bytecode, in hexadecimal. The phrase “*a b* $\Rightarrow$ *a+b*” shows the stack before and after the bytecode executes. Beforehand, the stack must contain at least two values, *a* and *b*; since the top of the stack is to the right, *b* is on the top of the stack, and *a* is underneath it. After execution, the bytecode will have popped *a* and *b* from the stack, and replaced them with a single value, *a+b*. There may be other values on the stack below those shown, but the bytecode affects only those shown.

Here is another example:

const8 (0x22) *n*: $\Rightarrow$ *n*

Push the 8-bit integer constant *n* on the stack, without sign extension.

In this example, the bytecode **const8** takes an operand *n* directly from the bytecode stream; the operand follows the **const8** bytecode itself. We write any such operands immediately after the name of the bytecode, before the colon, and describe the exact encoding of the operand in the bytecode stream in the body of the bytecode description.

For the **const8** bytecode, there are no stack items given before the $\Rightarrow$; this simply means that the bytecode consumes no values from the stack. If a bytecode consumes no values, or produces no values, the list on either side of the $\Rightarrow$ may be empty.

If a value is written as *a*, *b*, or *n*, then the bytecode treats it as an integer. If a value is written as *addr*, then the bytecode treats it as an address.

We do not fully describe the floating point operations here; although this design can be extended in a clean way to handle floating point values, they are not of immediate interest to the customer, so we avoid describing them, to save time.

float (0x01): $\Rightarrow$

Prefix for floating-point bytecodes. Not implemented yet.

add (0x02): $a\ b \Rightarrow a+b$

Pop two integers from the stack, and push their sum, as an integer.

sub (0x03): $a\ b \Rightarrow a-b$

Pop two integers from the stack, subtract the top value from the next-to-top value, and push the difference.

mul (0x04): $a\ b \Rightarrow a*b$

Pop two integers from the stack, multiply them, and push the product on the stack. Note that, when one multiplies two n -bit numbers yielding another n -bit number, it is irrelevant whether the numbers are signed or not; the results are the same.

div_signed (0x05): $a\ b \Rightarrow a/b$

Pop two signed integers from the stack; divide the next-to-top value by the top value, and push the quotient. If the divisor is zero, terminate with an error.

div_unsigned (0x06): $a\ b \Rightarrow a/b$

Pop two unsigned integers from the stack; divide the next-to-top value by the top value, and push the quotient. If the divisor is zero, terminate with an error.

rem_signed (0x07): $a\ b \Rightarrow a \text{ modulo } b$

Pop two signed integers from the stack; divide the next-to-top value by the top value, and push the remainder. If the divisor is zero, terminate with an error.

rem_unsigned (0x08): $a\ b \Rightarrow a \text{ modulo } b$

Pop two unsigned integers from the stack; divide the next-to-top value by the top value, and push the remainder. If the divisor is zero, terminate with an error.

lsh (0x09): $a\ b \Rightarrow a \ll b$

Pop two integers from the stack; let a be the next-to-top value, and b be the top value. Shift a left by b bits, and push the result.

rsh_signed (0x0a): $a\ b \Rightarrow (\text{signed})a \gg b$

Pop two integers from the stack; let a be the next-to-top value, and b be the top value. Shift a right by b bits, inserting copies of the top bit at the high end, and push the result.

rsh_unsigned (0x0b): $a\ b \Rightarrow a \gg b$

Pop two integers from the stack; let a be the next-to-top value, and b be the top value. Shift a right by b bits, inserting zero bits at the high end, and push the result.

log_not (0x0e): $a \Rightarrow !a$

Pop an integer from the stack; if it is zero, push the value one; otherwise, push the value zero.

bit_and (0x0f): $a\ b \Rightarrow a \& b$

Pop two integers from the stack, and push their bitwise **and**.

bit_or (0x10): $a\ b \Rightarrow a | b$

Pop two integers from the stack, and push their bitwise **or**.

bit_xor (0x11): $a\ b \Rightarrow a \wedge b$

Pop two integers from the stack, and push their bitwise exclusive-**or**.

bit_not (0x12): $a \Rightarrow \sim a$

Pop an integer from the stack, and push its bitwise complement.

equal (0x13): $a\ b \Rightarrow a = b$

Pop two integers from the stack; if they are equal, push the value one; otherwise, push the value zero.

less_signed (0x14): $a\ b \Rightarrow a < b$

Pop two signed integers from the stack; if the next-to-top value is less than the top value, push the value one; otherwise, push the value zero.

less_unsigned (0x15): $a\ b \Rightarrow a < b$

Pop two unsigned integers from the stack; if the next-to-top value is less than the top value, push the value one; otherwise, push the value zero.

ext (0x16) n : $a \Rightarrow a$, sign-extended from n bits

Pop an unsigned value from the stack; treating it as an n -bit twos-complement value, extend it to full length. This means that all bits to the left of bit $n-1$ (where the least significant bit is bit 0) are set to the value of bit $n-1$. Note that n may be larger than or equal to the width of the stack elements of the bytecode engine; in this case, the bytecode should have no effect.

The number of source bits to preserve, n , is encoded as a single byte unsigned integer following the **ext** bytecode.

zero_ext (0x2a) n : $a \Rightarrow a$, zero-extended from n bits

Pop an unsigned value from the stack; zero all but the bottom n bits.

The number of source bits to preserve, n , is encoded as a single byte unsigned integer following the **zero_ext** bytecode.

ref8 (0x17): $addr \Rightarrow a$

ref16 (0x18): $addr \Rightarrow a$

ref32 (0x19): $addr \Rightarrow a$

ref64 (0x1a): $addr \Rightarrow a$

Pop an address $addr$ from the stack. For bytecode **refn**, fetch an n -bit value from $addr$, using the natural target endianness. Push the fetched value as an unsigned integer.

Note that $addr$ may not be aligned in any particular way; the **refn** bytecodes should operate correctly for any address.

If attempting to access memory at $addr$ would cause a processor exception of some sort, terminate with an error.

`ref_float (0x1b):` $addr \Rightarrow d$
`ref_double (0x1c):` $addr \Rightarrow d$
`ref_long_double (0x1d):` $addr \Rightarrow d$
`l_to_d (0x1e):` $a \Rightarrow d$
`d_to_l (0x1f):` $d \Rightarrow a$

Not implemented yet.

`dup (0x28):` $a \Rightarrow a\ a$
 Push another copy of the stack's top element.

`swap (0x2b):` $a\ b \Rightarrow b\ a$
 Exchange the top two items on the stack.

`pop (0x29):` $a \Rightarrow$
 Discard the top value on the stack.

`pick (0x32)` $n: a \dots b \Rightarrow a \dots b\ a$
 Duplicate an item from the stack and push it on the top of the stack. n , a single byte, indicates the stack item to copy. If n is zero, this is the same as `dup`; if n is one, it copies the item under the top item, etc. If n exceeds the number of items on the stack, terminate with an error.

`rot (0x33):` $a\ b\ c \Rightarrow c\ a\ b$
 Rotate the top three items on the stack. The top item (c) becomes the third item, the next-to-top item (b) becomes the top item and the third item (a) from the top becomes the next-to-top item.

`if_goto (0x20)` $offset: a \Rightarrow$
 Pop an integer off the stack; if it is non-zero, branch to the given offset in the bytecode string. Otherwise, continue to the next instruction in the bytecode stream. In other words, if a is non-zero, set the `pc` register to `start + offset`. Thus, an offset of zero denotes the beginning of the expression.
 The *offset* is stored as a sixteen-bit unsigned value, stored immediately following the `if_goto` bytecode. It is always stored most significant byte first, regardless of the target's normal endianness. The offset is not guaranteed to fall at any particular alignment within the bytecode stream; thus, on machines where fetching a 16-bit on an unaligned address raises an exception, you should fetch the offset one byte at a time.

`goto (0x21)` $offset: \Rightarrow$
 Branch unconditionally to *offset*; in other words, set the `pc` register to `start + offset`.
 The offset is stored in the same way as for the `if_goto` bytecode.

`const8 (0x22)` $n: \Rightarrow n$
`const16 (0x23)` $n: \Rightarrow n$
`const32 (0x24)` $n: \Rightarrow n$
`const64 (0x25)` $n: \Rightarrow n$

Push the integer constant n on the stack, without sign extension. To produce a small negative value, push a small twos-complement value, and then sign-extend it using the `ext` bytecode.

The constant n is stored in the appropriate number of bytes following the `constb` bytecode. The constant n is always stored most significant byte first, regardless of the target's normal endianness. The constant is not guaranteed to fall at any particular alignment within the bytecode stream; thus, on machines where fetching a 16-bit on an unaligned address raises an exception, you should fetch n one byte at a time.

reg (0x26) $n: \Rightarrow a$

Push the value of register number n , without sign extension. The registers are numbered following GDB's conventions.

The register number n is encoded as a 16-bit unsigned integer immediately following the `reg` bytecode. It is always stored most significant byte first, regardless of the target's normal endianness. The register number is not guaranteed to fall at any particular alignment within the bytecode stream; thus, on machines where fetching a 16-bit on an unaligned address raises an exception, you should fetch the register number one byte at a time.

getv (0x2c) $n: \Rightarrow v$

Push the value of trace state variable number n , without sign extension.

The variable number n is encoded as a 16-bit unsigned integer immediately following the `getv` bytecode. It is always stored most significant byte first, regardless of the target's normal endianness. The variable number is not guaranteed to fall at any particular alignment within the bytecode stream; thus, on machines where fetching a 16-bit on an unaligned address raises an exception, you should fetch the register number one byte at a time.

setv (0x2d) $n: v \Rightarrow v$

Set trace state variable number n to the value found on the top of the stack. The stack is unchanged, so that the value is readily available if the assignment is part of a larger expression. The handling of n is as described for `getv`.

trace (0x0c): $addr\ size \Rightarrow$

Record the contents of the $size$ bytes at $addr$ in a trace buffer, for later retrieval by GDB.

trace_quick (0x0d) $size: addr \Rightarrow addr$

Record the contents of the $size$ bytes at $addr$ in a trace buffer, for later retrieval by GDB. $size$ is a single byte unsigned integer following the `trace` opcode.

This bytecode is equivalent to the sequence `dup const8 size trace`, but we provide it anyway to save space in bytecode strings.

trace16 (0x30) $size: addr \Rightarrow addr$

Identical to `trace_quick`, except that $size$ is a 16-bit big-endian unsigned integer, not a single byte. This should probably have been named `trace_quick16`, for consistency.

tracev (0x2e) $n: \Rightarrow a$

Record the value of trace state variable number n in the trace buffer. The handling of n is as described for `getv`.

tracenz (0x2f) *addr size* $\Rightarrow$

Record the bytes at *addr* in a trace buffer, for later retrieval by GDB. Stop at either the first zero byte, or when *size* bytes have been recorded, whichever occurs first.

printf (0x34) *numargs string* $\Rightarrow$

Do a formatted print, in the style of the C function **printf**). The value of *numargs* is the number of arguments to expect on the stack, while *string* is the format string, prefixed with a two-byte length. The last byte of the string must be zero, and is included in the length. The format string includes escaped sequences just as it appears in C source, so for instance the format string "**\t%d\n**" is six characters long, and the output will consist of a tab character, a decimal number, and a newline. At the top of the stack, above the values to be printed, this bytecode will pop a "function" and "channel". If the function is nonzero, then the target may treat it as a function and call it, passing the channel as a first argument, as with the C function **fprintf**. If the function is zero, then the target may simply call a standard formatted print function of its choice. In all, this bytecode pops $2 + \text{numargs}$ stack elements, and pushes nothing.

end (0x27): $\Rightarrow$

Stop executing bytecode; the result should be the top element of the stack. If the purpose of the expression was to compute an lvalue or a range of memory, then the next-to-top of the stack is the lvalue's address, and the top of the stack is the lvalue's size, in bytes.

F.3 Using Agent Expressions

Agent expressions can be used in several different ways by GDB, and the debugger can generate different bytecode sequences as appropriate.

One possibility is to do expression evaluation on the target rather than the host, such as for the conditional of a conditional tracepoint. In such a case, GDB compiles the source expression into a bytecode sequence that simply gets values from registers or memory, does arithmetic, and returns a result.

Another way to use agent expressions is for tracepoint data collection. GDB generates a different bytecode sequence for collection; in addition to bytecodes that do the calculation, GDB adds **trace** bytecodes to save the pieces of memory that were used.

- The user selects trace points in the program's code at which GDB should collect data.
- The user specifies expressions to evaluate at each trace point. These expressions may denote objects in memory, in which case those objects' contents are recorded as the program runs, or computed values, in which case the values themselves are recorded.
- GDB transmits the tracepoints and their associated expressions to the GDB agent, running on the debugging target.
- The agent arranges to be notified when a trace point is hit.
- When execution on the target reaches a trace point, the agent evaluates the expressions associated with that trace point, and records the resulting values and memory ranges.

- Later, when the user selects a given trace event and inspects the objects and expression values recorded, GDB talks to the agent to retrieve recorded data as necessary to meet the user's requests. If the user asks to see an object whose contents have not been recorded, GDB reports an error.

F.4 Varying Target Capabilities

Some targets don't support floating-point, and some would rather not have to deal with `long long` operations. Also, different targets will have different stack sizes, and different bytecode buffer lengths.

Thus, GDB needs a way to ask the target about itself. We haven't worked out the details yet, but in general, GDB should be able to send the target a packet asking it to describe itself. The reply should be a packet whose length is explicit, so we can add new information to the packet in future revisions of the agent, without confusing old versions of GDB, and it should contain a version number. It should contain at least the following information:

- whether floating point is supported
- whether `long long` is supported
- maximum acceptable size of bytecode stack
- maximum acceptable length of bytecode expressions
- which registers are actually available for collection
- whether the target supports disabled tracepoints

F.5 Rationale

Some of the design decisions apparent above are arguable.

What about stack overflow/underflow?

GDB should be able to query the target to discover its stack size. Given that information, GDB can determine at translation time whether a given expression will overflow the stack. But this spec isn't about what kinds of error-checking GDB ought to do.

Why are you doing everything in LONGEST?

Speed isn't important, but agent code size is; using LONGEST brings in a bunch of support code to do things like division, etc. So this is a serious concern.

First, note that you don't need different bytecodes for different operand sizes. You can generate code without *knowing* how big the stack elements actually are on the target. If the target only supports 32-bit ints, and you don't send any 64-bit bytecodes, everything just works. The observation here is that the MIPS and the Alpha have only fixed-size registers, and you can still get C's semantics even though most instructions only operate on full-sized words. You just need to make sure everything is properly sign-extended at the right times. So there is no need for 32- and 64-bit variants of the bytecodes. Just implement everything using the largest size you support.

GDB should certainly check to see what sizes the target supports, so the user can get an error earlier, rather than later. But this information is not necessary for correctness.

Why don't you have > or <= operators?

I want to keep the interpreter small, and we don't need them. We can combine the `less_` opcodes with `log_not`, and swap the order of the operands, yielding all four asymmetrical comparison operators. For example, `(x <= y)` is `! (x > y)`, which is `! (y < x)`.

Why do you have `log_not`?**Why do you have `ext`?****Why do you have `zero_ext`?**

These are all easily synthesized from other instructions, but I expect them to be used frequently, and they're simple, so I include them to keep bytecode strings short.

`log_not` is equivalent to `const8 0 equal`; it's used in half the relational operators.

`ext n` is equivalent to `const8 s-n lsh const8 s-n rsh_signed`, where *s* is the size of the stack elements; it follows `refm` and `reg` bytecodes when the value should be signed. See the next bulleted item.

`zero_ext n` is equivalent to `constm mask log_and`; it's used whenever we push the value of a register, because we can't assume the upper bits of the register aren't garbage.

Why not have sign-extending variants of the `ref` operators?

Because that would double the number of `ref` operators, and we need the `ext` bytecode anyway for accessing bitfields.

Why not have constant-address variants of the `ref` operators?

Because that would double the number of `ref` operators again, and `const32 address ref32` is only one byte longer.

Why do the `refn` operators have to support unaligned fetches?

GDB will generate bytecode that fetches multi-byte values at unaligned addresses whenever the executable's debugging information tells it to. Furthermore, GDB does not know the value the pointer will have when GDB generates the bytecode, so it cannot determine whether a particular fetch will be aligned or not.

In particular, structure bitfields may be several bytes long, but follow no alignment rules; members of packed structures are not necessarily aligned either.

In general, there are many cases where unaligned references occur in correct C code, either at the programmer's explicit request, or at the compiler's discretion. Thus, it is simpler to make the GDB agent bytecodes work correctly in all circumstances than to make GDB guess in each case whether the compiler did the usual thing.

Why are there no side-effecting operators?

Because our current client doesn't want them? That's a cheap answer. I think the real answer is that I'm afraid of implementing function calls. We should re-visit this issue after the present contract is delivered.

Why aren't the goto ops PC-relative?

The interpreter has the base address around anyway for PC bounds checking, and it seemed simpler.

Why is there only one offset size for the goto ops?

Offsets are currently sixteen bits. I'm not happy with this situation either:

Suppose we have multiple branch ops with different offset sizes. As I generate code left-to-right, all my jumps are forward jumps (there are no loops in expressions), so I never know the target when I emit the jump opcode. Thus, I have to either always assume the largest offset size, or do jump relaxation on the code after I generate it, which seems like a big waste of time.

I can imagine a reasonable expression being longer than 256 bytes. I can't imagine one being longer than 64k. Thus, we need 16-bit offsets. This kind of reasoning is so bogus, but relaxation is pathetic.

The other approach would be to generate code right-to-left. Then I'd always know my offset size. That might be fun.

Where is the function call bytecode?

When we add side-effects, we should add this.

Why does the reg bytecode take a 16-bit register number?

Intel's IA-64 architecture has 128 general-purpose registers, and 128 floating-point registers, and I'm sure it has some random control registers.

Why do we need trace and trace_quick?

Because GDB needs to record all the memory contents and registers an expression touches. If the user wants to evaluate an expression `x->y->z`, the agent must record the values of `x` and `x->y` as well as the value of `x->y->z`.

Don't the trace bytecodes make the interpreter less general?

They do mean that the interpreter contains special-purpose code, but that doesn't mean the interpreter can only be used for that purpose. If an expression doesn't use the `trace` bytecodes, they don't get in its way.

Why doesn't trace_quick consume its arguments the way everything else does?

In general, you do want your operators to consume their arguments; it's consistent, and generally reduces the amount of stack rearrangement necessary. However, `trace_quick` is a kludge to save space; it only exists so we needn't write `dup const8 SIZE trace` before every memory reference. Therefore, it's okay for it not to consume its arguments; it's meant for a specific context in which we know exactly what it should do with the stack. If we're going to have a kludge, it should be an effective kludge.

Why does trace16 exist?

That opcode was added by the customer that contracted Cygnus for the data tracing work. I personally think it is unnecessary; objects that large will be quite rare, so it is okay to use `dup const16 size trace` in those cases.

Whatever we decide to do with `trace16`, we should at least leave opcode 0x30 reserved, to remain compatible with the customer who added it.

Appendix G Target Descriptions

One of the challenges of using GDB to debug embedded systems is that there are so many minor variants of each processor architecture in use. It is common practice for vendors to start with a standard processor core — ARM, PowerPC, or MIPS, for example — and then make changes to adapt it to a particular market niche. Some architectures have hundreds of variants, available from dozens of vendors. This leads to a number of problems:

- With so many different customized processors, it is difficult for the GDB maintainers to keep up with the changes.
- Since individual variants may have short lifetimes or limited audiences, it may not be worthwhile to carry information about every variant in the GDB source tree.
- When GDB does support the architecture of the embedded system at hand, the task of finding the correct architecture name to give the `set architecture` command can be error-prone.

To address these problems, the GDB remote protocol allows a target system to not only identify itself to GDB, but to actually describe its own features. This lets GDB support processor variants it has never seen before — to the extent that the descriptions are accurate, and that GDB understands them.

GDB must be linked with the Expat library to support XML target descriptions. See [Expat], page 768.

G.1 Retrieving Descriptions

Target descriptions can be read from the target automatically, or specified by the user manually. The default behavior is to read the description from the target. GDB retrieves it via the remote protocol using ‘qXfer’ requests (see Section E.5 [General Query Packets], page 811). The *annex* in the ‘qXfer’ packet will be ‘target.xml’. The contents of the ‘target.xml’ annex are an XML document, of the form described in Section G.2 [Target Description Format], page 883.

Alternatively, you can specify a file to read for the target description. If a file is set, the target will not be queried. The commands to specify a file are:

set tdesc filename path

Read the target description from *path*.

unset tdesc filename

Do not read the XML target description from a file. GDB will use the description supplied by the current target.

show tdesc filename

Show the filename to read for a target description, if any.

G.2 Target Description Format

A target description annex is an XML (<http://www.w3.org/XML/>) document which complies with the Document Type Definition provided in the GDB sources in `gdb/features/gdb-target.dtd`. This means you can use generally available tools like `xmllint` to check that your feature descriptions are well-formed and valid. However, to

help people unfamiliar with XML write descriptions for their targets, we also describe the grammar here.

Target descriptions can identify the architecture of the remote target and (for some architectures) provide information about custom register sets. They can also identify the OS ABI of the remote target. GDB can use this information to autoconfigure for your target, or to warn you if you connect to an unsupported target.

Here is a simple target description:

```
<target version="1.0">
  <architecture>i386:x86-64</architecture>
</target>
```

This minimal description only says that the target uses the x86-64 architecture.

A target description has the following overall form, with [] marking optional elements and ... marking repeatable elements. The elements are explained further below.

```
<?xml version="1.0"?>
<!DOCTYPE target SYSTEM "gdb-target.dtd">
<target version="1.0">
  [architecture]
  [osabi]
  [compatible]
  [feature...]
</target>
```

The description is generally insensitive to whitespace and line breaks, under the usual common-sense rules. The XML version declaration and document type declaration can generally be omitted (GDB does not require them), but specifying them may be useful for XML validation tools. The ‘**version**’ attribute for ‘<target>’ may also be omitted, but we recommend including it; if future versions of GDB use an incompatible revision of `gdb-target.dtd`, they will detect and report the version mismatch.

G.2.1 Inclusion

It can sometimes be valuable to split a target description up into several different annexes, either for organizational purposes, or to share files between different possible target descriptions. You can divide a description into multiple files by replacing any element of the target description with an inclusion directive of the form:

```
<xi:include href="document"/>
```

When GDB encounters an element of this form, it will retrieve the named XML *document*, and replace the inclusion directive with the contents of that document. If the current description was read using ‘`qXfer`’, then so will be the included document; *document* will be interpreted as the name of an annex. If the current description was read from a file, GDB will look for *document* as a file in the same directory where it found the original description.

G.2.2 Architecture

An ‘<architecture>’ element has this form:

```
<architecture>arch</architecture>
```

arch is one of the architectures from the set accepted by `set architecture` (see Chapter 19 [Specifying a Debugging Target], page 313).

G.2.3 OS ABI

This optional field was introduced in GDB version 7.0. Previous versions of GDB ignore it.

An ‘<osabi>’ element has this form:

```
<osabi>abi-name</osabi>
```

abi-name is an OS ABI name from the same selection accepted by `set osabi` (see Section 22.7 [Configuring the Current ABI], page 380).

G.2.4 Compatible Architecture

This optional field was introduced in GDB version 7.0. Previous versions of GDB ignore it.

A ‘<compatible>’ element has this form:

```
<compatible>arch</compatible>
```

arch is one of the architectures from the set accepted by `set architecture` (see Chapter 19 [Specifying a Debugging Target], page 313).

A ‘<compatible>’ element is used to specify that the target is able to run binaries in some other than the main target architecture given by the ‘<architecture>’ element. For example, on the Cell Broadband Engine, the main architecture is `powerpc:common` or `powerpc:common64`, but the system is able to run binaries in the `spu` architecture as well. The way to describe this capability with ‘<compatible>’ is as follows:

```
<architecture>powerpc:common</architecture>
<compatible>spu</compatible>
```

G.2.5 Features

Each ‘<feature>’ describes some logical portion of the target system. Features are currently used to describe available CPU registers and the types of their contents. A ‘<feature>’ element has this form:

```
<feature name="name">
  [type...]
  reg...
</feature>
```

Each feature’s name should be unique within the description. The name of a feature does not matter unless GDB has some special knowledge of the contents of that feature; if it does, the feature should have its standard name. See Section G.5 [Standard Target Features], page 889.

G.2.6 Types

Any register’s value is a collection of bits which GDB must interpret. The default interpretation is a two’s complement integer, but other types can be requested by name in the register description. Some predefined types are provided by GDB (see Section G.3 [Predefined Target Types], page 887), and the description can define additional composite and enum types.

Each type element must have an ‘id’ attribute, which gives a unique (within the containing ‘<feature>’) name to the type. Types must be defined before they are used.

Some targets offer vector registers, which can be treated as arrays of scalar elements. These types are written as ‘<vector>’ elements, specifying the array element type, *type*, and the number of elements, *count*:

```
<vector id="id" type="type" count="count"/>
```

If a register's value is usefully viewed in multiple ways, define it with a union type containing the useful representations. The '`<union>`' element contains one or more '`<field>`' elements, each of which has a *name* and a *type*:

```
<union id="id">
  <field name="name" type="type"/>
  ...
</union>
```

If a register's value is composed from several separate values, define it with either a structure type or a flags type. A flags type may only contain bitfields. A structure type may either contain only bitfields or contain no bitfields. If the value contains only bitfields, its total size in bytes must be specified.

Non-bitfield values have a *name* and *type*.

```
<struct id="id">
  <field name="name" type="type"/>
  ...
</struct>
```

Both *name* and *type* values are required. No implicit padding is added.

Bitfield values have a *name*, *start*, *end* and *type*.

```
<struct id="id" size="size">
  <field name="name" start="start" end="end" type="type"/>
  ...
</struct>
<flags id="id" size="size">
  <field name="name" start="start" end="end" type="type"/>
  ...
</flags>
```

The *name* value is required. Bitfield values may be named with the empty string, `""`, in which case the field is "filler" and its value is not printed. Not all bits need to be specified, so "filler" fields are optional.

The *start* and *end* values are required, and *type* is optional. The field's *start* must be less than or equal to its *end*, and zero represents the least significant bit.

The default value of *type* is `bool` for single bit fields, and an unsigned integer otherwise.

Which to choose? Structures or flags?

Registers defined with '`flags`' have these advantages over defining them with '`struct`':

- Arithmetic may be performed on them as if they were integers.
- They are printed in a more readable fashion.

Registers defined with '`struct`' have one advantage over defining them with '`flags`':

- One can fetch individual fields like in 'C'.

```
(gdb) print $my_struct_reg.field3
$1 = 42
```

G.2.7 Registers

Each register is represented as an element with this form:

```
<reg name="name"
  bitsize="size"
  [regnum="num"]
  [save-restore="save-restore"]>
```

```
[type="type"]
[group="group"]/>
```

The components are as follows:

<i>name</i>	The register's name; it must be unique within the target description.
<i>bitsize</i>	The register's size, in bits.
<i>regnum</i>	The register's number. If omitted, a register's number is one greater than that of the previous register (either in the current feature or in a preceding feature); the first register in the target description defaults to zero. This register number is used to read or write the register; e.g. it is used in the remote p and P packets, and registers appear in the g and G packets in order of increasing register number.
<i>save-restore</i>	Whether the register should be preserved across inferior function calls; this must be either yes or no . The default is yes , which is appropriate for most registers except for some system control registers; this is not related to the target's ABI.
<i>type</i>	The type of the register. It may be a predefined type, a type defined in the current feature, or one of the special types int and float . int is an integer type of the correct size for <i>bitsize</i> , and float is a floating point type (in the architecture's normal floating point format) of the correct size for <i>bitsize</i> . The default is int .
<i>group</i>	The register group to which this register belongs. It can be one of the standard register groups general , float , vector or an arbitrary string. Group names should be limited to alphanumeric characters. If a group name is made up of multiple words the words may be separated by hyphens; e.g. special-group or ultra-special-group . If no <i>group</i> is specified, GDB will not display the register in info registers .

G.3 Predefined Target Types

Type definitions in the self-description can build up composite types from basic building blocks, but can not define fundamental types. Instead, standard identifiers are provided by GDB for the fundamental types. The currently supported types are:

bool	Boolean type, occupying a single bit.
int8	
int16	
int24	
int32	
int64	
int128	Signed integer types holding the specified number of bits.

uint8
 uint16
 uint24
 uint32
 uint64
 uint128 Unsigned integer types holding the specified number of bits.

code_ptr
 data_ptr Pointers to unspecified code and data. The program counter and any dedicated return address register may be marked as code pointers; printing a code pointer converts it into a symbolic address. The stack pointer and any dedicated address registers may be marked as data pointers.

ieee_half Half precision IEEE floating point.

ieee_single Single precision IEEE floating point.

ieee_double Double precision IEEE floating point.

bfloat16 The 16-bit *brain floating point* format used e.g. by x86 and ARM.

arm_fpa_ext The 12-byte extended precision format used by ARM FPA registers.

i387_ext The 10-byte extended precision format used by x87 registers.

i386_eflags 32bit EFLAGS register used by x86.

i386_mxcsr 32bit MXCSR register used by x86.

G.4 Enum Target Types

Enum target types are useful in ‘struct’ and ‘flags’ register descriptions. See Section G.2 [Target Description Format], page 883.

Enum types have a name, size and a list of name/value pairs.

```

<enum id="id" size="size">
  <eval name="name" value="value"/>
  ...
</enum>

```

Enums must be defined before they are used.

```

<enum id="levels_type" size="4">
  <eval name="low" value="0"/>
  <eval name="high" value="1"/>
</enum>
<flags id="flags_type" size="4">
  <field name="X" start="0"/>
  <field name="LEVEL" start="1" end="1" type="levels_type"/>
</flags>
<reg name="flags" bitsize="32" type="flags_type"/>

```

Given that description, a value of 3 for the ‘**flags**’ register would be printed as:

```
(gdb) info register flags
flags 0x3 [ X LEVEL=high ]
```

G.5 Standard Target Features

A target description must contain either no registers or all the target’s registers. If the description contains no registers, then GDB will assume a default register layout, selected based on the architecture. If the description contains any registers, the default layout will not be used; the standard registers must be described in the target description, in such a way that GDB can recognize them.

This is accomplished by giving specific names to feature elements which contain standard registers. GDB will look for features with those names and verify that they contain the expected registers; if any known feature is missing required registers, or if any required feature is missing, GDB will reject the target description. You can add additional registers to any of the standard features — GDB will display them just as if they were added to an unrecognized feature.

This section lists the known features and their expected contents. Sample XML documents for these features are included in the GDB source tree, in the directory `gdb/features`.

Names recognized by GDB should include the name of the company or organization which selected the name, and the overall architecture to which the feature applies; so e.g. the feature containing ARM core registers is named ‘`org.gnu.gdb.arm.core`’.

The names of registers are not case sensitive for the purpose of recognizing standard features, but GDB will only display registers using the capitalization used in the description.

G.5.1 AArch64 Features

G.5.1.1 AArch64 core registers feature

The ‘`org.gnu.gdb.aarch64.core`’ feature is required for AArch64 targets. It must contain the following:

- ‘`x0`’ through ‘`x30`’, the general purpose registers, with size of 64 bits. Register ‘`x30`’ is also known as the *link register*, or ‘`lr`’.
- ‘`sp`’, the stack pointer register or ‘`x31`’. It is 64 bits in size and has a type of ‘`data_ptr`’.
- ‘`pc`’, the program counter register. It is 64 bits in size and has a type of ‘`code_ptr`’.
- ‘`cpsr`’, the current program status register. It is 32 bits in size and has a custom flags type.

The semantics of the individual flags and fields in ‘`cpsr`’ can change as new architectural features are added. The current layout can be found in the `aarch64-core.xml` file.

Extra registers are allowed in this feature, but they will not affect GDB.

G.5.1.2 AArch64 floating-point registers feature

The ‘`org.gnu.gdb.aarch64.fpu`’ feature is optional. If present, it must contain the following registers:

- ‘`v0`’ through ‘`v31`’, the vector registers with size of 128 bits. The type is a custom vector type.

- ‘fpsr’, the floating-point status register. It is 32 bits in size and has a custom flags type.
- ‘fpcr’, the floating-point control register. It is 32 bits in size and has a custom flags type.

The semantics of the individual flags and fields in ‘fpsr’ and ‘fpcr’ can change as new architectural features are added.

The types for the vector registers, ‘fpsr’ and ‘fpcr’ registers can be found in the aarch64-fpu.xml file.

Extra registers are allowed in this feature, but they will not affect GDB.

G.5.1.3 AArch64 SVE registers feature

The ‘org.gnu.gdb.aarch64.sve’ feature is optional. If present, it means the target supports the Scalable Vector Extension and must contain the following registers:

- ‘z0’ through ‘z31’, the scalable vector registers. Their sizes are variable and a multiple of 128 bits up to a maximum of 2048 bit. Their type is a custom union type that helps visualize different sizes of sub-vectors.
- ‘fpsr’, the floating-point status register. It is 32 bits in size and has a custom flags type.
- ‘fpcr’, the floating-point control register. It is 32 bits in size and has a custom flags type.
- ‘p0’ through ‘p15’, the predicate registers. Their sizes are variable, based on the current vector length, and a multiple of 16 bits. Their types are a custom union to help visualize sub-elements.
- ‘ffr’, the First Fault register. It has a variable size based on the current vector length and is a multiple of 16 bits. The type is the same as the predicate registers.
- ‘vg’, the vector granule. It represents the number of 64 bits chunks in a ‘z’ register. It is closely associated with the current vector length. It has a type of ‘int’.

When GDB sees the SVE feature, it will assume the Scalable Vector Extension is supported, and will adjust the sizes of the ‘z’, ‘p’ and ‘ffr’ registers accordingly, based on the value of ‘vg’.

GDB will also create pseudo-registers equivalent to the ‘v’ vector registers from the ‘org.gnu.gdb.aarch64.fpu’ feature.

The first 128 bits of the ‘z’ registers overlap the 128 bits of the ‘v’ registers, so changing one will trigger a change to the other.

For the types of the ‘z’, ‘p’ and ‘ffr’ registers, please check the aarch64-sve.c file. No XML file is available for this feature because it is dynamically generated based on the current vector length.

The semantics of the individual flags and fields in ‘fpsr’ and ‘fpcr’ can change as new architectural features are added.

The types for the ‘fpsr’ and ‘fpcr’ registers can be found in the aarch64-sve.c file, and should match what is described in aarch64-fpu.xml.

Extra registers are allowed in this feature, but they will not affect GDB.

G.5.1.4 AArch64 Pointer Authentication registers feature

The `'org.gnu.gdb.aarch64.pauth'` optional feature was introduced so GDB could detect support for the Pointer Authentication extension. If present, it must contain one of two possible register sets.

Pointer Authentication masks for user-mode:

- `'pauth_dmask'`, the user-mode pointer authentication mask for data pointers. It is 64 bits in size.
- `'pauth_cmask'`, the user-mode pointer authentication mask for code pointers. It is 64 bits in size.

Pointer Authentication masks for user-mode and kernel-mode:

- `'pauth_dmask'`, the user-mode pointer authentication mask for data pointers. It is 64 bits in size.
- `'pauth_cmask'`, the user-mode pointer authentication mask for code pointers. It is 64 bits in size.
- `'pauth_dmask_high'`, the kernel-mode pointer authentication mask for data pointers. It is 64 bits in size.
- `'pauth_cmask_high'`, the kernel-mode pointer authentication mask for code pointers. It is 64 bits in size.

If GDB sees any of the two sets of registers in this feature, it will assume the target is capable of signing pointers. If so, GDB will decorate backtraces with a `'[PAC]'` marker alongside a function that has a signed link register value that needs to be unmasked/decoded.

GDB will also use the masks to remove non-address bits from pointers.

Extra registers are allowed in this feature, but they will not affect GDB.

Please note the `'org.gnu.gdb.aarch64.pauth'` feature string is deprecated and must only be used for backwards compatibility with older releases of GDB and `gdbserver`. Targets that support Pointer Authentication must advertise such capability by using the `'org.gnu.gdb.aarch64.pauth_v2'` feature string instead.

The `'org.gnu.gdb.aarch64.pauth_v2'` feature has the exact same contents as feature `'org.gnu.gdb.aarch64.pauth'`.

The reason for having feature `'org.gnu.gdb.aarch64.pauth_v2'` is a bug in previous versions of GDB (versions 9, 10, 11 and 12). This bug caused GDB to crash whenever the target reported support for Pointer Authentication (using feature string `'org.gnu.gdb.aarch64.pauth'`) and also reported additional system registers that were not accounted for by GDB. This is more common when using emulators and on bare-metal debugging scenarios.

It can also happen if a newer `gdbserver` is used with an old GDB that has the bug. In such a case, the newer `gdbserver` might report Pointer Authentication support via the `'org.gnu.gdb.aarch64.pauth'` feature string and also report additional registers the older GDB does not know about, potentially leading to a crash.

G.5.1.5 AArch64 TLS registers feature

The `'org.gnu.gdb.aarch64.tls'` optional feature was introduced to expose the TLS registers to GDB. If present, it must contain either one of the following register sets.

Only ‘tpidr’:

- ‘tpidr’, the software thread id register. It is 64 bits in size and has a type of ‘data_ptr’.

Both ‘tpidr’ and ‘tpidr2’:

- ‘tpidr’, the software thread id register. It is 64 bits in size and has a type of ‘data_ptr’.
- ‘tpidr2’, the second software thread id register. It is 64 bits in size and has a type of ‘data_ptr’. It may be used in the future alongside the Scalable Matrix Extension for a lazy restore scheme.

If GDB sees this feature, it will attempt to find one of the variations of the register set. If ‘tpidr2’ is available, GDB may act on it to display additional data in the future.

There is no XML for this feature as the presence of ‘tpidr2’ is determined dynamically at runtime.

Extra registers are allowed in this feature, but they will not affect GDB.

G.5.1.6 AArch64 MTE registers feature

The ‘org.gnu.gdb.aarch64.mte’ optional feature was introduced so GDB could detect support for the Memory Tagging Extension and control memory tagging settings. If present, this feature must have the following register:

- ‘tag_ctl’, the tag control register. It is 64 bits in size and has a type of ‘uint64’.

Memory Tagging detection is done via a runtime check though, so the presence of this feature and register is not enough to enable memory tagging support.

This restriction may be lifted in the future.

Extra registers are allowed in this feature, but they will not affect GDB.

G.5.1.7 AArch64 SME registers feature

The ‘org.gnu.gdb.aarch64.sme’ feature is optional. If present, it should contain registers ZA, SVG and SVCR. See [AArch64 SME], page 357.

- ZA is a register represented by a 2-dimensional array of bytes, with each dimension having size svl. See [svl], page 357.
- SVG is a 64-bit register containing the value of svg. See [svg], page 357.
- SVCR is a 64-bit status pseudo-register with two valid bits. Bit 0 (SM) shows whether the streaming SVE mode is enabled or disabled. Bit 1 (ZA) shows whether the ZA register state is active (in use) or not. See [aarch64 sme svcr], page 357.

The rest of the unused bits of the SVCR pseudo-register is undefined and reserved. Such bits should not be used and may be defined by future extensions of the architecture.

Extra registers are allowed in this feature, but they will not affect GDB.

The ‘org.gnu.gdb.aarch64.sme’ feature is required when the target also reports support for the ‘org.gnu.gdb.aarch64.sme2’ feature.

G.5.1.8 AArch64 SME2 registers feature

The ‘org.gnu.gdb.aarch64.sme2’ feature is optional. If present, then the ‘org.gnu.gdb.aarch64.sme’ feature must also be present. The ‘org.gnu.gdb.aarch64.sme2’ feature should contain the following: See [AArch64 SME2], page 359.

- ZT0 is a register of 512 bits (64 bytes). It is defined as a vector of bytes.

Extra registers are allowed in this feature, but they will not affect GDB.

G.5.1.9 AArch64 GCS registers feature

The `'org.gnu.gdb.aarch64.gcs'` feature is optional. If present, it means the target supports Guarded Control Stacks and must contain the following register:

- `gcspr`, which points to the thread's Guarded Control Stack. It is 64 bits in size and has a type of `'data_ptr'`.

The `'org.gnu.gdb.aarch64.gcs.linux'` feature is optional. If present, then the `'org.gnu.gdb.aarch64.gcs'` feature must also be present. The `'org.gnu.gdb.aarch64.gcs.linux'` feature represents facilities provided by the Linux kernel for GCS support and should contain the following:

- `gcs_features_enabled` shows the features currently enabled via the `prctl` or `ptrace` system calls. It is represented as if it were a 64-bit register with a custom flags type.
- `gcs_features_locked` shows the features currently locked via the `prctl` or `ptrace` system calls. It is represented as if it were a 64-bit register with a custom flags type.

The custom flags type allows GDB to print a human-friendly representation of the contents of `gcs_features_enabled` and `gcs_features_locked` and should contain:

- `PR_SHADOW_STACK_ENABLE`
- `PR_SHADOW_STACK_WRITE`
- `PR_SHADOW_STACK_PUSH`

For further information, see the documentation in the Linux kernel.

Extra registers are allowed in these features, but they will not affect GDB.

G.5.1.10 AArch64 FPMR feature

The `'org.gnu.gdb.aarch64.fpmr'` optional feature was introduced so GDB could detect support for the Floating Point Mode Register extension. If present, this feature must have the following register:

- `'fpmr'`, the floating point mode register. It is represented as if it were a 64-bit register with a custom flags type.

The custom flags type allows GDB to print a human-friendly representation of the contents of `fpmr` should contain:

- `F8S1`, which is either `E5M2` or `E4M3`
- `F8S2`, which is either `E5M2` or `E4M3`
- `F8D`, which is either `E5M2` or `E4M3`
- `OSM`, which is either `Inf` or `MaxNormal`
- `OSC`, which is either `Inf/NaN` or `MaxNormal`
- `LSCALE`, which is a 7-bit unsigned value
- `NSCALE`, which is actually an 8-bit signed value but represented as an 8-bit unsigned value
- `LSCALE2`, which is a 6-bit unsigned value

Extra registers are allowed in this feature, but they will not affect GDB.

For more information about FPMR, please refer to the official architecture registers documentation.

G.5.2 Alpha Features

The `'org.gnu.gdb.alpha.core'` feature is required for Alpha targets. It must contain the following 64-bit registers; note that GDB uses the software names for Alpha registers:

- `'v0'`: function return value
- `'t0'` through `'t12'`: temporary registers
- `'s0'` through `'s5'`: saved registers
- `'fp'`: frame pointer
- `'a0'` through `'a5'`: argument registers
- `'ra'`: return address
- `'at'`: assembler temporary register
- `'gp'`: global pointer
- `'sp'`: stack pointer
- `'zero'`: always zero
- `'f0'` through `'f30'`: floating-point registers
- `'fpcr'`: floating-point control register
- `'pc'`: program counter
- `'`: an anonymous register for historical purpose
- `'unique'`: PALcode memory slot

G.5.3 ARC Features

ARC processors are so configurable that even core registers and their numbers are not predetermined completely. Moreover, *flags* and *PC* registers, which are important to GDB, are not “core” registers in ARC. Therefore, there are two features that their presence is mandatory: `'org.gnu.gdb.arc.core'` and `'org.gnu.gdb.arc.aux'`.

The `'org.gnu.gdb.arc.core'` feature is required for all targets. It must contain registers:

- `'r0'` through `'r25'` for normal register file targets.
- `'r0'` through `'r3'`, and `'r10'` through `'r15'` for reduced register file targets.
- `'gp'`, `'fp'`, `'sp'`, `'r30'`¹, `'blink'`, `'lp_count'`, `'pcl'`.

In case of an ARCompact target (ARCV1 ISA), the `'org.gnu.gdb.arc.core'` feature may contain registers `'ilink1'` and `'ilink2'`. While in case of ARC EM and ARC HS targets (ARCV2 ISA), register `'ilink'` may be present. The difference between ARCV1 and ARCV2 is the naming of registers *29th* and *30th*. They are called `'ilink1'` and `'ilink2'` for ARCV1 and are optional. For ARCV2, they are called `'ilink'` and `'r30'` and only `'ilink'` is optional. The optionality of `'ilink*`' registers is because of their inaccessibility during user space debugging sessions.

¹ Not necessary for ARCV1.

Extension core registers ‘r32’ through ‘r59’ are optional and their existence depends on the configuration. When debugging GNU/Linux applications, i.e. user space debugging, these core registers are not available.

The ‘org.gnu.gdb.arc.aux’ feature is required for all ARC targets. Here is the list of registers pertinent to this feature:

- mandatory: ‘pc’ and ‘status32’.
- optional: ‘lp_start’, ‘lp_end’, and ‘bta’.

G.5.4 ARM Features

G.5.4.1 Core register set for non-M-profile

The ‘org.gnu.gdb.arm.core’ feature is required for non-M-profile ARM targets. It must contain the following registers:

- ‘r0’ through ‘r12’. The general purpose registers. They are 32 bits in size and have a type of ‘uint32’.
- ‘sp’, the stack pointer register, also known as ‘r13’. It is 32 bits in size and has a type of ‘data_ptr’.
- ‘lr’, the link register. It is 32 bits in size.
- ‘pc’, the program counter register. It is 32 bit in size and of type ‘code_ptr’.
- ‘cpsr’, the current program status register containing all the status bits. It is 32 bits in size. Historically this register was hardwired to number 25, but debugging stubs that report XML do not need to use this number anymore.

Extra registers are allowed in this feature, but they will not affect GDB.

G.5.4.2 Core register set for M-profile

For M-profile targets (e.g. Cortex-M3), the ‘org.gnu.gdb.arm.core’ feature is replaced by ‘org.gnu.gdb.arm.m-profile’, and it is a required feature. It must contain the following registers:

- ‘r0’ through ‘r12’, the general purpose registers. They have a size of 32 bits and a type of ‘uint32’.
- ‘sp’, the stack pointer register, also known as ‘r13’. It has a size of 32 bits and a type of ‘data_ptr’.
- ‘lr’, the link register. It has a size of 32 bits.
- ‘pc’, the program counter register. It has a size of 32 bits and a type of ‘code_ptr’.
- ‘xpsr’, the program status register containing all the status bits. It has a size of 32 bits. Historically this register was hardwired to number 25, but debugging stubs that report XML do not need to use this number anymore.

Upon seeing this feature, GDB will acknowledge that it is dealing with an M-profile target. This means GDB will use hooks and configurations that are meaningful to M-profiles.

Extra registers are allowed in this feature, but they will not affect GDB.

G.5.4.3 FPA registers feature (obsolete)

The `'org.gnu.gdb.arm.fpa'` feature is obsolete and should not be advertised by debugging stubs anymore. It used to advertise registers for the old FPA architecture that has long been discontinued in toolchains.

It is kept in GDB for backward compatibility purposes so older debugging stubs that don't support XML target descriptions still work correctly. One such example is the KGDB debugging stub from Linux or BSD kernels.

The description below is for historical purposes only. This feature used to contain the following registers:

- `'f0'` through `'f8'`. The floating point registers. They are 96 bits in size and of type `'arm_fpa_ext'`. `'f0'` is pinned to register number 16.
- `'fps'`, the status register. It has a size of 32 bits.

G.5.4.4 M-profile Vector Extension (MVE)

Also known as Helium, the M-profile Vector Extension is advertised via the optional `'org.gnu.gdb.arm.m-profile-mve'` feature.

It must contain the following:

- `'vpr'`, the vector predication status and control register. It is 32 bits in size and has a custom flags type. The flags type is laid out in a way that exposes the `'P0'` field from bits 0 to 15, the `'MASK01'` field from bits 16 to 19 and the `'MASK23'` field from bits 20 to 23.

Bits 24 through 31 are reserved.

When this feature is available, GDB will synthesize the `'p0'` pseudo-register from `'vpr'` contents.

This feature must only be advertised if the target is M-profile. Advertising this feature for targets that are not M-profile may cause GDB to assume the target is M-profile when it isn't.

If the `'org.gnu.gdb.arm.vfp'` feature is available alongside the `'org.gnu.gdb.arm.m-profile-mve'` feature, GDB will synthesize the `'q'` pseudo-registers from `'d'` register contents.

Extra registers are allowed in this feature, but they will not affect GDB.

G.5.4.5 XScale iwMMXt feature

The XScale `'org.gnu.gdb.xscale.iwmmxt'` feature is optional. If present, it must contain the following:

- `'wR0'` through `'wR15'`, registers with size 64 bits and a custom type `'iwmmxt_vec64i'`. `'iwmmxt_vec64i'` is a union of four other types: `'uint64'`, a 2-element vector of `'uint32'`, a 4-element vector of `'uint16'` and a 8-element vector of `'uint8'`.
- `'wCGR0'` through `'wCGR3'`, registers with size 32 bits and type `'int'`.

The following registers are optional:

- `'wCID'`, register with size of 32 bits and type `'int'`.
- `'wCon'`, register with size 32 bits and type `'int'`.
- `'wCSSF'`, register with size 32 bits and type `'int'`.

- ‘wCASF’, register with size 32 bit and type ‘int’.

This feature should only be reported if the target is XScale.

Extra registers are allowed in this feature, but they will not affect GDB.

G.5.4.6 Vector Floating-Point (VFP) feature

The ‘org.gnu.gdb.arm.vfp’ feature is optional. If present, it should contain one of two possible sets of values depending on whether VFP version 2 or VFP version 3 is in use.

For VFP v2:

- ‘d0’ through ‘d15’. The double-precision registers. They are 64 bits in size and have type ‘ieee_double’.
- ‘fpscr’, the floating-point status and control register. It has a size of 32 bits and a type of ‘int’.

For VFP v3:

- ‘d0’ through ‘d31’. The double-precision registers. They are 64 bits in size and have type ‘ieee_double’.
- ‘fpscr’, the floating-point status and control register. It has a size of 32 bits and a type of ‘int’.

If this feature is available, GDB will synthesize the single-precision floating-point registers from halves of the double-precision registers as pseudo-registers.

Extra registers are allowed in this feature, but they will not affect GDB.

G.5.4.7 NEON architecture feature

The ‘org.gnu.gdb.arm.neon’ feature is optional. It does not need to contain registers; it instructs GDB to display the VFP double-precision registers as vectors and to synthesize the quad-precision registers from pairs of double-precision registers. If this feature is present, ‘org.gnu.gdb.arm.vfp’ must also be present and include 32 double-precision registers.

Extra registers are allowed in this feature, but they will not affect GDB.

G.5.4.8 M-profile Pointer Authentication and Branch Target Identification feature

The ‘org.gnu.gdb.arm.m-profile-pacbti’ feature is optional, and acknowledges support for the ARMv8.1-m PACBTI extensions.

This feature doesn’t contain any required registers, and it only serves as a hint to GDB that the debugging stub supports the ARMv8.1-m PACBTI extensions.

When GDB sees this feature, it will track return address signing states and will decorate backtraces using the [PAC] marker, similar to AArch64’s PAC extension. See [AArch64 PAC], page 359.

Extra registers are allowed in this feature, but they will not affect GDB.

G.5.4.9 M-profile system registers feature

The ‘org.gnu.gdb.arm.m-system’ optional feature was introduced as a way to inform GDB about additional system registers.

At the moment this feature must contain the following:

- ‘msp’, the main stack pointer register. It is 32 bits in size with type ‘data_ptr’.
- ‘psp’, the process stack pointer register. It is 32 bits in size with type ‘data_ptr’.

This feature must only be advertised for M-profile targets. When GDB sees this feature, it will attempt to track the values of ‘msp’ and ‘psp’ across frames.

Extra registers are allowed in this feature, but they will not affect GDB.

G.5.4.10 M-profile Security Extensions feature

The ‘org.gnu.gdb.arm.secext’ optional feature was introduced so GDB could better support the switching of stack pointers and secure states in the Security Extensions.

At the moment this feature must contain the following:

- ‘msp_ns’, the main stack pointer register (non-secure state). It is 32 bits in size with type ‘data_ptr’.
- ‘psp_ns’, the process stack pointer register (non-secure state). It is 32 bits in size with type ‘data_ptr’.
- ‘msp_s’, the main stack pointer register (secure state). It is 32 bits in size with type ‘data_ptr’.
- ‘psp_s’, the process stack pointer register (secure state). It is 32 bits in size with type ‘data_ptr’.

When GDB sees this feature, it will attempt to track the values of all 4 stack pointers across secure state transitions, potentially improving unwinding when applications switch between security states.

Extra registers are allowed in this feature, but they will not affect GDB.

G.5.4.11 TLS registers feature

The optional ‘org.gnu.gdb.arm.tls’ feature contains TLS registers.

Currently it contains the following:

- ‘tpidruro’, the user read-only thread id register. It is 32 bits in size and has type ‘data_ptr’.

At the moment GDB looks for this feature, but doesn’t do anything with it other than displaying it.

Extra registers are allowed in this feature, but they will not affect GDB.

G.5.5 i386 Features

The ‘org.gnu.gdb.i386.core’ feature is required for i386/amd64 targets. It should describe the following registers:

- ‘eax’ through ‘edi’ plus ‘eip’ for i386
- ‘rax’ through ‘r15’ plus ‘rip’ for amd64
- ‘eflags’, ‘cs’, ‘ss’, ‘ds’, ‘es’, ‘fs’, ‘gs’
- ‘st0’ through ‘st7’
- ‘fctrl’, ‘fstat’, ‘ftag’, ‘fiseq’, ‘fioff’, ‘foseq’, ‘fooff’ and ‘fop’

The register sets may be different, depending on the target.

The `'org.gnu.gdb.i386.sse'` feature is optional. It should describe registers:

- `'xmm0'` through `'xmm7'` for i386
- `'xmm0'` through `'xmm15'` for amd64
- `'mxcsr'`

The `'org.gnu.gdb.i386.avx'` feature is optional and requires the `'org.gnu.gdb.i386.sse'` feature. It should describe the upper 128 bits of YMM registers:

- `'ymm0h'` through `'ymm7h'` for i386
- `'ymm0h'` through `'ymm15h'` for amd64

The `'org.gnu.gdb.i386.linux'` feature is optional. If the feature is present, then it should describe the 32 bit register, `'orig_eax'`.

Additionally, the `'org.gnu.gdb.i386.linux'` feature can optionally contain three 128 bit registers called `'i386_tls_gdt_0'`, `'i386_tls_gdt_1'`, and `'i386_tls_gdt_2'`. Each of these registers contains one 16 byte `'struct user_desc'` (see *man 2 get_thread_area*) object which describes one of the three TLS related GDT entries.

The `'org.gnu.gdb.i386.segments'` feature is optional. It should describe two system registers: `'fs_base'` and `'gs_base'`.

The `'org.gnu.gdb.i386.avx512'` feature is optional and requires the `'org.gnu.gdb.i386.avx'` feature. It should describe additional XMM registers:

- `'xmm16'` through `'xmm31'`, only valid for amd64.

It should describe the upper 128 bits of additional YMM registers:

- `'ymm16h'` through `'ymm31h'`, only valid for amd64.

It should describe the upper 256 bits of ZMM registers:

- `'zmm0h'` through `'zmm7h'` for i386.
- `'zmm0h'` through `'zmm15h'` for amd64.

It should describe the additional ZMM registers:

- `'zmm16h'` through `'zmm31h'`, only valid for amd64.

It should describe the additional 64-bit K registers:

- `'k0'` through `'k7'`, valid for amd64 and i386.

The `'org.gnu.gdb.i386.pkeys'` feature is optional. It should describe a single register, `'pkru'`. It is a 32-bit register valid for i386 and amd64.

The `'org.gnu.gdb.i386.pl3_ssp'` feature is optional. It should describe the user mode register `'pl3_ssp'` which has 64 bits on amd64, 32 bits on amd64 with 32-bit pointer size (X32) and 32 bits on i386. Following the restriction of the Linux kernel, only GDB for amd64 targets makes use of this feature for now.

G.5.6 LoongArch Features

The `'org.gnu.gdb.loongarch.base'` feature is required for LoongArch targets. It should contain the registers `'r0'` through `'r31'`, `'pc'`, and `'badv'`. Either the architectural names (`'r0'`, `'r1'`, etc) can be used, or the ABI names (`'zero'`, `'ra'`, etc).

The `'org.gnu.gdb.loongarch.fpu'` feature is optional. If present, it should contain registers `'f0'` through `'f31'`, `'fcc'`, and `'fcsr'`.

G.5.7 MicroBlaze Features

The `'org.gnu.gdb.microblaze.core'` feature is required for MicroBlaze targets. It should contain registers `'r0'` through `'r31'`, `'rpc'`, `'rmsr'`, `'rear'`, `'resr'`, `'rfsr'`, `'rbtr'`, `'rpvr'`, `'rpvr1'` through `'rpvr11'`, `'redr'`, `'rpid'`, `'rzpr'`, `'rtlbx'`, `'rtlbsx'`, `'rtlblo'`, and `'rtlbhi'`.

The `'org.gnu.gdb.microblaze.stack-protect'` feature is optional. If present, it should contain registers `'rshr'` and `'rslr'`.

G.5.8 MIPS Features

The `'org.gnu.gdb.mips.cpu'` feature is required for MIPS targets. It should contain registers `'r0'` through `'r31'`, `'lo'`, `'hi'`, and `'pc'`. They may be 32-bit or 64-bit depending on the target.

The `'org.gnu.gdb.mips.cp0'` feature is also required. It should contain at least the `'status'`, `'badvaddr'`, and `'cause'` registers. They may be 32-bit or 64-bit depending on the target.

The `'org.gnu.gdb.mips.fpu'` feature is currently required, though it may be optional in a future version of GDB. It should contain registers `'f0'` through `'f31'`, `'fcsr'`, and `'fir'`. They may be 32-bit or 64-bit depending on the target.

The `'org.gnu.gdb.mips.dsp'` feature is optional. It should contain registers `'hi1'` through `'hi3'`, `'lo1'` through `'lo3'`, and `'dspctl'`. The `'dspctl'` register should be 32-bit and the rest may be 32-bit or 64-bit depending on the target.

The `'org.gnu.gdb.mips.linux'` feature is optional. It should contain a single register, `'restart'`, which is used by the Linux kernel to control restartable syscalls.

G.5.9 M68K Features

`'org.gnu.gdb.m68k.core'`

`'org.gnu.gdb.coldfire.core'`

`'org.gnu.gdb.fido.core'`

One of those features must be always present. The feature that is present determines which flavor of m68k is used. The feature that is present should contain registers `'d0'` through `'d7'`, `'a0'` through `'a5'`, `'fp'`, `'sp'`, `'ps'` and `'pc'`.

`'org.gnu.gdb.coldfire.fp'`

This feature is optional. If present, it should contain registers `'fp0'` through `'fp7'`, `'fpcontrol'`, `'fpstatus'` and `'fpiaddr'`.

Note that, despite the fact that this feature's name says `'coldfire'`, it is used to describe any floating point registers. The size of the registers must match the main m68k flavor; so, for example, if the primary feature is reported as `'coldfire'`, then 64-bit floating point registers are required.

G.5.10 NDS32 Features

The `'org.gnu.gdb.nds32.core'` feature is required for NDS32 targets. It should contain at least registers `'r0'` through `'r10'`, `'r15'`, `'fp'`, `'gp'`, `'lp'`, `'sp'`, and `'pc'`.

The `'org.gnu.gdb.nds32.fpu'` feature is optional. If present, it should contain 64-bit double-precision floating-point registers `'fd0'` through `'fdN'`, which should be `'fd3'`, `'fd7'`, `'fd15'`, or `'fd31'` based on the FPU configuration implemented.

Note: The first sixteen 64-bit double-precision floating-point registers are overlapped with the thirty-two 32-bit single-precision floating-point registers. The 32-bit single-precision registers, if not being listed explicitly, will be synthesized from halves of the overlapping 64-bit double-precision registers. Listing 32-bit single-precision registers explicitly is deprecated, and the support to it could be totally removed some day.

G.5.11 Openrisc 1000 Features

The `'org.gnu.gdb.or1k.group0'` feature is required for OpenRISC 1000 targets. It should contain the 32 general purpose registers (`'r0'` through `'r31'`), `'ppc'`, `'npc'` and `'sr'`.

G.5.12 PowerPC Features

The `'org.gnu.gdb.power.core'` feature is required for PowerPC targets. It should contain registers `'r0'` through `'r31'`, `'pc'`, `'msr'`, `'cr'`, `'lr'`, `'ctr'`, and `'xer'`. They may be 32-bit or 64-bit depending on the target.

The `'org.gnu.gdb.power.fpu'` feature is optional. It should contain registers `'f0'` through `'f31'` and `'fpscr'`.

The `'org.gnu.gdb.power.altivec'` feature is optional. It should contain registers `'vr0'` through `'vr31'`, `'vscr'`, and `'vrsave'`. GDB will define pseudo-registers `'v0'` through `'v31'` as aliases for the corresponding `'vrX'` registers.

The `'org.gnu.gdb.power.vsx'` feature is optional. It should contain registers `'vs0h'` through `'vs31h'`. GDB will combine these registers with the floating point registers (`'f0'` through `'f31'`) and the altivec registers (`'vr0'` through `'vr31'`) to present the 128-bit wide registers `'vs0'` through `'vs63'`, the set of vector-scalar registers for POWER7. Therefore, this feature requires both `'org.gnu.gdb.power.fpu'` and `'org.gnu.gdb.power.altivec'`.

The `'org.gnu.gdb.power.spe'` feature is optional. It should contain registers `'ev0h'` through `'ev31h'`, `'acc'`, and `'spefsr'`. SPE targets should provide 32-bit registers in `'org.gnu.gdb.power.core'` and provide the upper halves in `'ev0h'` through `'ev31h'`. GDB will combine these to present registers `'ev0'` through `'ev31'` to the user.

The `'org.gnu.gdb.power.ppr'` feature is optional. It should contain the 64-bit register `'ppr'`.

The `'org.gnu.gdb.power.dscr'` feature is optional. It should contain the 64-bit register `'dscr'`.

The `'org.gnu.gdb.power.tar'` feature is optional. It should contain the 64-bit register `'tar'`.

The `'org.gnu.gdb.power.ebb'` feature is optional. It should contain registers `'bescr'`, `'ebbhr'` and `'ebbrr'`, all 64-bit wide.

The `'org.gnu.gdb.power.linux.pmu'` feature is optional. It should contain registers `'mmcr0'`, `'mmcr2'`, `'siar'`, `'sdar'` and `'sier'`, all 64-bit wide. This is the subset of the isa 2.07 server PMU registers provided by GNU/Linux.

The `'org.gnu.gdb.power.htm.spr'` feature is optional. It should contain registers `'tfhar'`, `'texasr'` and `'tfiar'`, all 64-bit wide.

The `'org.gnu.gdb.power.htm.core'` feature is optional. It should contain the check-pointed general-purpose registers `'cr0'` through `'cr31'`, as well as the checkpointed registers `'clr'` and `'cctr'`. These registers may all be either 32-bit or 64-bit depending on the target.

It should also contain the checkpointed registers ‘`ccr`’ and ‘`cxer`’, which should both be 32-bit wide.

The ‘`org.gnu.gdb.power.htm.fpu`’ feature is optional. It should contain the checkpointed 64-bit floating-point registers ‘`cf0`’ through ‘`cf31`’, as well as the checkpointed 64-bit register ‘`cfpscr`’.

The ‘`org.gnu.gdb.power.htm.altivec`’ feature is optional. It should contain the checkpointed altivec registers ‘`cvr0`’ through ‘`cvr31`’, all 128-bit wide. It should also contain the checkpointed registers ‘`cvscr`’ and ‘`cvrsave`’, both 32-bit wide.

The ‘`org.gnu.gdb.power.htm.vsx`’ feature is optional. It should contain registers ‘`cvs0h`’ through ‘`cvs31h`’. GDB will combine these registers with the checkpointed floating point registers (‘`cf0`’ through ‘`cf31`’) and the checkpointed altivec registers (‘`cvr0`’ through ‘`cvr31`’) to present the 128-bit wide checkpointed vector-scalar registers ‘`cvs0`’ through ‘`cvs63`’. Therefore, this feature requires both ‘`org.gnu.gdb.power.htm.altivec`’ and ‘`org.gnu.gdb.power.htm.fpu`’.

The ‘`org.gnu.gdb.power.htm.ppr`’ feature is optional. It should contain the 64-bit checkpointed register ‘`cppr`’.

The ‘`org.gnu.gdb.power.htm.dscr`’ feature is optional. It should contain the 64-bit checkpointed register ‘`cdscr`’.

The ‘`org.gnu.gdb.power.htm.tar`’ feature is optional. It should contain the 64-bit checkpointed register ‘`ctar`’.

G.5.13 RISC-V Features

The ‘`org.gnu.gdb.riscv.cpu`’ feature is required for RISC-V targets. It should contain the registers ‘`x0`’ through ‘`x31`’, and ‘`pc`’. Either the architectural names (‘`x0`’, ‘`x1`’, etc) can be used, or the ABI names (‘`zero`’, ‘`ra`’, etc).

The ‘`org.gnu.gdb.riscv.fpu`’ feature is optional. If present, it should contain registers ‘`f0`’ through ‘`f31`’, ‘`fflags`’, ‘`frm`’, and ‘`fcsr`’. As with the `cpu` feature, either the architectural register names, or the ABI names can be used.

The ‘`org.gnu.gdb.riscv.virtual`’ feature is optional. If present, it should contain registers that are not backed by real registers on the target, but are instead virtual, where the register value is derived from other target state. In many ways these are like GDBs pseudo-registers, except implemented by the target. Currently the only register expected in this set is the one byte ‘`priv`’ register that contains the target’s privilege level in the least significant two bits.

The ‘`org.gnu.gdb.riscv.csr`’ feature is optional. If present, it should contain all of the target’s standard CSRs. Standard CSRs are those defined in the RISC-V specification documents. There is some overlap between this feature and the `fpu` feature; the ‘`fflags`’, ‘`frm`’, and ‘`fcsr`’ registers could be in either feature. The expectation is that these registers will be in the `fpu` feature if the target has floating point hardware, but can be moved into the `csr` feature if the target has the floating point control registers, but no other floating point hardware.

The ‘`org.gnu.gdb.riscv.vector`’ feature is optional. If present, it should contain registers ‘`v0`’ through ‘`v31`’, all of which must be the same size.

G.5.14 RX Features

The `'org.gnu.gdb.rx.core'` feature is required for RX targets. It should contain the registers `'r0'` through `'r15'`, `'usp'`, `'isp'`, `'psw'`, `'pc'`, `'intb'`, `'bpsw'`, `'bpc'`, `'fintv'`, `'fpsw'`, and `'acc'`.

G.5.15 S/390 and System z Features

The `'org.gnu.gdb.s390.core'` feature is required for S/390 and System z targets. It should contain the PSW and the 16 general registers. In particular, System z targets should provide the 64-bit registers `'pswm'`, `'pswa'`, and `'r0'` through `'r15'`. S/390 targets should provide the 32-bit versions of these registers. A System z target that runs in 31-bit addressing mode should provide 32-bit versions of `'pswm'` and `'pswa'`, as well as the general register's upper halves `'r0h'` through `'r15h'`, and their lower halves `'r0l'` through `'r15l'`.

The `'org.gnu.gdb.s390.fpr'` feature is required. It should contain the 64-bit registers `'f0'` through `'f15'`, and `'fpc'`.

The `'org.gnu.gdb.s390.acr'` feature is required. It should contain the 32-bit registers `'acr0'` through `'acr15'`.

The `'org.gnu.gdb.s390.linux'` feature is optional. It should contain the register `'orig_r2'`, which is 64-bit wide on System z targets and 32-bit otherwise. In addition, the feature may contain the `'last_break'` register, whose width depends on the addressing mode, as well as the `'system_call'` register, which is always 32-bit wide.

The `'org.gnu.gdb.s390.tdb'` feature is optional. It should contain the 64-bit registers `'tdb0'`, `'tac'`, `'tct'`, `'atia'`, and `'tr0'` through `'tr15'`.

The `'org.gnu.gdb.s390.vx'` feature is optional. It should contain 64-bit wide registers `'v0l'` through `'v15l'`, which will be combined by GDB with the floating point registers `'f0'` through `'f15'` to present the 128-bit wide vector registers `'v0'` through `'v15'`. In addition, this feature should contain the 128-bit wide vector registers `'v16'` through `'v31'`.

The `'org.gnu.gdb.s390.gs'` feature is optional. It should contain the 64-bit wide guarded-storage-control registers `'gsd'`, `'gssm'`, and `'gsepla'`.

The `'org.gnu.gdb.s390.gsbc'` feature is optional. It should contain the 64-bit wide guarded-storage broadcast control registers `'bc_gsd'`, `'bc_gssm'`, and `'bc_gsepla'`.

G.5.16 Sparc Features

The `'org.gnu.gdb.sparc.cpu'` feature is required for sparc32/sparc64 targets. It should describe the following registers:

- `'g0'` through `'g7'`
- `'o0'` through `'o7'`
- `'l0'` through `'l7'`
- `'i0'` through `'i7'`

They may be 32-bit or 64-bit depending on the target.

Also the `'org.gnu.gdb.sparc.fpu'` feature is required for sparc32/sparc64 targets. It should describe the following registers:

- `'f0'` through `'f31'`
- `'f32'` through `'f62'` for sparc64

The `'org.gnu.gdb.sparc.cp0'` feature is required for sparc32/sparc64 targets. It should describe the following registers:

- `'y'`, `'psr'`, `'wim'`, `'tbr'`, `'pc'`, `'npc'`, `'fsr'`, and `'csr'` for sparc32
- `'pc'`, `'npc'`, `'state'`, `'fsr'`, `'fprs'`, and `'y'` for sparc64

G.5.17 TMS320C6x Features

The `'org.gnu.gdb.tic6x.core'` feature is required for TMS320C6x targets. It should contain registers `'A0'` through `'A15'`, registers `'B0'` through `'B15'`, `'CSR'` and `'PC'`.

The `'org.gnu.gdb.tic6x.gp'` feature is optional. It should contain registers `'A16'` through `'A31'` and `'B16'` through `'B31'`.

The `'org.gnu.gdb.tic6x.c6xp'` feature is optional. It should contain registers `'TSR'`, `'ILC'` and `'RILC'`.

Appendix H Operating System Information

Users of GDB often wish to obtain information about the state of the operating system running on the target—for example the list of processes, or the list of open files. This section describes the mechanism that makes it possible. This mechanism is similar to the target features mechanism (see Appendix G [Target Descriptions], page 883), but focuses on a different aspect of target.

Operating system information is retrieved from the target via the remote protocol, using ‘qXfer’ requests (see [qXfer osdata read], page 837). The object name in the request should be ‘osdata’, and the *annex* identifies the data to be fetched.

H.1 Process list

When requesting the process list, the *annex* field in the ‘qXfer’ request should be ‘processes’. The returned data is an XML document. The formal syntax of this document is defined in `gdb/features/osdata.dtd`.

An example document is:

```
<?xml version="1.0"?>
<!DOCTYPE target SYSTEM "osdata.dtd">
<osdata type="processes">
  <item>
    <column name="pid">1</column>
    <column name="user">root</column>
    <column name="command">/sbin/init</column>
    <column name="cores">1,2,3</column>
  </item>
</osdata>
```

Each item should include a column whose name is ‘pid’. The value of that column should identify the process on the target. The ‘user’ and ‘command’ columns are optional, and will be displayed by GDB. The ‘cores’ column, if present, should contain a comma-separated list of cores that this process is running on. Target may provide additional columns, which GDB currently ignores.

Appendix I Trace File Format

The trace file comes in three parts: a header, a textual description section, and a trace frame section with binary data.

The header has the form `\x7fTRACE0\n`. The first byte is `0x7f` so as to indicate that the file contains binary data, while the `0` is a version number that may have different values in the future.

The description section consists of multiple lines of ASCII text separated by newline characters (`0xa`). The lines may include a variety of optional descriptive or context-setting information, such as tracepoint definitions or register set size. GDB will ignore any line that it does not recognize. An empty line marks the end of this section.

R *size* Specifies the size of a register block in bytes. This is equal to the size of a `g` packet payload in the remote protocol. *size* is an ascii decimal number. There should be only one such line in a single trace file.

status *status*

Trace status. *status* has the same format as a `qTStatus` remote packet reply. There should be only one such line in a single trace file.

tp *payload*

Tracepoint definition. The *payload* has the same format as `qTfP/qTsP` remote packet reply payload. A single tracepoint may take multiple lines of definition, corresponding to the multiple reply packets.

tsv *payload*

Trace state variable definition. The *payload* has the same format as `qTfV/qTsV` remote packet reply payload. A single variable may take multiple lines of definition, corresponding to the multiple reply packets.

tdesc *payload*

Target description in XML format. The *payload* is a single line of the XML file. All such lines should be concatenated together to get the original XML file. This file is in the same format as `qXfer features` payload, and corresponds to the main `target.xml` file. Includes are not allowed.

The trace frame section consists of a number of consecutive frames. Each frame begins with a two-byte tracepoint number, followed by a four-byte size giving the amount of data in the frame. The data in the frame consists of a number of blocks, each introduced by a character indicating its type (at least register, memory, and trace state variable). The data in this section is raw binary, not a hexadecimal or other encoding; its endianness matches the target's endianness.

R *bytes* Register block. The number and ordering of bytes matches that of a `g` packet in the remote protocol. Note that these are the actual bytes, in target order, not a hexadecimal encoding.

M *address length bytes...*

Memory block. This is a contiguous block of memory, at the 8-byte address *address*, with a 2-byte length *length*, followed by *length* bytes.

V *number value*

Trace state variable block. This records the 8-byte signed value *value* of trace state variable numbered *number*.

Future enhancements of the trace file format may include additional types of blocks.

Appendix J .gdb_index section format

This section documents the index section that is created by `save gdb-index` (see Section 18.5 [Index Files], page 307). The index section is DWARF-specific; some knowledge of DWARF is assumed in this description.

The mapped index file format is designed to be directly `mmap`able on any architecture. In most cases, a datum is represented using a little-endian 32-bit integer value, called an `offset_type`. Big endian machines must byte-swap the values before using them. Exceptions to this rule are noted. The data is laid out such that alignment is always respected.

A mapped index consists of several areas, laid out in order.

1. The file header. This is a sequence of values, of `offset_type` unless otherwise noted:
 1. The version number, currently 9. Versions 1 through 6 are obsolete. Version 7 adds attributes to the CU indices in the symbol table. Version 8 specifies that symbols from DWARF type units ('`DW_TAG_type_unit`') refer to the type unit's symbol table and not the compilation unit ('`DW_TAG_comp_unit`') using the type. Version 9 adds the name and the language of the main function to the index.
GDB has a workaround for potentially broken version 7 indices so it is currently not flagged as deprecated.
 2. The offset, from the start of the file, of the CU list.
 3. The offset, from the start of the file, of the types CU list. Note that this area can be empty, in which case this offset will be equal to the next offset.
 4. The offset, from the start of the file, of the address area.
 5. The offset, from the start of the file, of the symbol table.
 6. The offset, from the start of the file, of the shortcut table. This field was introduced with index version 9.
 7. The offset, from the start of the file, of the constant pool.
2. The CU list. This is a sequence of pairs of 64-bit little-endian values, sorted by the CU offset. The first element in each pair is the offset of a CU in the `.debug_info` section. The second element in each pair is the length of that CU. References to a CU elsewhere in the map are done using a CU index, which is just the 0-based index into this table. Note that if there are type CUs, then conceptually CUs and type CUs form a single list for the purposes of CU indices.
3. The types CU list. This is a sequence of triplets of 64-bit little-endian values. In a triplet, the first value is the CU offset, the second value is the type offset in the CU, and the third value is the type signature. The types CU list is not sorted.
4. The address area. The address area consists of a sequence of address entries. Each address entry has three elements:
 1. The low address. This is a 64-bit little-endian value.
 2. The high address. This is a 64-bit little-endian value. Like `DW_AT_high_pc`, the value is one byte beyond the end.
 3. The CU index. This is an `offset_type` value.
5. The symbol table. This is an open-addressed hash table. The size of the hash table is always a power of 2.

Each slot in the hash table consists of a pair of `offset_type` values. The first value is the offset of the symbol's name in the constant pool. The second value is the offset of the CU vector in the constant pool.

If both values are 0, then this slot in the hash table is empty. This is ok because while 0 is a valid constant pool index, it cannot be a valid index for both a string and a CU vector.

The hash value for a table entry is computed by applying an iterative hash function to the symbol's name. Starting with an initial value of `r = 0`, each (unsigned) character 'c' in the string is incorporated into the hash using the formula depending on the index version:

Version 4 The formula is `r = r * 67 + c - 113`.

Versions 5 to 7

The formula is `r = r * 67 + tolower(c) - 113`.

The terminating '\0' is not incorporated into the hash.

The step size used in the hash table is computed via `((hash * 17) & (size - 1)) | 1`, where 'hash' is the hash value, and 'size' is the size of the hash table. The step size is used to find the next candidate slot when handling a hash collision.

The names of C++ symbols in the hash table are canonicalized. We don't currently have a simple description of the canonicalization algorithm; if you intend to create new index sections, you must read the code.

6. The shortcut table This is a data structure with the following fields:

Language of main

An `offset_type` value indicating the language of the main function as a `DW_LANG_` constant. This value will be zero if main function information is not present.

Name of main

An `offset_type` value indicating the offset of the main function's name in the constant pool. This value must be ignored if the value for the language of main is zero.

This data structure was introduced in index version 9.

7. The constant pool. This is simply a bunch of bytes. It is organized so that alignment is correct: CU vectors are stored first, followed by strings.

A CU vector in the constant pool is a sequence of `offset_type` values. The first value is the number of CU indices in the vector. Each subsequent value is the index and symbol attributes of a CU in the CU list. This element in the hash table is used to indicate which CUs define the symbol and how the symbol is used. See below for the format of each CU index+attributes entry.

A string in the constant pool is zero-terminated.

Attributes were added to CU index values in `.gdb_index` version 7. If a symbol has multiple uses within a CU then there is one CU index+attributes value for each use.

The format of each CU index+attributes entry is as follows (bit 0 = LSB):

Bits 0-23 This is the index of the CU in the CU list.

Bits 24-27 These bits are reserved for future purposes and must be zero.

Bits 28-30 The kind of the symbol in the CU.

- 0 This value is reserved and should not be used. By reserving zero the full `offset_type` value is backwards compatible with previous versions of the index.
- 1 The symbol is a type.
- 2 The symbol is a variable or an enum value.
- 3 The symbol is a function.
- 4 Any other kind of symbol.
- 5,6,7 These values are reserved.

Bit 31 This bit is zero if the value is global and one if it is static.

The determination of whether a symbol is global or static is complicated. The authoritative reference is the file `dwarf2read.c` in GDB sources.

This pseudo-code describes the computation of a symbol's kind and global/static attributes in the index.

```
is_external = get_attribute (die, DW_AT_external);
language = get_attribute (cu_die, DW_AT_language);
switch (die->tag)
{
  case DW_TAG_typedef:
  case DW_TAG_base_type:
  case DW_TAG_subrange_type:
    kind = TYPE;
    is_static = 1;
    break;
  case DW_TAG_enumerator:
    kind = VARIABLE;
    is_static = language != CPLUS;
    break;
  case DW_TAG_subprogram:
    kind = FUNCTION;
    is_static = ! (is_external || language == ADA);
    break;
  case DW_TAG_constant:
    kind = VARIABLE;
    is_static = ! is_external;
    break;
  case DW_TAG_variable:
    kind = VARIABLE;
    is_static = ! is_external;
    break;
  case DW_TAG_namespace:
    kind = TYPE;
    is_static = 0;
    break;
  case DW_TAG_class_type:
  case DW_TAG_interface_type:
  case DW_TAG_structure_type:
  case DW_TAG_union_type:
```

```
case DW_TAG_enumeration_type:
    kind = TYPE;
    is_static = language != CPLUS;
    break;
default:
    assert (0);
}
```

Appendix K Download debugging resources with Debuginfod

`debuginfod` is an HTTP server for distributing ELF, DWARF and source files.

With the `debuginfod` client library, `libdebuginfod`, GDB can query servers using the build IDs associated with missing debug info, executables and source files in order to download them on demand.

For instructions on building GDB with `libdebuginfod`, see Section C.5 [–with-debuginfod], page 771. `debuginfod` is packaged with `elfutils`, starting with version 0.178. See <https://sourceware.org/elfutils/Debuginfod.html> for more information regarding `debuginfod`.

K.1 Debuginfod Settings

GDB provides the following commands for configuring `debuginfod`.

`set debuginfod enabled`

`set debuginfod enabled on`

GDB may query `debuginfod` servers for missing debug info and source files. GDB may also download individual ELF/DWARF sections such as `.gdb_index` to help reduce the total amount of data downloaded from `debuginfod` servers; this can be controlled by `maint set debuginfod download-sections` (see Appendix D [Maintenance Commands], page 777).

`set debuginfod enabled off`

GDB will not attempt to query `debuginfod` servers when missing debug info or source files. By default, `debuginfod enabled` is set to `off` for non-interactive sessions.

`set debuginfod enabled ask`

GDB will prompt the user to enable or disable `debuginfod` before attempting to perform the next query. By default, `debuginfod enabled` is set to `ask` for interactive sessions.

`show debuginfod enabled`

Display whether `debuginfod enabled` is set to `on`, `off` or `ask`.

`set debuginfod urls`

`set debuginfod urls urls`

Set the space-separated list of URLs that `debuginfod` will attempt to query. Only `http://`, `https://` and `file://` protocols should be used. The default value of `debuginfod urls` is copied from the `DEBUGINFOD_URLS` environment variable.

`show debuginfod urls`

Display the list of URLs that `debuginfod` will attempt to query.

`set debuginfod verbose`

`set debuginfod verbose n`

Enable or disable `debuginfod`-related output. Use a non-zero value to enable and 0 to disable. `debuginfod` output is shown by default.

show debuginfod verbose

Show the current verbosity setting.

set progress-bars enabled [on|off]

Set whether GDB can display a progress bar when downloading a file from debuginfod. When ‘off’, GDB will not display a progress bar. When ‘on’, GDB will display a progress bar if GDB’s output console supports it.

show progress-bars enabled

Shows whether progress bars are currently enabled or not. Progress bars can be automatically disabled if GDB’s output console doesn’t support them, or if the terminal width is too small (see Section 22.4 [*set width* command], page 374).

Appendix L Manual pages

`gdb` man

`gdb` [OPTIONS] [*prog*|*prog procID*|*prog core*]

The purpose of a debugger such as GDB is to allow you to see what is going on “inside” another program while it executes – or what another program was doing at the moment it crashed.

GDB can do four main kinds of things (plus other things in support of these) to help you catch bugs in the act:

- Start your program, specifying anything that might affect its behavior.
- Make your program stop on specified conditions.
- Examine what has happened, when your program has stopped.
- Change things in your program, so you can experiment with correcting the effects of one bug and go on to learn about another.

You can use GDB to debug programs written in C, C++, Fortran and Modula-2.

GDB is invoked with the shell command `gdb`. Once started, it reads commands from the terminal until you tell it to exit with the GDB command `quit` or `exit`. You can get online help from GDB itself by using the command `help`.

You can run `gdb` with no arguments or options; but the most usual way to start GDB is with one argument or two, specifying an executable program as the argument:

```
gdb program
```

You can also start with both an executable program and a core file specified:

```
gdb program core
```

You can, instead, specify a process ID as a second argument or use option `-p`, if you want to debug a running process:

```
gdb program 1234
gdb -p 1234
```

would attach GDB to process 1234. With option `-p` you can omit the *program* filename.

Here are some of the most frequently needed GDB commands:

`break` [*file:*][*function*|*line*]

Set a breakpoint at *function* or *line* (in *file*).

`run` [*arglist*]

Start your program (with *arglist*, if specified).

`bt` Backtrace: display the program stack.

`print` *expr*

Display the value of an expression.

`c` Continue running your program (after stopping, e.g. at a breakpoint).

`next` Execute next program line (after stopping); step *over* any function calls in the line.

edit [*file:*]*function*
 look at the program line where it is presently stopped.

list [*file:*]*function*
 type the text of the program in the vicinity of where it is presently stopped.

step Execute next program line (after stopping); step *into* any function calls in the line.

help [*name*]
 Show information about GDB command *name*, or general information about using GDB.

quit

exit Exit from GDB.

Any arguments other than options specify an executable file and core file (or process ID); that is, the first argument encountered with no associated option flag is equivalent to a **--se** option, and the second, if any, is equivalent to a **-c** option if it's the name of a file. Many options have both long and abbreviated forms; both are shown here. The long forms are also recognized if you truncate them, so long as enough of the option is present to be unambiguous.

The abbreviated forms are shown here with '-' and long forms are shown with '--' to reflect how they are shown in **--help**. However, GDB recognizes all of the following conventions for most options:

```
--option=value
--option value
-option=value
-option value
--o=value
--o value
-o=value
-o value
```

All the options and command line arguments you give are processed in sequential order. The order makes a difference when the **-x** option is used.

```
--help
-h           List all options, with brief explanations.
```

```
--symbols=file
-s file      Read symbol table from file.
```

```
--write      Enable writing into executable and core files.
```

```
--exec=file
-e file      Use file as the executable file to execute when appropriate, and for examining pure data in conjunction with a core dump.
```

```
--se=file
             Read symbol table from file and use it as the executable file.
```

--core=*file*
-c *file* Use *file* as a core dump to examine.
--command=*file*
-x *file* Execute GDB commands from *file*.
--eval-command=*command*
-ex *command*
 Execute given GDB *command*.
--init-eval-command=*command*
-iex Execute GDB *command* before loading the inferior.
--directory=*directory*
-d *directory*
 Add *directory* to the path to search for source files.
--nh Do not execute commands from `~/.config/gdb/gdbinit`, `~/.gdbinit`,
 `~/.config/gdb/gdbearlyinit`, or `~/.gdbearlyinit`
--nx
-n Do not execute commands from any `.gdbinit` or `.gdbearlyinit` initialization
 files.
--quiet
--silent
-q “Quiet”. Do not print the introductory and copyright messages. These mes-
 sages are also suppressed in batch mode.
--batch Run in batch mode. Exit with status 0 after processing all the command files
 specified with **-x** (and `.gdbinit`, if not inhibited). Exit with nonzero status if
 an error occurs in executing the GDB commands in the command files.
 Batch mode may be useful for running GDB as a filter, for example to download
 and run a program on another computer; in order to make this more useful, the
 message
 Program exited normally.
 (which is ordinarily issued whenever a program running under GDB control
 terminates) is not issued when running in batch mode.
--batch-silent
 Run in batch mode, just like **--batch**, but totally silent. All GDB output is
 suppressed (stderr is unaffected). This is much quieter than **--silent** and
 would be useless for an interactive session.
 This is particularly useful when using targets that give ‘Loading section’ mes-
 sages, for example.
 Note that targets that give their output via GDB, as opposed to writing directly
 to `stdout`, will also be made silent.
--args *prog* [*arglist*]
--no-escape-args *prog* [*arglist*]
 Change interpretation of command line so that arguments following either of
 these options are passed as arguments to the inferior. As an example, take the
 following command:

```
gdb ./a.out -q
```

It would start GDB with `-q`, not printing the introductory message. On the other hand, using:

```
gdb --args ./a.out -q
```

starts GDB with the introductory message, and passes the option `-q` to the inferior.

The difference between `--args` and `--no-escape-args` is whether GDB applies escapes to the arguments it sees:

```
gdb --args ./a.out *.c
```

in this case the `*.c` is expanded by the shell that invokes GDB, the list of matching files will be fixed in the inferior argument list. If instead this is used:

```
gdb --args ./a.out '*.c'
```

then the shell that invokes GDB will not expand `*.c`, instead GDB will escape the `*` character, so when `a.out` is invoked it will be passed a literal `*.c`. If instead this is used:

```
gdb --no-escape-args ./a.out '*.c'
```

now GDB will not escape the `*` character. When the inferior is invoked the `*.c` will be expanded, and the inferior will be passed the list of files as present at the time the inferior is invoked.

This change of behaviour can be important if the list of matching files could change between the time that GDB is started, and the time the inferior is started.

--pid=*pid*

Attach GDB to an already running program, with the PID *pid*.

--tui Open the terminal user interface.

--readnow Read all symbols from the given symfile on the first access.

--readnever Do not read symbol files.

--return-child-result GDB's exit code will be the same as the child's exit code.

--configuration Print details about GDB configuration and then exit.

--version Print version information and then exit.

--cd=*directory* Run GDB using *directory* as its working directory, instead of the current directory.

--data-directory=*directory*

-D Run GDB using *directory* as its data directory. The data directory is where GDB searches for its auxiliary files.

- fullname**
- f** Emacs sets this option when it runs GDB as a subprocess. It tells GDB to output the full file name and line number in a standard, recognizable fashion each time a stack frame is displayed (which includes each time the program stops). This recognizable format looks like two ‘\032’ characters, followed by the file name, line number and character position separated by colons, and a newline. The Emacs-to-GDB interface program uses the two ‘\032’ characters as a signal to display the source code for the frame.
- b *baudrate***
Set the line speed (baud rate or bits per second) of any serial interface used by GDB for remote debugging.
- l *timeout***
Set timeout, in seconds, for remote debugging.
- tty=*device***
Run using *device* for your program’s standard input and output.

gdbserver man

`gdbserver comm prog [args...]`

`gdbserver -attach comm pid`

`gdbserver -multi comm`

gdbserver is a program that allows you to run GDB on a different machine than the one which is running the program being debugged.

Usage (server (target) side)

First, you need to have a copy of the program you want to debug put onto the target system. The program can be stripped to save space if needed, as **gdbserver** doesn’t care about symbols. All symbol handling is taken care of by the GDB running on the host system.

To use the server, you log on to the target system, and run the **gdbserver** program. You must tell it (a) how to communicate with GDB, (b) the name of your program, and (c) its arguments. The general syntax is:

```
target> gdbserver comm program [args...]
```

For example, using a serial port, you might say:

```
target> gdbserver /dev/com1 emacs foo.txt
```

This tells **gdbserver** to debug emacs with an argument of foo.txt, and to communicate with GDB via /dev/com1. **gdbserver** now waits patiently for the host GDB to communicate with it.

To use a TCP connection, you could say:

```
target> gdbserver host:2345 emacs foo.txt
```

This says pretty much the same thing as the last example, except that we are going to communicate with the **host** GDB via TCP. The **host:2345** argument means that we are expecting to see a TCP connection from **host** to local TCP port 2345. (Currently, the **host**

part is ignored.) You can choose any number you want for the port number as long as it does not conflict with any existing TCP ports on the target system. This same port number must be used in the host GDBs **target remote** command, which will be described shortly. Note that if you chose a port number that conflicts with another service, **gdbserver** will print an error message and exit.

gdbserver can also attach to running programs. This is accomplished via the **--attach** argument. The syntax is:

```
target> gdbserver --attach comm pid
```

pid is the process ID of a currently running process. It isn't necessary to point **gdbserver** at a binary for the running process.

To start **gdbserver** without supplying an initial command to run or process ID to attach, use the **--multi** command line option. In such case you should connect using **target extended-remote** to start the program you want to debug.

```
target> gdbserver --multi comm
```

Usage (host side)

You need an unstripped copy of the target program on your host system, since GDB needs to examine its symbol tables and such. Start up GDB as you normally would, with the target program as the first argument. (You may need to use the **--baud** option if the serial line is running at anything except 9600 baud.) That is **gdb TARGET-PROG**, or **gdb --baud BAUD TARGET-PROG**. After that, the only new command you need to know about is **target remote** (or **target extended-remote**). Its argument is either a device name (usually a serial device, like **/dev/ttyb**), or a **HOST:PORT** descriptor. For example:

```
(gdb) target remote /dev/ttyb
```

communicates with the server via serial line **/dev/ttyb**, and:

```
(gdb) target remote the-target:2345
```

communicates via a TCP connection to port 2345 on host 'the-target', where you previously started up **gdbserver** with the same port number. Note that for TCP connections, you must start up **gdbserver** prior to using the 'target remote' command, otherwise you may get an error that looks something like 'Connection refused'.

gdbserver can also debug multiple inferiors at once, described in Section 4.9 [Inferiors Connections and Programs], page 43. In such case use the **extended-remote** GDB command variant:

```
(gdb) target extended-remote the-target:2345
```

The **gdbserver** option **--multi** may or may not be used in such case.

There are three different modes for invoking **gdbserver**:

- Debug a specific program specified by its program name:

```
gdbserver comm prog [args...]
```

The *comm* parameter specifies how should the server communicate with GDB; it is either a device name (to use a serial line), a TCP port number (**:1234**), or **-** or **stdio** to use stdin/stdout of **gdbserver**. Specify the name of the program to debug in *prog*. Any remaining arguments will be passed to the program verbatim. When the program exits, GDB will close the connection, and **gdbserver** will exit.

- Debug a specific program by specifying the process ID of a running program:

```
gdbserver --attach comm pid
```

The *comm* parameter is as described above. Supply the process ID of a running program in *pid*; GDB will do everything else. Like with the previous mode, when the process *pid* exits, GDB will close the connection, and **gdbserver** will exit.

- Multi-process mode – debug more than one program/process:

```
gdbserver --multi comm
```

In this mode, GDB can instruct **gdbserver** which command(s) to run. Unlike the other 2 modes, GDB will not close the connection when a process being debugged exits, so you can debug several processes in the same session.

In each of the modes you may specify these options:

--help List all options, with brief explanations.

--version

This option causes **gdbserver** to print its version number and exit.

--attach **gdbserver** will attach to a running program. The syntax is:

```
target> gdbserver --attach comm pid
```

pid is the process ID of a currently running process. It isn't necessary to point **gdbserver** at a binary for the running process.

--multi To start **gdbserver** without supplying an initial command to run or process ID to attach, use this command line option. Then you can connect using *target extended-remote* and start the program you want to debug. The syntax is:

```
target> gdbserver --multi comm
```

--debug[=option1,option2,...]

Instruct **gdbserver** to display extra status information about the debugging process. This option is intended for **gdbserver** development and for bug reports to the developers.

Each *option* is the name of a component for which debugging should be enabled. The list of possible options is **all**, **threads**, **event-loop**, **remote**. The special option **all** enables all components. The option list is processed left to right, and an option can be prefixed with the - character to disable output for that component, so you could write:

```
target> gdbserver --debug=all,-event-loop
```

to turn on debug output for all components except **event-loop**. If no options are passed to **--debug** then this is treated as equivalent to **--debug=threads**. This could change in future releases of **gdbserver**.

--debug-file=filename

Instruct **gdbserver** to send any debug output to the given *filename*. This option is intended for **gdbserver** development and for bug reports to the developers.

--debug-format=option1[,option2,...]

Instruct **gdbserver** to include extra information in each line of debugging output. See [Other Command-Line Arguments for **gdbserver**], page 324.

--wrapper

Specify a wrapper to launch programs for debugging. The option should be followed by the name of the wrapper, then any command-line arguments to pass to the wrapper, then `--` indicating the end of the wrapper arguments.

--once

By default, `gdbserver` keeps the listening TCP port open, so that additional connections are possible. However, if you start `gdbserver` with the `--once` option, it will stop listening for any further connection attempts after connecting to the first GDB session.

--no-escape-args

By default, inferior arguments passed on the `gdbserver` command line will have any special shell characters escaped by `gdbserver`. This ensures that when `gdbserver` invokes the inferior, the arguments passed to the inferior are identical to the arguments passed to `gdbserver`.

To disable this escaping, use `--no-escape-args`. With this option special shell characters will not be escaped. When `gdbserver` starts a new shell in order to invoke the inferior, this new shell will expand any special shell characters.

gcore

```
gcore [-v | -version] [-h | -help] [-a] [-o prefix] [-d directory]
      pid1 [pid2...pidN]
```

Generate core dumps of one or more running programs with process IDs *pid1*, *pid2*, etc. A core file produced by `gcore` is equivalent to one produced by the kernel when the process crashes (and when `ulimit -c` was used to set up an appropriate core dump limit). However, unlike after a crash, after `gcore` finishes its job the program remains running without any change.

-a Dump all memory mappings. The actual effect of this option depends on the Operating System. On GNU/Linux, it will disable `use-coredump-filter` (see [set use-coredump-filter], page 189) and enable `dump-excluded-mappings` (see [set dump-excluded-mappings], page 189).

-o *prefix* The optional argument *prefix* specifies the prefix to be used when composing the file names of the core dumps. The file name is composed as *prefix.pid*, where *pid* is the process ID of the running program being analyzed by `gcore`. If not specified, *prefix* defaults to *core*.

-g *gdb* Use *gdb* as the executable binary to invoke GDB for running the `gcore` command. This argument is optional, and defaults to invoking the GDB binary that is installed alongside `gcore`.

-d *directory*

Use *directory* as the data directory when invoking GDB for running the `gcore` command. This argument is optional.

--help

-h List all options, with brief explanations.

--version

-v Print version information and then exit.

gstack

`gstack [-h | -help] [-v | -version] pid`

Print a stack trace of a running program with process ID *pid*. If the process is multi-threaded, **gstack** outputs backtraces for every thread which exists in the process.

The script invokes GDB, attaches to the given process ID, prints the stack trace, and detaches from the process.

gstack exits with non-zero status if **gdb** was unable to attach to the given process ID for any reason, such as a non-existent process ID or insufficient privileges to attach to the process.

--help

-h List all options, with brief explanations.

--version

-v Print version information and then exit.

AWK Full file name for an Awk interpreter to use. If not set, **PATH** will be searched for an **awk** program.

GDB Full file name for a GDB executable to use to generate stack backtraces. If not set, **PATH** will be searched for a **gdb** program.

GDBARGS Optional arguments to be passed to the **gdb** program.

gdbinit

`~/.config/gdb/gdbinit`

`~/.gdbinit`

`./gdbinit`

These files contain GDB commands to automatically execute during GDB startup. The lines of contents are canned sequences of commands, described in Section 23.1 [Sequences], page 397.

Please read more in Section 2.1.3 [Startup], page 17.

(not enabled with **--with-system-gdbinit** during compilation)

System-wide initialization file. It is executed unless user specified GDB option **-nx** or **-n**. See more in

(not enabled with **--with-system-gdbinit-dir** during compilation)

System-wide initialization directory. All files in this directory are executed on startup unless user specified GDB option **-nx** or **-n**, as long as they have a recognized file extension. See more in Section C.6 [System-wide configuration], page 775.

`~/.config/gdb/gdbinit` or `~/.gdbinit`

User initialization file. It is executed unless user specified GDB options **-nx**, **-n** or **-nh**.

.gdbinit Initialization file for current directory. It may need to be enabled with GDB security command `set auto-load local-gdbinit`. See more in Section 22.8.1 [Init File in the Current Directory], page 383.

gdb-add-index

`gdb-add-index [-h | -help] [-v | -version] [-dwarf-5] filename`

When GDB finds a symbol file, it scans the symbols in the file in order to construct an internal symbol table. This lets most GDB operations work quickly—at the cost of a delay early on. For large programs, this delay can be quite lengthy, so GDB provides a way to build an index, which speeds up startup.

To determine whether a file contains such an index, use the command `readelf -S filename`: the index is stored in a section named `.gdb_index` (pre-DWARF 5) or `.debug_names` and `.debug_str` (DWARF 5). Indexes can only be produced on systems which use ELF binaries and DWARF debug information (i.e., sections named `.debug_*`).

By default `gdb-add-index` will add a pre-DWARF 5 `.gdb_index` section to *filename*. With `--dwarf-5` the DWARF 5 section `.debug_names` is added instead.

filename must be writable.

`gdb-add-index` uses GDB, `objcopy`, and `readelf` found in the `PATH` environment variable. If you want to use different versions of these programs, you can specify them through the appropriate environment variables (see below).

`gdb-add-index` exits with status 0 if it succeeds in creating the index for *filename* or greater than 0 if an error occurs.

See more in Section 18.5 [Index Files], page 307.

--dwarf-5

Add DWARF 5 sections instead of previous `.gdb_index` section.

--help

-h List all options, with brief explanations.

--version

-v Print version information and then exit.

GDB Full file name of the `gdb` program to use for index generation. If not set, the `PATH` will be searched for a `gdb` program.

OBJCOPY Full file name of the `objcopy` program to use to copy section information into the given file. If not set, the `PATH` will be searched for a `objcopy` program.

READELF Full file name of the `readelf` program to use to inspect properties of the given file. If not set, the `PATH` will be searched for a `readelf` program.

Appendix M GNU GENERAL PUBLIC LICENSE

Version 3, 29 June 2007

Copyright © 2007 Free Software Foundation, Inc. <http://fsf.org/>

Everyone is permitted to copy and distribute verbatim copies of this license document, but changing it is not allowed.

Preamble

The GNU General Public License is a free, copyleft license for software and other kinds of works.

The licenses for most software and other practical works are designed to take away your freedom to share and change the works. By contrast, the GNU General Public License is intended to guarantee your freedom to share and change all versions of a program—to make sure it remains free software for all its users. We, the Free Software Foundation, use the GNU General Public License for most of our software; it applies also to any other work released this way by its authors. You can apply it to your programs, too.

When we speak of free software, we are referring to freedom, not price. Our General Public Licenses are designed to make sure that you have the freedom to distribute copies of free software (and charge for them if you wish), that you receive source code or can get it if you want it, that you can change the software or use pieces of it in new free programs, and that you know you can do these things.

To protect your rights, we need to prevent others from denying you these rights or asking you to surrender the rights. Therefore, you have certain responsibilities if you distribute copies of the software, or if you modify it: responsibilities to respect the freedom of others.

For example, if you distribute copies of such a program, whether gratis or for a fee, you must pass on to the recipients the same freedoms that you received. You must make sure that they, too, receive or can get the source code. And you must show them these terms so they know their rights.

Developers that use the GNU GPL protect your rights with two steps: (1) assert copyright on the software, and (2) offer you this License giving you legal permission to copy, distribute and/or modify it.

For the developers' and authors' protection, the GPL clearly explains that there is no warranty for this free software. For both users' and authors' sake, the GPL requires that modified versions be marked as changed, so that their problems will not be attributed erroneously to authors of previous versions.

Some devices are designed to deny users access to install or run modified versions of the software inside them, although the manufacturer can do so. This is fundamentally incompatible with the aim of protecting users' freedom to change the software. The systematic pattern of such abuse occurs in the area of products for individuals to use, which is precisely where it is most unacceptable. Therefore, we have designed this version of the GPL to prohibit the practice for those products. If such problems arise substantially in other domains, we stand ready to extend this provision to those domains in future versions of the GPL, as needed to protect the freedom of users.

Finally, every program is threatened constantly by software patents. States should not allow patents to restrict development and use of software on general-purpose computers, but in those that do, we wish to avoid the special danger that patents applied to a free program could make it effectively proprietary. To prevent this, the GPL assures that patents cannot be used to render the program non-free.

The precise terms and conditions for copying, distribution and modification follow.

TERMS AND CONDITIONS

0. Definitions.

“This License” refers to version 3 of the GNU General Public License.

“Copyright” also means copyright-like laws that apply to other kinds of works, such as semiconductor masks.

“The Program” refers to any copyrightable work licensed under this License. Each licensee is addressed as “you”. “Licensees” and “recipients” may be individuals or organizations.

To “modify” a work means to copy from or adapt all or part of the work in a fashion requiring copyright permission, other than the making of an exact copy. The resulting work is called a “modified version” of the earlier work or a work “based on” the earlier work.

A “covered work” means either the unmodified Program or a work based on the Program.

To “propagate” a work means to do anything with it that, without permission, would make you directly or secondarily liable for infringement under applicable copyright law, except executing it on a computer or modifying a private copy. Propagation includes copying, distribution (with or without modification), making available to the public, and in some countries other activities as well.

To “convey” a work means any kind of propagation that enables other parties to make or receive copies. Mere interaction with a user through a computer network, with no transfer of a copy, is not conveying.

An interactive user interface displays “Appropriate Legal Notices” to the extent that it includes a convenient and prominently visible feature that (1) displays an appropriate copyright notice, and (2) tells the user that there is no warranty for the work (except to the extent that warranties are provided), that licensees may convey the work under this License, and how to view a copy of this License. If the interface presents a list of user commands or options, such as a menu, a prominent item in the list meets this criterion.

1. Source Code.

The “source code” for a work means the preferred form of the work for making modifications to it. “Object code” means any non-source form of a work.

A “Standard Interface” means an interface that either is an official standard defined by a recognized standards body, or, in the case of interfaces specified for a particular programming language, one that is widely used among developers working in that language.

The “System Libraries” of an executable work include anything, other than the work as a whole, that (a) is included in the normal form of packaging a Major Component, but which is not part of that Major Component, and (b) serves only to enable use of the work with that Major Component, or to implement a Standard Interface for which an implementation is available to the public in source code form. A “Major Component”, in this context, means a major essential component (kernel, window system, and so on) of the specific operating system (if any) on which the executable work runs, or a compiler used to produce the work, or an object code interpreter used to run it.

The “Corresponding Source” for a work in object code form means all the source code needed to generate, install, and (for an executable work) run the object code and to modify the work, including scripts to control those activities. However, it does not include the work’s System Libraries, or general-purpose tools or generally available free programs which are used unmodified in performing those activities but which are not part of the work. For example, Corresponding Source includes interface definition files associated with source files for the work, and the source code for shared libraries and dynamically linked subprograms that the work is specifically designed to require, such as by intimate data communication or control flow between those subprograms and other parts of the work.

The Corresponding Source need not include anything that users can regenerate automatically from other parts of the Corresponding Source.

The Corresponding Source for a work in source code form is that same work.

2. Basic Permissions.

All rights granted under this License are granted for the term of copyright on the Program, and are irrevocable provided the stated conditions are met. This License explicitly affirms your unlimited permission to run the unmodified Program. The output from running a covered work is covered by this License only if the output, given its content, constitutes a covered work. This License acknowledges your rights of fair use or other equivalent, as provided by copyright law.

You may make, run and propagate covered works that you do not convey, without conditions so long as your license otherwise remains in force. You may convey covered works to others for the sole purpose of having them make modifications exclusively for you, or provide you with facilities for running those works, provided that you comply with the terms of this License in conveying all material for which you do not control copyright. Those thus making or running the covered works for you must do so exclusively on your behalf, under your direction and control, on terms that prohibit them from making any copies of your copyrighted material outside their relationship with you.

Conveying under any other circumstances is permitted solely under the conditions stated below. Sublicensing is not allowed; section 10 makes it unnecessary.

3. Protecting Users’ Legal Rights From Anti-Circumvention Law.

No covered work shall be deemed part of an effective technological measure under any applicable law fulfilling obligations under article 11 of the WIPO copyright treaty adopted on 20 December 1996, or similar laws prohibiting or restricting circumvention of such measures.

When you convey a covered work, you waive any legal power to forbid circumvention of technological measures to the extent such circumvention is effected by exercising rights under this License with respect to the covered work, and you disclaim any intention to limit operation or modification of the work as a means of enforcing, against the work's users, your or third parties' legal rights to forbid circumvention of technological measures.

4. Conveying Verbatim Copies.

You may convey verbatim copies of the Program's source code as you receive it, in any medium, provided that you conspicuously and appropriately publish on each copy an appropriate copyright notice; keep intact all notices stating that this License and any non-permissive terms added in accord with section 7 apply to the code; keep intact all notices of the absence of any warranty; and give all recipients a copy of this License along with the Program.

You may charge any price or no price for each copy that you convey, and you may offer support or warranty protection for a fee.

5. Conveying Modified Source Versions.

You may convey a work based on the Program, or the modifications to produce it from the Program, in the form of source code under the terms of section 4, provided that you also meet all of these conditions:

- a. The work must carry prominent notices stating that you modified it, and giving a relevant date.
- b. The work must carry prominent notices stating that it is released under this License and any conditions added under section 7. This requirement modifies the requirement in section 4 to "keep intact all notices".
- c. You must license the entire work, as a whole, under this License to anyone who comes into possession of a copy. This License will therefore apply, along with any applicable section 7 additional terms, to the whole of the work, and all its parts, regardless of how they are packaged. This License gives no permission to license the work in any other way, but it does not invalidate such permission if you have separately received it.
- d. If the work has interactive user interfaces, each must display Appropriate Legal Notices; however, if the Program has interactive interfaces that do not display Appropriate Legal Notices, your work need not make them do so.

A compilation of a covered work with other separate and independent works, which are not by their nature extensions of the covered work, and which are not combined with it such as to form a larger program, in or on a volume of a storage or distribution medium, is called an "aggregate" if the compilation and its resulting copyright are not used to limit the access or legal rights of the compilation's users beyond what the individual works permit. Inclusion of a covered work in an aggregate does not cause this License to apply to the other parts of the aggregate.

6. Conveying Non-Source Forms.

You may convey a covered work in object code form under the terms of sections 4 and 5, provided that you also convey the machine-readable Corresponding Source under the terms of this License, in one of these ways:

- a. Convey the object code in, or embodied in, a physical product (including a physical distribution medium), accompanied by the Corresponding Source fixed on a durable physical medium customarily used for software interchange.
- b. Convey the object code in, or embodied in, a physical product (including a physical distribution medium), accompanied by a written offer, valid for at least three years and valid for as long as you offer spare parts or customer support for that product model, to give anyone who possesses the object code either (1) a copy of the Corresponding Source for all the software in the product that is covered by this License, on a durable physical medium customarily used for software interchange, for a price no more than your reasonable cost of physically performing this conveying of source, or (2) access to copy the Corresponding Source from a network server at no charge.
- c. Convey individual copies of the object code with a copy of the written offer to provide the Corresponding Source. This alternative is allowed only occasionally and noncommercially, and only if you received the object code with such an offer, in accord with subsection 6b.
- d. Convey the object code by offering access from a designated place (gratis or for a charge), and offer equivalent access to the Corresponding Source in the same way through the same place at no further charge. You need not require recipients to copy the Corresponding Source along with the object code. If the place to copy the object code is a network server, the Corresponding Source may be on a different server (operated by you or a third party) that supports equivalent copying facilities, provided you maintain clear directions next to the object code saying where to find the Corresponding Source. Regardless of what server hosts the Corresponding Source, you remain obligated to ensure that it is available for as long as needed to satisfy these requirements.
- e. Convey the object code using peer-to-peer transmission, provided you inform other peers where the object code and Corresponding Source of the work are being offered to the general public at no charge under subsection 6d.

A separable portion of the object code, whose source code is excluded from the Corresponding Source as a System Library, need not be included in conveying the object code work.

A “User Product” is either (1) a “consumer product”, which means any tangible personal property which is normally used for personal, family, or household purposes, or (2) anything designed or sold for incorporation into a dwelling. In determining whether a product is a consumer product, doubtful cases shall be resolved in favor of coverage. For a particular product received by a particular user, “normally used” refers to a typical or common use of that class of product, regardless of the status of the particular user or of the way in which the particular user actually uses, or expects or is expected to use, the product. A product is a consumer product regardless of whether the product has substantial commercial, industrial or non-consumer uses, unless such uses represent the only significant mode of use of the product.

“Installation Information” for a User Product means any methods, procedures, authorization keys, or other information required to install and execute modified versions of a covered work in that User Product from a modified version of its Corresponding Source.

The information must suffice to ensure that the continued functioning of the modified object code is in no case prevented or interfered with solely because modification has been made.

If you convey an object code work under this section in, or with, or specifically for use in, a User Product, and the conveying occurs as part of a transaction in which the right of possession and use of the User Product is transferred to the recipient in perpetuity or for a fixed term (regardless of how the transaction is characterized), the Corresponding Source conveyed under this section must be accompanied by the Installation Information. But this requirement does not apply if neither you nor any third party retains the ability to install modified object code on the User Product (for example, the work has been installed in ROM).

The requirement to provide Installation Information does not include a requirement to continue to provide support service, warranty, or updates for a work that has been modified or installed by the recipient, or for the User Product in which it has been modified or installed. Access to a network may be denied when the modification itself materially and adversely affects the operation of the network or violates the rules and protocols for communication across the network.

Corresponding Source conveyed, and Installation Information provided, in accord with this section must be in a format that is publicly documented (and with an implementation available to the public in source code form), and must require no special password or key for unpacking, reading or copying.

7. Additional Terms.

“Additional permissions” are terms that supplement the terms of this License by making exceptions from one or more of its conditions. Additional permissions that are applicable to the entire Program shall be treated as though they were included in this License, to the extent that they are valid under applicable law. If additional permissions apply only to part of the Program, that part may be used separately under those permissions, but the entire Program remains governed by this License without regard to the additional permissions.

When you convey a copy of a covered work, you may at your option remove any additional permissions from that copy, or from any part of it. (Additional permissions may be written to require their own removal in certain cases when you modify the work.) You may place additional permissions on material, added by you to a covered work, for which you have or can give appropriate copyright permission.

Notwithstanding any other provision of this License, for material you add to a covered work, you may (if authorized by the copyright holders of that material) supplement the terms of this License with terms:

- a. Disclaiming warranty or limiting liability differently from the terms of sections 15 and 16 of this License; or
- b. Requiring preservation of specified reasonable legal notices or author attributions in that material or in the Appropriate Legal Notices displayed by works containing it; or
- c. Prohibiting misrepresentation of the origin of that material, or requiring that modified versions of such material be marked in reasonable ways as different from the original version; or

- d. Limiting the use for publicity purposes of names of licensors or authors of the material; or
- e. Declining to grant rights under trademark law for use of some trade names, trademarks, or service marks; or
- f. Requiring indemnification of licensors and authors of that material by anyone who conveys the material (or modified versions of it) with contractual assumptions of liability to the recipient, for any liability that these contractual assumptions directly impose on those licensors and authors.

All other non-permissive additional terms are considered “further restrictions” within the meaning of section 10. If the Program as you received it, or any part of it, contains a notice stating that it is governed by this License along with a term that is a further restriction, you may remove that term. If a license document contains a further restriction but permits relicensing or conveying under this License, you may add to a covered work material governed by the terms of that license document, provided that the further restriction does not survive such relicensing or conveying.

If you add terms to a covered work in accord with this section, you must place, in the relevant source files, a statement of the additional terms that apply to those files, or a notice indicating where to find the applicable terms.

Additional terms, permissive or non-permissive, may be stated in the form of a separately written license, or stated as exceptions; the above requirements apply either way.

8. Termination.

You may not propagate or modify a covered work except as expressly provided under this License. Any attempt otherwise to propagate or modify it is void, and will automatically terminate your rights under this License (including any patent licenses granted under the third paragraph of section 11).

However, if you cease all violation of this License, then your license from a particular copyright holder is reinstated (a) provisionally, unless and until the copyright holder explicitly and finally terminates your license, and (b) permanently, if the copyright holder fails to notify you of the violation by some reasonable means prior to 60 days after the cessation.

Moreover, your license from a particular copyright holder is reinstated permanently if the copyright holder notifies you of the violation by some reasonable means, this is the first time you have received notice of violation of this License (for any work) from that copyright holder, and you cure the violation prior to 30 days after your receipt of the notice.

Termination of your rights under this section does not terminate the licenses of parties who have received copies or rights from you under this License. If your rights have been terminated and not permanently reinstated, you do not qualify to receive new licenses for the same material under section 10.

9. Acceptance Not Required for Having Copies.

You are not required to accept this License in order to receive or run a copy of the Program. Ancillary propagation of a covered work occurring solely as a consequence of using peer-to-peer transmission to receive a copy likewise does not require acceptance.

However, nothing other than this License grants you permission to propagate or modify any covered work. These actions infringe copyright if you do not accept this License. Therefore, by modifying or propagating a covered work, you indicate your acceptance of this License to do so.

10. Automatic Licensing of Downstream Recipients.

Each time you convey a covered work, the recipient automatically receives a license from the original licensors, to run, modify and propagate that work, subject to this License. You are not responsible for enforcing compliance by third parties with this License.

An “entity transaction” is a transaction transferring control of an organization, or substantially all assets of one, or subdividing an organization, or merging organizations. If propagation of a covered work results from an entity transaction, each party to that transaction who receives a copy of the work also receives whatever licenses to the work the party’s predecessor in interest had or could give under the previous paragraph, plus a right to possession of the Corresponding Source of the work from the predecessor in interest, if the predecessor has it or can get it with reasonable efforts.

You may not impose any further restrictions on the exercise of the rights granted or affirmed under this License. For example, you may not impose a license fee, royalty, or other charge for exercise of rights granted under this License, and you may not initiate litigation (including a cross-claim or counterclaim in a lawsuit) alleging that any patent claim is infringed by making, using, selling, offering for sale, or importing the Program or any portion of it.

11. Patents.

A “contributor” is a copyright holder who authorizes use under this License of the Program or a work on which the Program is based. The work thus licensed is called the contributor’s “contributor version”.

A contributor’s “essential patent claims” are all patent claims owned or controlled by the contributor, whether already acquired or hereafter acquired, that would be infringed by some manner, permitted by this License, of making, using, or selling its contributor version, but do not include claims that would be infringed only as a consequence of further modification of the contributor version. For purposes of this definition, “control” includes the right to grant patent sublicenses in a manner consistent with the requirements of this License.

Each contributor grants you a non-exclusive, worldwide, royalty-free patent license under the contributor’s essential patent claims, to make, use, sell, offer for sale, import and otherwise run, modify and propagate the contents of its contributor version.

In the following three paragraphs, a “patent license” is any express agreement or commitment, however denominated, not to enforce a patent (such as an express permission to practice a patent or covenant not to sue for patent infringement). To “grant” such a patent license to a party means to make such an agreement or commitment not to enforce a patent against the party.

If you convey a covered work, knowingly relying on a patent license, and the Corresponding Source of the work is not available for anyone to copy, free of charge and under the terms of this License, through a publicly available network server or other readily accessible means, then you must either (1) cause the Corresponding Source to be so

available, or (2) arrange to deprive yourself of the benefit of the patent license for this particular work, or (3) arrange, in a manner consistent with the requirements of this License, to extend the patent license to downstream recipients. “Knowingly relying” means you have actual knowledge that, but for the patent license, your conveying the covered work in a country, or your recipient’s use of the covered work in a country, would infringe one or more identifiable patents in that country that you have reason to believe are valid.

If, pursuant to or in connection with a single transaction or arrangement, you convey, or propagate by procuring conveyance of, a covered work, and grant a patent license to some of the parties receiving the covered work authorizing them to use, propagate, modify or convey a specific copy of the covered work, then the patent license you grant is automatically extended to all recipients of the covered work and works based on it.

A patent license is “discriminatory” if it does not include within the scope of its coverage, prohibits the exercise of, or is conditioned on the non-exercise of one or more of the rights that are specifically granted under this License. You may not convey a covered work if you are a party to an arrangement with a third party that is in the business of distributing software, under which you make payment to the third party based on the extent of your activity of conveying the work, and under which the third party grants, to any of the parties who would receive the covered work from you, a discriminatory patent license (a) in connection with copies of the covered work conveyed by you (or copies made from those copies), or (b) primarily for and in connection with specific products or compilations that contain the covered work, unless you entered into that arrangement, or that patent license was granted, prior to 28 March 2007.

Nothing in this License shall be construed as excluding or limiting any implied license or other defenses to infringement that may otherwise be available to you under applicable patent law.

12. No Surrender of Others’ Freedom.

If conditions are imposed on you (whether by court order, agreement or otherwise) that contradict the conditions of this License, they do not excuse you from the conditions of this License. If you cannot convey a covered work so as to satisfy simultaneously your obligations under this License and any other pertinent obligations, then as a consequence you may not convey it at all. For example, if you agree to terms that obligate you to collect a royalty for further conveying from those to whom you convey the Program, the only way you could satisfy both those terms and this License would be to refrain entirely from conveying the Program.

13. Use with the GNU Affero General Public License.

Notwithstanding any other provision of this License, you have permission to link or combine any covered work with a work licensed under version 3 of the GNU Affero General Public License into a single combined work, and to convey the resulting work. The terms of this License will continue to apply to the part which is the covered work, but the special requirements of the GNU Affero General Public License, section 13, concerning interaction through a network will apply to the combination as such.

14. Revised Versions of this License.

The Free Software Foundation may publish revised and/or new versions of the GNU General Public License from time to time. Such new versions will be similar in spirit to the present version, but may differ in detail to address new problems or concerns.

Each version is given a distinguishing version number. If the Program specifies that a certain numbered version of the GNU General Public License “or any later version” applies to it, you have the option of following the terms and conditions either of that numbered version or of any later version published by the Free Software Foundation. If the Program does not specify a version number of the GNU General Public License, you may choose any version ever published by the Free Software Foundation.

If the Program specifies that a proxy can decide which future versions of the GNU General Public License can be used, that proxy’s public statement of acceptance of a version permanently authorizes you to choose that version for the Program.

Later license versions may give you additional or different permissions. However, no additional obligations are imposed on any author or copyright holder as a result of your choosing to follow a later version.

15. Disclaimer of Warranty.

THERE IS NO WARRANTY FOR THE PROGRAM, TO THE EXTENT PERMITTED BY APPLICABLE LAW. EXCEPT WHEN OTHERWISE STATED IN WRITING THE COPYRIGHT HOLDERS AND/OR OTHER PARTIES PROVIDE THE PROGRAM “AS IS” WITHOUT WARRANTY OF ANY KIND, EITHER EXPRESSED OR IMPLIED, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE. THE ENTIRE RISK AS TO THE QUALITY AND PERFORMANCE OF THE PROGRAM IS WITH YOU. SHOULD THE PROGRAM PROVE DEFECTIVE, YOU ASSUME THE COST OF ALL NECESSARY SERVICING, REPAIR OR CORRECTION.

16. Limitation of Liability.

IN NO EVENT UNLESS REQUIRED BY APPLICABLE LAW OR AGREED TO IN WRITING WILL ANY COPYRIGHT HOLDER, OR ANY OTHER PARTY WHO MODIFIES AND/OR CONVEYS THE PROGRAM AS PERMITTED ABOVE, BE LIABLE TO YOU FOR DAMAGES, INCLUDING ANY GENERAL, SPECIAL, INCIDENTAL OR CONSEQUENTIAL DAMAGES ARISING OUT OF THE USE OR INABILITY TO USE THE PROGRAM (INCLUDING BUT NOT LIMITED TO LOSS OF DATA OR DATA BEING RENDERED INACCURATE OR LOSSES SUSTAINED BY YOU OR THIRD PARTIES OR A FAILURE OF THE PROGRAM TO OPERATE WITH ANY OTHER PROGRAMS), EVEN IF SUCH HOLDER OR OTHER PARTY HAS BEEN ADVISED OF THE POSSIBILITY OF SUCH DAMAGES.

17. Interpretation of Sections 15 and 16.

If the disclaimer of warranty and limitation of liability provided above cannot be given local legal effect according to their terms, reviewing courts shall apply local law that most closely approximates an absolute waiver of all civil liability in connection with the Program, unless a warranty or assumption of liability accompanies a copy of the Program in return for a fee.

END OF TERMS AND CONDITIONS

How to Apply These Terms to Your New Programs

If you develop a new program, and you want it to be of the greatest possible use to the public, the best way to achieve this is to make it free software which everyone can redistribute and change under these terms.

To do so, attach the following notices to the program. It is safest to attach them to the start of each source file to most effectively state the exclusion of warranty; and each file should have at least the “copyright” line and a pointer to where the full notice is found.

```
one line to give the program's name and a brief idea of what it does.
Copyright (C) year name of author
```

```
This program is free software: you can redistribute it and/or modify
it under the terms of the GNU General Public License as published by
the Free Software Foundation, either version 3 of the License, or (at
your option) any later version.
```

```
This program is distributed in the hope that it will be useful, but
WITHOUT ANY WARRANTY; without even the implied warranty of
MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU
General Public License for more details.
```

```
You should have received a copy of the GNU General Public License
along with this program. If not, see http://www.gnu.org/licenses/.
```

Also add information on how to contact you by electronic and paper mail.

If the program does terminal interaction, make it output a short notice like this when it starts in an interactive mode:

```
program Copyright (C) year name of author
This program comes with ABSOLUTELY NO WARRANTY; for details type 'show w'.
This is free software, and you are welcome to redistribute it
under certain conditions; type 'show c' for details.
```

The hypothetical commands ‘show w’ and ‘show c’ should show the appropriate parts of the General Public License. Of course, your program’s commands might be different; for a GUI interface, you would use an “about box”.

You should also get your employer (if you work as a programmer) or school, if any, to sign a “copyright disclaimer” for the program, if necessary. For more information on this, and how to apply and follow the GNU GPL, see <http://www.gnu.org/licenses/>.

The GNU General Public License does not permit incorporating your program into proprietary programs. If your program is a subroutine library, you may consider it more useful to permit linking proprietary applications with the library. If this is what you want to do, use the GNU Lesser General Public License instead of this License. But first, please read <http://www.gnu.org/philosophy/why-not-lgpl.html>.

Appendix N GNU Free Documentation License

Version 1.3, 3 November 2008

Copyright © 2000, 2001, 2002, 2007, 2008 Free Software Foundation, Inc.

<http://fsf.org/>

Everyone is permitted to copy and distribute verbatim copies of this license document, but changing it is not allowed.

0. PREAMBLE

The purpose of this License is to make a manual, textbook, or other functional and useful document *free* in the sense of freedom: to assure everyone the effective freedom to copy and redistribute it, with or without modifying it, either commercially or non-commercially. Secondly, this License preserves for the author and publisher a way to get credit for their work, while not being considered responsible for modifications made by others.

This License is a kind of “copyleft”, which means that derivative works of the document must themselves be free in the same sense. It complements the GNU General Public License, which is a copyleft license designed for free software.

We have designed this License in order to use it for manuals for free software, because free software needs free documentation: a free program should come with manuals providing the same freedoms that the software does. But this License is not limited to software manuals; it can be used for any textual work, regardless of subject matter or whether it is published as a printed book. We recommend this License principally for works whose purpose is instruction or reference.

1. APPLICABILITY AND DEFINITIONS

This License applies to any manual or other work, in any medium, that contains a notice placed by the copyright holder saying it can be distributed under the terms of this License. Such a notice grants a world-wide, royalty-free license, unlimited in duration, to use that work under the conditions stated herein. The “Document”, below, refers to any such manual or work. Any member of the public is a licensee, and is addressed as “you”. You accept the license if you copy, modify or distribute the work in a way requiring permission under copyright law.

A “Modified Version” of the Document means any work containing the Document or a portion of it, either copied verbatim, or with modifications and/or translated into another language.

A “Secondary Section” is a named appendix or a front-matter section of the Document that deals exclusively with the relationship of the publishers or authors of the Document to the Document’s overall subject (or to related matters) and contains nothing that could fall directly within that overall subject. (Thus, if the Document is in part a textbook of mathematics, a Secondary Section may not explain any mathematics.) The relationship could be a matter of historical connection with the subject or with related matters, or of legal, commercial, philosophical, ethical or political position regarding them.

The “Invariant Sections” are certain Secondary Sections whose titles are designated, as being those of Invariant Sections, in the notice that says that the Document is released

under this License. If a section does not fit the above definition of Secondary then it is not allowed to be designated as Invariant. The Document may contain zero Invariant Sections. If the Document does not identify any Invariant Sections then there are none.

The “Cover Texts” are certain short passages of text that are listed, as Front-Cover Texts or Back-Cover Texts, in the notice that says that the Document is released under this License. A Front-Cover Text may be at most 5 words, and a Back-Cover Text may be at most 25 words.

A “Transparent” copy of the Document means a machine-readable copy, represented in a format whose specification is available to the general public, that is suitable for revising the document straightforwardly with generic text editors or (for images composed of pixels) generic paint programs or (for drawings) some widely available drawing editor, and that is suitable for input to text formatters or for automatic translation to a variety of formats suitable for input to text formatters. A copy made in an otherwise Transparent file format whose markup, or absence of markup, has been arranged to thwart or discourage subsequent modification by readers is not Transparent. An image format is not Transparent if used for any substantial amount of text. A copy that is not “Transparent” is called “Opaque”.

Examples of suitable formats for Transparent copies include plain ASCII without markup, Texinfo input format, LaTeX input format, SGML or XML using a publicly available DTD, and standard-conforming simple HTML, PostScript or PDF designed for human modification. Examples of transparent image formats include PNG, XCF and JPG. Opaque formats include proprietary formats that can be read and edited only by proprietary word processors, SGML or XML for which the DTD and/or processing tools are not generally available, and the machine-generated HTML, PostScript or PDF produced by some word processors for output purposes only.

The “Title Page” means, for a printed book, the title page itself, plus such following pages as are needed to hold, legibly, the material this License requires to appear in the title page. For works in formats which do not have any title page as such, “Title Page” means the text near the most prominent appearance of the work’s title, preceding the beginning of the body of the text.

The “publisher” means any person or entity that distributes copies of the Document to the public.

A section “Entitled XYZ” means a named subunit of the Document whose title either is precisely XYZ or contains XYZ in parentheses following text that translates XYZ in another language. (Here XYZ stands for a specific section name mentioned below, such as “Acknowledgements”, “Dedications”, “Endorsements”, or “History”.) To “Preserve the Title” of such a section when you modify the Document means that it remains a section “Entitled XYZ” according to this definition.

The Document may include Warranty Disclaimers next to the notice which states that this License applies to the Document. These Warranty Disclaimers are considered to be included by reference in this License, but only as regards disclaiming warranties: any other implication that these Warranty Disclaimers may have is void and has no effect on the meaning of this License.

2. VERBATIM COPYING

You may copy and distribute the Document in any medium, either commercially or noncommercially, provided that this License, the copyright notices, and the license notice saying this License applies to the Document are reproduced in all copies, and that you add no other conditions whatsoever to those of this License. You may not use technical measures to obstruct or control the reading or further copying of the copies you make or distribute. However, you may accept compensation in exchange for copies. If you distribute a large enough number of copies you must also follow the conditions in section 3.

You may also lend copies, under the same conditions stated above, and you may publicly display copies.

3. COPYING IN QUANTITY

If you publish printed copies (or copies in media that commonly have printed covers) of the Document, numbering more than 100, and the Document's license notice requires Cover Texts, you must enclose the copies in covers that carry, clearly and legibly, all these Cover Texts: Front-Cover Texts on the front cover, and Back-Cover Texts on the back cover. Both covers must also clearly and legibly identify you as the publisher of these copies. The front cover must present the full title with all words of the title equally prominent and visible. You may add other material on the covers in addition. Copying with changes limited to the covers, as long as they preserve the title of the Document and satisfy these conditions, can be treated as verbatim copying in other respects.

If the required texts for either cover are too voluminous to fit legibly, you should put the first ones listed (as many as fit reasonably) on the actual cover, and continue the rest onto adjacent pages.

If you publish or distribute Opaque copies of the Document numbering more than 100, you must either include a machine-readable Transparent copy along with each Opaque copy, or state in or with each Opaque copy a computer-network location from which the general network-using public has access to download using public-standard network protocols a complete Transparent copy of the Document, free of added material. If you use the latter option, you must take reasonably prudent steps, when you begin distribution of Opaque copies in quantity, to ensure that this Transparent copy will remain thus accessible at the stated location until at least one year after the last time you distribute an Opaque copy (directly or through your agents or retailers) of that edition to the public.

It is requested, but not required, that you contact the authors of the Document well before redistributing any large number of copies, to give them a chance to provide you with an updated version of the Document.

4. MODIFICATIONS

You may copy and distribute a Modified Version of the Document under the conditions of sections 2 and 3 above, provided that you release the Modified Version under precisely this License, with the Modified Version filling the role of the Document, thus licensing distribution and modification of the Modified Version to whoever possesses a copy of it. In addition, you must do these things in the Modified Version:

- A. Use in the Title Page (and on the covers, if any) a title distinct from that of the Document, and from those of previous versions (which should, if there were any,

- be listed in the History section of the Document). You may use the same title as a previous version if the original publisher of that version gives permission.
- B. List on the Title Page, as authors, one or more persons or entities responsible for authorship of the modifications in the Modified Version, together with at least five of the principal authors of the Document (all of its principal authors, if it has fewer than five), unless they release you from this requirement.
 - C. State on the Title page the name of the publisher of the Modified Version, as the publisher.
 - D. Preserve all the copyright notices of the Document.
 - E. Add an appropriate copyright notice for your modifications adjacent to the other copyright notices.
 - F. Include, immediately after the copyright notices, a license notice giving the public permission to use the Modified Version under the terms of this License, in the form shown in the Addendum below.
 - G. Preserve in that license notice the full lists of Invariant Sections and required Cover Texts given in the Document's license notice.
 - H. Include an unaltered copy of this License.
 - I. Preserve the section Entitled "History", Preserve its Title, and add to it an item stating at least the title, year, new authors, and publisher of the Modified Version as given on the Title Page. If there is no section Entitled "History" in the Document, create one stating the title, year, authors, and publisher of the Document as given on its Title Page, then add an item describing the Modified Version as stated in the previous sentence.
 - J. Preserve the network location, if any, given in the Document for public access to a Transparent copy of the Document, and likewise the network locations given in the Document for previous versions it was based on. These may be placed in the "History" section. You may omit a network location for a work that was published at least four years before the Document itself, or if the original publisher of the version it refers to gives permission.
 - K. For any section Entitled "Acknowledgements" or "Dedications", Preserve the Title of the section, and preserve in the section all the substance and tone of each of the contributor acknowledgements and/or dedications given therein.
 - L. Preserve all the Invariant Sections of the Document, unaltered in their text and in their titles. Section numbers or the equivalent are not considered part of the section titles.
 - M. Delete any section Entitled "Endorsements". Such a section may not be included in the Modified Version.
 - N. Do not retitle any existing section to be Entitled "Endorsements" or to conflict in title with any Invariant Section.
 - O. Preserve any Warranty Disclaimers.

If the Modified Version includes new front-matter sections or appendices that qualify as Secondary Sections and contain no material copied from the Document, you may at your option designate some or all of these sections as invariant. To do this, add their

titles to the list of Invariant Sections in the Modified Version's license notice. These titles must be distinct from any other section titles.

You may add a section Entitled "Endorsements", provided it contains nothing but endorsements of your Modified Version by various parties—for example, statements of peer review or that the text has been approved by an organization as the authoritative definition of a standard.

You may add a passage of up to five words as a Front-Cover Text, and a passage of up to 25 words as a Back-Cover Text, to the end of the list of Cover Texts in the Modified Version. Only one passage of Front-Cover Text and one of Back-Cover Text may be added by (or through arrangements made by) any one entity. If the Document already includes a cover text for the same cover, previously added by you or by arrangement made by the same entity you are acting on behalf of, you may not add another; but you may replace the old one, on explicit permission from the previous publisher that added the old one.

The author(s) and publisher(s) of the Document do not by this License give permission to use their names for publicity for or to assert or imply endorsement of any Modified Version.

5. COMBINING DOCUMENTS

You may combine the Document with other documents released under this License, under the terms defined in section 4 above for modified versions, provided that you include in the combination all of the Invariant Sections of all of the original documents, unmodified, and list them all as Invariant Sections of your combined work in its license notice, and that you preserve all their Warranty Disclaimers.

The combined work need only contain one copy of this License, and multiple identical Invariant Sections may be replaced with a single copy. If there are multiple Invariant Sections with the same name but different contents, make the title of each such section unique by adding at the end of it, in parentheses, the name of the original author or publisher of that section if known, or else a unique number. Make the same adjustment to the section titles in the list of Invariant Sections in the license notice of the combined work.

In the combination, you must combine any sections Entitled "History" in the various original documents, forming one section Entitled "History"; likewise combine any sections Entitled "Acknowledgements", and any sections Entitled "Dedications". You must delete all sections Entitled "Endorsements."

6. COLLECTIONS OF DOCUMENTS

You may make a collection consisting of the Document and other documents released under this License, and replace the individual copies of this License in the various documents with a single copy that is included in the collection, provided that you follow the rules of this License for verbatim copying of each of the documents in all other respects.

You may extract a single document from such a collection, and distribute it individually under this License, provided you insert a copy of this License into the extracted document, and follow this License in all other respects regarding verbatim copying of that document.

7. AGGREGATION WITH INDEPENDENT WORKS

A compilation of the Document or its derivatives with other separate and independent documents or works, in or on a volume of a storage or distribution medium, is called an “aggregate” if the copyright resulting from the compilation is not used to limit the legal rights of the compilation’s users beyond what the individual works permit. When the Document is included in an aggregate, this License does not apply to the other works in the aggregate which are not themselves derivative works of the Document.

If the Cover Text requirement of section 3 is applicable to these copies of the Document, then if the Document is less than one half of the entire aggregate, the Document’s Cover Texts may be placed on covers that bracket the Document within the aggregate, or the electronic equivalent of covers if the Document is in electronic form. Otherwise they must appear on printed covers that bracket the whole aggregate.

8. TRANSLATION

Translation is considered a kind of modification, so you may distribute translations of the Document under the terms of section 4. Replacing Invariant Sections with translations requires special permission from their copyright holders, but you may include translations of some or all Invariant Sections in addition to the original versions of these Invariant Sections. You may include a translation of this License, and all the license notices in the Document, and any Warranty Disclaimers, provided that you also include the original English version of this License and the original versions of those notices and disclaimers. In case of a disagreement between the translation and the original version of this License or a notice or disclaimer, the original version will prevail.

If a section in the Document is Entitled “Acknowledgements”, “Dedications”, or “History”, the requirement (section 4) to Preserve its Title (section 1) will typically require changing the actual title.

9. TERMINATION

You may not copy, modify, sublicense, or distribute the Document except as expressly provided under this License. Any attempt otherwise to copy, modify, sublicense, or distribute it is void, and will automatically terminate your rights under this License.

However, if you cease all violation of this License, then your license from a particular copyright holder is reinstated (a) provisionally, unless and until the copyright holder explicitly and finally terminates your license, and (b) permanently, if the copyright holder fails to notify you of the violation by some reasonable means prior to 60 days after the cessation.

Moreover, your license from a particular copyright holder is reinstated permanently if the copyright holder notifies you of the violation by some reasonable means, this is the first time you have received notice of violation of this License (for any work) from that copyright holder, and you cure the violation prior to 30 days after your receipt of the notice.

Termination of your rights under this section does not terminate the licenses of parties who have received copies or rights from you under this License. If your rights have been terminated and not permanently reinstated, receipt of a copy of some or all of the same material does not give you any rights to use it.

10. FUTURE REVISIONS OF THIS LICENSE

The Free Software Foundation may publish new, revised versions of the GNU Free Documentation License from time to time. Such new versions will be similar in spirit to the present version, but may differ in detail to address new problems or concerns. See <http://www.gnu.org/copyleft/>.

Each version of the License is given a distinguishing version number. If the Document specifies that a particular numbered version of this License “or any later version” applies to it, you have the option of following the terms and conditions either of that specified version or of any later version that has been published (not as a draft) by the Free Software Foundation. If the Document does not specify a version number of this License, you may choose any version ever published (not as a draft) by the Free Software Foundation. If the Document specifies that a proxy can decide which future versions of this License can be used, that proxy’s public statement of acceptance of a version permanently authorizes you to choose that version for the Document.

11. RELICENSING

“Massive Multiauthor Collaboration Site” (or “MMC Site”) means any World Wide Web server that publishes copyrightable works and also provides prominent facilities for anybody to edit those works. A public wiki that anybody can edit is an example of such a server. A “Massive Multiauthor Collaboration” (or “MMC”) contained in the site means any set of copyrightable works thus published on the MMC site.

“CC-BY-SA” means the Creative Commons Attribution-Share Alike 3.0 license published by Creative Commons Corporation, a not-for-profit corporation with a principal place of business in San Francisco, California, as well as future copyleft versions of that license published by that same organization.

“Incorporate” means to publish or republish a Document, in whole or in part, as part of another Document.

An MMC is “eligible for relicensing” if it is licensed under this License, and if all works that were first published under this License somewhere other than this MMC, and subsequently incorporated in whole or in part into the MMC, (1) had no cover texts or invariant sections, and (2) were thus incorporated prior to November 1, 2008.

The operator of an MMC Site may republish an MMC contained in the site under CC-BY-SA on the same site at any time before August 1, 2009, provided the MMC is eligible for relicensing.

ADDENDUM: How to use this License for your documents

To use this License in a document you have written, include a copy of the License in the document and put the following copyright and license notices just after the title page:

```
Copyright (C)  year  your name.
Permission is granted to copy, distribute and/or modify this document
under the terms of the GNU Free Documentation License, Version 1.3
or any later version published by the Free Software Foundation;
with no Invariant Sections, no Front-Cover Texts, and no Back-Cover
Texts. A copy of the license is included in the section entitled ‘‘GNU
Free Documentation License’’.
```

If you have Invariant Sections, Front-Cover Texts and Back-Cover Texts, replace the “with...Texts.” line with this:

```
with the Invariant Sections being list their titles, with
the Front-Cover Texts being list, and with the Back-Cover Texts
being list.
```

If you have Invariant Sections without Cover Texts, or some other combination of the three, merge those two alternatives to suit the situation.

If your document contains nontrivial examples of program code, we recommend releasing these examples in parallel under your choice of free software license, such as the GNU General Public License, to permit their use in free software.

Concept Index

!

'!' packet 797

#

in Modula-2 252

\$

\$ 172

\$\$ 172

\$_ and commands that search source files 133

\$_ and info breakpoints 63

\$_ and info line 138

\$_, \$_, and value history 156

&

&, background execution of commands 98

—

--annotate 15

--args 15

--attach, gdbserver option 323

--batch 14

--batch-silent 14

--baud 16

--binary-output 15

--cd 14

--command 12

--configuration 17

--core 12

--data-directory 15

--debug, gdbserver option 324

--debug-file, gdbserver option 325

--debug-format, gdbserver option 325

--directory 13

--early-init-command 13

--early-init-eval-command 13

--eval-command 12

--exec 12

--fullname 15

--init-command 12

--init-eval-command 13

--interpreter 16

--multi, gdbserver option 317

--nh 13

--no-escape-args 16

--nowindows 14

--nx 13

--once, gdbserver option 324

--pid 12

--quiet 13

--readnever, command-line option 13

--readnow 13

--return-child-result 14

--se 12

--selftest 325

--silent 13

--statistics 17

--symbols 12

--tty 16

--tui 16

--version 17

--windows 14

'--with-gdb-datadir' 312

'--with-relocated-sources' 136

'--with-sysroot' 301

--wrapper, gdbserver option 325

--write 16

-b 16

-c 12

-d 13

-D 15

-e 12

-eiex 13

-eix 13

-ex 12

-f 15

-iex 13

-info-gdb-mi-command 704

-ix 12

-l 16

-n 13

-nw 14

-p 12

-q 13

-r 13

--readnever, option for symbol-file command .. 294

-s 12

-t 16

-w 14

-x 12

•

., Modula-2 scope operator 251

.build-id directory 303

.debug subdirectories 303

.debug_gdb_scripts section 593

' .debug_names' section 309

' .gdb_index' section 307

.gdb_index section format 909

.gdbinit 19

' .gnu_debugdata' section 306

.gnu_debuglink sections 304

<code>.note.gnu.build-id</code> sections	304
<code>.o</code> files, reading symbols from	295
/	
<code>/proc</code>	339
:	
<code>::</code> , context for variables/functions	149
<	
<code><architecture></code>	884
<code><compatible></code>	885
<code><feature></code>	885
<code><flags></code>	886
<code><not saved></code> values	183
<code><osabi></code>	885
<code><reg></code>	886
<code><struct></code>	886
<code><union></code>	885
<code><vector></code>	885
<code><xi:include></code>	884
?	
<code>'?</code> packet	797
-	
<code>_NSPrintForDebugger</code> , and printing Objective-C objects	241
‘	
“No symbol “foo” in current context”	150
{	
<code>{type}</code>	147

A

<code>'A'</code> packet	798
AArch64 support	356
abbreviation	23
acknowledgment, for GDB remote	853
active targets	313
Ada	252
Ada exception catching	72
Ada exception handlers catching	72
Ada mode, general	252
Ada task switching	259
Ada tasking and core file debugging	260
Ada, deviations from	254
Ada, omissions from	252
Ada, problems	262
Ada, source character set	261
Ada, tasking	256
add directory to source search path	135
add new commands for external monitor	322
address locations	132
address of a symbol	266
address size for remote targets	328
addressable memory unit	156
aggregates (Ada)	253
AIX threads	388
aliases for commands	405
aliases for commands, default arguments	406
alignment of remote memory accesses	800, 805
all-stop mode	96
Alpha stack	362
always-read-ctf	276
ambiguous expressions	148
AMD GPU debugging info	388
AMD GPU precise memory event reporting	368
AMD GPU support	365
annotations	715
annotations for errors, warnings and interrupts	717
annotations for invalidation messages	717
annotations for prompts	716
annotations for running programs	717
annotations for source display	718
append data to a file	188
Application Data Integrity	364
apply a command to all frames (ignoring errors and empty output)	124
apply a command to all frames of all threads (ignoring errors and empty output)	51
apply command to all threads (ignoring errors and empty output)	51
apply command to several frames	122
apply command to several threads	51
ARC EM	350
ARC HS	350
ARC specific commands	350
ARC600	350
ARC700	350
architecture debugging info	387

argument count in user-defined commands 398
 arguments (to your program) 38
 arguments, to **gdbserver** 323
 arguments, to user-defined commands 397
 ARM 32-bit mode 351
 ARM AArch64 387
 array aggregates (Ada) 253
 arrays 152
 arrays in expressions 147
 arrays slices (Fortran) 244
 artificial array 152
 assembly instructions 138
 assignment 279
 async output in GDB/MI 615
 async records in GDB/MI 618
 asynchronous execution 98, 611
 asynchronous execution, and process
 record and replay 106
 AT&T disassembly flavor 142
 attach 41
 attach to a program, **gdbserver** 323
 auto-loading 381
 auto-loading extensions 592
 auto-loading init file in the current directory . . 383
 auto-loading libthread_db.so.1 383
 auto-loading safe-path 384
 auto-loading verbose mode 385
 auto-retry, for remote TCP target 330
 automatic display 158
 automatic hardware breakpoints 66
 automatic overlay debugging 226
 automatic symbol index cache 308
 automatic thread selection 96
 auxiliary vector 183
 AVR 355

B

‘b’ packet 798
 ‘B’ packet 798
 background execution 98, 611
 backtrace beyond **main** function 118
 backtrace limit 119
 base name differences 302
 baud rate for remote targets 328
 ‘bc’ packet 798
 bcache statistics 786
 bits in remote address 328
 blocks in guile 572
 blocks in python 493
 bookmark 56
 branch trace configuration format 870
 branch trace format 869
 branch trace store 106
 break in overloaded functions 237
 break on a system call 72
 break on fork/exec 72
 BREAK signal instead of Ctrl-C 328

breakpoint address adjusted 85
 breakpoint at static probe point 130
 breakpoint commands 79
 breakpoint commands for GDB/MI 627
 breakpoint commands, in remote protocol 830
 breakpoint conditions 78
 breakpoint debugging info 393
 breakpoint kinds, ARM 840
 breakpoint kinds, MIPS 840
 breakpoint lists 59
 breakpoint numbers 59
 breakpoint on events 59
 breakpoint on memory address 59
 breakpoint on variable modification 59
 breakpoint ranges 59
breakpoint subroutine, remote 336
 breakpointing Ada elaboration code 256
 breakpoints 59
 breakpoints and inferiors 47
 breakpoints and tasks, in Ada 260
 breakpoints and threads 99
 breakpoints at functions matching a regexp . . . 63
 breakpoints in guile 578
 breakpoints in overlays 226
 breakpoints in python 503
 breakpoints, multiple locations 64
 ‘bs’ packet 798
 bug criteria 729
 bug reports 729
 bugs in GDB 729
 build ID sections 304
 build ID, and separate debugging files 303
 building GDB, requirements for 767
 built-in simulator target 315
 builtin Go functions 240
 builtin Go types 240

C

‘c’ packet 798
 C and C++ 233
 C and C++ checks 237
 C and C++ constants 235
 C and C++ defaults 237
 C and C++ operators 233
 ‘C’ packet 798
 C++ 233
 C++ compilers 236
 C++ demangling 238
 C++ exception handling 237
 C++ overload debugging info 391
 C++ scope resolution 150
 C++ symbol decoding style 169
 C++ symbol display 238
 caching data of targets 192
 caching of bfd objects 302
 caching of opened files 302
 call dummy stack unwinding 283

call dummy stack unwinding on timeout.....	284	commands for C++.....	237
call dummy stack unwinding on unhandled exception.....	284	commands in guile.....	561
call overloaded functions.....	236	commands in python, CLI.....	471
call stack.....	115	commands in python, GDB/MI.....	474
call stack traces.....	116	commands to access guile.....	538
call-clobbered registers.....	183	commands to access python.....	408
caller-saved registers.....	183	comment.....	23
calling functions.....	283	common targets.....	314
calling functions in the program, disabling.....	284	COMMON blocks, Fortran.....	244
calling make.....	21	compatibility, GDB/MI and CLI.....	615
case sensitivity in symbol names.....	265	compilation directory.....	134
case-insensitive symbol names.....	265	compile C++ type conversion.....	288
casts, in expressions.....	147	compile command debugging info.....	288
casts, to view memory.....	147	compile command driver filename override.....	292
catch Ada exceptions.....	72	compile command options override.....	289
catch Ada exceptions when handled.....	72	compiling code.....	287
catch syscalls from inferior, remote request....	818	completion.....	24
catchpoints.....	59	completion of Guile commands.....	562
catchpoints, setting.....	71	completion of Python commands.....	472
change GDB's working directory.....	40	completion of quoted strings.....	26
change inferior's working directory.....	40	completion of structure field names.....	26
character sets.....	190	completion of union field names.....	26
charset.....	190	compressed debug sections.....	768
check if a given address is in a memory tagged region.....	817	conditional breakpoints.....	78
checkpoint.....	56	conditional tracepoints.....	209
checkpoints and process id.....	58	configure debuginfod URLs.....	913
checks, range.....	232	configuring GDB.....	769
checks, type.....	231	confirmation.....	387
checksum, for GDB remote.....	795	connection timeout, for remote TCP target....	330
choosing target byte order.....	316	connections in python.....	514
circular trace buffer.....	215	console i/o as part of file-i/o.....	856
clearing breakpoints, watchpoints, catchpoints..	75	console interpreter.....	597
CLI commands in python.....	471	console output in GDB/MI.....	615
close, file-i/o system call.....	858	constants, in file-i/o protocol.....	864
closest symbol and offset for an address.....	266	continuing.....	85
code address and its source line.....	138	continuing threads.....	95
code compression, MIPS.....	363	control C, and remote debugging.....	337
code location.....	128	controlling terminal.....	41
COFF/PE exported symbols.....	388	convenience functions.....	177
collected data discarded.....	214	convenience functions in python.....	483
colon, doubled as scope operator.....	251	convenience variables.....	173
colon-colon, context for variables/functions....	149	convenience variables for tracepoints.....	220
color space.....	176	convenience variables, and trace state variables.....	209
colors.....	375	convenience variables, initializing.....	173
colors in python.....	509	core dump file.....	293
command editing.....	733	core dump file target.....	314
command files.....	401	crash of debugger.....	729
command history.....	372	CRC algorithm definition.....	305
command hooks.....	400	CRC of memory block, remote request.....	812
command interpreters.....	597	CRIS.....	355
command line editing.....	371	CRIS mode.....	355
command options.....	28	CRIS version.....	355
command options, boolean.....	28	CTF debug messages.....	388
command options, raw input.....	28	CTF info, when to read.....	276
command scripts, debugging.....	387	Ctrl-BREAK, MS-Windows.....	344
command tracing.....	387	ctrl-c message, in file-i/o protocol.....	856
		current Ada task ID.....	258

current directory	134
current Go package	240
current thread	48
current thread, remote request	812
custom JIT debug info	722
Cygwin DLL, debugging	344
Cygwin-specific commands	344

D

'd' packet	799
'D' packet	799
DAP	597
Darwin	349
data breakpoints	59
data manipulation, in GDB/MI	671
dcache line-size	193
dcache size	193
dcache, flushing	193
dead names, GNU Hurd	348
debug expression parser	391
debug formats and C++	236
debug link sections	304
debug remote protocol	391
Debugger Adapter Protocol	597
debugger crash	729
debugging agent	725
debugging C++ programs	236
debugging information directory, global	303
debugging information in separate files	303
debugging <code>libthread_db</code>	53
debugging multiple processes	55
debugging optimized code	197
debugging stub, example	335
debugging target	313
debugging the Cygwin DLL	344
debugging <code>threads</code>	53
debuginfod	913
debuginfod verbosity	913
debuginfod, maintenance commands	780
decimal floating point format	239
default behavior of commands, changing	23
default collection action	212
default data directory	312
default settings, changing	23
default source path substitution	136
default system root	301
define trace state variable, remote request	842
defining macros interactively	201
definition of a macro, showing	201
delete breakpoints	76
deleting breakpoints, watchpoints, catchpoints	75
deliver a signal to a program	281
demangle	267
demangler crashes	780, 781
demangling C++ names	168
deprecated commands	780
derived type of an object, printing	169
descriptor tables display	342
detach from task, GNU Hurd	348
detach from thread, GNU Hurd	349
direct memory access (DMA) on MS-DOS	343
directories for source files	133
directory, compilation	134
directory, current	134
disable address space randomization, remote request	813
disabling calling functions in the program	284
disassembler in Python, global vs. specific	524
disassembler options	141
disconnected tracing	214
displaced stepping debugging info	389
displaced stepping support	779
displaced stepping, and process record and replay	106
display command history	374
display derived types	169
display disabled out of scope	159
display GDB copyright	31
display of expressions	158
display remote monitor communications	315
display remote packets	391
DJGPP debugging	341
DLLs with no debugging symbols	345
do not print frame arguments	162
documentation	765
don't repeat command	399
don't repeat Guile command	562
don't repeat Python command	471, 482
D	240
DOS file-name semantics of file names	301
DOS serial data link, remote debugging	343
DOS serial port status	343
DPMI	341
dprintf	81
dump all data collected at tracepoint	219
dump core from inferior	188
dump data to a file	188
dump/restore files	188
DVC register	354
DWARF compilation units cache	787
DWARF DIEs	389
DWARF Line Tables	389
DWARF Reading	389
DWARF-2 CFI and CRIS	355
dynamic linking	295
dynamic printf	81
dynamic varobj	663

E

early initialization	18
early initialization file	17
editing	372
editing command lines	733
editing source files	132
eight-bit characters in strings	167
elaboration phase	35
ELinOS system-wide configuration script	776
Emacs	607
empty response, for unsupported packets	796
enable debuginfod	913
enable/disable a breakpoint	76
enabling and disabling probes	84
entering numbers	379
environment (of your program)	38
errno values, in file-i/o protocol	864
error on valid input	729
event debugging info	389
event designators	759
event handling	71
event-loop debugging	389
examine process image	339
examining data	143
examining memory	154
exception handlers	71
exceptions, guile	543
exceptions, python	416
exec events, remote reply	810
executable file	293
executable file target	314
executable file, for remote target	329
execute commands from a file	401
execute forward or backward in time	104
execute remote command, remote request	821
execution, foreground, background and asynchronous	98, 611
exit status of shell commands	176
exiting GDB	20
expand macro once	201
expanding preprocessor macros	201
explicit locations	131
explore type	147
explore value	147
exploring hierarchical data structures	145
expression debugging info	389
expression parser, debugging info	391
expressions	147
expressions in Ada	252
expressions in C or C++	233
expressions in C++	236
expressions in Modula-2	246
extend GDB for remote targets	322
extending GDB	397
extra signal information	95

F

'F' packet	799
F reply packet	855
F request packet	855
fast tracepoints	205
fast tracepoints, setting	207
fatal signal	729
fatal signals	92
features of the remote protocol	822
fetch memory tags	816
file name canonicalization	302
file names, quoting and escaping	27
file transfer	322
file transfer, remote protocol	847
file-i/o examples	865
file-i/o overview	854
file-i/o reply packet	855
file-i/o request packet	855
File-I/O remote protocol extension	854
filename-display	119
find trace snapshot	217
flinching	387
float promotion	380
floating point	183
floating point mode register, AArch64	361
floating point registers	181
floating point, MIPS remote	353
focus of debugging	48
foo	311
foreground execution	98, 611
fork events, remote reply	809
fork, debugging programs which call	54
format options	159
formatted output	152
Fortran	1
fortran array slicing debugging info	390
Fortran Defaults	242
Fortran Intrinsics	243
Fortran modules, information about	274
Fortran operators and expressions	243
Fortran Types	242
Fortran-specific support in GDB	242
FPMR, AArch64	361
frame debugging info	390
frame decorator api	438
frame filters api	436
frame information, printing	165
frame level	115
frame number	115
frame pointer	115
frame pointer register	181
frame, definition	115
frameless execution	115
frames in guile	569
frames in python	490
frames of inlined functions	783
free memory information (MS-DOS)	342
FreeBSD	350

FreeBSD LWP debug messages..... 389
 FreeBSD native target debug messages..... 390
 fstat, file-i/o system call..... 860
 Fujitsu..... 336
 full symbol tables, listing GDB's internal..... 275
 function call arguments, optimized out..... 118
 function entry/exit, wrong
 values of variables..... 150
 functions and variables by Fortran module.... 274
 functions without line info, and stepping..... 87

G

'g' packet..... 799
g++, GNU C++ compiler..... 233
 'G' packet..... 800
 garbled pointers..... 342
 GCC and C++..... 236
 GCS, AArch64..... 360
 GDB bugs, reporting..... 729
 GDB internal error..... 781
 gdb module..... 410
 gdb objects..... 541
 GDB reference card..... 765
 GDB startup..... 17
 GDB version number..... 31
gdb.ini..... 19
 gdb.printing..... 533
 gdb.prompt..... 535
 gdb.ptwrite..... 536
 gdb.types..... 533
gdb.Value..... 417
 GDB/MI development..... 615
 GDB/MI, async records..... 618
 GDB/MI, breakpoint commands..... 627
 GDB/MI, compatibility with CLI..... 615
 GDB/MI, data manipulation..... 671
 GDB/MI, input syntax..... 613
 GDB/MI, its purpose..... 609
 GDB/MI, output syntax..... 613
 GDB/MI, result records..... 617
 GDB/MI, simple examples..... 626
 GDB/MI, stream records..... 617
 GDB/MI General Design..... 609
 gdbarch debugging info..... 387
 GDBHISTFILE, environment variable..... 372
 GDBHISTSIZE, environment variable..... 373
gdbinit..... 19
gdbserver, command-line arguments..... 323
gdbserver, connecting..... 317
gdbserver, search path for **libthread_db**..... 327
gdbserver, send all debug output
 to a single file..... 325
gdbserver, **target extended-remote** mode.... 317
gdbserver, **target remote** mode..... 317
gdbserver, types of connections..... 317
 GDT..... 342
 general initialization..... 18

get thread information block address..... 816
 get thread-local storage address,
 remote request..... 815
 gettimeofday, file-i/o system call..... 861
 getting structure elements using **gdb.Field**
 objects as subscripts..... 418
 global debugging information directories..... 303
 global thread identifier (GDB)..... 49
 global thread number..... 49
 GNAT descriptive types..... 262
 GNAT encoding..... 262
 GNU C++..... 233
 GNU Emacs..... 607
 GNU Hurd debugging..... 347
 GNU/Hurd debug messages..... 390
 GNU/Linux namespaces debug messages..... 391
 GNU/Linux native target debug messages..... 391
 GNU ifunc debug messages..... 390
 Go (programming language)..... 240
 Guarded Control Stack, AArch64..... 360
 guile api..... 539
 guile architectures..... 583
 guile auto-loading..... 590
 guile colors..... 589
 guile commands..... 538, 561
 guile configuration..... 541
 guile exceptions..... 543
 guile gdb module..... 539
 guile iterators..... 587
 guile modules..... 590
 guile pagination..... 539
 guile parameters..... 565
 guile pretty printing api..... 556
 guile scripting..... 538
 guile scripts directory..... 538
 guile stdout..... 539
 guile, working with types..... 551
 guile, working with values from inferior..... 545

H

'H' packet..... 800
 handling signals..... 93
 hardware breakpoints..... 62
 hardware debug registers..... 789
 hardware watchpoints..... 68
 hash mark while downloading..... 315
heuristic-fence-post (Alpha, MIPS)..... 362
 history events..... 759
 history expansion..... 759
 history expansion, turn on/off..... 373
 history file..... 372
 history number..... 172
 history of values printed by GDB..... 172
 history size..... 373
 history substitution..... 372
 hooks, for commands..... 400
 hooks, post-command..... 400

hooks, pre-command	400
host character set	190
Host I/O, remote protocol	847
how many arguments (user-defined commands)	398
HPPA support	364

I

‘i’ packet	800
i/o	41
i386	335
i386-stub.c	335
‘I’ packet	800
I/O registers (Atmel AVR)	355
ID list	43
IDT	342
ignore count (of breakpoint)	79
in-process agent protocol	725
incomplete type	270
indentation in structure display	167
index files	307, 309
index section format	909
inferior	43
inferior command line with embedded newlines	38
inferior debugging info	390
inferior events in Python	458
inferior function call debugging info	390
inferior functions, calling	283
inferior tty	41
inferior-specific breakpoints	47
inferiors in Python	455
infinite recursion in user-defined commands ...	400
info for known .debug_gdb_scripts-loaded scripts	786
info for known object files	786
info line, repeated calls	138
info proc cmdline	340
info proc cwd	340
info proc environ	340
info proc exe	340
info proc files	340
information about static tracepoint markers ...	213
information about tracepoints	212
inheritance	237
init file	17
init file name	18
initial frame	115
initialization file	18
initialization file, readline	736
injecting code	287
inline functions, debugging	197
innermost frame	115
input syntax for GDB/MI	613
installation	767
instructions, assembly	138
integral datatypes, in file-i/o protocol	862

Intel	335
Intel Control-Flow Enforcement Technology ...	361
Intel disassembly flavor	142
Intel Processor Trace	106
interaction, readline	733
internal commands	777
internal errors, control of GDB behavior	781
internal GDB breakpoints	67
internal TLS address lookup	54
interrupt	20
interrupt debuggee on MS-Windows	344
interrupt remote programs	328, 330
interrupting remote programs	321
interrupting remote targets	337
interrupts (remote protocol)	849
invalid input	729
invoke another interpreter	597
ipa protocol commands	727
ipa protocol objects	726
isatty, file-i/o system call	861

J

JIT compilation interface	721
JIT debug info reader	722
just-in-time compilation	721
just-in-time compilation, debugging messages	390

K

‘k’ packet	800
kernel crash dump	339
kernel memory image	339
kill ring	734
killing text	734

L

languages	229
last tracepoint number	208
latest breakpoint	60
lazy strings in guile	582
lazy strings in python	508
LDT	342
leaving GDB	20
libkvm	339
library list format, remote protocol	865, 866
limit hardware breakpoints and watchpoints ...	329
limit hardware watchpoints length	329
limit on number of printed array elements	162
limit on number of printed string characters ...	162
limits, in file-i/o protocol	865
line tables in python	502
line tables, listing GDB’s internal	276
linespec locations	129
linker namespaces	299
Linux native targets	391

list active threads, remote request 815
 list of supported file-i/o calls 856
 list output in GDB/MI 615
list, how many lines to display 127
 listing GDB's internal line tables 276
 listing GDB's internal symbol tables 275
 listing machine instructions 138
 listing mapped overlays 225
 lists of breakpoints 59
 load address, overlay's 223
 load shared library 298
 load symbols from memory 296
 local socket, **target remote** 319
 local variables 271
 locate address 153
 location resolution 128
 location spec. 128
 lock scheduler 96
 locspec 128
 log output in GDB/MI 615
 logging file name 22
 logging GDB output 22
 look up of disassembler in Python 524
 lseek flags, in file-i/o protocol 865
 lseek, file-i/o system call 859

M

'm' packet 800
 m680x0 336
m68k-stub.c 336
 'M' packet 801
 Mach-O symbols processing 391
 machine instructions 138
 macro definition, showing 201
 macro expansion, showing the results
 of preprocessor 201
 macros, example of debugging with 202
 macros, from debug info 201
 macros, user-defined 201
 mailing lists 617
 maintenance commands 777
 Man pages 915
 managing frame filters 124
 manual overlay debugging 225
 map an overlay 225
 mapped address 223
 mapped overlays 223
 markers, static tracepoints 205
 maximum value for offset of closest symbol 160
 member functions 236
 memory address space mappings 341, 782
 memory map format 867
 memory region attributes 186
 memory tag types, ARM 840
 Memory Tagging Extension, AArch64 360
 memory tracing 59
 memory transfer, in file-i/o protocol 862

memory used by commands 792
 memory used for symbol tables 298
 memory, alignment and size of
 remote accesses 800, 805
 memory, viewing as typed object 147
 mi interpreter 597
 mi2 interpreter 597
 mi3 interpreter 597
 MI commands in python 474
 MI notifications in python 477
 minimal language 263
 minimal symbol dump 275
 Minimal symbols and DLLs 345
 MIPS addresses, masking 363
 MIPS remote floating point 353
 MIPS stack 362
 miscellaneous settings 393
 MMX registers (x86) 182
 mode_t values, in file-i/o protocol 864
 Modula-2 1
 Modula-2 built-ins 247
 Modula-2 checks 251
 Modula-2 constants 248
 Modula-2 defaults 250
 Modula-2 operators 246
 Modula-2 types 249
 Modula-2, deviations from 251
 Modula-2, GDB support 246
 module functions and variables 274
 modules 274
 monitor commands, for **gdbserver** 326
 Motorola 680x0 336
 MS Windows debugging 344
 MS-DOS system info 342
 MS-DOS-specific commands 341
 MTE, AArch64 360
 multiple locations, breakpoints 64
 multiple processes 54
 multiple targets 313
 multiple threads 48
 multiple threads, backtrace 117
 multiple-symbols menu 148
 multiprocess extensions, in remote protocol 829

N

name a thread 52
 names of symbols 265
 namespace in C++ 236
 native Cygwin debugging 344
 native DJGPP debugging 341
 native script auto-loading 405
 native target 315
 negative breakpoint numbers 67
 never read symbols 294
New systag message 48
 new user interface 597

Newlib OS ABI and its influence on the	
longjmp handling	380
newlines in inferior command lines	38
no debug info functions	286
no debug info variables	151
non-member C++ functions, set breakpoint in ...	63
non-stop mode	97
non-stop mode, and process	
record and replay	106
non-stop mode, and ‘set	
displaced-stepping’	779
non-stop mode, remote request	818
noninvasive task options	348
notation, readline	733
notational conventions, for GDB/MI	609
notification packets	850
notifications in python, GDB/MI	477
notify output in GDB/MI	615
NULL elements in arrays	167
number of array elements to print	162
number of string characters to print	162
number representation	379
numbers for breakpoints	59

O

object files, relocatable, reading	
symbols from	295
Objective-C	240
Objective-C, classes and selectors	274
Objective-C, print objects	241
objfile-gdb.gdb	592
objfile-gdb.py	592
objfile-gdb.scm	592
objfiles in guile	568
objfiles in python	487
observer debugging info	391
octal escapes in strings	167
online documentation	29
opaque data types	274
open flags, in file-i/o protocol	864
open, file-i/o system call	856
OpenCL C	241
OpenCL C Datatypes	242
OpenCL C Expressions	242
OpenCL C Operators	242
OpenRISC 1000	353
operate-and-get-next	372
operating system information	905
operating system information, process list	905
optimized code, debugging	197
optimized code, wrong values of variables	150
optimized out value in guile	546
optimized out value in Python	418
optimized out, in backtrace	118
optional debugging messages	387
optional warnings	386
OS ABI	380

OS information	183
out-of-line single-stepping	779
outermost frame	115
output formats	152
output syntax of GDB/MI	613
overlay area	223
overlay example program	227
overlays	223
overlays, setting breakpoints in	226
overloaded functions, calling	236
overloaded functions, overload resolution	238
overloading in C++	237
overloading, Ada	255

P

‘p’ packet	801
‘P’ packet	801
PAC, AArch64	359
packet acknowledgment, for GDB remote	853
packet size, remote protocol	827
packet size, remote, configuring	334
packets, notification	850
packets, reporting on stdout	391
packets, tracepoint	840
page size	374
page tables display (MS-DOS)	342
pagination	374
parameters in guile	565
parameters in python	477
Pascal	1
Pascal objects, static members display	169
Pascal support in GDB, limitations	245
pass signals to inferior, remote request	819
patching binaries	287
patching object files	293
pause current task (GNU Hurd)	348
pause current thread (GNU Hurd)	348
pauses in output	374
pending breakpoints	65
physical address from linear address	343
physname	388
pipe, target remote to	321
pipes	34
Pointer Authentication, AArch64	359
pointer values, in file-i/o protocol	862
pointer, finding referent	160
port rights, GNU Hurd	348
port sets, GNU Hurd	348
PowerPC architecture	364
prefix for data files	312
prefix for executable and shared	
library file names	300
premature return from system calls	100
preprocessor macro expansion,	
showing the results of	201
pretty print arrays	161
pretty print C++ virtual function tables	169

pretty-printer commands 171
 print all frame argument values 162
 print an Objective-C object description 241
 print array indexes 161
 print binary values in groups of four bits 161
 print frame argument values for scalars only ... 162
 print list of auto-loaded canned sequences
 of commands scripts 405
 print list of auto-loaded Guile scripts 590
 print list of auto-loaded Python scripts 532
 print messages on inferior start and exit 46
 print messages on thread start and exit 52
 print messages when symbols are loaded 275
 print settings 159
 print structures in indented form 167
 print/don't print memory addresses 159
 printing byte arrays 153
 printing data 143
 printing frame argument values 162
 printing frame information 165
 printing memory tag violation information 166
 printing nested structures 166
 printing strings 153
 probe static tracepoint marker 207
 probing markers, static tracepoints 205
 process detailed status information 341
 process ID 340
 process info via `/proc` 339
 process record and replay 105
 process status register 181
 processes, multiple 54
procfs API calls 341
 profiling GDB 789
 program counter register 181
 program entry point 118
 programming in guile 539
 programming in python 409
 progress bars, disabling 914
 progspaces in guile 567
 progspaces in python 484
 prologue-end 277
 prompt 371
 protocol basics, file-i/o 854
 protocol, GDB remote serial 795
 protocol-specific representation of
 datatypes, in file-i/o protocol 862
 python api 409
 Python architectures 512
 Python auto-loading 532
 python commands 408
 python commands, CLI 471
 python commands, GDB/MI 474
 python convenience functions 483
 python directory 407
 python exceptions 416
 python finish breakpoints 507
 python functions 410
 python instruction disassembly 517

python module 410
 python modules 533
 python notifications, GDB/MI 477
 python pagination 410
 python parameters 477
 python pretty printing api 430
 python scripting 407
 python stdout 410
 Python TUI Windows 516
 python, core files 531
 python, handle missing debug information 526
 python, handle missing objfiles 528
 Python, working with types 425
 python, working with values from inferior 417

Q

'q' packet 801
 'Q' packet 801
 'QAllow' packet 812
 'qAttached' packet 838
 'qC' packet 812
 'QCatchSyscalls' packet 818
 'qCRC' packet 812
 'QDisableRandomization' packet 813
 'QEnvironmentHexEncoded' packet 813
 'QEnvironmentReset' packet 814
 'QEnvironmentUnset' packet 814
 'qExecAndArgs' packet 838
 'qfThreadInfo' packet 815
 'qGetTIBAddr' packet 816
 'qGetTLSAddr' packet 815
 'qIsAddressTagged' packet 817
 'qMemTags' packet 816
 'QMemTags' packet 817
 'QNonStop' packet 818
 'qOffsets' packet 817
 'qP' packet 818
 'QPassSignals' packet 819
 'QProgramSignals' packet 819
 'qRcmd' packet 821
 'qSearch memory' packet 821
 'qSearch:memory' packet 821
 'QSetWorkingDir' packet 815
 'QStartNoAckMode' packet 822
 'QStartupWithShell' packet 813
 'qsThreadInfo' packet 815
 'qSupported' packet 822
 'qSymbol' packet 832
 'qTBuffer' packet 846
 'QTBuffer size' packet 846
 'QTDisable' packet 843
 'QTDIsconnected' packet 844
 'QTDP' packet 840
 'QTDPsrc' packet 842
 'QTDV' packet 842
 'QTEnable' packet 843
 'qTfP' packet 845

'QTFrame' packet	842
'qTfSTM' packet	846
'qTfV' packet	846
'QThreadEvents' packet	820
'qThreadExtraInfo' packet	832
'QThreadOptions' packet	820
'qTinit' packet	844
'qTMinFTPILen' packet	843
'QTNotes' packet	847
'qTP' packet	845
'qTro' packet	844
'QTSave' packet	846
'qTsP' packet	845
'qTsSTM' packet	846
'QTStart' packet	843
'qTStatus' packet	844
'qTSTMat' packet	846
'QTStop' packet	843
'qTsV' packet	846
'qTV' packet	845
qualified thread ID	49
query attached, remote request	838
query executable, remote request	838
query program arguments, remote request	838
quotes in commands	26
quoting Ada internal identifiers	255
quoting names	265
'qXfer' packet	833

R

'r' packet	801
'R' packet	801
range checking	232
range stepping	89
ranged breakpoint	354
ranges of breakpoints	59
Ravenscar Profile	261
Ravenscar thread	261
raw printing	153
read special object, remote request	833
read, file-i/o system call	858
read-only sections	297
reading symbols from relocatable object files ..	295
reading symbols immediately	294
readline	371
Readline application name	372
receive rights, GNU Hurd	348
recent tracepoint number	208
record aggregates (Ada)	253
record mode	105
record serial communications on file	329
recording a session script	730
recording inferior's execution	
and replaying it	105
recordings in python	467
redirection	41
reference card	765

reference declarations	236
register cache, flushing	785
register packet format, MIPS	840
registers	181
Registers In Python	513
regular expression	63
reloading the overlay table	225
relocatable object files, reading	
symbols from	295
remote async notification debugging info	391
remote connection commands	319
remote connection without stubs	322
remote debugging	317
remote debugging, connecting	317
remote debugging, detach and program exit ..	317
remote debugging, symbol files	318
remote debugging, types of connections	317
remote memory comparison	156
remote packets, enabling and disabling	331
remote packets, standard replies	796
remote programs, interrupting	321
remote protocol debugging	391
remote protocol, binary data	795
remote protocol, field separator	795
remote query requests	811
remote serial debugging summary	338
remote serial debugging, overview	335
remote serial protocol	795
remote serial stub	336
remote serial stub list	335
remote serial stub, initialization	336
remote serial stub, main routine	336
remote stub, example	335
remote stub, support routines	336
remote target	314
remote target, file transfer	322
remote target, limit break- and watchpoints ..	329
remote target, limit watchpoints length	329
remote timeout	329
remove actions from a tracepoint	210
remove duplicate history	373
rename, file-i/o system call	859
Reneas	336
repeated array elements	165
repeating command sequences	23
repeating commands	23
replay log events, remote reply	809
replay mode	105
reporting bugs in GDB	729
reprint the last value	145, 288
reset environment, remote request	814
resolution of location spec	128
resources used by commands	791
response time, MIPS debugging	362
restart	56
restore data from a file	188
restrictions on Go expressions	240
result records in GDB/MI	617

resume threads of multiple processes
 simultaneously 97
 resuming execution 85
 returning from a function 282
 reverse execution 103
 rewind program state 56
 RISC-V support 369
 run to first instruction 35
 run to main procedure 35
 run until specified location 87
 running 34
 running programs backward 103

S

‘s’ packet 801
 ‘S’ packet 801
 S12Z support 365
 save breakpoints to a file for future sessions 83
 save command history 373
 save GDB output to a file 22
 save tracepoints for future sessions 220
 Scalable Matrix Extension 2, AArch64 359
 Scalable Matrix Extension, AArch64 357
 Scalable Vector Extension, AArch64 356
 scheduler locking mode 96
 scheduler-locking 96
 scope 251
 screen size 374
 scripting commands 401
 scripting with guile 538
 scripting with python 407
 search for a thread 52
 search order for disassembler in Python 524
 search path for `libthread_db` 52
 searching memory 193
 searching memory, in remote debugging 821
 searching source files 133
 section offsets, remote request 817
 segment descriptor tables 342
 select Ctrl-C, BREAK or BREAK-g 330
 select trace snapshot 217
 selected frame 115
 selecting guile pretty-printers 558
 selecting python pretty-printers 432
 self tests 787
 semaphores on static probe points 83
 send command to remote monitor 322
 send command to simulator 350
 send interrupt-sequence on start 330
 send rights, GNU Hurd 348
 send the output of a gdb command to
 a shell command 21
 sending files to remote systems 322
 separate debug sections 306
 separate debugging information files 303
 sequence-id, for GDB remote 795
 serial connections, debugging 391

serial line, **target remote** 319
 serial protocol, GDB remote 795
 server prefix 716
server, command prefix 372
 set ABI for MIPS 362
 set breakpoints in many functions 63
 set breakpoints on all functions 63
 set environment variable, remote request 813
 set exec-file-mismatch 42
 set fast tracepoint 207
 set inferior controlling terminal 41
 set static tracepoint 207
 set tdesc filename 883
 set tracepoint 206
 set working directory, remote request 815
 setting variables 279
 setting watchpoints 68
 settings 24
sh-stub.c 336
 shared libraries 298
 shared library events, remote reply 809
 shell command, exit code 176
 shell command, exit signal 176
 shell escape 20
 show all convenience functions 181
 show all user variables and functions 173
 show exec-file-mismatch 42
 show inferior’s working directory 40
 show last commands 374
 show screen characteristics 792
 show tdesc filename 883
 SH 336
 signals 92
 signals the inferior may see, remote request 819
 SIGQUIT signal, dump core of GDB 780
 SingleKey keymap name 602
 size of remote memory accesses 800, 805
 size of screen 374
 skipping over files via glob-style patterns 90
 skipping over functions and files 89
 skipping over functions via
 regular expressions 90
 SME, AArch64 357
 SME2, AArch64 359
 snapshot of a process 56
 software watchpoints 68
 source code, caching 786
 source code, disable access 142
 source directories, substitution rules 135
 source file and line of a symbol 160
 source line and its code address 137
 source location 128
 source path, add directory 135
 source search path 133
 Sparc 336
sparc-stub.c 336
 Sparc64 support 364
sparc1-stub.c 336

SparcLite	336
Special Fortran commands	244
specifying location	128
SSE registers (x86)	182
stack frame	115
stack on Alpha	362
stack on MIPS	362
stack pointer register	181
stacking targets	313
standard registers	181
standard responses for remote packets	796
start a new independent interpreter	597
start a new trace experiment	214
starting	34
startup code, and backtrace	118
startup with shell, remote request	813
stat, file-i/o system call	860
static members of C++ objects	169
static members of Pascal objects	169
static probe point, DTrace	83
static probe point, SystemTap	83
static tracepoints	205
static tracepoints, in remote protocol	830
static tracepoints, setting	207
status of trace data collection	214
status output in GDB/MI	615
stepping	85
stepping and signal handlers	94
stepping into functions with no line info	87
stop a running trace experiment	214
stop on C++ exceptions	71
stop reply packets	807
stopped threads	95
store memory tags	817
stream records in GDB/MI	617
string tracing, in remote protocol	830
struct gdb_reader_funcs	723
struct gdb_symbol_callbacks	724
struct gdb_unwind_callbacks	724
struct return convention	361
struct stat, in file-i/o protocol	863
struct timeval, in file-i/o protocol	863
struct/union returned in registers	361
structure field name completion	26
stub example, remote debugging	335
stupid questions	387
styles in python	510
styling	375
substitution rules, for source search path	135
Super-H	356
supported GDB/MI features, list	704
supported packets, remote query	822
SVE, AArch64	356
svg	357
svl	357
svq	357
switching threads	48
switching threads automatically	96
symbol cache size	276
symbol cache, flushing	277
symbol cache, printing its contents	277
symbol cache, printing usage statistics	277
symbol decoding style, C++	169
symbol dump	275
symbol file functions	392
symbol files, remote debugging	318
symbol from address	266
symbol lookup	392
symbol lookup, remote request	832
symbol names	265
symbol table	293
symbol table creation	392
symbol tables in guile	577
symbol tables in python	500
symbol tables, listing GDB's internal	275
symbol, source file and line	160
symbols in guile	574
symbols in python	495
symbols, never read	294
symbols, reading from	
relocatable object files	295
symbols, reading immediately	294
Synopsys ARC	350
syscall DS0	296
system calls and thread breakpoints	100
system root, alternate	300
system, file-i/o system call	861
system-wide configuration scripts	776
system-wide init file	775
T	
‘t’ packet	802
‘T’ packet	802
‘T’ packet reply	808
tail call frames, debugging	198
target architecture	313
target byte order	316
target character set	190
target debugging info	393
target descriptions	883
target descriptions, AArch64 features	889
target descriptions, Alpha Features	894
target descriptions, ARC Features	894
target descriptions, ARM features	895
target descriptions, enum types	888
target descriptions, i386 features	898
target descriptions, inclusion	884
target descriptions, LoongArch Features	899
target descriptions, M68K features	900
target descriptions, MicroBlaze features	900
target descriptions, MIPS features	900
target descriptions, NDS32 features	900
target descriptions, OpenRISC 1000 features ..	901
target descriptions, PowerPC features	901
target descriptions, predefined types	887

target descriptions, RISC-V Features	902
target descriptions, RX Features	903
target descriptions, S/390 features	903
target descriptions, sparc32 features	903
target descriptions, sparc64 features	903
target descriptions, standard features	889
target descriptions, System z features	903
target descriptions, TIC6x features	904
target descriptions, TMS320C6x features	904
target descriptions, XML format	883
target memory comparison	156
target output in GDB/MI	615
target stack description	786
target-assisted range stepping	89
task attributes (GNU Hurd)	348
task breakpoints, in Ada	260
task exception port, GNU Hurd	348
task suspend count	348
task switching with program using Ravenscar Profile	261
TCP port, target remote	320
temporarily change settings	24
terminal	41
Text User Interface	599
thread attributes info, remote request	832
thread breakpoints	99
thread breakpoints and system calls	100
thread clone events, remote reply	810
thread create event, remote reply	810
thread create/exit events, remote request	820
thread default settings, GNU Hurd	349
thread exit event, remote reply	811
thread ID lists	49
thread identifier (GDB)	48
thread identifier (system)	48
thread info (Solaris)	50
thread information, remote request	818
thread list format	868
thread local storage	53
thread number, per inferior	48
thread options, remote request	820
thread properties, GNU Hurd	348
thread suspend count, GNU Hurd	349
<i>thread-id</i> , in remote protocol	797
threads and watchpoints	70
threads in python	465
threads of execution	48
threads, automatic switching	96
threads, continuing	95
threads, stopped	95
time of command execution	792
timeout for called functions	285, 286
timeout for commands	794
timeout for serial communications	329
timeout, for remote target connection	330
timestamping debugging info	393
TLS	53
trace experiment, status of	214
trace file format	907
trace files	221
trace state variable value, remote request	845
trace state variables	209
traceback	116
traceframe info format	869
tracepoint actions	210
tracepoint conditions	209
tracepoint data, display	219
tracepoint deletion	208
tracepoint number	208
tracepoint packets	840
tracepoint pass count	208
tracepoint restrictions	216
tracepoint status, remote request	845
tracepoint variables	220
tracepoints	205
tracepoints support in gdbserver	327
trailing underscore, in Fortran symbols	242
translating between character sets	190
TUI	599
TUI commands	602
TUI configuration variables	605
TUI key bindings	600
TUI mouse support	602
TUI single key mode	601
type casting memory	147
type chain of a data type	786
type checking	231
type conversions in C++	236
type printer	435
type printing API for Python	435
types in guile	551
types in Python	425
U	
UDP port, target remote	321
unavailable values in Python	418
union field name completion	26
unions in structures, printing	168
Unix domain socket	319
unknown address, locating	153
unknown type	270
unlink, file-i/o system call	860
unlinked object files	293
unload symbols from shared libraries	299
unmap an overlay	225
unmapped overlays	223
unset environment variable, remote request	814
unset tdesc filename	883
unsupported languages	263
unsupported packets, empty response for	796
unwind stack in called functions	283
unwind stack in called functions when timing out	284
unwind stack in called functions with unhandled exceptions	284

unwinding frames in Python	445
use only software watchpoints	69
user registers	786
user-defined command	397
user-defined macros	201
user-defined variables	173

V

value history	172
values from inferior, in guile	545
values from inferior, with Python	417
variable name conflict	149
variable object debugging info	393
variable objects in GDB/MI	660
variable values, wrong	150
variables, readline	736
variables, setting	279
‘vAttach’ packet	802
‘vCont’ packet	802
‘vCont?’ packet	803
‘vCtrlC’ packet	803
vector unit	183
vector, auxiliary	183
verbose operation	386
verify remote memory image	156
verify target memory image	156
‘vFile’ packet	804
‘vFlashDone’ packet	804
‘vFlashErase’ packet	804
‘vFlashWrite’ packet	804
vfork events, remote reply	809
vforkdone events, remote reply	810
vg	356
virtual functions (C++) display	169
‘vKill’ packet	804
vl	356
‘vMustReplyEmpty’ packet	805
volatile registers	183
vq	356
‘vRun’ packet	805
‘vStopped’ packet	805
VTBL display	169

W

watchdog timer	794
watchpoints	59
watchpoints and threads	70
wavefronts	366
where to look for shared libraries	300
wild pointer, interpreting	160
Wind River Linux system-wide configuration script	776

word completion	24
working directory	134
working directory (of your program)	40
working language	229
write data into object, remote request	837
write, file-i/o system call	858
writing a frame filter	441
writing a Guile pretty-printer	559
writing a pretty-printer	433
writing convenience functions	483
writing into corefiles	287
writing into executables	287
writing JIT debug info readers	723
writing xmethods in Python	452
wrong values	150

X

x command, default address	138
‘x’ packet	805
‘X’ packet	806
Xilinx MicroBlaze	352
XInclude	884
XMD, Xilinx Microprocessor Debugger	352
xmethod API	451
xmethods in Python	450
XML parser debugging	393

Y

yanking text	734
--------------------	-----

Z

‘z’ packet	806
‘z0’ packet	806
‘z1’ packet	807
‘z2’ packet	807
‘z3’ packet	807
‘z4’ packet	807
‘Z’ packets	806
‘Z0’ packet	806
‘Z1’ packet	807
‘Z2’ packet	807
‘Z3’ packet	807
‘Z4’ packet	807

Command, Variable, and Function Index

!		\$bpnum, convenience variable	60
!	20	\$cdir, convenience variable	134
		\$cwd, convenience variable	134
#		\$tpnum	208
# (a comment)	23	\$trace_file	220
		\$trace_frame	220
		\$trace_func	220
		\$trace_line	220
		\$tracepoint	220
\$		—	
\$_, convenience variable	174	-ada-task-info	646
\$__, convenience variable	174	-add-inferior	710
\$_ada_exception, convenience variable	72	-break-after	627
\$_any_caller_is, convenience function	180	-break-commands	628
\$_any_caller_matches,		-break-condition	628
convenience function	180	-break-delete	629
\$_as_string, convenience function	180	-break-disable	630
\$_caller_is, convenience function	180	-break-enable	630
\$_caller_matches, convenience function	180	-break-info	631
\$_cimag, convenience function	181	-break-insert	631
\$_coloursupport, convenience variable	176	-break-list	634
\$_creal, convenience function	181	-break-passcount	635
\$_exception, convenience variable	71	-break-watch	635
\$_exitcode, convenience variable	174	-catch-assert	639
\$_exitsignal, convenience variable	174	-catch-catch	642
\$_gdb_maint_setting, convenience function	178	-catch-exception	639
\$_gdb_maint_setting_str,		-catch-handlers	640
convenience function	178	-catch-load	638
\$_gdb_major, convenience variable	176	-catch-rethrow	641
\$_gdb_minor, convenience variable	176	-catch-throw	640
\$_gdb_setting, convenience function	178	-catch-unload	638
\$_gdb_setting_str, convenience function	177	-complete	713
\$_gthread, convenience variable	49	-data-disassemble	671
\$_hit_bpnum, convenience variable	60	-data-evaluate-expression	674
\$_hit_locno, convenience variable	60	-data-list-changed-registers	675
\$_inferior, convenience variable	44	-data-list-register-names	675
\$_inferior_thread_count,		-data-list-register-values	676
convenience variable	49	-data-read-memory	677
\$_isvoid, convenience function	177	-data-read-memory-bytes	679
\$_linker_namespace, convenience variable	176	-data-write-memory-bytes	680
\$_linker_namespace_count,		-dprintf-insert	633
convenience variable	176	-enable-frame-filters	654
\$_memeq, convenience function	179	-enable-pretty-printing	662
\$_probe_arg, convenience variable	84	-enable-timings	712
\$_regex, convenience function	179	-environment-cd	643
\$_sdata, collect	211	-environment-directory	643
\$_sdata, inspect, convenience variable	175	-environment-path	644
\$_shell, convenience function	179	-environment-pwd	644
\$_shell_exitcode, convenience variable	176	-exec-arguments	642
\$_shell_exitsignal, convenience variable	176	-exec-continue	647
\$_siginfo, convenience variable	175	-exec-finish	648
\$_streq, convenience function	179	-exec-interrupt	649
\$_strlen, convenience function	180		
\$_thread, convenience variable	49		
\$_tlb, convenience variable	175		

-exec-jump.....	650	-trace-save.....	684
-exec-next.....	650	-trace-start.....	684
-exec-next-instruction.....	650	-trace-status.....	685
-exec-return.....	651	-trace-stop.....	686
-exec-run.....	652	-var-assign.....	667
-exec-step.....	653	-var-create.....	662
-exec-step-instruction.....	653	-var-delete.....	663
-exec-until.....	654	-var-evaluate-expression.....	667
-file-exec-and-symbols.....	694	-var-info-expression.....	666
-file-exec-file.....	695	-var-info-num-children.....	664
-file-list-exec-source-file.....	695	-var-info-path-expression.....	666
-file-list-exec-source-files.....	695	-var-info-type.....	666
-file-list-shared-libraries.....	698	-var-list-children.....	664
-file-symbol-file.....	698	-var-set-format.....	664
-gdb-exit.....	706	-var-set-frozen.....	669
-gdb-set.....	706	-var-set-update-range.....	670
-gdb-show.....	706	-var-set-visualizer.....	670
-gdb-version.....	707	-var-show-attributes.....	667
-inferior-tty-set.....	712	-var-show-format.....	664
-inferior-tty-show.....	712	-var-update.....	668
-info-ada-exceptions.....	703		
-info-gdb-mi-command.....	704	:	
-info-os.....	709	::, in Modula-2.....	251
-interpreter-exec.....	711		
-list-features.....	704	<	
-list-target-features.....	705	<gdb:arch>.....	583
-list-thread-groups.....	707	<gdb:block>.....	572
-remove-inferior.....	711	<gdb:breakpoint>.....	578
-stack-info-depth.....	655	<gdb:color>.....	589
-stack-info-frame.....	655	<gdb:iterator>.....	587
-stack-list-arguments.....	655	<gdb:lazy-string>.....	582
-stack-list-frames.....	657	<gdb:objfile>.....	568
-stack-list-locals.....	659	<gdb:progspace>.....	567
-stack-list-variables.....	659	<gdb:sal>.....	577
-stack-select-frame.....	660	<gdb:symbol>.....	574
-symbol-info-functions.....	686	<gdb:symtab>.....	577
-symbol-info-module-functions.....	688	<gdb:type>.....	551
-symbol-info-module-variables.....	689	<gdb:value>.....	545
-symbol-info-modules.....	690		
-symbol-info-types.....	691	@	
-symbol-info-variables.....	692	@, referencing memory as an array.....	152
-symbol-list-lines.....	694		
-target-attach.....	698	^	
-target-detach.....	699	^connected.....	617
-target-disconnect.....	699	^done.....	617
-target-download.....	699	^error.....	617
-target-file-delete.....	703	^exit.....	617
-target-file-get.....	702	^running.....	617
-target-file-put.....	702		
-target-flash-erase.....	701	-	
-target-select.....	701	__init__ on TypePrinter.....	535
-thread-info.....	645		
-thread-list-ids.....	645		
-thread-select.....	646		
-trace-define-variable.....	682		
-trace-find.....	681		
-trace-frame-collected.....	682		
-trace-list-variables.....	684		

| 21

A

abort (C-g) 755
 accept-line (Newline or Return) 750
 actions 210
 active-region-end-color 737
 active-region-start-color 737
 ada-task-info 705
 add-auto-load-safe-path 384
 add-auto-load-scripts-directory 593
 add-inferior 45
 add-symbol-file 295
 add-symbol-file-from-memory 296
 adi assign 365
 adi examine 365
 advance *locs*pec 88
 alias 405
 append 188
 append-pretty-printer! 591
 apropos 30
 arch-bool-type 584
 arch-char-type 584
 arch-charset 583
 arch-disassemble 585
 arch-double-type 584
 arch-float-type 584
 arch-int-type 584
 arch-int16-type 584
 arch-int32-type 585
 arch-int64-type 585
 arch-int8-type 584
 arch-long-type 584
 arch-longdouble-type 584
 arch-longlong-type 584
 arch-name 583
 arch-schar-type 584
 arch-short-type 584
 arch-uchar-type 584
 arch-uint-type 584
 arch-uint16-type 585
 arch-uint32-type 585
 arch-uint64-type 585
 arch-uint8-type 584
 arch-ulong-type 584
 arch-ulonglong-type 584
 arch-ushort-type 584
 arch-void-type 583
 arch-wide-charset 583
 arch? 583
 Architecture.disassemble 512
 Architecture.integer_type 513
 Architecture.name 512
 Architecture.register_groups 513
 Architecture.registers 513
 Architecture.void_type 513

attach 41
 attach& 98
 awatch 69

B

b (break) 60
 backtrace 116
 backward-char (C-b) 749
 backward-delete-char (Rubout) 752
 backward-kill-line (C-x Rubout) 753
 backward-kill-word (M-DEL) 753
 backward-word (M-b) 749
 beginning-of-history (M-<) 750
 beginning-of-line (C-a) 749
 bell-style 737
 bfd caching 302, 303
 bind-tty-special-chars 737
 blink-matching-paren 737
 block-end 573
 block-function 573
 block-global-block 573
 block-global? 573
 block-start 573
 block-static-block 573
 block-static? 573
 block-superblock 573
 block-symbols 573
 block-symbols-progress? 573
 block-valid? 572
 Block.end 494
 Block.function 495
 Block.global_block 495
 Block.is_global 495
 Block.is_static 495
 Block.is_valid 494
 Block.ranges 494
 Block.start 494
 Block.static_block 495
 Block.subblocks 495
 Block.superblock 495
 block? 572
 BP_ACCESS_WATCHPOINT 504, 579
 BP_BREAKPOINT 503, 579
 BP_CATCHPOINT 504, 579
 BP_HARDWARE_BREAKPOINT 503
 BP_HARDWARE_WATCHPOINT 504, 579
 BP_READ_WATCHPOINT 504, 579
 BP_WATCHPOINT 503, 579
 bracketed-paste-begin () 752
 break 60
 break ... inferior *inferior-id* 47
 break ... task *taskno* (Ada) 260
 break ... thread *thread-id* 99
 break, and Objective-C 241
 break-range 354
 breakpoint annotation 718
 breakpoint-commands 582

colored-completion-prefix..... 737
 colored-stats 738
 COLORSPACE_AIXTERM_16COLOR..... 510, 590
 COLORSPACE_ANSI_8COLOR..... 510, 590
 COLORSPACE_MONOCHROME 510, 589
 COLORSPACE_RGB_24BIT..... 510, 590
 COLORSPACE_XTERM_256COLOR..... 510, 590
 Command.__init__..... 471
 Command.complete..... 472
 Command.dont_repeat 471
 Command.invoke..... 471
 command?..... 562
 COMMAND_BREAKPOINTS..... 473, 563
 COMMAND_DATA..... 472, 563
 COMMAND_FILES..... 473, 563
 COMMAND_MAINTENANCE..... 473, 564
 COMMAND_NONE..... 472, 563
 COMMAND_OBSCURE..... 473, 564
 COMMAND_RUNNING..... 472, 563
 COMMAND_STACK..... 472, 563
 COMMAND_STATUS..... 473, 563
 COMMAND_SUPPORT..... 473, 563
 COMMAND_TRACEPOINTS..... 473, 564
 COMMAND_TUI..... 473
 COMMAND_USER..... 473, 564
 commands..... 80
 commands annotation..... 716
 comment-begin..... 738
 compare-sections..... 156
 compile code..... 287
 compile file..... 288
 complete..... 30
 complete (TAB)..... 754
 COMPLETE_COMMAND..... 474, 564
 COMPLETE_EXPRESSION..... 474, 564
 COMPLETE_FILENAME..... 474, 564
 COMPLETE_LOCATION..... 474, 564
 COMPLETE_NONE..... 474, 564
 COMPLETE_SYMBOL..... 474, 564
 completion-display-width..... 738
 completion-ignore-case..... 738
 completion-map-case..... 738
 completion-prefix-display-length..... 738
 completion-query-items..... 738
 condition..... 78
 ConnectionEvent.connection..... 463
 continue..... 85
 continue&..... 98
 convert-meta..... 738
 copy-backward-word ()..... 754
 copy-forward-word ()..... 754
 copy-region-as-kill ()..... 754
 core-file..... 294
 Corefile.filename..... 531
 Corefile.is_valid..... 531
 Corefile.mapped_files..... 531
 CorefileChangedEvent.inferior..... 464
 CorefileMappedFile.build_id..... 531

CorefileMappedFile.filename..... 531
 CorefileMappedFile.is_main_executable.... 532
 CorefileMappedFile.regions..... 532
 CorefileMappedFileRegion.end..... 532
 CorefileMappedFileRegion.file_offset.... 532
 CorefileMappedFileRegion.start..... 532
 ctf..... 221
 Ctrl-o (operate-and-get-next)..... 23
 current-arch..... 583
 current-objfile..... 569
 current-progspace..... 568
 cwd..... 134

D

d (delete)..... 76
 d (SingleKey TUI key)..... 601
 data-directory..... 541
 data-disassemble-a-option..... 705
 data-read-memory-bytes..... 705
 default-visualizer..... 558
 define..... 398
 define-prefix..... 398
 delete..... 76
 delete checkpoint *checkpoint-id*..... 57
 delete display..... 158
 delete mem..... 186
 delete skip..... 92
 delete tracepoint..... 208
 delete tvariable..... 210
 delete-breakpoint!..... 580
 delete-char (C-d)..... 752
 delete-char-or-list ()..... 755
 delete-horizontal-space ()..... 754
 demangle..... 267
 detach..... 42
 detach (remote)..... 321
 detach inferiors *infno*... 46
 digit-argument (*M-0*, *M-1*, ... *M--*)..... 754
 dir..... 135
 directory..... 135
 dis (disable)..... 77
 disable..... 77
 disable display..... 158
 disable frame-filter..... 124
 disable mem..... 186
 disable missing-objfile-handler..... 530
 disable pretty-printer..... 171
 disable probes..... 84
 disable skip..... 92
 disable tracepoint..... 208
 disable type-printer..... 271
 disable-completion..... 739
 disassemble..... 138
 DisassembleInfo.__init__..... 518
 DisassembleInfo.address..... 518
 DisassembleInfo.address_part..... 519
 DisassembleInfo.architecture..... 518

DisassembleInfo.is_valid.....	518
DisassembleInfo.progspace.....	518
DisassembleInfo.read_memory.....	518
DisassembleInfo.text_part.....	519
Disassembler.__call__.....	519
Disassembler.__init__.....	519
DisassemblerAddressPart.address.....	522
DisassemblerPart.string.....	521
DisassemblerResult.__init__.....	520
DisassemblerResult.length.....	520
DisassemblerResult.parts.....	521
DisassemblerResult.string.....	521
DisassemblerTextPart.style.....	522
disconnect.....	321
display.....	158
do (down).....	120
do-lowercase-version (M-A, M-B, M-x, ...)	755
document.....	398
dont-repeat.....	399, 562
Down.....	601
down.....	120
down-silently.....	121
downcase-word (M-l).....	752
dprintf.....	81
dprintf-style agent.....	82
dprintf-style call.....	81
dprintf-style gdb.....	81
dump.....	188
dump-functions ().....	756
dump-macros ().....	757
dump-variables ().....	757

E

e (edit).....	132
echo.....	403
echo-control-characters.....	739
edit.....	132
editing-mode.....	739
else.....	402
emacs-editing-mode (C-e).....	757
emacs-mode-string.....	739
enable.....	77
enable display.....	158
enable frame-filter.....	125
enable mem.....	186
enable missing-objfile-handler.....	530
enable pretty-printer.....	171
enable probes.....	84
enable skip.....	92
enable tracepoint.....	208
enable type-printer.....	271
enable-active-region.....	739
enable-bracketed-paste.....	739
enable-keypad.....	739
enabled.....	451
enabled of type_printer.....	436

end (breakpoint commands).....	80
end (if/else/while commands).....	402
end (user-defined commands).....	398
end-kbd-macro (C-x).....	755
end-of-file (usually C-d).....	751
end-of-history (M->).....	750
end-of-iteration.....	588
end-of-iteration?.....	588
end-of-line (C-e).....	749
error annotation.....	717
error-begin annotation.....	717
error-port.....	586
eval.....	404
EventRegistry.connect.....	458
EventRegistry.disconnect.....	458
exception-args.....	544
exception-key.....	544
exception?.....	544
exceptionHandler.....	337
exchange-point-and-mark (C-x C-x).....	756
exec-file.....	293
exec-file-mismatch.....	42
exec-run-start-option.....	705
ExecutableChangedEvent.progspace.....	463
ExecutableChangedEvent.reload.....	464
execute.....	540
exit [expression].....	20
exited annotation.....	717
ExitedEvent.exit_code.....	459
ExitedEvent.inferior.....	459
expand-tilde.....	740
explore.....	145

F

f (frame).....	119
f (SingleKey TUI key).....	601
F (SingleKey TUI key).....	601
faas.....	124
fetch-history ().....	751
fg (resume foreground execution).....	85
field-artificial?.....	556
field-base-class?.....	556
field-bitpos.....	556
field-bitsize.....	556
field-enumval.....	555
field-name.....	555
field-type.....	555
field?.....	555
file.....	293
fin (finish).....	87
find.....	194
find-pc-line.....	578
finish.....	87
finish&.....	98
FinishBreakpoint.__init__.....	508
FinishBreakpoint.out_of_scope.....	508
FinishBreakpoint.return_value.....	508

flash-erase 316
 flush_i_cache 337
 flushregs 785
 fo (forward-search) 133
 focus 604
 forward-backward-delete-char () 752
 forward-char (C-f) 749
 forward-search 133
 forward-search-history (C-s) 750
 forward-word (M-f) 749
 frame address 119
 frame apply 122
 frame function 120
 frame level 119
 frame view 120
 frame, selecting 119
 frame-arch 569
 frame-block 571
 frame-function 571
 frame-name 569
 frame-newer 571
 frame-older 571
 frame-pc 571
 frame-read-register 571
 frame-read-var 571
 frame-sal 571
 frame-select 571
 frame-type 569
 frame-unwind-stop-reason 570
 frame-valid? 569
 Frame.architecture 490
 Frame.block 492
 Frame.find_sal 492
 Frame.function 492
 Frame.is_valid 490
 Frame.language 493
 Frame.level 493
 Frame.name 490
 Frame.newer 492
 Frame.older 492
 Frame.pc 492
 Frame.read_register 492
 Frame.read_var 493
 Frame.select 493
 Frame.static_link 493
 Frame.type 490
 Frame.unwind_stop_reason 491
 frame? 569
 FrameDecorator.address 439
 FrameDecorator.elided 439
 FrameDecorator.filename 439
 FrameDecorator.frame_args 440
 FrameDecorator.frame_locals 440
 FrameDecorator.function 439
 FrameDecorator.inferior_frame 441
 FrameDecorator.line 439
 FrameFilter.enabled 438
 FrameFilter.filter 437

FrameFilter.name 438
 FrameFilter.priority 438
 frames-invalid annotation 717
 FreeObjFileEvent.objfile 460
 FreeProgspaceEvent.progspace 464
 frozen-varobjs 705
 ftrace 207
 Function 483
 Function.__init__ 483
 Function.invoke 484

G

gcore 189
 gdb-object-kind 541
 gdb-version 541
 gdb.add_history 412
 gdb.architecture_names 414
 gdb.Block 493
 gdb.block_for_pc 494
 gdb.blocked_signals 415
 gdb.BP_ACCESS_WATCHPOINT 504
 gdb.BP_BREAKPOINT 503
 gdb.BP_CATCHPOINT 504
 gdb.BP_HARDWARE_BREAKPOINT 503
 gdb.BP_HARDWARE_WATCHPOINT 504
 gdb.BP_READ_WATCHPOINT 504
 gdb.BP_WATCHPOINT 504
 gdb.Breakpoint 503
 gdb.breakpoints 411
 gdb.Color 509
 gdb.COLORSPACE_AIXTERM_16COLOR 510
 gdb.COLORSPACE_ANSI_8COLOR 510
 gdb.COLORSPACE_MONOCHROME 510
 gdb.COLORSPACE_RGB_24BIT 510
 gdb.COLORSPACE_XTERM_256COLOR 510
 gdb.COMMAND_BREAKPOINTS 473
 gdb.COMMAND_DATA 472
 gdb.COMMAND_FILES 473
 gdb.COMMAND_MAINTENANCE 473
 gdb.COMMAND_NONE 472
 gdb.COMMAND_OBSCURE 473
 gdb.COMMAND_RUNNING 472
 gdb.COMMAND_STACK 472
 gdb.COMMAND_STATUS 473
 gdb.COMMAND_SUPPORT 473
 gdb.COMMAND_TRACEPOINTS 473
 gdb.COMMAND_TUI 473
 gdb.COMMAND_USER 473
 gdb.COMPLETE_COMMAND 474
 gdb.COMPLETE_EXPRESSION 474
 gdb.COMPLETE_FILENAME 474
 gdb.COMPLETE_LOCATION 474
 gdb.COMPLETE_NONE 474
 gdb.COMPLETE_SYMBOL 474
 gdb.connections 414
 gdb.convenience_variable 412
 gdb.current_language 415

<code>gdb.current_objfile</code>	487	<code>gdb.missing_</code>	
<code>gdb.current_progspace</code>	484	<code>objfile.MissingObjfileHandler</code>	528
<code>gdb.current_recording</code>	467	<code>gdb.missing_objfile.register_handler</code>	530
<code>gdb.decode_line</code>	414	<code>gdb.newest_frame</code>	490
<code>gdb.default_visualizer</code>	432	<code>gdb.notify_mi</code>	477
<code>gdb.disassembler.DisassembleInfo</code>	517	<code>gdb.Objfile</code>	487
<code>gdb.disassembler.Disassembler</code>	519	<code>gdb.objfiles</code>	487
<code>gdb.disassembler.DisassemblerAddressPart</code>	522	<code>gdb.PARAM_AUTO_BOOLEAN</code>	479
<code>gdb.disassembler.DisassemblerPart</code>	521	<code>gdb.PARAM_BOOLEAN</code>	479
<code>gdb.disassembler.DisassemblerResult</code>	520	<code>gdb.PARAM_COLOR</code>	480
<code>gdb.disassembler.DisassemblerTextPart</code>	521	<code>gdb.PARAM_ENUM</code>	480
<code>gdb.disassembler.STYLE_ADDRESS</code>	523	<code>gdb.PARAM_FILENAME</code>	480
<code>gdb.disassembler.STYLE_ADDRESS_OFFSET</code>	523	<code>gdb.PARAM_INTEGER</code>	480
<code>gdb.disassembler.STYLE_</code>		<code>gdb.PARAM_OPTIONAL_FILENAME</code>	480
<code>ASSEMBLER_DIRECTIVE</code>	523	<code>gdb.PARAM_STRING</code>	480
<code>gdb.disassembler.STYLE_COMMENT_START</code>	524	<code>gdb.PARAM_STRING_NOESCAPE</code>	480
<code>gdb.disassembler.STYLE_IMMEDIATE</code>	523	<code>gdb.PARAM_UINTEGER</code>	480
<code>gdb.disassembler.STYLE_MNEMONIC</code>	522	<code>gdb.PARAM_ZINTEGER</code>	480
<code>gdb.disassembler.STYLE_REGISTER</code>	523	<code>gdb.PARAM_ZUINTEGER</code>	480
<code>gdb.disassembler.STYLE_SUB_MNEMONIC</code>	522	<code>gdb.PARAM_ZUINTEGER_UNLIMITED</code>	480
<code>gdb.disassembler.STYLE_SYMBOL</code>	524	<code>gdb.Parameter</code>	477
<code>gdb.disassembler.STYLE_TEXT</code>	522	<code>gdb.parameter</code>	411
<code>gdb.error</code>	416	<code>gdb.parse_and_eval</code>	412
<code>gdb.execute</code>	410	<code>gdb.post_event</code>	416
<code>gdb.execute_mi</code>	476	<code>gdb.pretty_printers</code>	433
<code>gdb.find_pc_line</code>	412	<code>gdb.print_options</code>	432
<code>gdb.FinishBreakpoint</code>	507	<code>gdb.Progspace</code>	484
<code>gdb.flush</code>	413	<code>gdb.progspace</code>	484
<code>gdb.format_address</code>	414	<code>gdb.prompt_hook</code>	414
<code>gdb.frame_stop_reason_string</code>	490	<code>gdb.ptwrite.get_filter</code>	537
<code>gdb.FrameDecorator</code>	439	<code>gdb.ptwrite.register_filter_factory</code>	536
<code>gdb.Function</code>	483	<code>gdb.PYTHONDIR</code>	410
<code>gdb.GdbError</code>	417	<code>gdb.rbreak</code>	411
<code>gdb.history</code>	411	<code>gdb.register_window_type</code>	516
<code>gdb.history_count</code>	412	<code>gdb.selected_frame</code>	490
<code>gdb.host_charset</code>	414	<code>gdb.selected_inferior</code>	455
<code>gdb.Inferior</code>	455	<code>gdb.selected_thread</code>	465
<code>gdb.InferiorCallPostEvent</code>	461	<code>gdb.set_convenience_variable</code>	412
<code>gdb.InferiorCallPreEvent</code>	461	<code>gdb.set_parameter</code>	411
<code>gdb.inferiors</code>	455	<code>gdb.solib_name</code>	414
<code>gdb.InferiorThread</code>	465	<code>gdb.start_recording</code>	467
<code>gdb.INTENSITY_BOLD</code>	512	<code>gdb.STDERR</code>	413
<code>gdb.INTENSITY_DIM</code>	512	<code>gdb.STDLOG</code>	413
<code>gdb.INTENSITY_NORMAL</code>	512	<code>gdb.STDOUT</code>	413
<code>gdb.interrupt</code>	416	<code>gdb.stop_recording</code>	467
<code>gdb.invalidate_cached_frames</code>	490	<code>gdb.string_to_argv</code>	472
<code>gdb.LazyString</code>	508	<code>gdb.Style</code>	510
<code>gdb.LineTable</code>	502	<code>gdb.Symbol</code>	495
<code>gdb.lookup_global_symbol</code>	496	<code>gdb.SYMBOL_COMMON_BLOCK_DOMAIN</code>	498
<code>gdb.lookup_objfile</code>	487	<code>gdb.SYMBOL_FUNCTION_DOMAIN</code>	498
<code>gdb.lookup_static_symbol</code>	496	<code>gdb.SYMBOL_LABEL_DOMAIN</code>	498
<code>gdb.lookup_static_symbols</code>	496	<code>gdb.SYMBOL_LOC_ARG</code>	499
<code>gdb.lookup_symbol</code>	495	<code>gdb.SYMBOL_LOC_BLOCK</code>	499
<code>gdb.lookup_type</code>	425	<code>gdb.SYMBOL_LOC_COMMON_BLOCK</code>	500
<code>gdb.MemoryError</code>	417	<code>gdb.SYMBOL_LOC_COMPUTED</code>	500
<code>gdb.missing_debug.MissingDebugHandler</code>	526	<code>gdb.SYMBOL_LOC_CONST</code>	499
<code>gdb.missing_debug.register_handler</code>	528	<code>gdb.SYMBOL_LOC_CONST_BYTES</code>	499
		<code>gdb.SYMBOL_LOC_LABEL</code>	499

gdb.SYMBOL_LOC_LOCAL 499
 gdb.SYMBOL_LOC_OPTIMIZED_OUT 500
 gdb.SYMBOL_LOC_REF_ARG 499
 gdb.SYMBOL_LOC_REGISTER 499
 gdb.SYMBOL_LOC_REGPARM_ADDR 499
 gdb.SYMBOL_LOC_STATIC 499
 gdb.SYMBOL_LOC_TYPEDEF 499
 gdb.SYMBOL_LOC_UNDEF 499
 gdb.SYMBOL_LOC_UNRESOLVED 499
 gdb.SYMBOL_MODULE_DOMAIN 498
 gdb.SYMBOL_STRUCT_DOMAIN 498
 gdb.SYMBOL_TYPE_DOMAIN 498
 gdb.SYMBOL_UNDEF_DOMAIN 498
 gdb.SYMBOL_VAR_DOMAIN 498
 gdb.Symtab 500
 gdb.Symtab_and_line 500
 gdb.target_charset 413
 gdb.target_wide_charset 414
 gdb.Thread 415
 gdb.Type 425
 gdb.TYPE_CODE_ARRAY 429
 gdb.TYPE_CODE_BITSTRING 429
 gdb.TYPE_CODE_BOOL 430
 gdb.TYPE_CODE_CHAR 430
 gdb.TYPE_CODE_COMPLEX 430
 gdb.TYPE_CODE_DECFLOAT 430
 gdb.TYPE_CODE_ENUM 429
 gdb.TYPE_CODE_ERROR 429
 gdb.TYPE_CODE_FIXED_POINT 430, 555
 gdb.TYPE_CODE_FLAGS 429
 gdb.TYPE_CODE_FLT 429
 gdb.TYPE_CODE_FUNC 429
 gdb.TYPE_CODE_INT 429
 gdb.TYPE_CODE_INTERNAL_FUNCTION 430
 gdb.TYPE_CODE_MEMBERPTR 429
 gdb.TYPE_CODE_METHOD 429
 gdb.TYPE_CODE_METHODPTR 429
 gdb.TYPE_CODE_NAMESPACE 430, 555
 gdb.TYPE_CODE_PTR 429
 gdb.TYPE_CODE_RANGE 429
 gdb.TYPE_CODE_REF 430
 gdb.TYPE_CODE_RVALUE_REF 430
 gdb.TYPE_CODE_SET 429
 gdb.TYPE_CODE_STRING 429
 gdb.TYPE_CODE_STRUCT 429
 gdb.TYPE_CODE_TYPEDEF 430
 gdb.TYPE_CODE_UNION 429
 gdb.TYPE_CODE_VOID 429
 gdb.TYPE_CODE_XMETHOD 430, 555
 gdb.unwinder.enabled 448
 gdb.unwinder.FrameId 448
 gdb.unwinder.FrameId.__init__ 448
 gdb.unwinder.name 448
 gdb.unwinder.pc 448
 gdb.unwinder.register_unwinder 449
 gdb.unwinder.sp 448
 gdb.unwinder.special 448
 gdb.unwinder.Unwinder 448

gdb.unwinder.Unwinder.__init__ 448
 gdb.UnwindInfo.add_saved_register 447
 gdb.warning 413
 gdb.with_parameter 411
 gdb.WP_ACCESS 504
 gdb.WP_READ 504
 gdb.WP_WRITE 504
 gdb.write 413
 gdb:error 544
 gdb:invalid-object 544
 gdb:memory-error 544
 gdb:pp-type-error 544
 gdb_init_reader 723
 GdbExitingEvent.exit_code 463
 gdbserver 322
 generate-core-file 189
 get-basic-type 591
 get_filter 537
 getDebugChar 337
 gnu_debuglink_crc32 305
 gr 538
 gu 538
 guile 538
 guile-data-directory 541
 guile-repl 538

H

h (help) 29
 handle 93
 handle_exception 336
 hbreak 62
 help 29
 help function 181
 help target 314
 help user-defined 399
 history-append! 541
 history-preserve-point 740
 history-ref 540
 history-search-backward () 750
 history-search-forward () 750
 history-size 740
 history-substring-search-backward () 751
 history-substring-search-forward () 751
 hook 400
 hookpost 400
 horizontal-scroll-mode 740
 host-config 541

I

i (info)	31
i (SingleKey TUI key)	602
I (SingleKey TUI key)	602
if	402
ignore	79
inferior	44
inferior <i>info</i>	44
Inferior.architecture	456
Inferior.arguments	456
Inferior.clear_env	457
Inferior.connection	455
Inferior.connection_num	455
Inferior.corefile	456
Inferior.is_valid	456
Inferior.main_name	455
Inferior.num	455
Inferior.pid	455
Inferior.progspace	456
Inferior.read_memory	456
Inferior.search_memory	457
Inferior.set_env	457
Inferior.thread_from_handle	457
Inferior.thread_from_thread_handle	457
Inferior.threads	456
Inferior.unset_env	457
Inferior.was_attached	455
Inferior.write_memory	456
InferiorCallPostEvent.address	461
InferiorCallPostEvent.ptid	461
InferiorCallPreEvent.address	461
InferiorCallPreEvent.ptid	461
InferiorDeletedEvent.inferior	462
InferiorThread.details	465
InferiorThread.global_num	465
InferiorThread.handle	466
InferiorThread.inferior	465
InferiorThread.is_exited	466
InferiorThread.is_running	466
InferiorThread.is_stopped	466
InferiorThread.is_valid	466
InferiorThread.name	465
InferiorThread.num	465
InferiorThread.ptid	465
InferiorThread.ptid_string	465
InferiorThread.switch	466
info	31
info address	266
info all-registers	181
info args	121
info auto-load	382
info auto-load gdb-scripts	405
info auto-load guile-scripts	590
info auto-load libthread-db	384
info auto-load local-gdbinit	383
info auto-load python-scripts	532
info auxv	184
info breakpoints	63
info checkpoints	57
info classes	274
info common	244
info connections [<i>id</i>...]	44
info copying	31
info dcache	193
info display	159
info dll	299
info dos	342
info exceptions	256
info extensions	231
info f (info frame)	121
info files	296
info float	183
info frame	121
info frame, show the source language	231
info frame-filter	124
info functions	272
info handle	93
info inferiors [<i>id</i>...]	43
info io_registers, AVR	355
info line	137
info line, and Objective-C	241
info linker-namespaces	299
info locals	122
info macro	201
info macros	201
info main	274
info mem	186
info missing-objfile-handlers	530
info module	274
info modules	274
info os	184
info os cpus	184
info os files	184
info os modules	184
info os msg	184
info os processes	184
info os procgroups	185
info os semaphores	185
info os shm	185
info os sockets	185
info os threads	185
info pretty-printer	171
info probes	83
info proc	340
info program	59
info record	110
info registers	181
info scope	271
info selectors	274
info serial	343
info set	31
info share	298
info sharedlibrary	298
info signals	93
info skip	91
info source	272

info source, show the source language..... 231
 info sources..... 272, 695
 info stack..... 117
 info static-tracepoint-markers..... 213
 info symbol..... 266
 info target..... 296
 info task taskno..... 258
 info tasks..... 257
 info terminal..... 41
 info threads..... 50
 info tp [n...]...... 212
 info tracepoints [n...]...... 212
 info tvariables..... 210
 info type-printers..... 271
 info types..... 271
 info variables..... 273
 info vector..... 183
 info w32..... 344
 info warranty..... 31
 info watchpoints [list...]...... 69
 info win..... 603
 info-gdb-mi-command..... 705
 init-if-undefined..... 173
 input-meta..... 740
 input-port..... 586
 insert-comment (M-#)..... 756
 insert-completions (M-*)..... 755
 inspect..... 143
 instantiate on type_printer..... 436
 Instruction.data..... 468
 Instruction.decoded..... 468
 Instruction.pc..... 468
 Instruction.size..... 468
 INTENSITY_BOLD..... 512
 INTENSITY_DIM..... 512
 INTENSITY_NORMAL..... 512
 interpreter-exec..... 597
 interrupt..... 99
 isearch-terminators..... 740
 iterator->list..... 588
 iterator-filter..... 588
 iterator-for-each..... 588
 iterator-map..... 588
 iterator-next!..... 588
 iterator-object..... 588
 iterator-progress..... 588
 iterator-until..... 588
 iterator?..... 588

J

j (jump)..... 280
 jit-reader-load..... 723
 jit-reader-unload..... 723
 jump..... 280
 jump, and Objective-C..... 241

K

KeyboardInterrupt..... 417
 keymap..... 740
 kill..... 42
 kill inferiors *info*..... 46
 kill-line (C-k)..... 753
 kill-region ()..... 754
 kill-whole-line ()..... 753
 kill-word (M-d)..... 753
 kvm..... 339

L

l (list)..... 127
 language-option..... 705
 layout..... 603
 lazy-string->value..... 583
 lazy-string-address..... 583
 lazy-string-encoding..... 583
 lazy-string-length..... 583
 lazy-string-type..... 583
 lazy-string?..... 583
 LazyString.address..... 508
 LazyString.encoding..... 509
 LazyString.length..... 509
 LazyString.type..... 509
 LazyString.value..... 508
 Left..... 601
 LineTable.has_line..... 502
 LineTable.line..... 502
 LineTable.source_lines..... 502
 LineTableEntry.line..... 502
 LineTableEntry.pc..... 502
 list..... 127
 list, and Objective-C..... 241
 load *filename offset*..... 315
 lookup-block..... 573
 lookup-global-symbol..... 575
 lookup-symbol..... 575
 lookup-type..... 551
 loop_break..... 402
 loop_continue..... 402

M

macro define..... 201
 macro exp1..... 201
 macro expand..... 201
 macro list..... 202
 macro undef..... 202
 maint ada set ignore-descriptive-types.... 263
 maint ada show ignore-descriptive-types... 263
 maint agent..... 777
 maint agent-eval..... 777
 maint agent-printf..... 777
 maint btrace clear..... 778
 maint btrace clear-packet-history..... 778
 maint btrace packet-history..... 778

<code>maint canonicalize</code>	777	<code>maint print symbols</code>	275
<code>maint check libthread-db</code>	782	<code>maint print target-stack</code>	786
<code>maint check symtabs</code>	780	<code>maint print type</code>	786
<code>maint check xml-descriptions</code>	782	<code>maint print unwind, HPPA</code>	364
<code>maint cplus first_component</code>	780	<code>maint print user-registers</code>	786
<code>maint cplus namespace</code>	780	<code>maint print xml-tdesc</code>	782
<code>maint demangler-warning</code>	780	<code>maint selftest</code>	787
<code>maint deprecate</code>	780	<code>maint set backtrace-on-fatal-signal</code>	793
<code>maint dump-me</code>	780	<code>maint set bfd-sharing</code>	302
<code>maint expand-symtabs</code>	780	<code>maint set btrace pt skip-pad</code>	778
<code>maint flush dcache</code>	193	<code>maint set catch-demangler-crashes</code>	780
<code>maint flush register-cache</code>	785	<code>maint set check-libthread-db</code>	791
<code>maint flush source-cache</code>	786	<code>maint set debuginfod download-sections</code>	780
<code>maint flush symbol-cache</code>	277	<code>maint set demangler-warning</code>	781
<code>maint flush-symbol-cache</code>	277	<code>maint set dwarf always-disassemble</code>	787
<code>maint frame-unwinder disable</code>	788	<code>maint set dwarf max-cache-age</code>	787
<code>maint frame-unwinder enable</code>	788	<code>maint set dwarf synchronous</code>	788
<code>maint ignore-probes</code>	794	<code>maint set</code>	
<code>maint info bcdcsr, SI2Z</code>	365	<code>force-internal-tls-address-lookup</code>	54
<code>maint info bfds</code>	302	<code>maint set gnu-source-highlight enabled</code>	791
<code>maint info blocks</code>	784	<code>maint set ignore-prologue-end-flag</code>	277
<code>maint info breakpoints</code>	777	<code>maint set internal-error</code>	781
<code>maint info btrace</code>	778	<code>maint set internal-warning</code>	781
<code>maint info frame-unwinders</code>	788	<code>maint set libopcodes-styling enabled</code>	792
<code>maint info inline-frames</code>	783	<code>maint set per-command</code>	791
<code>maint info jit</code>	778	<code>maint set profile</code>	789
<code>maint info line-table</code>	276	<code>maint set selftest verbose</code>	787
<code>maint info linux-lwps</code>	779	<code>maint set show-all-tib</code>	789
<code>maint info program-spaces</code>	47	<code>maint set show-debug-regs</code>	789
<code>maint info python-disassemblers</code>	778	<code>maint set symbol-cache-size</code>	276
<code>maint info screen</code>	792	<code>maint set target-async</code>	789
<code>maint info sections</code>	296	<code>maint set target-non-stop mode</code>	
<code>maint info selftests</code>	787	<code>[on off auto]</code>	789
<code>maint info sol-threads</code>	50	<code>maint set test-settings</code>	793
<code>maint info symtabs</code>	275	<code>maint set tui-left-margin-verbose</code>	790
<code>maint info target-sections</code>	297	<code>maint set tui-resize-message</code>	790
<code>maint internal-error</code>	780	<code>maint set worker-threads</code>	788
<code>maint internal-warning</code>	780	<code>maint show backtrace-on-fatal-signal</code>	793
<code>maint packet</code>	515, 781	<code>maint show bfd-sharing</code>	302
<code>maint print arc arc-instruction</code>	350	<code>maint show btrace pt skip-pad</code>	778
<code>maint print architecture</code>	781	<code>maint show catch-demangler-crashes</code>	780
<code>maint print c-tdesc</code>	781	<code>maint show check-libthread-db</code>	791
<code>maint print cooked-registers</code>	785	<code>maint show debuginfod download-sections</code>	780
<code>maint print core-file-backed-mappings</code>	782	<code>maint show demangler-warning</code>	781
<code>maint print dummy-frames</code>	782	<code>maint show dwarf always-disassemble</code>	787
<code>maint print frame-id</code>	782	<code>maint show dwarf max-cache-age</code>	787
<code>maint print msymbols</code>	275	<code>maint show dwarf synchronous</code>	788
<code>maint print objfiles</code>	786	<code>maint show</code>	
<code>maint print raw-registers</code>	785	<code>force-internal-tls-address-lookup</code>	54
<code>maint print record-instruction</code>	787	<code>maint show gnu-source-highlight enabled</code> ...	791
<code>maint print reggroups</code>	785	<code>maint show ignore-prologue-end-flag</code>	277
<code>maint print register-groups</code>	785	<code>maint show internal-error</code>	781
<code>maint print registers</code>	785	<code>maint show internal-warning</code>	781
<code>maint print remote-registers</code>	785	<code>maint show libopcodes-styling enabled</code>	792
<code>maint print section-scripts</code>	786	<code>maint show per-command</code>	791
<code>maint print statistics</code>	786	<code>maint show profile</code>	789
<code>maint print symbol-cache</code>	277	<code>maint show selftest verbose</code>	787
<code>maint print symbol-cache-statistics</code>	277	<code>maint show show-all-tib</code>	789

maint show show-debug-regs 789
 maint show symbol-cache-size 276
 maint show target-async 789
 maint show target-non-stop 789
 maint show
 test-options-completion-result 793
 maint show test-settings 793
 maint show tui-left-margin-verbose 790
 maint show tui-resize-message 790
 maint show worker-threads 788
 maint space 792
 maint test-options 792
 maint test-remote-args 793
 maint time 792
 maint translate-address 792
 maint undeprecate 780
 maint wait-for-index-cache 793
 maint with 794
 make 21
 make-block-symbols-iterator 573
 make-breakpoint 579
 make-color 589
 make-command 561
 make-enum-hashtable 591
 make-exception 544
 make-field-iterator 553
 make-iterator 587
 make-lazy-value 550
 make-list-iterator 588
 make-parameter 565
 make-pretty-printer 556
 make-pretty-printer-worker 556
 make-value 545
 mark-modified-lines 741
 mark-symlinked-directories 741
 match-hidden-files 741
 may-insert-breakpoints 101
 may-insert-fast-tracepoints 101
 may-insert-tracepoints 101
 may-interrupt 101
 may-write-memory 101
 may-write-registers 101
 mem 186
 memory-port-range 587
 memory-port? 586
 memory-tag check 157
 memory-tag print-allocation-tag 157
 memory-tag print-logical-tag 157
 memory-tag setatag 157
 memory-tag with-logical-tag 157
 MemoryChangedEvent.address 461
 MemoryChangedEvent.length 461
 memset 337
 menu-complete () 755
 menu-complete-backward () 755
 menu-complete-display-prefix 741
 meta-flag 740
 methods 451

M ICommand. __init__ 474
 M ICommand.installed 475
 M ICommand.invoke 475
 M ICommand.name 475
 MissingDebugHandler. __call__ 527
 MissingDebugHandler. __init__ 527
 MissingDebugHandler.enabled 528
 MissingDebugHandler.name 527
 MissingObjfileHandler. __call__ 529
 MissingObjfileHandler. __init__ 529
 MissingObjfileHandler.enabled 530
 MissingObjfileHandler.name 530
 monitor 322

N

n (next) 86
 n (SingleKey TUI key) 601
 N (SingleKey TUI key) 601
 name 451
 name of type_printer 436
 new-ui 597
 newest-frame 571
 NewInferiorEvent.inferior 462
 NewObjFileEvent.new_objfile 460
 NewProgspaceEvent.progspace 464
 NewThreadEvent.inferior_thread 462
 next 86
 next& 98
 next-history (C-n) 750
 next-screen-line () 749
 nexti 89
 nexti& 98
 ni (nexti) 89
 non-incremental-forward-
 search-history (M-n) 750
 non-incremental-reverse-
 search-history (M-p) 750
 nosharedlibrary 299

O

o (SingleKey TUI key) 601
 O (SingleKey TUI key) 601
 Objfile 487
 objfile-filename 568
 objfile-pretty-printers 569
 objfile-progspace 569
 objfile-valid? 568
 Objfile.add_separate_debug_file 489
 Objfile.build_id 488
 Objfile.filename 488
 Objfile.frame_filters 489
 Objfile.is_file 488
 Objfile.is_valid 489
 Objfile.lookup_global_symbol 489
 Objfile.lookup_static_symbol 489
 Objfile.owner 488

Objfile.pretty_printers.....	488
Objfile.progspace.....	488
Objfile.type_printers.....	488
Objfile.username.....	488
objfile?.....	568
objfiles.....	569
observer.....	100
open-memory.....	586
operate-and-get-next (C-o).....	751
output.....	403
output-meta.....	741
output-port.....	586
overlay.....	225
overload-choice annotation.....	716
overwrite-mode ().....	753

P

page-completions.....	742
PARAM_AUTO_BOOLEAN.....	479, 566
PARAM_BOOLEAN.....	479, 566
PARAM_COLOR.....	480, 567
PARAM_ENUM.....	480, 567
PARAM_FILENAME.....	480, 567
PARAM_INTEGER.....	480
PARAM_OPTIONAL_FILENAME.....	480, 567
PARAM_STRING.....	480, 567
PARAM_STRING_NOESCAPE.....	480, 567
PARAM_UIINTEGER.....	480, 566
PARAM_ZINTEGER.....	480, 567
PARAM_ZUIINTEGER.....	480, 567
PARAM_ZUIINTEGER_UNLIMITED.....	480, 567
Parameter.....	477, 565
parameter-value.....	566
Parameter.__init__.....	478
Parameter.get_set_string.....	479
Parameter.get_show_string.....	479
Parameter.set_doc.....	478
Parameter.show_doc.....	478
Parameter.value.....	479
parameter?.....	566
ParameterPrefix.__init__.....	481
ParameterPrefix.dont_repeat.....	482
ParameterPrefix.invoke_set.....	481
ParameterPrefix.invoke_show.....	481
parse-and-eval.....	541
passcount.....	208
path.....	39
pending-breakpoints.....	705
PendingFrame.architecture.....	447
PendingFrame.block.....	447
PendingFrame.create_unwind_info.....	446
PendingFrame.find_sal.....	447
PendingFrame.function.....	447
PendingFrame.is_valid.....	447
PendingFrame.language.....	447
PendingFrame.level.....	447
PendingFrame.name.....	447

PendingFrame.pc.....	447
PendingFrame.read_register.....	446
PgDn.....	601
PgUp.....	601
pi.....	408
pipe.....	21
po (print-object).....	241
possible-completions (M-?).....	754
post-commands annotation.....	716
post-overload-choice annotation.....	716
post-prompt annotation.....	716
post-prompt-for-continue annotation.....	716
post-query annotation.....	716
pre-commands annotation.....	716
pre-overload-choice annotation.....	716
pre-prompt annotation.....	716
pre-prompt-for-continue annotation.....	716
pre-query annotation.....	716
prefix-meta (ESC).....	756
prepend-pretty-printer!.....	591
pretty-printer-enabled?.....	556
pretty-printer?.....	556
pretty-printers.....	556
pretty_printer.child.....	432
pretty_printer.children.....	431
pretty_printer.display_hint.....	431
pretty_printer.num_children.....	432
pretty_printer.to_string.....	431
previous-history (C-p).....	750
previous-screen-line ().....	749
print.....	143
print-last-kbd-macro ().....	755
print-object.....	241
printf.....	403
proc-trace-entry.....	341
proc-trace-exit.....	341
proc-untrace-entry.....	341
proc-untrace-exit.....	341
Progspace.....	484
progspace-filename.....	568
progspace-objfiles.....	568
progspace-pretty-printers.....	568
progspace-valid?.....	567
Progspace.block_for_pc.....	486
Progspace.executable_filename.....	485
Progspace.filename.....	485
Progspace.find_pc_line.....	486
Progspace.frame_filters.....	485
Progspace.is_valid.....	486
Progspace.missing_file_handlers.....	485
Progspace.objfile_for_address.....	486
Progspace.objfiles.....	486
Progspace.pretty_printers.....	485
Progspace.solib_name.....	486
Progspace.symbol_file.....	485
Progspace.type_printers.....	485
progspace?.....	567
progspace.....	568

prompt annotation 716
 prompt-for-continue annotation 716
 ptype 270
 putDebugChar 337
 pwd 40
 py 408
 python 408, 705
 python-interactive 408

Q

q (quit) 20
 q (SingleKey TUI key) 601
 query annotation 716
 queue-signal 281
 quit [expression] 20
 quit annotation 717
 quoted-insert (C-q or C-v) 752

R

r (run) 34
 r (SingleKey TUI key) 601
 rbreak 63
 rc (reverse-continue) 103
 re-read-init-file (C-x C-r) 755
 readnow 294
 rec 105
 rec btrace 105
 rec btrace bts 105
 rec btrace pt 105
 rec bts 105
 rec del 111
 rec full 105
 rec function-call-history 112
 rec instruction-history 111
 rec pt 105
 rec s 106
 recognize on type_recognizer 436
 record 105
 record btrace 105
 record btrace bts 105
 record btrace pt 105
 record bts 105
 record delete 111
 record full 105
 record function-call-history 112
 record goto 107
 record instruction-history 111
 record pt 105
 record restore 107
 record save 107
 record stop 106
 Record.begin 467
 Record.clear 468
 Record.end 467
 Record.format 467
 Record.function_call_history 468

Record.goto 468
 Record.instruction_history 468
 Record.method 467
 Record.replay_position 467
 RecordAuxiliary.data 469
 RecordAuxiliary.number 469
 RecordFunctionSegment.instructions 469
 RecordFunctionSegment.level 469
 RecordFunctionSegment.next 469
 RecordFunctionSegment.number 469
 RecordFunctionSegment.prev 469
 RecordFunctionSegment.symbol 469
 RecordFunctionSegment.up 469
 RecordGap.error_code 469
 RecordGap.error_string 469
 RecordGap.number 468
 RecordInstruction.is_speculative 468
 RecordInstruction.number 468
 RecordInstruction.sal 468
 redraw-current-line () 750
 refresh 604
 register-breakpoint! 580
 register-command! 561
 register-parameter! 566
 register_disassembler 524
 register_filter_factory 537
 register_xmethod_matcher 452
 RegisterChangedEvent.frame 461
 RegisterChangedEvent.regnum 461
 RegisterDescriptor.name 513
 RegisterDescriptorIterator.find 514
 RegisterGroup.name 514
 remote delete 322
 remote get 322
 remote put 322
 RemoteTargetConnection.send_packet 515
 remove-inferiors 46
 remove-symbol-file 296
 restart checkpoint-id 57
 restore 188
 RET (repeat last command) 23
 return 282
 reverse-continue 103
 reverse-finish 104
 reverse-next 104
 reverse-nexti 104
 reverse-search 133
 reverse-search-history (C-r) 750
 reverse-step 103
 reverse-stepi 103
 revert-all-at-newline 742
 revert-line (M-r) 756
 Right 601
 rn (reverse-next) 104
 rni (reverse-nexti) 104
 rs (step) 103
 rsi (reverse-stepi) 103
 run 34

run&..... 98
 rwatch..... 69

S

s (SingleKey TUI key)..... 601
 s (step)..... 86
 S (SingleKey TUI key)..... 602
 sal-last..... 578
 sal-line..... 578
 sal-pc..... 578
 sal-symtab..... 578
 sal-valid?..... 578
 sal?..... 578
 save breakpoints..... 83
 save gdb-index..... 308
 save history..... 373
 save skip..... 92
 save tracepoints..... 220
 save user..... 400
 save-tracepoints..... 220
 search..... 133
 section..... 296
 select-frame..... 121
 selected-frame..... 571
 SelectedContextEvent.frame..... 463
 SelectedContextEvent.inferior..... 463
 SelectedContextEvent.thread..... 463
 self..... 562
 self-insert (a, b, A, 1, !, ...)..... 752
 set..... 31
 set ada print-signatures..... 256
 set ada source-charset..... 261
 set ada trust-PAD-over-XVS..... 262
 set agent off..... 725
 set agent on..... 725
 set always-read-ctf [on|off]..... 276
 set amdgpu precise-memory..... 368
 set annotate..... 715
 set architecture..... 313
 set args..... 38
 set arm..... 351
 set auto-connect-native-target..... 36
 set auto-load gdb-scripts..... 405
 set auto-load guile-scripts..... 590
 set auto-load libthread-db..... 383
 set auto-load local-gdbinit..... 383
 set auto-load off..... 381
 set auto-load python-scripts..... 532
 set auto-load safe-path..... 384
 set auto-load scripts-directory..... 593
 set auto-solib-add..... 298
 set backtrace..... 118
 set basenames-may-differ..... 302
 set breakpoint always-inserted..... 66
 set breakpoint auto-hw..... 66
 set breakpoint condition-evaluation..... 67
 set breakpoint pending..... 66

set can-use-hw-watchpoints..... 69
 set case-sensitive..... 265
 set charset..... 190
 set check range..... 232
 set check type..... 232
 set circular-trace-buffer..... 215
 set code-cache..... 193
 set coerce-float-to-double..... 381
 set com1base..... 343
 set com1irq..... 343
 set com2base..... 343
 set com2irq..... 343
 set com3base..... 343
 set com3irq..... 343
 set com4base..... 343
 set com4irq..... 343
 set complaints..... 386
 set confirm..... 387
 set cp-abi..... 381
 set cwd..... 40
 set cygwin-exceptions..... 344
 set data-directory..... 312
 set dcache line-size..... 193
 set dcache size..... 193
 set debug..... 387
 set debug aarch64..... 356
 set debug arc..... 350
 set debug auto-load..... 386
 set debug bfd-cache level..... 303
 set debug darwin..... 349
 set debug entry-values..... 198
 set debug hppa..... 364
 set debug libthread-db..... 53
 set debug mach-o..... 349
 set debug mips..... 364
 set debug monitor..... 315
 set debug py-breakpoint..... 409
 set debug py-unwind..... 409
 set debug skip..... 92
 set debug threads..... 53
 set debug tui..... 606
 set debug-file-directory..... 304
 set debugevents..... 345
 set debugexceptions..... 345
 set debugexec..... 345
 set debuginfod enabled..... 913
 set debuginfod urls..... 913
 set debuginfod verbose..... 913
 set debugmemory..... 345
 set default-collect..... 212
 set demangle-style..... 169
 set detach-on-fork..... 55
 set direct-call-timeout..... 285
 set directories..... 136
 set disable-randomization..... 37
 set disassemble-next-line..... 142
 set disassembler-options..... 141
 set disassembly-flavor..... 142

set disconnected-dprintf	82	set print	159
set disconnected-tracing	215	set print entry-values	163
set displaced-stepping	779	set print finish	87
set dump-excluded-mappings	189	set print frame-arguments	162
set editing	372	set print frame-info	165
set endian	316	set print inferior-events	46
set environment	39	set print symbol-loading	275
set exceptions, Hurd command	347	set print thread-events	52
set exec-direction	104	set print type hex	266
set exec-done-display	387	set print type methods	265
set exec-wrapper	35	set print type nested-type-limit	265
set extended-prompt	371	set print type typedefs	266
set extension-language	231	set processor	313
set follow-exec-mode	55	set procfs-file	341
set follow-fork-mode	54	set procfs-trace	341
set fortran repack-array-slices	244	set progress-bars enabled	914
set frame-filter priority	126	set prompt	371
set gnutarget	314	set python dont-write-bytecode	409
set guile print-stack	543	set python ignore-environment	408
set hash, for remote monitors	315	set python print-stack	408
set height	374	set radix	380
set history expansion	374	set range-stepping	89
set history filename	372	set ravenscar task-switching off	261
set history remove-duplicates	373	set ravenscar task-switching on	261
set history save	373	set record	111
set history size	373	set record btrace	108
set host-charset	190	set record btrace bts	109
set index-cache	308	set record btrace pt	109
set indirect-call-timeout	285	set record full	107
set inferior-tty	41	set remote	328
set input-radix	379	set remote system-call-allowed	862
set interactive-mode	393	set remote-mips64-transfers-32bit-regs	363
set language	230	set remotecache	193
set libthread-db-search-path	52	set remoteflow	329
set listsize	127	set riscv numeric-register-names	369
set local-environment	394	set schedule-multiple	97
set logging enabled	22	set script-extension	397
set mach-exceptions	350	set sh calling-convention	356
set max-completions	25	set shell	345
set max-user-call-depth	400	set signal-thread	347
set max-value-size	195	set signals, Hurd command	347
set may-call-functions	284	set sigs, Hurd command	347
set mem inaccessible-by-default	187	set sigthread	347
set mi-async	611	set solib-absolute-prefix	300
set mips abi	362	set solib-search-path	301
set mips compression	363	set source open	142
set mips mask-address	363	set stack-cache	193
set mipsfpu	353	set startup-quietly	13
set multiple-symbols	148	set startup-with-shell	36
set new-console	345	set step-mode	87
set new-group	345	set stop-on-solib-events	299
set non-stop	97	set stopped, Hurd command	347
set opaque-type-resolution	274	set struct-convention	361
set osabi	380	set style	375
set output-radix	379	set substitute-path	136
set overload-resolution	238	set suppress-cli-notifications	394
set pagination	375	set sysroot	300
set powerpc	354	set target-charset	190

set target-file-system-kind (unix dos-based auto)	301	show arm	351
set target-wide-charset	190	show auto-load	382
set task, Hurd commands	348	show auto-load gdb-scripts	405
set tcp	330	show auto-load guile-scripts	590
set thread, Hurd command	348	show auto-load libthread-db	383
set trace-buffer-size	215	show auto-load local-gdbinit	383
set trace-commands	387	show auto-load python-scripts	532
set trace-notes	216	show auto-load safe-path	384
set trace-stop-notes	216	show auto-load scripts-directory	593
set trace-user	216	show auto-solib-add	298
set trust-readonly-sections	297	show backtrace	118
set tui active-border-mode	605	show basenames-may-differ	302
set tui border-kind	605	show breakpoint always-inserted	66
set tui border-mode	605	show breakpoint auto-hw	66
set tui compact-source	606	show breakpoint condition-evaluation	67
set tui mouse-events	606	show breakpoint pending	66
set tui tab-width	605	show can-use-hw-watchpoints	69
set unwind-on-signal	283	show case-sensitive	265
set unwind-on-terminating-exception	284	show charset	190
set unwind-on-timeout	284	show check range	232
set unwindonsignal	283	show check type	232
set use-coredump-filter	189	show circular-trace-buffer	215
set variable	279	show code-cache	193
set verbose	386	show coerce-float-to-double	381
set watchdog	794	show com1base	343
set width	374	show com1irq	343
set write	287	show com2base	343
set-breakpoint-condition!	582	show com2irq	343
set-breakpoint-enabled!	581	show com3base	343
set-breakpoint-hit-count!	581	show com3irq	343
set-breakpoint-ignore-count!	581	show com4base	343
set-breakpoint-silent!	581	show com4irq	343
set-breakpoint-stop!	582	show commands	374
set-breakpoint-task!	582	show complaints	386
set-breakpoint-thread!	581	show configuration	31
set-iterator-progress!	588	show confirm	387
set-mark (C-@)	756	show convenience	173
set-objfile-pretty-printers!	569	show copying	31
set-parameter-value!	566	show cp-abi	381
set-pretty-printer-enabled!	556	show cwd	40
set-pretty-printers!	556	show cygwin-exceptions	345
set-progspace-pretty-printers!	568	show data-directory	312
set_debug_traps	336	show dcache line-size	193
share	299	show dcache size	193
sharedlibrary	299	show debug	387
shell	20	show debug arc	350
shell-transpose-words (M-C-t)	753	show debug auto-load	386
show	31	show debug bfd-cache	303
show ada print-signatures	256	show debug darwin	349
show ada source-charset	261	show debug entry-values	199
show ada trust-PAD-over-XVS	262	show debug libthread-db	53
show agent	725	show debug mach-o	349
show always-read-ctf	276	show debug mips	364
show amdgpu precise-memory	368	show debug monitor	315
show annotate	715	show debug py-breakpoint	409
show architecture	313	show debug py-unwind	409
show args	38	show debug skip	92
		show debug threads	53

- show debug tui 606
- show debug-file-directory 304
- show debuginfod enabled 913
- show debuginfod urls 913
- show debuginfod verbose 913
- show default-collect 212
- show detach-on-fork 55
- show direct-call-timeout 285
- show directories 136
- show disassemble-next-line 142
- show disassembler-options 141
- show disassembly-flavor 142
- show disconnected-dprintf 82
- show disconnected-tracing 215
- show displaced-stepping 779
- show editing 372
- show environment 39
- show exceptions, Hurd command 348
- show exec-done-display 387
- show extended-prompt 371
- show follow-fork-mode 55
- show fortran repack-array-slices 244
- show frame-filter priority 126
- show gnutarget 314
- show hash, for remote monitors 315
- show height 374
- show history 374
- show host-charset 190
- show index-cache 309
- show indirect-call-timeout 286
- show inferior-tty 41
- show input-radix 380
- show interactive-mode 393
- show language 231
- show libthread-db-search-path 53
- show listsize 127
- show local-environment 394
- show logging 22
- show mach-exceptions 350
- show max-completions 26
- show max-user-call-depth 400
- show max-value-size 195
- show may-call-functions 284
- show mem inaccessible-by-default 187
- show mi-async 611
- show mips abi 363
- show mips compression 363
- show mips mask-address 363
- show mipsfp 353
- show multiple-symbols 149
- show new-console 345
- show new-group 345
- show non-stop 97
- show opaque-type-resolution 275
- show osabi 380
- show output-radix 380
- show overload-resolution 238
- show pagination 375
- show paths 39
- show print 160
- show print finish 87
- show print inferior-events 46
- show print symbol-loading 275
- show print thread-events 52
- show print type hex 266
- show print type methods 265
- show print type nested-type-limit 265
- show print type typedefs 266
- show processor 313
- show procfs-file 341
- show procfs-trace 341
- show progress-bars enabled 914
- show prompt 371
- show radix 380
- show range-stepping 89
- show ravenscar task-switching 261
- show record 112
- show record btrace 109
- show record full 107
- show remote 328
- show remote system-call-allowed 862
- show remote-mips64-transfers-32bit-regs 364
- show remotecache 193
- show remoteflow 329
- show script-extension 397
- show sh calling-convention 356
- show shell 345
- show signal-thread 347
- show signals, Hurd command 347
- show sigs, Hurd command 347
- show sigthread 347
- show solib-search-path 301
- show source open 142
- show stack-cache 193
- show startup-quietly 13
- show stop-on-solib-events 299
- show stopped, Hurd command 347
- show struct-convention 361
- show style 375
- show substitute-path 137
- show suppress-cli-notifications 395
- show sysroot 301
- show target-charset 190
- show target-file-system-kind 301
- show target-wide-charset 190
- show task, Hurd commands 348
- show tcp 330
- show thread, Hurd command 348
- show trace-buffer-size 216
- show trace-notes 216
- show trace-stop-notes 216
- show trace-user 216
- show unwind-on-signal 284
- show unwind-on-terminating-exception 284
- show unwind-on-timeout 284
- show unwindonsignal 284

show user	399	STYLE_ASSEMBLER_DIRECTIVE	523
show values	173	STYLE_COMMENT_START	524
show verbose	386	STYLE_IMMEDIATE	523
show version	31	STYLE_MNEMONIC	522
show warranty	31	STYLE_REGISTER	523
show width	374	STYLE_SUB_MNEMONIC	522
show write	287	STYLE_SYMBOL	524
show-all-if-ambiguous	742	STYLE_TEXT	522
show-all-if-unmodified	742	StyleParameterSet.__init__(name,	483
show-mode-in-prompt	742	StyleParameterSet.apply(string)	483
si (stepi)	88	StyleParameterSet.style	483
signal	281	StyleParameterSet.value	483
signal annotation	718	symbol-addr-class	574
signal-event	344	symbol-argument?	574
signal-name annotation	717	symbol-constant?	575
signal-name-end annotation	717	symbol-file	293
signal-string annotation	717	symbol-function?	575
signal-string-end annotation	717	symbol-line	574
SignalEvent.stop_signal	460	symbol-linkage-name	574
signalled annotation	717	symbol-name	574
silent	80	symbol-needs-frame?	574
sim, a command	350	symbol-print-name	574
simple-values-ref-types	705	symbol-symtab	574
skip	90	symbol-type	574
skip delete	92	symbol-valid?	574
skip disable	92	symbol-value	575
skip enable	92	symbol-variable?	575
skip file	91	Symbol.addr_class	497
skip function	91	Symbol.domain	497
skip-completed-text	742	Symbol.is_argument	497
skip-csi-sequence ()	756	Symbol.is_artificial	497
source	401	Symbol.is_constant	497
source annotation	718	Symbol.is_function	497
start	35	Symbol.is_valid	498
start-kbd-macro (C-x ())	755	Symbol.is_variable	497
starti	35	Symbol.line	497
starting annotation	717	Symbol.linkage_name	497
STDERR	413	Symbol.name	497
stdio-port?	586	Symbol.needs_frame	497
STDLOG	413	Symbol.print_name	497
STDOUT	413	Symbol.symtab	497
step	86	Symbol.type	496
step&	98	Symbol.value	498
stepi	88	symbol?	574
stepi&	98	SYMBOL_COMMON_BLOCK_DOMAIN	498
stop, a pseudo-command	400	SYMBOL_FUNCTION_DOMAIN	498, 575
StopEvent.details	459	SYMBOL_FUNCTIONS_DOMAIN	576
stopping annotation	717	SYMBOL_LABEL_DOMAIN	498, 576
strace	207	SYMBOL_LOC_ARG	499, 576
string->argv	562	SYMBOL_LOC_BLOCK	499, 577
Style.__init__	510	SYMBOL_LOC_COMMON_BLOCK	500
Style.apply	511	SYMBOL_LOC_COMPUTED	500, 577
Style.background	511	SYMBOL_LOC_CONST	499, 576
Style.escape_sequence	511	SYMBOL_LOC_CONST_BYTES	499, 577
Style.foreground	511	SYMBOL_LOC_LABEL	499
Style.intensity	512	SYMBOL_LOC_LOCAL	499, 577
STYLE_ADDRESS	523	SYMBOL_LOC_OPTIMIZED_OUT	499, 577
STYLE_ADDRESS_OFFSET	523	SYMBOL_LOC_REF_ARG	499, 576

SYMBOL_LOC_REGISTER..... 499, 576
 SYMBOL_LOC_REGPARAM_ADDR..... 499, 577
 SYMBOL_LOC_STATIC..... 499, 576
 SYMBOL_LOC_TPEDEF..... 499, 577
 SYMBOL_LOC_UNDEF..... 499, 576
 SYMBOL_LOC_UNRESOLVED..... 499, 577
 SYMBOL_MODULE_DOMAIN..... 498
 SYMBOL_STRUCT_DOMAIN..... 498, 576
 SYMBOL_TYPE_DOMAIN..... 498, 576
 SYMBOL_TYPES_DOMAIN..... 576
 SYMBOL_UNDEF_DOMAIN..... 498, 575
 SYMBOL_VAR_DOMAIN..... 498, 575
 SYMBOL_VARIABLES_DOMAIN..... 576
 symtab-filename..... 577
 symtab-fullname..... 577
 symtab-global-block..... 578
 symtab-objfile..... 578
 symtab-static-block..... 578
 symtab-valid?..... 577
 Symtab.filename..... 500
 Symtab.fullname..... 501
 Symtab.global_block..... 501
 Symtab.is_valid..... 501
 Symtab.linetable..... 501
 Symtab.objfile..... 500
 Symtab.producer..... 501
 Symtab.source_lines..... 501
 Symtab.static_block..... 501
 symtab?..... 577
 Symtab_and_line.is_valid..... 500
 Symtab_and_line.last..... 500
 Symtab_and_line.line..... 500
 Symtab_and_line.pc..... 500
 Symtab_and_line.symtab..... 500
 sysinfo..... 342

T

taas..... 51
 tab-insert (M-TAB)..... 752
 tabset..... 605
 target..... 314
 target ctf..... 221
 target record..... 105
 target record-btrace..... 105
 target record-full..... 105
 target sim..... 354
 target tfile..... 221
 target-config..... 541
 TargetConnection.description..... 515
 TargetConnection.details..... 515
 TargetConnection.is_valid..... 514
 TargetConnection.num..... 515
 TargetConnection.type..... 515
 task (Ada)..... 258
 tbreak..... 62
 tcatch..... 75
 tdump..... 219

teval (tracepoints)..... 212
 tfaas..... 51
 tfile..... 221
 tfind..... 217
 thbreak..... 63
 this, inside C++ member functions..... 236
 thread apply..... 51
 thread find..... 52
 thread name..... 52
 thread thread-id..... 50
 thread-info..... 705
 ThreadEvent.inferior_thread..... 459
 ThreadExitedEvent.inferior_thread..... 462
 throw-user-error..... 562
 tilde-expand (M-~)..... 756
 trace..... 206
 transpose-chars (C-t)..... 752
 transpose-words (M-t)..... 752
 tsave..... 221
 tstart [notes]..... 214
 tstatus..... 214
 tstop [notes]..... 214
 tty..... 41
 tui disable..... 602
 tui enable..... 602
 tui layout..... 603
 tui new-layout..... 603
 tui refresh..... 604
 tui reg..... 604
 tui window height..... 604
 tui window width..... 605
 TuiEnabledEvent.enabled..... 464
 TuiWindow.erase..... 516
 TuiWindow.height..... 516
 TuiWindow.is_valid..... 516
 TuiWindow.title..... 516
 TuiWindow.width..... 516
 TuiWindow.write..... 516
 tvariable..... 210
 type-array..... 552
 type-code..... 551
 type-const..... 553
 type-field..... 553
 type-fields..... 553
 type-has-field-deep?..... 591
 type-has-field?..... 553
 type-name..... 552
 type-num-fields..... 553
 type-pointer..... 552
 type-print-name..... 552
 type-range..... 552
 type-reference..... 552
 type-sizeof..... 552
 type-strip-typedefs..... 552
 type-tag..... 552
 type-target..... 553
 type-unqualified..... 553
 type-vector..... 552

type-volatile	553
Type.alignof	425
Type.array	427
Type.code	425
Type.const	428
Type.dynamic	425
Type.fields	426
Type.is_array_like	426
Type.is_scalar	426
Type.is_signed	426
Type.is_string_like	426
Type.name	426
Type.objfile	426
Type.optimized_out	428
Type.pointer	428
Type.range	428
Type.reference	428
Type.sizeof	426
Type.strip_typedefs	428
Type.tag	426
Type.target	428
Type.template_argument	428
Type.unqualified	428
Type.vector	427
Type.volatile	428
type?	551
TYPE_CODE_ARRAY	429, 554
TYPE_CODE_BITSTRING	429, 554
TYPE_CODE_BOOL	430, 555
TYPE_CODE_CHAR	430, 555
TYPE_CODE_COMPLEX	430, 555
TYPE_CODE_DECFLOAT	430, 555
TYPE_CODE_ENUM	429, 554
TYPE_CODE_ERROR	429, 554
TYPE_CODE_FIXED_POINT	430, 555
TYPE_CODE_FLAGS	429, 554
TYPE_CODE_FLT	429, 554
TYPE_CODE_FUNC	429, 554
TYPE_CODE_INT	429, 554
TYPE_CODE_INTERNAL_FUNCTION	430, 555
TYPE_CODE_MEMBERPTR	429, 554
TYPE_CODE_METHOD	429, 554
TYPE_CODE_METHODPTR	429, 554
TYPE_CODE_NAMESPACE	430, 555
TYPE_CODE_PTR	429, 554
TYPE_CODE_RANGE	429, 554
TYPE_CODE_REF	429, 555
TYPE_CODE_RVALUE_REF	430, 555
TYPE_CODE_SET	429, 554
TYPE_CODE_STRING	429, 554
TYPE_CODE_STRUCT	429, 554
TYPE_CODE_TYPEDEF	430, 555
TYPE_CODE_UNION	429, 554
TYPE_CODE_VOID	429, 554
TYPE_CODE_XMETHOD	430, 555

U

u (SingleKey TUI key)	602
u (until)	87
undefined-command-error-code	705
undisplay	158
undo (C-_ or C-x C-u)	756
universal-argument ()	754
unix-filename-rubout ()	753
unix-line-discard (C-u)	753
unix-word-rubout (C-w)	753
unset environment	39
unset local-environment	395
unset substitute-path	137
until	87
until&	99
unwind-stop-reason-string	572
Up	601
up	120
up-silently	121
uppercase-word (M-u)	752
update	604

V

v (SingleKey TUI key)	602
value->bool	548
value->bytevector	549
value->integer	548
value->lazy-string	549
value->real	549
value->string	549
value-abs	551
value-add	550
value-address	546
value-call	548
value-cast	547
value-const-value	548
value-dereference	547
value-div	550
value-dynamic-cast	547
value-dynamic-type	546
value-fetch-lazy!	550
value-field	548
value-lazy?	550
value-logand	551
value-logior	551
value-lognot	551
value-logxor	551
value-lsh	551
value-max	551
value-min	551
value-mod	550
value-mul	550
value-neg	551
value-not	551
value-optimized-out?	546
value-pos	551
value-pow	551

value-print 550
 value-reference-value 548
 value-referenced-value 548
 value-reinterpret-cast 547
 value-rem 550
 value-rsh 551
 value-rvalue-reference-value 548
 value-sub 550
 value-subscript 548
 value-type 546
 Value.__init__ 419, 420
 Value.address 418
 Value.assign 420
 Value.bytes 419
 Value.cast 420
 Value.const_value 421
 Value.dereference 420
 Value.dynamic_cast 421
 Value.dynamic_type 419
 Value.fetch_lazy 424
 Value.format_string 422
 Value.is_lazy 419
 Value.is_optimized_out 418
 Value.is_unavailable 418
 Value.lazy_string 424
 Value.reference_value 421
 Value.referenced_value 421
 Value.reinterpret_cast 422
 Value.string 424
 Value.to_array 424
 Value.type 419
 value<=? 551
 value<? 551
 value=? 551
 value>=? 551
 value>? 551
 value? 545
 vi-cmd-mode-string 743
 vi-editing-mode (M-C-j) 757
 vi-ins-mode-string 743
 visible-stats 743

W

w (SingleKey TUI key) 602
 w (with) 24
 warn-language-frame-mismatch 230
 watch 68
 watchpoint annotation 718
 whatis 267
 where 117
 while 402
 while-stepping (tracepoints) 212
 Window.click 517
 Window.close 517
 Window.hscroll 517
 Window.render 517
 Window.vscroll 517
 winheight 604
 winwidth 605
 with command 24
 WP_ACCESS 504, 580
 WP_READ 504, 580
 WP_WRITE 504, 580

X

x (examine memory) 154
 x(examine), and info line 138
 XMethod.__init__ 451
 XMethodMatcher.__init__ 451
 XMethodMatcher.match 451
 XMethodWorker.__call__ 452
 XMethodWorker.get_arg_types 451
 XMethodWorker.get_result_type 452

Y

yank (C-y) 754
 yank-last-arg (M-. or M-_) 751
 yank-nth-arg (M-C-y) 751
 yank-pop (M-y) 754

The body of this manual is set in
 `cmr10`,
with headings in **`cmbx10`**
and examples in **`cmtt10`**.
 `cmti10`,
 `cmbx10`, and
 `cmsl10`
are used for emphasis.